

Avalanche Overview Specification

Revision1.2

Original Release Date: 01 NOV 1999
Revised: 03 SEP 2000

Transportation Systems Group
Motorola Japan Ltd.

Motorola reserves the right to make changes without further notice to any products herein to improve reliability, function or design. Motorola does not assume any liability arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights nor the rights of others. Motorola products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the Motorola product could create a situation where personal injury or death may occur. Should Buyer purchase or use Motorola products for any such unintended or unauthorized application, Buyer shall indemnify and hold Motorola and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that Motorola was negligent regarding the design or manufacture of the part.

Revision History

Release Number	Date	Author	Summary of Changes
0.1	01 NOV 99	Haru Y.	- Original creation from SIKA
0.2	01 JUN 00	Haru Y.	- fixed EBI(24 address/32 data /8 CS) -Corrected DSRC interface pins -Removed 16 GPIO -Removed TIMER2 -Added 32Khz OSC -Changed PKG from 196MAPBGA to 160QFP -Operatin frequency from 20 to 32.768MHz -Timer2 module select has swapped by DSRC module -Added 3 DSRC interrupts -Only updates Section 1
0.3	08 JUN 00	Haru Y.	- Corrected CS address decoding - Removed PORTC/D - Corrected X'tal OSC PINS definition - Removed Appendix a,b,c and d
0.4	19 JUN 00	Haru Y.	-Update all chapters based on SIKA Rev1.2
1.0	07 AUG 00	Haru Y.	-Release version of the Avalanche Specification
1.1	03 SEP 00	Haru Y.	- Added DSRC module spec - Corrected pin number on signal descriptions on Section2 - Recover Single chip mode on Appendix Test
1.2	19 SEP 00	Haru Y.	- Corrected DSRC bufferRAM address

Section 1 Introduction

1.1	Overview.	27
1.2	Avalanche Description	30
1.3	Avalanche Address Map.	32

Section 2 Signal Descriptions

2.1	Overview.	35
2.2	Avalanche Specific Implementation Pin Issues	37
2.2.1	RSTOUT Pin Functions	37
2.2.2	INT Pin Functions.	37
2.2.3	SPI Pin Functions.	38
2.2.4	SCI1 and SCI2 Pin Functions	38
2.2.5	Timer Pin Functions	39
2.2.6	DSRC Pin Functions(customer designed module).	39
2.3	Signal Descriptions.	40
2.3.1	Reset Signals	40
2.3.1.1	Reset In (RESET)	40
2.3.1.2	Reset Out (RSTOUT)	40
2.3.2	PLL and Clock Signals.	40
2.3.2.1	External Clock In (EXTAL).	40
2.3.2.2	Crystal (XTAL)	40
2.3.2.3	Clock Out (CLKOUT)	40
2.3.2.4	Synthesizer Power (VDDsyn, VSSsyn)	40
2.3.2.5	External OSC In (EXOSCL).	41
2.3.2.6	OSC (OSC)	41
2.3.2.7	Oscillator Power (VDDosc, VSSosc)	41
2.3.3	External Memory Interface Signals	41
2.3.3.1	Data Bus (D[31:0])	41
2.3.3.2	Show Cycle Strobe (SHS)	41
2.3.3.3	Transfer Acknowledge (TA)	41
2.3.3.4	Transfer Error Acknowledge (TEA)	42
2.3.3.5	Transfer Code (TC[2:0]).	42
2.3.3.6	Read/Write (R/W)	42
2.3.3.7	Address Bus (A[23:0])	42

2.3.3.8	Enable Byte (EB[3:0])	42
2.3.3.9	Chip Select (CS7 - CS0)	42
2.3.3.10	Output Enable (OE)	42
2.3.4	Edge Port Signals	42
2.3.4.1	External Interrupts (INT[7:6])	42
2.3.4.2	External Interrupts (INT[5:2])	43
2.3.4.3	External Interrupts (INT[1:0])	43
2.3.5	Serial Peripheral Interface Module Signals	43
2.3.5.1	Master Out/Slave In (MOSI)	43
2.3.5.2	Master In/Slave Out (MISO)	43
2.3.5.3	Serial Clock (SCK)	43
2.3.5.4	Slave Select (SS)	43
2.3.6	Serial Communications Interface Module Signals	43
2.3.6.1	Receive Data (RXD1, RXD2)	44
2.3.6.2	Transmit Data (TXD1, TXD2)	44
2.3.7	Timer Signals	44
2.3.7.1	ICOC[3:0]	44
2.3.8	Debug and Emulation Support Signals	44
2.3.8.1	Test Reset (TRST)	44
2.3.8.2	Test Clock (TCLK)	44
2.3.8.3	Test Mode Select (TMS)	44
2.3.8.4	Test Data Input (TDI)	44
2.3.8.5	Test Data Output (TDO)	45
2.3.8.6	Debug Event (DE)	45
2.3.9	Test Signal	46
2.3.9.1	TEST	46
2.3.10	DSRC Pins	46
2.3.10.1	DSRCO[2:0]	46
2.3.10.2	DSRCI[1:0]	46
2.3.11	Power and Ground Pins	46
2.3.11.1	Standby Power (VSTBY)	46
2.3.11.2	VDDSYN	46
2.3.11.3	VSSSYN	46
2.3.11.4	VDDOSC	46

2.3.11.5	VSSOSC	47
2.3.11.6	Positive Supply (VDD)	47
2.3.11.7	Ground (VSS)	47

Section 3 Low-Power Modes

3.1	Overview.	49
3.1.1	Low-Power Modes	49
3.1.1.1	RUN Mode	49
3.1.1.2	WAIT Mode	50
3.1.1.3	DOZE Mode	50
3.1.1.4	STOP Mode	50
3.1.1.5	Peripheral Shut Down	50
3.1.2	Peripheral Behavior in Low-Power Modes	50
3.1.2.1	Reset	50
3.1.2.2	Clocks	51
3.1.2.3	OnCE	51
3.1.2.4	JTAG	52
3.1.2.5	Interrupt Controller	52
3.1.2.6	Edge Port	52
3.1.2.7	RAM	52
3.1.2.8	Watchdog Timer	52
3.1.2.9	PIT1/2	53
3.1.2.10	SPI	53
3.1.2.11	SCI1/2	53
3.1.2.12	Timer	53
3.1.2.13	Time Of the Day timer	54
3.1.2.14	DSRC module	54
3.1.2.15	Peripheral State During Low-Power Modes Summary	54

Section 4 Clocks

4.1	System Clock Modes	58
4.2	System Clocks Generation	59
4.2.1	Programming System Clocks Frequency	59
4.3	PLL Lock Detection	60

4.3.1	PLL Loss of Lock Conditions	61
4.3.2	PLL Loss of Lock Reset	62
4.4	Loss of Clock Detection	62
4.4.1	Alternate Clock Selection	62
4.4.2	Loss of Clock Reset	63
4.5	Clock Operation During Reset	63
4.6	Clock Operation During Low Power Modes	64
4.7	Programming Model	68
4.7.1	Synthesizer Control Register (SYNCR)	68
4.7.2	Synthesizer Status Register (SYNSR)	71
4.7.3	Synthesizer Test Register (SYNTR)	73
4.7.4	Synthesizer Test Register 2 (SYNTR2)	73
4.8	PLL Operation	74
4.8.1	Phase/Frequency Detector (PFD)	75
4.8.2	VCO	76
4.8.3	MFD	76

Section 5 Reset

5.1	Overview.	77
5.2	Sources of Reset	78
5.2.1	Power On Reset.	78
5.2.2	External Reset	78
5.2.3	Watchdog Timer Reset	79
5.2.4	Loss of Clock Reset	79
5.2.5	Loss of Lock Reset.	79
5.2.6	Software Reset.	79
5.3	Reset Control Flow.	79
5.3.1	Synchronous Reset Requests	80
5.3.2	Internal Reset Request	80
5.3.3	Power-On Reset.	80
5.4	Programming Model	82
5.4.1	Reset Control Register (RCR)	82
5.4.2	Reset Status Register (RSR)	83
5.4.3	Reset Test Register (RTR).	84

5.5	Concurrent Resets	84
5.5.1	Reset Flow	84
5.5.2	Reset Status Flags	85
5.6	Half Reset.	85

Section 6 Interrupt Controller

6.1	Overview.	87
6.1.1	Interrupt Sources and Prioritization	88
6.1.2	Fast and Normal Interrupt Requests	89
6.1.3	Autovectored and Vectored Interrupt Requests	90
6.1.4	Low Power Mode Operation.	91
6.2	Programming Model	91
6.2.1	Interrupt Control Register (ICR)	92
6.2.2	Interrupt Status Register (ISR)	94
6.2.3	Interrupt Force Registers (IFRH/L)	95
6.2.4	Interrupt Pending Register (IPR)	96
6.2.5	Normal Interrupt Enable Register (NIER)	96
6.2.6	Normal Interrupt Pending Register (NIPR)	97
6.2.7	Fast Interrupt Enable Register (FIER)	97
6.2.8	Fast Interrupt Pending Register (FIPR)	98
6.2.9	Priority Level Select Registers (PLSR39 - PLSR0)	99
6.3	Interrupt Source Assignments	100
6.4	Interrupt Configuration	101
6.4.1	M•CORE Processor Setup	101
6.4.2	Interrupt Controller	102
6.4.3	Interrupt Source Setup	102

Section 7 M210 CPU and OnCE Debug

7.1	M•CORE Overview.	103
7.2	Features	103
7.3	Microarchitecture Summary	104
7.4	Programming Model	106
7.5	Data Format Summary	107
7.6	Operand Addressing Capabilities	108

7.7	Instruction Set Overview.	109
7.8	OnCE Debug Module	112
7.8.1	Operation	112
7.8.2	OnCE Pins	114
7.8.2.1	Debug Serial Input (TDI)	114
7.8.2.2	Debug Serial Clock (TCK)	114
7.8.2.3	Debug Serial Output (TDO)	114
7.8.2.4	Debug Mode Select (TMS)	115
7.8.2.5	Test Reset (TRST).	115
7.8.2.6	Debug Event (DE)	115
7.8.3	OnCE Controller and Serial Interface.	115
7.8.4	OnCE Interface Signals	116
7.8.4.1	Internal Debug Request Input (IDR)	116
7.8.4.2	CPU Debug Request (DBGREQ).	117
7.8.4.3	CPU Debug Acknowledge (DBGACK).	117
7.8.4.4	CPU Breakpoint Request (BRKREQ)	117
7.8.4.5	CPU Address, Attributes (ADDR, ATTR).	117
7.8.4.6	CPU Status (PSTAT)	117
7.8.4.7	OnCE Debug Output (DEBUG)	117
7.8.5	OnCE Controller Registers.	118
7.8.5.1	OnCE Command Register (OCMR).	118
7.8.5.2	OnCE Control Register (OCR).	120
7.8.5.3	OnCE Status Register (OSR)	123
7.8.6	OnCE Decoder (ODEC).	125
7.8.7	Memory Breakpoint Logic	125
7.8.7.1	Memory Address Latch (MAL)	127
7.8.7.2	Breakpoint Address Base Registers (BABA, BABB)	127
7.8.7.3	Breakpoint Address Mask Registers (BAMA, BAMB)	127
7.8.7.4	Breakpoint Address Comparators	127
7.8.7.5	Memory Breakpoint Counters (MBCA, MBCB)	127
7.8.8	OnCE Trace Logic	128
7.8.8.1	Trace Counter (OTC).	128
7.8.8.2	Trace Operation.	128
7.8.9	Methods of Entering Debug Mode	129

7.8.9.1	Debug Request During RESET	129
7.8.9.2	Debug Request During Normal Activity	129
7.8.9.3	Debug Request During Stop, Doze, or Wait Mode	130
7.8.9.4	Software Request During Normal Activity	130
7.8.10	Enabling OnCE Trace Mode	130
7.8.11	Enabling OnCE Memory Breakpoints	130
7.8.12	Pipeline Information and Write-Back Bus Register	130
7.8.12.1	Program Counter Register (PC)	131
7.8.12.2	Instruction Register (IR)	131
7.8.12.3	Control State Register (CTL)	132
7.8.12.4	Write-Back Bus Register (WBBR)	133
7.8.12.5	Processor Status Register (PSR)	133
7.8.13	Instruction Address FIFO Buffer (PC FIFO)	133
7.8.14	Reserved Test Control Registers (MEM_BIST, FTCCR, LSRL)	135
7.8.15	Serial Protocol	135
7.8.16	OnCE Commands	135
7.8.17	Target Site Debug System Requirements	135
7.8.18	Interface Connector For JTAG/OnCE Serial Port	136

Section 8 Static Random Access Memory

8.1	Overview.	137
8.2	Programming Model	137
8.3	Read / Write Transactions	137
8.4	Reset Operation	137
8.5	Aborted Accesses.	138
8.6	Stand-by Operation	138

Section 9 Ports

9.1	Overview.	141
9.1.1	Port Operation	141
9.1.2	Port Digital I/O Timing	143
9.2	Programming Model	143
9.2.1	Port Output Data Registers (PORTA-B)	144
9.2.2	Port Data Direction Registers (DDRA-B)	145

9.2.3	Port Pin/Set Data Registers (PORTAP-BP/SETA-B)	145
9.2.4	Port Clear Output Data Registers (CLRA-B)	146

Section 10 Edge Port

10.1	Overview.	147
10.2	Programming Model	147
10.2.1	Edge Port Pin Assignment Register (EPPAR)	148
10.2.2	Edge Port Data Direction Register (EPDDR)	149
10.2.3	Edge Port Flag Enable Register (EPFER)	149
10.2.4	Edge Port Data Register (EPDR)	150
10.2.5	Edge Port Pin Data Register (EPPDR)	150
10.2.6	Edge Port Flag Register (EPFR)	151
10.3	Interrupt/General-Purpose I/O Pin Descriptions	151
10.4	Low-Power Mode Operation	152
10.5	Software Generated Interrupts	152

Section 11 Watchdog Timer

11.1	Overview.	153
11.1.1	Time-out Specifications	153
11.2	Programming Model	153
11.2.1	Watchdog Control Register (WCR)	154
11.2.2	Watchdog Modulus Register (WMR)	155
11.2.3	Watchdog Count Register (WCNTR)	156
11.2.4	Watchdog Service Register (WSR)	156

Section 12 Programmable Interval Timer

12.1	Overview.	157
12.1.1	PIT as a "Set-and-Forget" Timer	157
12.1.2	PIT as a "Free-Running" Timer	158
12.1.3	Time-out Specifications	158
12.2	Programming Model	158
12.2.1	PIT Control/Status Register (PCSR)	159
12.2.2	PIT Modulus Register (PMR)	161
12.2.3	PIT Count Register (PCNTR)	162

Section 13 Timer

13.1	Overview	163
13.2	Features	163
13.3	Modes of Operation	164
13.3.1	Supervisor Option	164
13.3.2	Low-Power Options	164
13.3.3	Test Option	164
13.4	Block Diagram	164
13.5	Timer Signal Description	166
13.5.1	Overview	166
13.5.2	Detailed Signal Descriptions	166
13.5.2.1	ICOC[2:0]	166
13.5.2.2	ICOC[3]	166
13.5.2.3	sync	166
13.6	Timer Memory Map and Registers	167
13.6.1	Overview	167
13.6.2	Module Memory Map	167
13.6.3	Register Quick Reference	168
13.6.4	Register Descriptions	170
13.6.4.1	Timer Input Capture/Output Compare (IC/OC) Select Register	171
13.6.4.2	Compare Force Register	171
13.6.4.3	Output Compare 3 Mask Register	172
13.6.4.4	Output Compare 3 Data Register	172
13.6.4.5	Timer Counter Registers	173
13.6.4.6	Timer System Control Register 1	174
13.6.4.7	Timer Toggle On Overflow (TOV) Register	175
13.6.4.8	Timer Control Register 1	176
13.6.4.9	Timer Control Register 2	176
13.6.4.10	Timer Interrupt Enable Register	177
13.6.4.11	Timer System Control Register 2	178
13.6.4.12	Timer Flag Register 1	179
13.6.4.13	Timer Flag Register 2	180
13.6.4.14	Timer Channel Registers	181
13.6.4.15	Pulse Accumulator Control Register	182

13.6.4.16	Pulse Accumulator Flag Register	184
13.6.4.17	Pulse Accumulator Counter Registers	185
13.6.4.18	Timer Port Data Register	186
13.6.4.19	Timer Port Data Direction Register	186
13.6.4.20	Timer Test Register	187
13.7	Timer Functional Description	188
13.7.1	General	188
13.7.2	Prescaler	188
13.7.3	Input Capture	188
13.7.4	Output Compare	188
13.7.5	Pulse Accumulator	189
13.7.5.1	Event Counter Mode	189
13.7.5.2	Gated Time Accumulation Mode	190
13.7.6	General-Purpose I/O Ports	191
13.7.6.1	Timer Port	191
13.8	Timer Reset	193
13.8.1	General	193
13.8.2	sync Signal Reset	193
13.9	Timer Interrupts	194
13.9.1	General	194
13.9.2	Description of Interrupt Operation	194
13.9.2.1	timchxint_b	194
13.9.2.2	Channel Flag Clearing	195
13.9.2.3	timpavint_b	195
13.9.2.4	Pulse Accumulator Overflow Flag Clearing	195
13.9.2.5	timpaint_b	196
13.9.2.6	Pulse Accumulator Input Flag Clearing	196
13.9.2.7	timovint_b	196
13.9.2.8	Timer Overflow Flag Clearing	197

Section 14 Time Of Day timer

14.1	Time-of-Day Timer	199
14.1.1	TOD Operation	199
14.1.2	TOD in Low-Power Modes	200

14.1.3	Time-of-Day Control/Status Register (TODCSR)	200
14.1.4	TOD Seconds Register (TODSR)	201
14.1.5	TOD Fraction Register (TODFR)	201
14.1.6	TOD Seconds Alarm Register (TODSAR)	202
14.1.7	TOD Fraction Alarm Register (TODFAR)	202

Section 15 Serial Communications Interface

15.1	Overview	205
15.2	Features	205
15.3	Block Diagram	206
15.4	Register Map	206
15.5	Functional Description	208
15.5.1	Data Format	208
15.5.2	Baud Rate Generation	209
15.5.3	Transmitter	211
15.5.3.1	Character Length	211
15.5.3.2	Character Transmission	211
15.5.3.3	Break Characters	213
15.5.3.4	Idle Characters	213
15.5.4	Receiver	214
15.5.4.1	Character Length	214
15.5.4.2	Character Reception	214
15.5.4.3	Data Sampling	215
15.5.4.4	Framing Errors	220
15.5.4.5	Baud Rate Tolerance	220
15.5.4.6	Receiver Wakeup	222
15.5.5	Single-Wire Operation	223
15.5.6	Loop Operation	224
15.6	Register Descriptions	225
15.6.1	SCI Baud Rate Registers	225
15.6.2	SCI Control Register 1	226
15.6.3	SCI Control Register 2	228
15.6.4	SCI Status Register 1	230
15.6.5	SCI Status Register 2	232

15.6.6	SCI Data Registers	232
15.7	External Pin Descriptions	233
15.7.1	TXD Pin	233
15.7.2	RXD Pin	233
15.8	Reset Initialization	233
15.9	Modes of Operation	234
15.10	Low-Power Options	234
15.10.1	Run Mode	234
15.10.2	Wait Mode	234
15.10.3	Stop Mode	234
15.11	Interrupt Operation	235
15.11.1	Interrupt Sources	235
15.11.2	Recovery from Wait Mode	235
15.12	General-Purpose I/O Ports	235
15.12.1	SCI Port Data Register	235
15.12.2	SCI Data Direction Register	236
15.12.3	SCI Pullup and Reduced Drive Register	236

Section 16 Serial Peripheral Interface Module

16.1	Introduction	239
16.2	Features	239
16.3	Block Diagram	240
16.4	Register Map	240
16.5	Functional Description	242
16.5.1	Master Mode	243
16.5.2	Slave Mode	243
16.5.3	Transmission Formats	244
16.5.3.1	Clock Phase and Polarity Controls	245
16.5.3.2	CPHA = 0 Transfer Format	245
16.5.3.3	CPHA = 1 Transfer Format	247
16.5.4	SPI Baud Rate Generation	249
16.5.5	Special Features	249
16.5.5.1	$\overline{SS}$ Output	249
16.5.5.2	Bidirectional Mode (MOMI or SISO)	249

16.5.6	Error Conditions	250
16.5.6.1	Write Collision Error	250
16.5.6.2	Mode Fault Error	251
16.6	Register Descriptions	251
16.6.1	SPI Control Register 1	251
16.6.2	SPI Control Register 2	253
16.6.3	SPI Baud Rate Register	254
16.6.4	SPI Status Register	255
16.6.5	SPI Data Register	256
16.7	External Pin Descriptions	256
16.7.1	MISO (Master In/Slave Out)	256
16.7.2	MOSI (Master Out/Slave In)	257
16.7.3	SCK (Serial Clock)	257
16.7.4	$\overline{SS}$ (Slave Select)	257
16.8	Reset Initialization	257
16.9	Modes of Operation	258
16.10	Low-Power Options	258
16.10.1	SPI in Run Mode	258
16.10.2	SPI in Wait Mode	258
16.10.3	SPI in Stop Mode	258
16.11	Interrupt Operation	258
16.12	General-Purpose I/O Ports	259
16.12.1	SPI Port Reduced Drive and Pullup Select Register	259
16.12.2	SPI Port Data Register	260
16.12.3	SPI Port Data Direction Register	261

Section 17 DSRC

17.1	DSRC	263
17.1.1	DSRC Operation	263
17.1.2	DSRC in Low-Power Modes	264
17.1.3	DSRC MyLIDH register(MyLIDH)	264
17.1.4	DSRC MyLIDL register(MyLIDL)	264
17.1.5	DSRC GID register(GIDR)	265
17.1.6	DSRC ACTC register(ACTCR)	265

17.1.7	DSRC MyKey register(MyKeyR)	266
17.1.8	DSRC BrSLT register(BrSLTR)	266
17.1.9	DSRC INTF register(INTFR)	266
17.1.10	DSRC INTE register(INTER)	267
17.1.11	DSRC INTC register(INTEC)	267
17.1.12	DSRC DRV register(DRVR)	267
17.1.13	DSRC BMG register(BMGR)	268
17.1.14	DSRC Status register(STATR)	268
17.1.15	DSRC Soft Reset(SWRR)	269
17.1.16	DSRC MDCE(MDCER)	270
17.1.17	DSRC ACTE(ACTER)	270
17.1.18	TxBUF	270
17.1.19	RxBUF	270

Section 18 Chip Selects

18.1	Overview.	271
18.1.1	Operation	273
18.2	Programming Model	274
18.2.1	Chip Select Control Registers (CSCR0 - CSCR7)	274

Section 19 External Bus Interface

19.1	Overview.	279
19.2	Bus Master Operation.	279
19.2.1	Operand Transfer Overview.	279
19.2.1.1	Byte Enable Pins (EB[3:0])	280
19.2.2	Bus Master Cycles	281
19.2.2.1	Read Cycles	281
19.2.2.2	Write Cycles.	284
19.2.3	Bus Exception Operation	288
19.2.3.1	Transfer Error Termination	288
19.2.3.2	Transfer Abort Termination	288
19.2.4	Show Strobe (SHS)	288
19.3	Monitors	290
19.3.1	Bus Monitor	290

19.3.2	Abort Monitor	291
--------	-------------------------	-----

Section 20 JTAG

20.1	IEEE 1149.1 Test Access Port	293
20.2	Overview.	294
20.3	TAP Controller	296
20.4	Instruction Shift Register	297
20.4.1	EXTEST Instruction	299
20.4.2	IDCODE Instruction	299
20.4.3	SAMPLE/PRELOAD Instruction.	299
20.4.4	ENABLE_MCU_ONCE Instruction.	300
20.4.5	HIGHZ Instruction	300
20.4.6	CLAMP Instruction	300
20.4.7	BYPASS Instruction	300
20.5	IDcode Register	301
20.6	Bypass Register	301
20.7	Boundary SCAN Register.	301
20.8	Restrictions.	302
20.9	Non-SCAN Chain Operation.	302
20.10	Boundary Scan.	302

Appendix A Electrical Characteristics

A.1	Maximum Ratings.	314
A.2	Thermal Characteristics	315
A.3	ESD Protection.	316
A.4	DC Electrical Specifications	317
A.5	Phase Lock Loop Electrical Specifications.	320
A.6	External Interface Timing Characteristics.	322
A.7	General Purpose I/O Timing.	327
A.8	Reset and Configuration Override Timing	328
A.9	SPI Timing Characteristics	329
A.10	OnCE, JTAG and Boundary Scan Timing	332

Appendix B Package

B.1 Pinout Table 338

Figure 1-1	Avalanche Block Diagram	31
Figure 1-2	Avalanche Address Map	32
Figure 4-1	System Clocks Block Diagram.	57
Figure 4-2	Lock Detect Sequence.	61
Figure 4-3	Synthesizer Control Register.	68
Figure 4-4	Synthesizer Status Register	71
Figure 4-5	Synthesizer Test Register	73
Figure 4-6	Synthesizer Test Register 2.	73
Figure 4-7	PLL Block Diagram	74
Figure 4-8	Crystal Oscillator Network	75
Figure 5-1	Reset Block Diagram.	77
Figure 5-3	Reset Control Register	82
Figure 5-4	Reset Status Register	83
Figure 5-5	Reset Test Register.	84
Figure 6-1	Interrupt Controller Block Diagram	88
Figure 6-2	Interrupt Control Register	93
Figure 6-3	Interrupt Status Register	94
Figure 6-4	Interrupt Force Register High	95
Figure 6-5	Interrupt Force Register Low.	95
Figure 6-6	Interrupt Pending Register.	96
Figure 6-7	Normal Interrupt Enable Register High	96
Figure 6-8	Normal Interrupt Pending Register	97
Figure 6-9	Fast Interrupt Enable Register.	98
Figure 6-10	Fast Interrupt Pending Register.	98
Figure 6-11	Priority Level Select Register x	99
Figure 7-2	Programming Model	107
Figure 7-3	Data Organization in Memory	108
Figure 7-4	Data Organization in Registers	108
Figure 7-5	OnCE Block Diagram.	112
Figure 7-6	OnCE Controller	113
Figure 7-7	OnCE Controller and Serial Interface	116
Figure 7-8	OnCE Command Register.	118
Figure 7-9	OnCE Control Register	121
Figure 7-10	OnCE Status Register	124

Figure 7-11	OnCE Memory Breakpoint Logic	126
Figure 7-12	OnCE Trace Logic Block Diagram.	128
Figure 7-13	CPU Scan Chain Register (CPUSCR).	131
Figure 7-14	Control State Register	132
Figure 7-15	OnCE PC FIFO	134
Figure 7-16	Recommended Connector Interface to JTAG/OnCE Port . . .	136
Figure 8-1	Read / Write / Read Sequence	138
Figure 9-1	Block Diagram	141
Figure 9-2	Digital Input Timing	143
Figure 9-3	Digital Output Timing	143
Figure 9-4	Port Output Data Registers	145
Figure 9-5	Port Data Direction Registers	145
Figure 9-6	Port Pin/Set Registers	146
Figure 9-7	Port Clear Output Data Registers	146
Figure 10-1	Edge Port Block Diagram	147
Figure 10-2	Edge Port Pin Assignment Register	148
Figure 10-3	Edge Port Data Direction Register.	149
Figure 10-4	Edge Port Flag Enable Register	150
Figure 10-5	Edge Port Data Register	150
Figure 10-6	Edge Port Pin Data Register	151
Figure 10-7	Edge Port Flag Register	151
Figure 11-1	Watchdog Timer Block Diagram	153
Figure 11-2	Watchdog Control Register	154
Figure 11-3	Watchdog Modulus Register	155
Figure 11-4	Watchdog Count Register	156
Figure 11-5	Watchdog Service Register.	156
Figure 12-1	PIT Block Diagram.	157
Figure 12-3	Counter in Free-Running Mode	158
Figure 12-4	PIT Control and Status Register	159
Figure 12-5	PIT Modulus Register	161
Figure 12-6	PIT Count Register	162
Figure 13-2	Timer IC/OC Select Register (TIMIOS).	171
Figure 13-3	Compare Force Register (TIMCFORC).	171
Figure 13-4	Output Compare 3 Mask Register (TIMOC3M).	172

Figure 13-5	Output Compare 3 Data Register (TIMOC3D)	172
Figure 13-6	Timer Counter Register High (TIMCNTH)	173
Figure 13-7	Timer Counter Register Low (TIMCNTL)	173
Figure 13-8	Timer System Control Register (TIMSCR1)	174
Figure 13-9	Fast Clear Flag Logic	175
Figure 13-10	Timer TOV Register (TIMTOV)	175
Figure 13-11	Timer Control Register 1 (TIMCTL1)	176
Figure 13-12	Timer Control Register 2 (TIMCTL2)	176
Figure 13-13	Timer Interrupt Enable Register (TIMIE)	177
Figure 13-14	Timer System Control Register 2 (TIMSCR2)	178
Figure 13-15	Timer Flag Register 1 (TIMFLG1)	179
Figure 13-16	Timer Flag Register 2 (TIMFLG2)	180
Figure 13-17	Timer Channel [0-3] High Register (TIMCxH)	181
Figure 13-18	Timer Channel [0-3] Low Register (TIMCxL)	181
Figure 13-19	Pulse Accumulator Control Register (TIMPACTL)	182
Figure 13-20	Pulse Accumulator Flag Register (TIMPAFLG)	184
Figure 13-21	Pulse Accumulator Counter High Register (TIMPACNTH)	185
Figure 13-22	Pulse Accumulator Counter Low Register (TIMPACNTL)	185
Figure 13-23	Timer Port Data Register (TIMPORT)	186
Figure 13-24	Timer Port Data Direction Register (TIMDDR)	186
Figure 13-25	Timer Test Register (TIMTST)	187
Figure 13-26	Channel 3 Output Compare/Pulse Accumulator Logic	191
Figure 13-27	Timer Pin Functions	192
Figure 13-28	Pin Data Direction	193
Figure 14-1	TOD Block Diagram	199
Figure 14-2	TOD Control/Status Register	200
Figure 14-3	TOD Seconds Register	201
Figure 14-4	TOD Fraction Register	202
Figure 14-5	TOD Seconds Alarm Register	202
Figure 14-6	TOD Fraction Alarm Register	203
Figure 15-2	. SCI I/O Register Summary	207
Figure 15-3	. SCI Data Formats	208
Figure 15-13	. Slow Data	221
Figure 15-14	. Fast Data	222

Figure 15-15 . Single-Wire Operation (LOOPS = 1, RSRC = 1)	224
Figure 15-16 . Loop Operation (LOOPS = 1, RSRC = 0)	225
Figure 15-17 . SCI Baud Rate Registers (SCBDH/L)	225
Figure 15-18 . SCI Control Register 1 (SCICR1)	226
Figure 15-19 . SCI Control Register 2 (SCICR2)	228
Figure 15-20 . SCI Status Register 1 (SCISR1)	230
Figure 15-21 . SCI Status Register 2 (SCISR2)	232
Figure 15-23 . SCI Port Data Register (SCIPORT)	235
Figure 15-24 . SCI Data Direction Register (SCIDDR)	236
Figure 15-25 . SCI Pullup and Reduced Drive Register (SCIPURD)	237
Figure 16-2 . I/O Register Summary	241
Figure 16-3 . Master/Slave Transfer Block Diagram	245
Figure 16-6 . SPI Control Register 1 (SPICR1)	251
Figure 16-7 . SPI Control Register 2 (SPICR2)	253
Figure 16-8 . SPI Baud Rate Register (SPIBR)	254
Figure 16-9 . SPI Status Register (SPISR)	255
Figure 16-10 . SPI Data Register (SPIDR)	256
Figure 16-11 . SPI Port Reduced Drive and Pullup Select (SPIPURD)	259
Figure 16-12 . SPI Port Data Register (SPIPORT)	260
Figure 16-13 . SPI Port Data Direction Register (SPIDDR)	261
Figure 17-1 . DSRC Block Diagram	263
Figure 17-2 . MyLID high Register	264
Figure 17-3 . MyLID low Register	264
Figure 17-4 . GID Register	265
Figure 17-5 . ACTC Register	265
Figure 17-6 . MyKey Register	266
Figure 17-7 . BrSLT Register	266
Figure 17-8 . INT Flag egister	266
Figure 17-9 . INT Enable egister	267
Figure 17-10 . INTF Clear register	267
Figure 17-11 . BrSLT Register	267
Figure 17-12 . BrSLT Register	268
Figure 17-13 . INT Flag egister	268
Figure 17-14 . BrSLT Register	269

Figure 17-15 BrSLT Register	270
Figure 17-16 BrSLT Register	270
Figure 18-1 Chip Select Functional Block Diagram	272
Figure 19-1 Read Cycle Flowchart	283
Figure 19-2 Write Cycle Flowchart	285
Figure 19-3 Master Mode - 1 Clock Read and Write Cycle.	286
Figure 19-4 Master Mode - 2 Clock Read and Write Cycle.	287
Figure 19-5 Internal (Show) Cycle followed by External 1 - Clock Read .	289
Figure 19-6 Internal (Show) Cycle followed by External 1 - Clock Write. .	290
Figure 20-1 Avalanche's JTAG Connections	295
Figure 20-2 Avalanche's Top Level JTAG Block Diagram	296
Figure 20-3 TAP Controller State Machine	297
Figure 20-4 IDcode Register Bit Specification	301
Figure 20-5 Bidirectional Pin Cell (bnd_bicell)	303
Figure 20-6 Control Cell for Bidi Pin Cell (bnd_dicell).	304

Table 1-1	Avalanche Address Map	33
Table 2-1	Avalanche Pin Descriptions	35
Table 3-1	MCORE and Peripherals in Low-Power Modes	54
Table 4-1	Clock-out vs. Clock-in Relationships	58
Table 4-2	System Frequency Multiplier of the Reference	60
Table 4-3	Loss of Clock Summary	63
Table 4-4	STOP Mode Operation	65
Table 4-5	Clock Address Map	68
Table 4-6	STOP Mode Operation	70
Table 4-7	System Clock Status Per Mode	71
Table 4-8	Charge pump vs. MFD in Normal Mode Operation	76
Table 5-1	Reset Source Summary	78
Table 5-2	Reset Controller Address Map	82
Table 6-1	Fast Interrupt Vector Number	90
Table 6-2	Vector Table Mapping	91
Table 6-3	Interrupt Controller Address Map	92
Table 6-4	MASK Encoding	94
Table 6-5	Priority Select Encoding	99
Table 6-6	Interrupt Source Assignment	100
Table 7-1	M•CORE Instruction Set	110
Table 7-2	OnCE Register Addressing	120
Table 7-3	Sequential Control Field Settings	121
Table 7-4	Memory Breakpoint Control Field Settings	123
Table 7-5	Processor Mode Field Settings	125
Table 9-1	Port Address Map	144
Table 10-1	Edge Port Address Map	148
Table 10-2	EPPAx Field Settings	149
Table 11-1	Watchdog Address Map	154
Table 12-1	PIT Address Map	158
Table 12-2	Pre-scalar Select Encoding	159
Table 13-1	Signal Properties	166
Table 13-2	Module Memory Map	167
Table 13-3	Output Compare Action Selection	176
Table 13-4	Input Capture Edge Selection	177

Table 13-5	Prescaler Selection	179
Table 13-6	Clock Selection	183
Table 13-7	TIMPORT I/O Function	192
Table 13-8	Interrupts	194
Table 13-9	Timer Interrupt Sources	197
Table 15-1	. Example 8-Bit Data Formats	209
Table 15-2	. Example 9-Bit Data Formats	209
Table 15-3	. Baud Rates (Module Clock = 10.2 Mhz)	210
Table 15-4	. Start Bit Verification	216
Table 15-5	. Data Bit Recovery	216
Table 15-6	. Stop Bit Recovery	217
Table 15-7	. Loop Functions	227
Table 15-8	. SCI Interrupt Sources	235
Table 15-9	. SCI Port Control Summary	238
Table 16-1	. Normal Mode and Bidirectional Mode	250
Table 16-2	. $\overline{SS}$ Output Selection	252
Table 16-3	. Bidirectional Pin Configurations	254
Table 16-4	. SPI Baud Rate Selection	255
Table 16-5	. SPI Interrupt Vectors	259
Table 16-6	. SPI Port Summary	261
Table 17-1	Status code	268
Table 18-1	Chip Select Address Range Encoding	273
Table 18-2	Chip Select Address Map	274
Table 18-3	Chip Select Wait States Encoding	277
Table 19-1	Data Transfer Cases	280
Table 19-2	EB[3:0] Assertion Encoding	281
Table 20-1	JTAG Instructions	298
Table 20-2	List of Pins Not Scanned in JTAG Mode	305
Table 20-3	Boundary Scan Bit Definition	306
Table 20-4	Ball Map Table	338

Section 1 Introduction

1.1 Overview

Avalanche is the member of a family of microcontrollers based on the M•CORE processor.

- M•CORE M210S Integer Processor
 - 32-bit RISC architecture
 - Low power, high performance
- OnCE Debug Support
- On chip, 8 Kbyte SRAM
 - One clock per access (including bytes, half-words, and words)
 - Byte, half-word (16-bits), or word (32-bit) read/write accesses
 - Standby power supply
- EEPROM support via SPI
- All Peripherals IPbus compliant
- Serial Peripheral Interface
 - Master mode and slave mode
 - Wired-OR mode
 - Slave select output
 - Mode fault error flag with CPU interrupt capability
 - Double-buffered operation
 - Serial clock with programmable polarity and phase
 - Control of SPI operation during wait mode
 - Reduced drive control for lower power consumption
- Two Serial Communications Interfaces
 - Full-duplex operation
 - Standard mark/space non-return-to-zero (NRZ) format
 - 13-bit baud rate selection
 - Programmable 8-bit or 9-bit data format
 - Separately enabled transmitter and receiver

- Separate receiver and transmitter CPU interrupt requests
- Programmable transmitter output polarity
- Two receiver wake-up methods (idle line and address mark)
- Interrupt-driven operation with eight flags
- Receiver framing error detection
- Hardware parity checking
- 1/16 bit-time noise detection
- General-purpose I/O (input/output) port
- Timer
 - Four 16-bit input capture/output compare channels
 - 16-bit architecture
 - 16-bit pulse accumulator
 - Pulse widths variable from microseconds to seconds
 - Prescaler
 - Toggle on overflow feature for PWM generation
 - Timer port pullups enabled on reset
- Interrupt Controller
 - Up to 40 interrupt sources
 - 32 unique programmable priority levels for each interrupt source
 - Independent enable/disable of pending interrupts based on priority level
 - Select normal or fast interrupt request for each priority level
 - Fast interrupt requests always have priority over normal interrupts
 - Ability to mask interrupts at and below a defined priority level
 - Ability to select between autovectored or vectored interrupt requests
 - Vectored interrupts generated based on priority level
 - Ability to generate a separate vector number for normal and fast interrupts
 - Ability for software to self-schedule interrupts
 - Software visibility of pending interrupts and interrupt signals to core
 - Asynchronous operation to support wake-up from low power modes
- External interrupts supported
 - Rising/falling edge select

- Low level sensitive
- Ability for software generation of external interrupt event
- General purpose I/O support
- Time Of Day Timer
 - The master clock signal for the time-of-day timer is driven from a 32.768-kHz oscillator
 - Provides time-of-day information as well as an alarm clock function
- Periodic Interval Timers
 - 16 bit counter
 - Selectable as free running or count down
- Watchdog Timer
 - 16 bit counter
 - Low power mode support
- 32KHz Oscillator
 - 32KHz clock is generated by onchip oscillator
 - TOD is driven by this 32KHz clock
- PLL
 - Reference crystal 2 to 10 MHz
 - Low power modes supported
 - Separate Clock-Out signal
- Reset
 - Separate Reset In and Reset Out signals
 - Six sources of reset (POR, External, Software, Watchdog, Loss of clock/lock)
 - Status flag indication of source of last reset
- Chip Configurations
 - Support for single chip, master, emulation, and test modes
 - System configuration during reset
 - Bus Monitor, Abort Monitor
 - Configurable output pad drive strength control
 - Avalanche Part Identification (h'17); Initial Part Revision (h'00)
- General Purpose I/O
 - 16 bits of dedicate general purpose I/O

- Coherent 16 bit control
- Bit manipulation supported via set/clear functions
- Unused peripheral pins may be used as extra GPIO
- DSRC interface
 - 2 input and 3 output ports are available for direct connection to 5.8GHz DSRC RF module
 - 144+512 byte data buffer for receive and transmitt data
- External Interface
 - Provides for direct support of asynchronous RAM, ROM, and FLASH
 - Support interfacing to 16 and 32 bit data buses
 - 32 bit external bi-directional data bus
 - 24 bit address bus
 - Eight chip selects
 - Byte/write enables
 - Ability to boot from internal or external memories
 - Internal bus activity is visible via the “show-cycle” mode
 - Special chip selects support replacement of general purpose I/O with external logic (port replacement logic)
 - Emulation of internal page mode flash support
- JTAG support for system level board testing
 - Avalanche Part Identification (h'001); Initial Part Revision (h'0)32bit

As a low-voltage part, Avalanche operates at voltages between 2.7 and 3.6 volts. It is particularly suited for use in battery-powered applications. The operating frequency is up to a maximum of 32MHz over a temperature range of -40 to 85 C. Available packages a 192MAPBGA for applications requiring the full external memory interface support or large number of GPIO.

1.2 Avalanche Description

The basic structure of the Avalanche is shown in **Figure 1-1**.

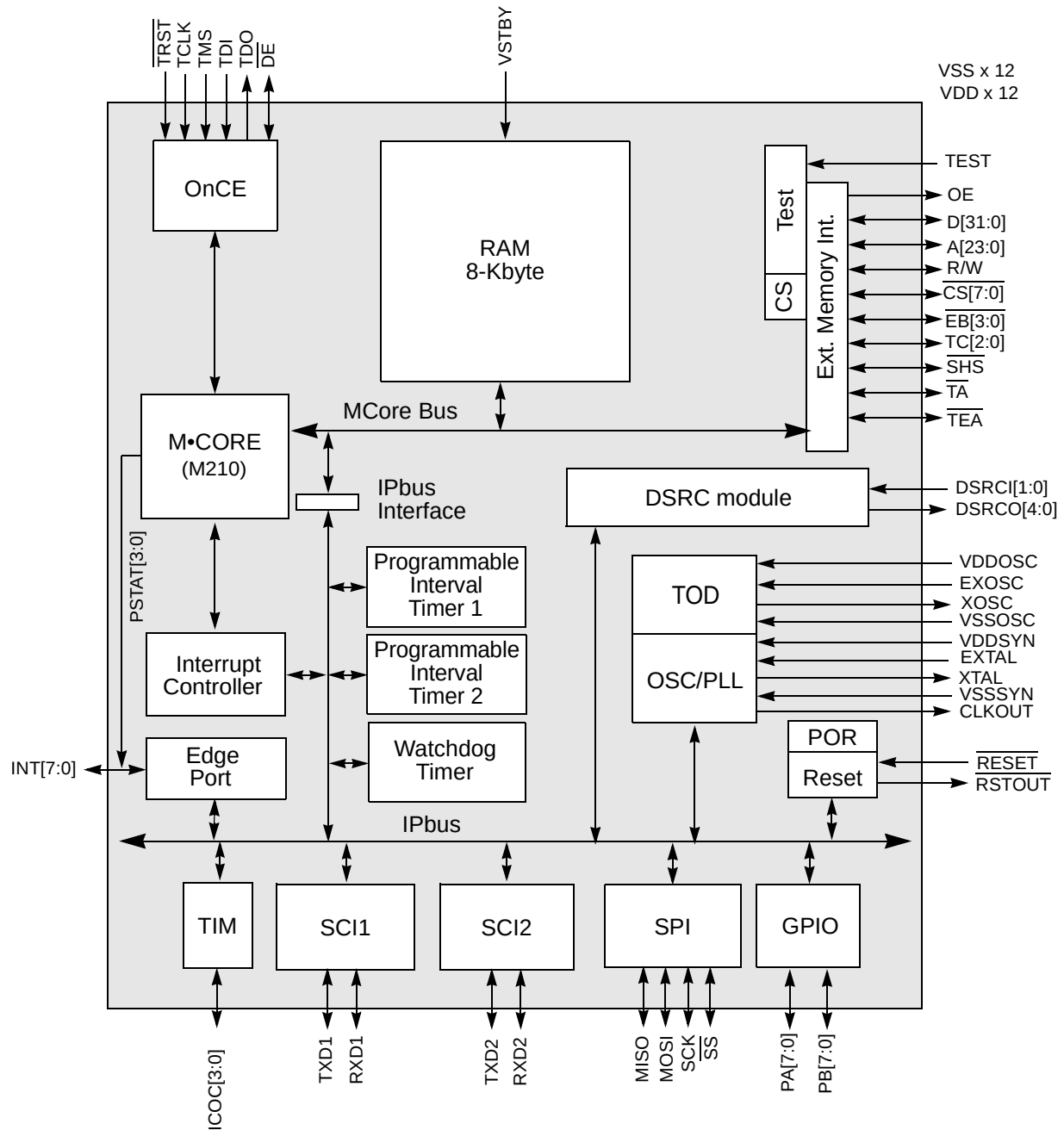


Figure 1-1 Avalanche Block Diagram

1.3 Avalanche Address Map

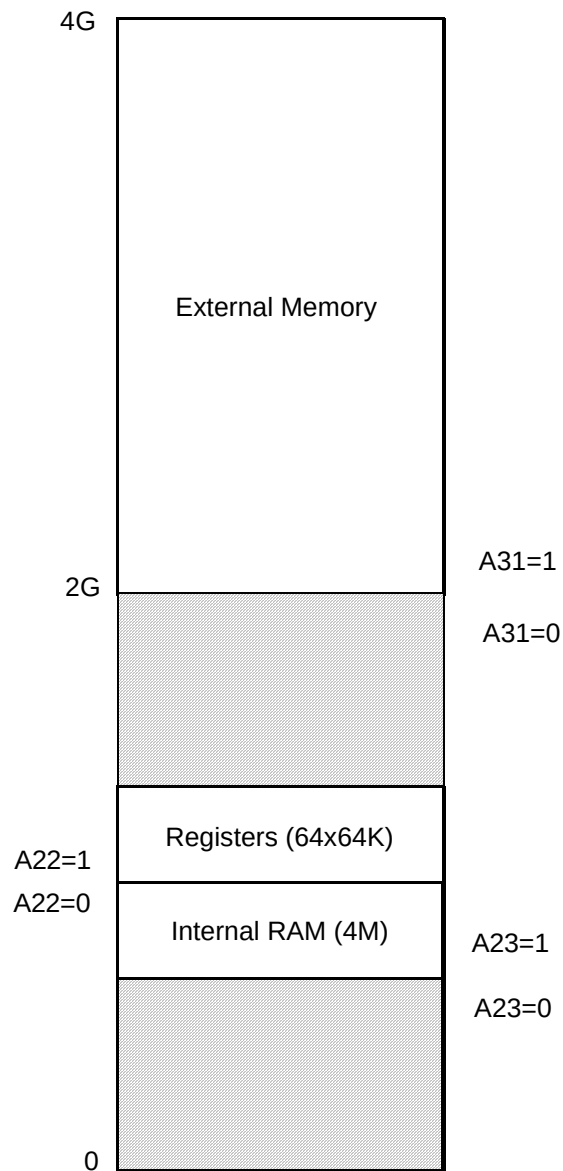


Figure 1-2 Avalanche Address Map

Table 1-1 Avalanche Address Map¹

Base Address (Hex)	Maximum Size	Use
0x0080_0000	4 Mbyte	On-Chip RAM Array (8Kbyte)
0x00c0_0000	64 Kbyte	Ports
0x00c1_0000	64 Kbyte	Chip Configuration
0x00c2_0000	64 Kbyte	Chip Selects
0x00c3_0000	64 Kbyte	Clocks
0x00c4_0000	64 Kbyte	Reset
0x00c5_0000	64 Kbyte	Interrupt Controller
0x00c6_0000	64 Kbyte	Edge Port
0x00c7_0000	64 Kbyte	Watchdog Timer
0x00c8_0000	64 Kbyte	Programmable Interval Timer 1
0x00c9_0000	64 Kbyte	Programmable Interval Timer 2
0x00ca_0000	64 Kbyte	Time Of Day Timer
0x00cb_0000	64 Kbyte	SPI
0x00cc_0000	64 Kbyte	SCI 1
0x00cd_0000	64 Kbyte	SCI 2
0x00ce_0000	64 Kbyte	Timer
0x00cf_0000	64 Kbyte	DSRC module
0x00d1_0000	64 Kbyte x 15	Reserved
0x00e0_0000	64 Kbyte x 16	Reserved
0x00f0_0000	64 Kbyte x 16	Reserved
0x8000_0000	2GB	External Memory

NOTES:

1. See module specifications for details of how much of each block is being decoded. Accesses to addresses outside the module memory maps (and also the Reserved area 0x00d1_0000 - 0x00f0_0000) will not be responded to and will result in a Bus Monitor transfer error exception.

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Section 2 Signal Descriptions

2.1 Overview

This section contains brief descriptions of Avalanche external signal and power connections. Specific details of signal operation are provided in the individual module descriptions.

Table 2-1 Avalanche Pin Descriptions

Name	Alt	Qty	Dir	Input Hyst	Input Sync ¹	Drive Strength Control ²	Pull-Up / Pull-Down ³	Output Driver (ST/OD/SP) ⁴
Reset								
RESET	—	1	I	Y	Y	—	pull-up	—
RSTOUT	SHOWINT	1	I/O ⁵	—	—	LOAD	—	ST
Clock								
EXTAL	—	1	I	N	N	—	—	SP
XTAL	—	1	I/O ⁵	—	—	—	—	SP
VDDSYN	—	1	I	N	N	—	—	—
VSSSYN	—	1	I	N	N	—	—	—
EXOSC	—	1	I	—	—	—	—	SP
XOSC	—	1	I/O ⁵	—	—	—	—	SP
VDDOSC	—	1	I	N	N	—	—	—
VSSOSC	—	1	I	N	N	—	—	—
CLKOUT	—	1	I/O ⁵	—	—	LOAD	—	ST
External Memory Interface and Ports								
D[31:0]	—	32	I/O	Y	Y	LOAD	—	ST
SHS	RCON	1	I/O	Y	Y	LOAD	pull-up	ST
TA	—	1	I/O ⁵	Y	Y	LOAD	pull-up	ST
TEA	—	1	I/O ⁵	Y	Y	LOAD	pull-up	ST
TC[2:0]	—	3	I/O ⁵	Y	Y	LOAD	pull-up ⁵	ST
R/W	—	1	I/O ⁵	Y	Y	LOAD	pull-up ⁵	ST
A[23:0]	—	24	I/O ⁵	Y	Y	LOAD	pull-up ⁵	ST
EB[3:0]	—	4	I/O ⁵	Y	Y	LOAD	pull-up ⁵	ST
CS7 - CS0	—	8	I/O ⁵	Y	Y	LOAD	pull-up ⁵	ST
OE	—	1	I/O ⁵	—	—	LOAD	—	ST
Edge Port								
INT[7:6]	TSIZ[1:0]	2	I/O	Y	Y	LOAD	—	—
INT[5:2]	PSTAT[3:0]	4	I/O	Y	Y	LOAD	—	—
INT[1:0]	—	2	I/O	Y	Y	LOAD	—	—

Table 2-1 Avalanche Pin Descriptions (Continued)

Name	Alt	Qty	Dir	Input Hyst	Input Sync ¹	Drive Strength Control ²	Pull-Up / Pull-Down ³	Output Driver (ST/OD/SP) ⁴
GPIO								
PA/PB[7:0]		16	I/O	Y	Y	LOAD	pull-up	ST
SPI								
MOSI		1	I/O	Y	Y	RDPSP0	pull-up ⁶	ST / OD ⁶
MISO		1	I/O	Y	Y	RDPSP0	pull-up ⁶	ST / OD ⁶
SCK		1	I/O	Y	Y	RDPSP0	pull-up ⁶	ST / OD ⁶
SS		1	I/O	Y	Y	RDPSP0	pull-up ⁶	ST / OD ⁶
SCI 1 and SCI 2								
TXD1		1	I/O	Y	Y	RDPSCI0	pull-up ⁶	ST / OD ⁶
RXD1		1	I/O	Y	Y	RDPSCI0	pull-up ⁶	ST / OD ⁶
TXD2		1	I/O	Y	Y	RDPSCI0	pull-up ⁶	ST / OD ⁶
RXD2		1	I/O	Y	Y	RDPSCI0	pull-up ⁶	ST / OD ⁶
DSRC Logic								
DSRCI[1:0]		2	I/O	Y	Y	—	—	-
DSRCO[4:0]		5	I/O ⁵	—	—	RDETC	—	ST / OD ⁶
Timer								
ICOC[3]	IC / OC / PAI	1	I/O	Y	Y	RDPT	pull-up ⁶	ST
ICOC[2:0]	IC / OC	3	I/O	Y	Y	RDPT	pull-up ⁶	ST
Debug and JTAG Test Port Control								
TRST	—	1	I	Y	N	—	pull-up	—
TCLK	—	1	I	Y	N	—	pull-up	—
TMS	—	1	I	Y	N	—	pull-up	—
TDI	—	1	I	Y	N	—	pull-up	—
TDO	—	1	O ⁷	—	—	LOAD	—	ST
DE	—	1	I/O	Y	N	LOAD	pull-up	OD
Test								
TEST	—	1	I	Y	N	—	—	—
Power Supplies								
VSTBY	—	1	I	—	—	—	—	—
VDD	—	12	I	—	—	—	—	—
VSS	—	12	I	—	—	—	—	—
Total		175						

NOTES:

1. Synchronized input used only if pin configured as a digital I/O. $\overline{\text{RESET}}$ pin is always synchronized, except for in low power STOP mode.

2. LOAD (Chip Configuration register bit), RDPSP0 (SPIPURD register bit in SPI), RDPSCI0 (SCIPURD register bit in both SCIs), RDPT (TIMSCR2 register bit in both Timers).
3. All pull-ups and pull-downs are disconnected when the pin is programmed as a standard output.
4. Output driver type: ST=Standard, OD=Standard driver with open drain pull down option selected, SP=Special.
5. Digital input function used only for JTAG boundary scan.
6. Open drain and pull-up function selectable via programmer's model in module configuration registers.
7. Three-state output with no input function.

2.2 Avalanche Specific Implementation Pin Issues

Most modules are designed to allow expanded capabilities if all the module signals to the pads are implemented. The paragraphs below discuss issues with how these modules are implemented on Avalanche.

2.2.1 $\overline{\text{RSTOUT}}$ Pin Functions

The $\overline{\text{RSTOUT}}$ pin has multiple functions.

- Whenever the internal system reset is asserted, the $\overline{\text{RSTOUT}}$ pin will always be asserted to indicate the reset condition to the system.
- If the internal reset is not asserted, then setting the FRCRSTOUT bit in the Reset Control Register will assert the $\overline{\text{RSTOUT}}$ pin for as long as the FRCRSTOUT bit is set.
- If the internal reset is not asserted and the FRCRSTOUT bit is not set, then setting the SHOWINT bit in the Chip Configuration Register will reflect internal interrupt requests out to the $\overline{\text{RSTOUT}}$ pin.
- If the internal reset is not asserted and the FRCRSTOUT bit is not set and the SHOWINT bit is not set, then the $\overline{\text{RSTOUT}}$ pin will be negated to indicate to the system that there is no reset condition.

Caution: External logic used to drive reset configuration data during reset needs to be considered when using the $\overline{\text{RSTOUT}}$ pin for a function other than an indication of reset.

2.2.2 $\overline{\text{INT}}$ Pin Functions

The $\overline{\text{INT}}$ pins have multiple functions.

- If the SZEN bit in the Chip Configuration Register is set, then $\overline{\text{INT}}[7:6]$ will be used to reflect the state of the TSIZ[1:0] signals from the M-Core.

- If the PSTEN bit in the Chip Configuration Register is set, then $\overline{\text{INT}}[5:2]$ will be used to reflect the state of the PSTAT[3:0] signals from the M-Core.

Note: If the SZEN or PSTEN bits are set during Emulation mode, then the corresponding Edge Port $\overline{\text{INT}}$ functions are lost and will not be emulated externally.

2.2.3 SPI Pin Functions

The SPI module can support up to 8 external pins, but only the 4 pins required for the SPI interface are implemented.

Full SPI interface capabilities and GPIO functions using the MISO, MOSI, SCK, and $\overline{\text{SS}}$ pins are supported.

- SWOM register bit controls whether output buffers behave as open-drain outputs. Default is not open-drain outputs.
- PUPSP0 register bit enables internal weak pad pull-up devices. Default is pull-ups disabled.
- RDPSP0 register bit controls reduced drive function of output buffers. Default is full drive.

GPIO 7-4 of SPI module not implemented.

- Writes to bits 7-4 of the SPIPORT and SPIDDR registers have no effect except to change the register bit values.
- Reads of bits 7-4 of the SPIPORT when the corresponding SPIDDR bits are set for inputs always return 0.
- PUPSP1 and RDPSP1 register bits have no effect.

The default reset values for PUPSP1 and PUPSP0 will be zero. Thus, the pull-up function is disabled by default.

2.2.4 SCI1 and SCI2 Pin Functions

The SCI modules can support up to 8 external pins each, but only the 2 pins required for each SCI interface are implemented.

Note: The pins associated with each SCI are only controlled by the register bits for the corresponding SCI. Thus, the WOMS register bit from SCI1 only affects TXD1 and RXD1 and has no effect on TXD2 and RXD2.

Full SCI interface capabilities and GPIO functions using the TXD1/2 and RXD1/2 pins are supported.

- WOMS register bit controls whether output buffers behave as open-drain outputs. Default is not open-drain outputs.
- PUPSCI0 register bit enables internal weak pad pull-up devices. Default is pull-ups disabled.
- RDPSCI0 register bit controls reduced drive function of output buffers. Default is full drive.

GPIO 7-2 of SCI modules not implemented.

- Writes to bits 7-2 of the SCI0PORT and SCIDDR registers have no effect except to change the register bit values.
- Reads of bits 7-2 of the SCI0PORT when the corresponding SCIDDR bits are set for inputs always return 0.
- PUPSCI1 and RDPSCI1 register bits have no effect.

The default reset values for PUPSCI1 and PUPSCI0 will be zero. Thus, the pull-up function is disabled by default.

2.2.5 Timer Pin Functions

The Timer module can support up to 4 external pins and all 4 pins are implemented.

Note: The pins associated with each Timer are only controlled by the register bits for the corresponding Timer.

Full Timer port pin functions are supported.

- PUPT register bit enables internal weak pad pull-up devices. Default is pull-ups disabled.
- RDPT register bit controls reduced drive function of output buffers. Default is full drive.

The default reset values for PUPT will be zero. Thus, the pull-up function is disabled by default.

The sync input is tied off and this function is not supported.

2.2.6 DSRC Pin Functions(customer designed module)

The DSRC module has two inputs(RxD, Wakeup) and three outputs(TxD, TxMASK,RxMASK).

Signal Descriptions

2.3 Signal Descriptions

2.3.1 Reset Signals

These signals are used to either reset the chip or as a reset indication.

2.3.1.1 Reset In ($\overline{RESET}$)

This active low input signal is used as an external reset request.

2.3.1.2 Reset Out ($\overline{RSTOUT}$)

This active low output signal is an indication that the internal reset controller has reset the chip. When $\overline{RSTOUT}$ is active, the user may drive override options on the data bus. $\overline{RSTOUT}$ is three-stated in PLL test mode.

$\overline{RSTOUT}$ may also be used to reflect an indication of an internal interrupt request.

2.3.2 PLL and Clock Signals

These signals are used to support the on chip clock generation circuitry. Refer to clock section for more information.

2.3.2.1 External Clock In ($EXTAL$)

This input signal is always driven by an external clock input except when used as a connection to the external crystal when the internal oscillator circuit is used. The clock source is configured during reset.

2.3.2.2 Crystal ($XTAL$)

This output signal is used as a connection to the external crystal when the internal oscillator circuit is used.

2.3.2.3 Clock Out ($CLKOUT$)

This output signal reflects the internal system clock.

2.3.2.4 Synthesizer Power (VDD_{syn} , VSS_{syn})

These are dedicated quiet power and ground supply pins for the frequency synthesizer circuit.

2.3.2.5 External OSC In (EXOSCL)

This input signal is always driven by an external clock input to drive TOD module with 32.768KHz.

2.3.2.6 OSC (OSC)

This output signal is used as a connection to the external 32.768KHz crystal.

2.3.2.7 Oscillator Power (VDDosc, VSSosc)

These are dedicated quiet power and ground supply pins for the TOD clock generator circuit.

2.3.3 External Memory Interface Signals

In addition to the function stated below, the following pins can also be configured to be discrete I/O pins.

2.3.3.1 Data Bus (D[31:0])

These three-state bi-directional signals provide the general purpose data path between the MCU and all other devices.

2.3.3.2 Show Cycle Strobe ($\overline{SHS}$)

This output signal is used in Emulation mode as a strobe for capturing address, controls, and data during show cycles.

This pin is also used as $\overline{RCON}$. This input signal, used only during reset, indicates whether or not the states on the external pins affect the chip configuration.

2.3.3.3 Transfer Acknowledge ($\overline{TA}$)

This signal indicates that the external data transfer is complete. During a read cycle, when the processor recognizes $\overline{TA}$, it latches the data and then terminates the bus cycle. During a write cycle, when the processor recognizes $\overline{TA}$, the bus cycle is terminated. This signal is an input in Master and Emulation modes, and is an output in FAST mode.

Signal Descriptions

2.3.3.4 Transfer Error Acknowledge ($\overline{\text{TEA}}$)

This signal indicates an error condition exists for the bus transfer. The bus cycle is terminated and the CPU begins execution of the access error exception. This signal is an input in Master and Emulation modes, and is an output in FAST mode.

2.3.3.5 Transfer Code ($\text{TC}[2:0]$)

These output signals indicate the data transfer code for the current bus cycle.

2.3.3.6 Read/Write ($\text{R}/\overline{\text{W}}$)

This output signal indicates the direction of the data transfer on the bus. A logic 1 indicates a read from a slave device and a logic 0 indicates a write to a slave device.

2.3.3.7 Address Bus ($\text{A}[23:0]$)

These output signals provide the address for the current bus transfer.

2.3.3.8 Enable Byte ($\overline{\text{EB}}[3:0]$)

These output signals indicate which byte of data is valid during external cycles.

2.3.3.9 Chip Select ($\overline{\text{CS}}7 - \overline{\text{CS}}0$)

These output signals select external devices for external bus transactions.

2.3.3.10 Output Enable ($\overline{\text{OE}}$)

This output signal indicates when an external device can drive data during external read cycles.

2.3.4 Edge Port Signals

2.3.4.1 External Interrupts ($\text{INT}[7:6]$)

These bidirectional pins comprise function as either external interrupt sources or general purpose I/O.

May also use these pins to reflect the internal $\text{TSIZ}[1:0]$ signals external to provide an external indication of the M-CORE transfer size signals.

2.3.4.2 External Interrupts (INT[5:2])

These bidirectional pins comprise function as either external interrupt sources or general purpose I/O.

May also use these pins to reflect the internal PSTAT[3:0] signals external to provide an external indication of the M-CORE processor status.

2.3.4.3 External Interrupts (INT[1:0])

These bidirectional pins comprise function as either external interrupt sources or general purpose I/O.

2.3.5 Serial Peripheral Interface Module Signals

The following signals are used by the SPI module and may also be configured to be discrete I/O pins.

2.3.5.1 Master Out/Slave In (MOSI)

This signal is the serial data output from the SPI in master mode and the serial data input in slave mode.

2.3.5.2 Master In/Slave Out (MISO)

This signal is the serial data input to the SPI in master mode and the serial data output in slave mode.

2.3.5.3 Serial Clock (SCK)

The serial clock synchronizes data transmissions between master and slave devices. SCK is an output if the SPI is configured as a master and SCK is an input if the SPI is configured as a slave.

2.3.5.4 Slave Select ($\overline{SS}$)

This I/O signal is the peripheral chip select pin in master mode and is an active low slave select in slave mode.

2.3.6 Serial Communications Interface Module Signals

The following signals are used by the two SCI modules.

2.3.6.1 Receive Data (RXD1, RXD2)

These signals are used for the SCI receiver data input and are also available for general-purpose I/O when not configured for receiver operation.

2.3.6.2 Transmit Data (TXD1, TXD2)

These signals are used for the SCI transmitter data output and are also available for general-purpose I/O when not configured for transmitter operation.

2.3.7 Timer Signals

2.3.7.1 ICOC[3:0],

These pins provide the external interface to the Timer functions. They may be configured as general-purpose I/O if the Timer output function is not needed. The default state at reset is general-purpose input.

2.3.8 Debug and Emulation Support Signals

These signals are used as the interface to the on-chip JTAG controller and also to interface to the OnCE logic.

2.3.8.1 Test Reset ($\overline{TRST}$)

The active-low input signal is used to initialize the JTAG and OnCE logic asynchronously.

2.3.8.2 Test Clock (TCLK)

This input signal is the test clock used to synchronize the JTAG and OnCE logic.

2.3.8.3 Test Mode Select (TMS)

This input signal is used to sequence the JTAG state machine. TMS is sampled on the rising edge of TCLK.

2.3.8.4 Test Data Input (TDI)

This input signal is the serial input for test instructions and data. TDI is sampled on the rising edge of TCLK.

2.3.8.5 Test Data Output (TDO)

This output signal is the serial output for test instructions and data. TDO is three-stateable and is actively driven in the shift-IR and shift-DR controller states. TDO changes on the falling edge of TCLK.

2.3.8.6 Debug Event ($\overline{DE}$)

This is a bidirectional, active low signal.

As an output, this pin will be asserted for 3 system clocks, synchronous to the rising CLKOUT edge, to acknowledge that the CPU has entered debug mode as a result of a debug request or a breakpoint condition.

As an input, this pin provides multiple functions. The main function is a means of entering debug mode from an external command controller. This pin, when asserted, causes the CPU to finish the current instruction being executed, save the instruction pipeline information, enter debug mode, and wait for commands to be entered from the serial debug input line. This input must be asserted for at least 3 system clocks, sampled with the rising CLKOUT edge. This function is ignored during reset. While the processor is in debug mode, this pin is still sampled but has no effect until debug mode is exited.

Another input function is to enable OnCE. This is an alternate method to the ENABLE_MCU_ONCE jtag command to enable the OnCE logic to be accessible via the jtag interface. This input pin must be asserted low (while in the test-logic-reset state with POR/TRST not asserted) for at least 2 TCLK rising edges. Once enabled, the OnCE will remain enabled until the next POR or TRST resets.

Another input function is as a wake-up event from a low power mode of operation. Asynchronously asserting this pin will cause the clock controller to restart. This pin must be held asserted until the MCore receives 3 valid rising edges on the system clock. Then the processor will exit the low power mode and go into debug mode.

Note: If used to enter debug mode, $\overline{DE}$ must be pulled negated before the processor exits debug mode to prevent a still low signal as being unintentionally recognized as another debug request. Also, asserting this pin to enter debug mode may prevent external logic from seeing the processor output acknowledgment since the external pullup may not be able to pull the pin negated before the handshake is asserted. Finally, if using this pin to enable OnCE outside of reset, then this may also be seen as a request to enter debug mode.

2.3.9 Test Signal

2.3.9.1 TEST

This input signal is reserved for factory testing only and should be connected to VSS to prevent unintentional activation of test functions.

2.3.10 DSRC Pins

2.3.10.1 DSRCO[2:0]

These pins are provided to control DSRC RF module in outside of the chip.

2.3.10.2 DSRCI[1:0]

These pins are provided to receive signals from DSRC RF module in outside of the chip.

2.3.11 Power and Ground Pins

These pins provide system power and ground to the chip. Multiple pins are provided for adequate current capability. All power supply pins must have adequate bypass capacitance for high-frequency noise suppression.

2.3.11.1 Standby Power (V_{STBY})

This pin is used to provide standby voltage to the RAM array if VDD is lost.

2.3.11.2 VDDSYN

This pin supplies positive power to the Frequency synthesizer.

2.3.11.3 VSSSYN

This pin is the negative supply to the Frequency synthesizer.

2.3.11.4 VDDOSC

This pin supplies positive power to the 32.768KHz Oscillator and TOD logic.

2.3.11.5 VSSOSC

This pin supplies negative power to the 32.768KHz Oscillator and TOD logic.

2.3.11.6 Positive Supply (VDD)

This pin supplies positive power to the core logic and I/O pads.

2.3.11.7 Ground (VSS)

This pin is the negative supply (ground) to the chip.

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Section 3 Low-Power Modes

3.1 Overview

The following features are available to support applications which require low power.

- Four modes of operation:
 - RUN
 - WAIT
 - DOZE
 - STOP
- Ability to shut down most peripherals independently.
- Ability to shut down the external CLKOUT pin.

3.1.1 Low-Power Modes

There are four modes of operation: RUN, WAIT, DOZE, and STOP. The system enters a low power mode by the MCore executing a STOP, WAIT, or DOZE instruction. This idles the MCore with no cycles active. The LPMD output signals indicate to the system and clock controller to power down and stop the clocks appropriately. During STOP mode, the system clock is stopped low.

A wakeup event is required to exit a low power mode and return back to RUN mode. Wakeup events consist of any of the following conditions. See the following sections for more details.

1. Any type of reset.
2. Assertion of the $\overline{DE}$ pin to request entry into Debug mode.
3. Debug request bit set in the OnCE control register to request entry into Debug mode.
4. Any valid interrupt request.

3.1.1.1 RUN Mode

RUN mode is the normal system operating mode. Current consumption in this mode is related directly to the frequency chosen for the system clock.

Low-Power Modes

3.1.1.2 WAIT Mode

WAIT mode is intended to be used to stop only the MCore and memory clocks until a wakeup event is detected. In this mode, peripherals may be programmed to continue operating and can generate interrupts, which cause the MCore to exit from WAIT mode.

3.1.1.3 DOZE Mode

DOZE mode affects the MCore in the same manner as WAIT mode, but with a different code on the MCore LPMD output signals, which are monitored by the peripherals. Each peripheral defines individual operational characteristics in DOZE mode. Peripherals which continue to run and have the capability of producing interrupts may cause the MCore to exit the DOZE mode and return to the RUN mode. Peripherals which are stopped will restart operation on exit from DOZE mode as defined for each peripheral.

3.1.1.4 STOP Mode

STOP mode affects the MCore in the same manner as the WAIT and DOZE modes, but with a different code on the MCore LPMD output signals. In this mode, all clocks to the system are stopped and the peripherals cease operation.

STOP mode must be entered in a controlled manner to ensure that any current operation is properly terminated. When exiting STOP mode, most peripherals retain their pre-stop status and resume operation.

3.1.1.5 Peripheral Shut Down

Most peripherals may be disabled by software in order to cease internal clock generation and remain in a static state. Each peripheral has its own specific disabling sequence (refer to each peripheral description for further details). A peripheral may be disabled at anytime and will remain disabled during any low power mode of operation. The only exception is TOD. TOD can operate anytime even main power (VDD) is shutted off. TOD is driven its own clock source and power source.

3.1.2 Peripheral Behavior in Low-Power Modes

3.1.2.1 Reset

A POR reset will always cause a chip reset and exit from any low power mode.

In WAIT and DOZE modes, asserting the external RESET pin for at least four clocks will cause an external reset that will reset the chip and exit any low power modes. In STOP mode, the

$\overline{\text{RESET}}$ pin synchronization is disabled and asserting the external $\overline{\text{RESET}}$ pin will asynchronously generate an internal reset and exit any low power modes.

If the PLL is active, then any loss of clock or loss of lock reset will reset the chip and exit any low power modes.

If the watchdog timer is still enabled during WAIT or DOZE modes, then a watchdog timer time-out may generate a reset to exit these low power modes.

Since the MCore is inactive, a software reset can not be generated to exit any low power mode.

3.1.2.2 Clocks

During the low power WAIT and DOZE modes, the clocks to the MCore, Flash, and RAM will be stopped and the system clocks to the peripherals are enabled. Each module may disable the module clocks locally at the module level. During the low power STOP mode, all clocks to the system will be stopped.

During STOP mode, there are several options for enabling/disabling the PLL and/or crystal oscillator (OSC) at a price of wakeup recovery time. The PLL may be disabled during STOP, but then there will be a wakeup time required for the PLL to re-lock. The OSC may also be disabled during STOP, but then there will be a wakeup time required for the OSC to restart.

The external CLKOUT signal may also be left enabled during low power STOP to support systems using this signal as the clock source.

There is also an option of enabling the system clocks during wakeup from STOP mode without waiting for the PLL to lock. This eliminates the wakeup recovery time but at a price of sending a potentially unstable clock to the system. It is recommended that if this option is used that the RFD is changed before entering into the low power STOP to set the system frequency to at most a half of maximum allowable to account for any frequency overshoot of an unlocked PLL.

In external clock mode, there are no wait times for the OSC start-up or PLL lock.

The external CLKOUT output pin may be disabled in the low state to lower power consumption via the disable CLKOUT (DISCLK) bit in the clock module. The external CLKOUT pin function is enabled by default at reset.

3.1.2.3 OnCE

The OnCE logic is clocked using the TCLK input and is not affected by the system clock. Entering DEBUG mode via the OnCE port (or asserting the external $\overline{\text{DE}}$ pin) will cause the MCore to exit

any low power mode. Toggling TCLK during any low power mode will increase the system current consumption.

3.1.2.4 JTAG

The JTAG controller logic is clocked using the TCLK input and is not affected by the system clock. The JTAG can not cause an event to cause the MCore to exit any low power mode. Toggling TCLK during any low power mode will increase the system current consumption.

3.1.2.5 Interrupt Controller

The Interrupt Controller is not affected by any low power modes. All logic between the input sources and generating the interrupt to the M•CORE processor will be combinational to allow the ability to wake up the M•CORE processor during low power STOP mode when all system clocks are stopped.

A fast interrupt request will cause the MCore to exit a low power mode only if the FE bit in the MCore's PSR register is set. A normal interrupt request will cause the MCore to exit a low power mode only if the IE and EE bits in the MCore's PSR register are set.

3.1.2.6 Edge Port

In WAIT and DOZE modes, the Edge Port continues to operate normally and may be configured to generate interrupts (either an edge transition or low level on an external pin) to exit the low power modes.

In STOP mode, there are no clocks available to perform the edge detect function. Thus, only the level detect logic is active (if configured) to allow any low level on the external interrupt pin to generate an interrupt (if enabled) to exit the STOP mode.

3.1.2.7 RAM

The RAM is disabled during any low power mode. No recovery time is required when exiting any low power mode.

3.1.2.8 Watchdog Timer

In STOP mode (or in WAIT/DOZE mode, if so programmed), the watchdog ceases operation and freezes at the current value. When exiting these modes, the watchdog resumes operation from the stopped value. It is the responsibility of software to avoid erroneous operation.

When not stopped, the watchdog may generate a reset to exit the low power modes.

3.1.2.9 PIT1/2

In STOP mode (or in DOZE mode, if so programmed), the PIT ceases operation, and freezes at the current value. When exiting these modes, the PIT resumes operation from the stopped value. It is the responsibility of software to avoid erroneous operation.

When not stopped, the PIT may generate an interrupt to exit the low power modes.

3.1.2.10 SPI

When not stopped, the SPI may generate an interrupt to exit the low power modes.

Clearing the SPI enable bit (SPE) disables the SPI function.

The SPI is unaffected by WAIT mode and may generate an interrupt to exit this mode.

SPI operation in DOZE mode is programmable. Depending on the state of internal bits, the SPI can operate normally when the MCore is in DOZE mode or the SPI clock generation can be turned off and the SPI module enters a power conservation state during DOZE mode. During DOZE mode, any master transmission in progress stops. Reception and transmission of a byte as slave continues so that the slave is synchronized to the master.

The SPI is inactive in STOP mode for reduced power consumption.

3.1.2.11 SCI1/2

When not stopped, the SCIs may generate an interrupt to exit the low power modes.

Clearing the transmit enable bit (TE) or the receiver enable bit (RE) disables SCI functions.

The SCIs are unaffected by WAIT mode and may generate an interrupt to exit this mode.

In STOP mode (or DOZE mode, if so programmed), the SCIs stop immediately and freeze their operation, register values, state machines, and external pins. During these modes, the SCI clocks are shut down. Coming out of the STOP or DOZE modes, returns the SCIs to operation from the state prior to the low-power mode entry.

3.1.2.12 Timer

When not stopped, the Timers may generate an interrupt to exit the low power modes.

Clearing the timer enable bit (TE) or the pulse accumulator enable bit (PAE) disables timer functions. TE is in timer system control register 1 (TIMSCR1). PAE is in the pulse accumulator control register (TIMPACTL). Timer and pulse accumulator registers are still accessible, but the remaining functions of the timer are disabled.

Low-Power Modes

The Timer is unaffected by either the WAIT or DOZE modes and may generate an interrupt to exit these modes.

In STOP mode, the Timers stop immediately and freeze their operation, register values, state machines, and external pins. Upon exiting STOP mode, the Timer will resume operation unless STOP mode was exited by reset.

3.1.2.13 Time Of the Day timer

The TOD timer is not affected by any of the low power modes and TOD may generate an interrupt to exit the low power mode.

3.1.2.14 DSRC module

The DSRC is not stopped in lowpower mode except STOP mode.

3.1.2.15 Peripheral State During Low-Power Modes Summary

The functionality of each of the peripherals and MCore during the various low power modes is summarized in **Table 3-1**. The status of each peripheral during a given mode refers to the condition the peripheral automatically assumes when the particular instruction (WAIT, DOZE, or STOP) is executed. Individual peripherals may be disabled always (and thus reduce power) by programming its dedicated control bits. The wake-up capability field refers to the ability of an interrupt or reset from that peripheral to force the MCore into RUN mode.

Table 3-1 MCore and Peripherals in Low-Power Modes

Module	Peripheral Status ¹ / Wake-up Capability						
	RUN	WAIT		DOZE		STOP	
MCore	Enabled	Stopped	No	Stopped	No	Stopped	No
Reset	Enabled	Enabled	Yes ²	Enabled	Yes ²	Enabled	Yes ²
Clock	Enabled	Enabled	Yes ²	Enabled	Yes ²	Prog.	Yes ²
OnCE	Enabled	Enabled	Yes ³	Enabled	Yes ³	Enabled	Yes ³
JTAG	Enabled	Enabled	No	Enabled	No	Enabled	No
Interrupt Controller	Enabled	Enabled	Yes ⁴	Enabled	Yes ⁴	Enabled	Yes ⁴
Edge Port	Enabled	Enabled	Yes ⁴	Enabled	Yes ⁴	Stopped	Yes ⁴
RAM	Enabled	Stopped	No	Stopped	No	Stopped	No
Watchdog Timer	Enabled	Prog.	Yes ²	Prog.	Yes ²	Stopped	No
PIT1/2	Enabled	Enabled	Yes ⁴	Prog.	Yes ⁴	Stopped	No
SPI	Enabled	Enabled	Yes ⁴	Prog.	Yes ⁴	Stopped	No

Table 3-1 MCore and Peripherals in Low-Power Modes

Module	Peripheral Status ¹ / Wake-up Capability						
	RUN	WAIT		DOZE		STOP	
SCI1/2	Enabled	Enabled	Yes ⁴	Prog.	Yes ⁴	Stopped	No
Timer	Enabled	Enabled	Yes ⁴	Enabled	Yes ⁴	Stopped	No
DSRC module	Enabled	Enabled	Yes ⁴	Enabled	Yes ⁴	Stopped	No
Time Of Day Timer	Enabled	Enabled	Yes ⁴	Enabled	Yes ⁴	Enabled	Yes

NOTES:

1. "Prog." indicates that the peripheral function during the low power mode is dependent on programmable bits in the peripheral register map.
2. These modules can generate a reset which will exit any low power mode.
3. The OnCE logic is clocked by a separate TCLK clock. Entering DEBUG mode via the OnCE port (or assertion of the external DE pin) exits any low power mode. Upon exit from DEBUG mode, the previous low-power mode will be re-entered and the changes made during DEBUG mode will remain in effect.
4. These modules can generate an interrupt which will exit any low power mode. The MCore will begin to service the interrupt exception after wakeup.

Section 4 Clocks

The clock module contains the crystal oscillator reference (OSC), phase lock loop (PLL), reduced frequency divider (RFD), status/control registers, and clock/PLL control logic. Figure 4-1 System Clocks Block Diagram shows the overall block diagram for the clock module.

To improve noise immunity, the PLL and OSC has its own set of power supply pins, VDDsyn and VSSsyn. All other circuits are powered by the normal internal supply pins, VDD and VSS.

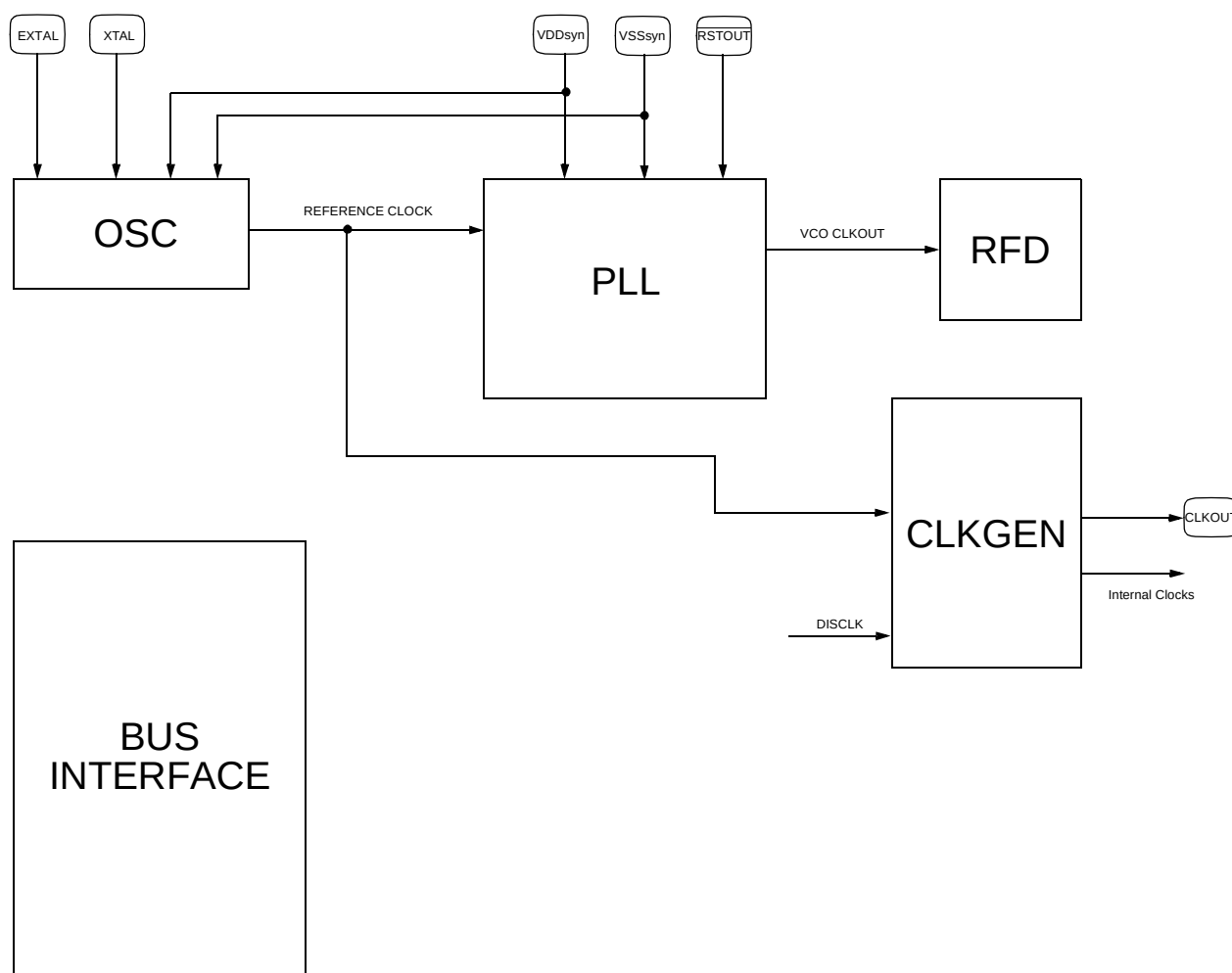


Figure 4-1 System Clocks Block Diagram

Clocks

4.1 System Clock Modes

The system clock source is determined during reset as shown in Table 3-6 Configuration During Reset. The value of VDDsyn is latched during reset and is expected to remain at that state after reset is negated.If VDDsyn is changed during a reset other than power-on reset, the internal clocks may glitch as the clock source is changed between external clock mode and PLL clock mode. Whenever VDDsyn is changed in reset, an immediate loss of lock condition occurs.

Shown in Table 4-1 Clock-out vs. Clock-in Relationships is the clock-out frequency to clock-in frequency relationships for the possible clock modes.

Table 4-1 Clock-out vs. Clock-in Relationships

Clock Mode	PLL Options
Normal PLL Clock Mode	$f_{sys} = f_{ref} * (MFD + 2) / 2^{RFD}$
1:1 PLL Clock Mode	$f_{sys} = f_{ref}$
External Clock Mode	$f_{sys} = f_{ref} / 2$

NOTES:
 fref = input reference frequency
 fsys = CLKOUT frequency
 MFD ranges from 0 to 7. See Table 4-2 System Frequency Multiplier of the Reference.
 RFD ranges from 0 to 7. See Table 4-2 System Frequency Multiplier of the Reference.

When normal PLL clock mode is selected, the PLL is fully programmable. It can synthesize frequencies ranging from 2X to 9X and has a post divider capable of reducing the PLL clock frequency without disturbing the PLL. In addition, the PLL reference can be either a crystal oscillator reference or an external clock reference.

When 1:1 PLL clock mode is selected, the PLL synthesizes the input reference frequency and the post divider is not active. The input reference frequency is an external clock reference.

When external clock mode is selected, the PLL is bypassed and the external clock is applied to EXTAL.

CAUTION:

XTAL must be tied low when external clock mode is selected and RESET is asserted. If it is not, clocks could be suspended indefinitely.

The external clock is divided by 2 internally to produce the various system clocks. Refer to electrical specifications for external clock input requirements.

4.2 System Clocks Generation

System clocks include the clock used to generate CLKOUT and also the various internal clocks.

4.2.1 Programming System Clocks Frequency

In normal PLL clock mode, the default system frequency is two times the reference frequency after reset. The system frequency multiplier that can be selected are shown in Table 4-2 System Frequency Multiplier of the Reference. The frequency multiple is determined by the RFD and Multiplication Frequency Divisor (MFD) bits in the SYNCR.

When programming the PLL, the system designer must be sure not to violate the maximum system clocks frequency specification. See electrical specifications for the maximum allowable frequency. The following steps are recommended to accommodate the frequency overshoot that occurs when the MFD bits are changed.

1. Determine the appropriate value for the MFD and RFD fields in the synthesizer control register (SYNCR). Note that the amount of jitter in the system clocks can be minimized by selecting the maximum MFD factor that can be paired with an RFD factor to provide the desired frequency.
2. Write a value of $RFD = RFD \text{ (from step 1)} + 1$ to the RFD field of the SYNCR.
3. Write the value determined in step 1 for the MFD field to the SYNCR.
4. Monitor the synthesizer lock bit (LOCK) in the synthesizer status register (SYNSR). When the PLL achieves lock, write the RFD value determined in step 1 to the RFD field of the SYNCR. This changes the system clocks frequency to the desired frequency.

Table 4-2 System Frequency Multiplier of the Reference

RFD[2:0]	MFD[2:0] =							
	000 (2X)	001 (3X)	010 (4X)*	011 (5X)	100 (6X)	101 (7X)	110 (8X)	111 (9X)
0 = 000 (+1)	2	3	4	5	6	7	8	9
1 = 001 (+2)*	1	1.5	2	2.5	3	3.5	4	4.5
2 = 010 (+4)	0.5	.75	1	1.25	1.5	1.75	2	2.25
3 = 011 (+8)	0.25	0.375	0.5	0.625	0.75	0.875	1	1.125
4 = 100 (+16)	0.125	0.188	0.25	0.313	0.375	0.4385	0.5	0.563
5 = 101 (+32)	0.0625	0.094	0.125	0.156	0.188	0.218	0.25	0.281
6 = 110 (+64)	0.031	0.047	0.063	0.078	0.094	0.109	0.125	0.141
7 = 111 (+128)	0.016	0.023	0.031	0.039	0.047	0.055	0.062	0.070

* Denotes default value out of reset.

NOTE: Keep maximum system clock frequency below specified limit given in the electrical specifications.

4.3 PLL Lock Detection

The lock detect logic monitors the reference frequency and the PLL feedback frequency to determine when frequency lock has been achieved. Phase lock is inferred by the frequency relationship, but is not guaranteed. The PLL lock status is reflected in the LOCK status bit in the SYNSR. A sticky lock status indication, LOCKS, is also provided.

The lock detect function utilizes two counters which are clocked by the reference and PLL feedback respectively. When the reference counter has counted N cycles, the feedback counter's count is compared. If the feedback counter has also counted N cycles, the process is repeated for N + K counts. Then if the two counters' counts still match, the lock criteria is relaxed by 1/2 and the system is notified that the PLL has achieved frequency lock.

After lock has been detected, the lock circuitry continues to monitor the reference and feedback frequencies using the alternate count and compare process. If the counters do not match at any comparison time, then the LOCK status bit is cleared to indicate that the PLL has lost lock. At this point, the lock criteria is tightened and the lock detect process is repeated.

The alternate count sequences prevent false lock detects due to frequency aliasing while the PLL tries to lock. Alternating between a tight and relaxed lock criteria prevents the lock detect function from randomly toggling between locked and not locked status due to phase sensitivities. Figure 4-2 Lock Detect Sequence illustrates the sequence for detecting locked and not locked conditions.

In external clock mode, the PLL cannot lock since the PLL is disabled.

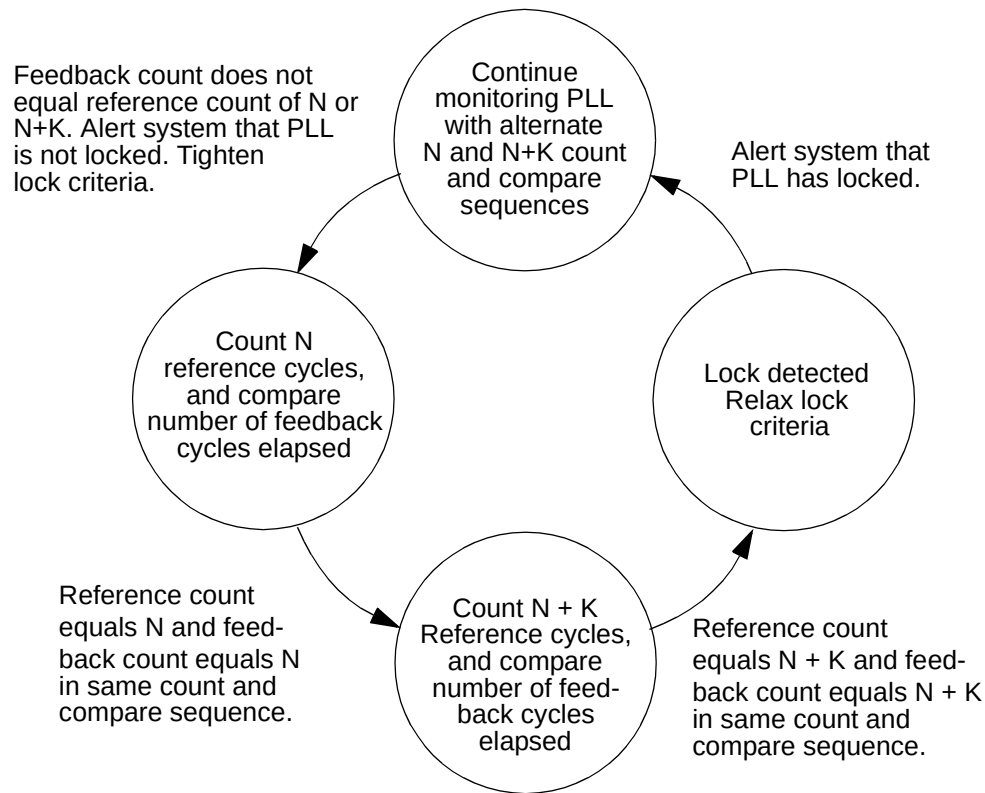


Figure 4-2 Lock Detect Sequence

4.3.1 PLL Loss of Lock Conditions

Once the PLL acquires lock after reset, the LOCK and LOCKS status bits are set. If the MFD is changed or if an unexpected loss of lock condition occurs, the LOCK and LOCKS status bits are negated. While the PLL is in an unlocked condition, the system clocks continue to be sourced from the PLL as the PLL attempts to relock. Consequently, during the relocking process, the system clocks frequency is not well defined and may possibly exceed the maximum system frequency violating the system clock timing specifications.

However, once the PLL has relocked, the LOCK bit is set. The LOCKS bit remains cleared if the loss of lock was unexpected. The LOCKS bit is set to one when the loss of lock was caused by changing MFD. If the PLL is intentionally disabled during low power STOP, then after exit from the STOP, the LOCKS bit will reflect the value prior to entering low power STOP mode once lock is regained.

Clocks

4.3.2 PLL Loss of Lock Reset

The system provides the ability to assert reset when a loss of lock condition occurs by programming the LOLRE bit in the SYNCR. Reset is asserted if LOLRE is set. Since the LOCK and LOCKS bits in the SYNSR are reinitialized after reset, the LOL bit must be read in the reset status register (RSR) to determine that a loss of lock condition occurred.

To exit reset in PLL mode, the reference must be present and the PLL must acquire lock.

In external clock mode, the PLL cannot lock. Therefore a loss of lock condition cannot occur, and LOLRE has no effect.

4.4 Loss of Clock Detection

When enabled by the LOCEN bit in the SYNCR, the loss of clock (LOC) detection circuit monitors the input clocks to the phase/frequency detector (PFD). When either the reference or feedback clock frequency falls below a minimum frequency, the LOC circuitry considers the clock to have failed and a loss of clock status is reflected by the sticky LOCS bit in the SYNSR. See electrical specifications for the minimum clock frequency.

In external clock mode, the loss of clock circuitry is disabled.

4.4.1 Alternate Clock Selection

Depending upon which clock source has failed, the LOC circuitry switches the system clocks source to the remaining operational clock. The system clocks are derived from the alternate clock source until reset is asserted. As shown in Table 4-3 Loss of Clock Summary, if the reference fails, the PLL goes out of lock and into self-clocked mode (SCM). The PLL remains in SCM until the next reset. Note, when the PLL is operating in SCM the system frequency is dependent upon the value in the RFD. The SCM system frequency stated in electrical specifications assumes that the RFD has been programmed to \$0. If the loss of clock condition is due to a PLL failure, the PLL reference becomes the system clocks source until the next reset, even if the PLL regains itself and re-locks.

Table 4-3 Loss of Clock Summary

Clock Mode	System Clock Source before Failure	REFERENCE FAILURE Alternate Clock selected by LOC circuitry until Reset	PLL FAILURE Alternate Clock selected by LOC circuitry until Reset
PLL	PLL	PLL Self-Clocked Mode	PLL reference
External	External Clock	None	NA

Note: The LOC circuit monitors the inputs to the PFD: reference, and feedback clocks. See Figure 4-7 PLL Block Diagram on page 74.

A special loss of clock condition occurs when both the reference and the PLL fail. The failures may be simultaneous or the PLL may fail first. In either case, the reference clock failure takes priority and the PLL attempts to operate in SCM. If successful, the PLL remains in SCM until the next reset. If the PLL cannot operate in SCM, the system remains static until the next reset. Both the reference and the PLL must be functioning properly to exit reset.

4.4.2 Loss of Clock Reset

When a loss of clock condition is recognized, reset is asserted if the LOCRE bit in the SYNCR is set. The LOCS bit in the SYNCR is cleared after reset, therefore, the LOC bit must be read in the RSR to determine that a loss of clock condition occurred. LOCRE has no effect in external clock mode.

To exit reset in PLL mode, the reference must be present and the PLL must acquire lock.

4.5 Clock Operation During Reset

If operating in external clock mode, the system is static and does not recognize reset until a clock is applied to EXTAL.

If operating in PLL mode, the PLL operates in SCM until the input reference clock to the PLL begins operating within the specified limits, see electrical specifications.

If a PLL failure caused the reset as outlined in the loss of clock discussion above, the system enters reset via the reference clock. Then the clock source changes to the PLL operating in SCM. If SCM is not functional, the system becomes static. Alternately, if LOCEN=0 when the PLL fails, the system becomes static. If external reset is asserted, the system cannot enter reset unless the PLL is capable of operating in SCM.

Clocks

4.6 Clock Operation During Low Power Modes

During the low power WAIT and DOZE modes, the clocks to the CPU, Flash, and RAM will be stopped and the system clocks to the peripherals are enabled. Each module may disable the module clocks locally at the module level. During the low power STOP mode, all clocks to the system will be stopped.

During STOP mode, there are several options for enabling/disabling the PLL and/or crystal oscillator (OSC) at a price of wakeup recovery time. The PLL may be disabled during STOP, but then there will be a wakeup time required for the PLL to re-lock. The OSC may also be disabled during STOP, but then there will be an additional wakeup time required for the OSC to restart. For more information on operation in STOP mode, please see **Table 4-4**

The external CLKOUT signal may also be left enabled during low power STOP (if the PLL is still enabled) to support systems using this signal as the clock source.

There is also an option of enabling the system clocks during wakeup from STOP mode quickly. This eliminates the wakeup recovery time but at a price of sending a potentially unstable clock to the system. If this option is used, it is recommended that the RFD is changed to (current RFD value plus 1) before entering into the low power STOP in order to prevent any frequency overshoot of an unlocked PLL.

In external clock mode, there are no wait times for the OSC start-up or PLL lock.

During wakeup from low power modes, the Flash clock will always clock through at least 16 cycles before the core clocks are enabled. This allows the flash module time to recover from the low power mode. Thus, software may immediately continue to fetch instructions from the Flash memory.

Table 4-4 STOP Mode Operation

MODE In	LOCEN	LOCRE	LOLRE	PLL	OSC	FWKUP	Expected PLL Action at Stop	PLL Action during Stop	MODE Out	LOCKS	LOCK	LOCS	Comments
EXT	X	X	X	X	X	X	—	—	EXT	0	0	0	
							Lose ref. clock	Stuck	Stuck	—	—	—	
							Regain	NRM	NRM	'LK	1	'LC	
							No regain	Stuck	Stuck	—	—	—	
NRM	0	0	0	Off	Off	0	Lose lock, f.b. clock, ref. clock	Regain clocks, but don't regain lock	SCM→ unstable NRM	0→'LK	0→1	1→'LC	Block LOCS and LOCKS until clock and lock respectively regain; enter SCM regardless of LOCEN bit until ref. regained
	X	0	0	Off	Off	1	Lose lock, f.b. clock, ref. clock	No ref. clock regain	SCM→	0→	0→	1→	Block LOCS and LOCKS until clock and lock respectively regain; enter SCM regardless of LOCEN bit
							No f.b. clock regain	No f.b. clock regain	Stuck	—	—	—	
							Regain	Regain	NRM	'LK	1	'LC	Block LOCKS from being cleared
NRM	0	0	0	Off	On	0	Lose lock	Lose ref. clock or No lock regain	Stuck	—	—	—	
							Lose ref. clock, regain	Lose ref. clock, regain	NRM	'LK	1	'LC	Block LOCKS from being cleared
							No lock regain	No lock regain	Unstable NRM	0→'LK	0→1	'LC	Block LOCKS until lock regained
							Lose ref. clock or No f.b. clock regain	Lose ref. clock or No f.b. clock regain	Stuck	—	—	—	
NRM	0	0	0	Off	On	1	Lose lock	Lose ref. clock, regain	Unstable NRM	0→'LK	0→1	'LC	LOCS not set because LOCEN = 0
							—	—	NRM	'LK	1	'LC	
							Lose lock or clock	Lose lock or clock	Stuck	—	—	—	
							Lose lock, regain	Lose lock, regain	NRM	0	1	'LC	
NRM	0	0	0	On	On	0	Lose lock and lock, regain	Lose clock and lock, regain	NRM	0	1	'LC	LOCS not set because LOCEN = 0
							—	—	NRM	'LK	1	'LC	
							Lose lock	Lose lock	Unstable NRM	0	0→1	'LC	
							Lose lock, regain	Lose lock, regain	NRM	0	1	'LC	
							Lose clock	Lose clock	Stuck	—	—	—	
							Lose clock, regain without lock	Lose clock, regain without lock	Unstable NRM	0	0→1	'LC	
							Lose clock, regain with lock	Lose clock, regain with lock	NRM	0	1	'LC	

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Clocks

Table 4-4 STOP Mode Operation

MODE In	LOCEN	LOCREF	LOLREF	PLL	OSC	FWKUP	Expected PLL Action at Stop	PLL Action during Stop	MODE Out	LOCKS	LOCK	LOCS	Comments
NRM	X	X	1	Off	X	X	Lose lock, f.b. clock, ref. clock	RESET	RESET	—	—	—	Reset immediately
NRM	0	0	1	On	On	X	—	—	NRM	'LK	1	'LC	Reset immediately
NRM	1	0	0	Off	Off	0	Lose lock, f.b. clock, ref. clock	Regain	NRM	'LK	1	'LC	REF not entered during stop; SCM entered during stop only during OSC startup
NRM	1	0	0	Off	On	0	Lose lock, f.b. clock	No regain	Stuck	—	—	—	—
NRM	1	0	0	Off	On	0	Lose lock, f.b. clock	Regain	NRM	'LK	1	'LC	REF mode not entered during stop
NRM	1	0	0	Off	On	0	Lose lock, f.b. clock	No f.b. clock or lock regain	Stuck	—	—	—	—
NRM	1	0	0	Off	On	1	Lose lock, f.b. clock	Regain f.b. clock	SCM	0	0	1	Wake up without lock
NRM	1	0	0	Off	On	1	Lose lock, f.b. clock	No f.b. clock regain	Unstable NRM	0→'LK	0→1	'LC	REF mode not entered during stop
NRM	1	0	0	Off	On	1	Lose lock, f.b. clock	Lose ref. clock	Stuck	—	—	—	—
NRM	1	0	0	On	On	0	—	—	SCM	0	0	1	Wake up without lock
NRM	1	0	0	On	On	0	—	Lose ref. clock	NRM	'LK	1	'LC	—
NRM	1	0	0	On	On	0	—	Lose ref. clock	SCM	0	0	1	Wake up without lock
NRM	1	0	0	On	On	0	—	Lose f.b. clock	REF	0	X	1	Wake up without lock
NRM	1	0	0	On	On	0	—	Lose lock	Stuck	—	—	—	—
NRM	1	0	0	On	On	0	—	Lose lock, regain	NRM	0	1	'LC	—
NRM	1	0	0	On	On	1	—	—	NRM	'LK	1	'LC	—
NRM	1	0	0	On	On	1	—	Lose ref. clock	SCM	0	0	1	Wake up without lock
NRM	1	0	0	On	On	1	—	Lose f.b. clock	REF	0	X	1	Wake up without lock
NRM	1	0	0	On	On	1	—	Lose lock	Unstable NRM	0	0→1	'LC	—
NRM	1	0	1	On	On	X	—	—	NRM	'LK	1	'LC	—
NRM	1	0	1	On	On	X	—	Lose lock or clock	RESET	—	—	—	Reset immediately
NRM	1	1	X	Off	X	X	Lose lock, f.b. clock, ref. clock	RESET	RESET	—	—	—	Reset immediately
NRM	1	1	0	On	On	0	—	—	NRM	'LK	1	'LC	—
NRM	1	1	0	On	On	0	—	Lose clock	RESET	—	—	—	Reset immediately
NRM	1	1	0	On	On	0	—	Lose lock	Stuck	—	—	—	—
NRM	1	1	0	On	On	0	—	Lose lock, regain	NRM	0	1	'LC	—

Table 4-4 STOP Mode Operation

MODE In	LOCEN	LOCREF	LOLREF	PLL	OSC	FWKUP	Expected PLL Action at Stop	PLL Action during Stop	MODE Out	LOCKS	LOCK	LOCKS	Comments
NRM	1	1	0	On	On	1	—	—	NRM	'LK	1	'LC	
								Lose clock	RESET	—	—	—	Reset immediately
								Lose lock	Unstable NRM	0	0->1	'LC	
								Lose lock, regain	NRM	0	1	'LC	
NRM	1	1	1	On	On	X	—	—	NRM	'LK	1	'LC	
								Lose clock or lock	RESET	—	—	—	Reset immediately
REF	1	0	0	X	X	X	—	—	REF	0	X	1	
								Lose ref. clock	Stuck	—	—	—	
SCM	1	0	0	Off	X	0	PLL disabled	Regain SCM	SCM	0	0	1	Wake up without lock
SCM	1	0	0	Off	X	1	PLL disabled	Regain SCM	SCM	0	0	1	
SCM	1	0	0	On	On	0	—	—	SCM	0	0	1	Wake up without lock
								Lose ref. clock	SCM	0	0	1	
SCM	1	0	0	On	On	1	—	—	SCM	0	0	1	
								Lose ref. clock	SCM	0	0	1	

- PLL = PLL enabled during STOP mode. PLL = On when STPMD[1:0] = 00 or 01.
- OSC = OSC enabled during STOP mode. OSC = On when STPMD[1:0] = 00, 01, or 10.
- MODES
 - NRM = normal PLL crystal clock reference or normal PLL external reference or PLL 1:1 mode. During PLL 1:1 or normal external reference mode, the oscillator is never enabled. Therefore, during these modes, refer to the OSC = On case regardless of STPMD values.
 - EXT = external clock mode
 - REF = PLL reference mode due to losing PLL clock or lock from NRM mode
 - SCM = PLL self-clocked mode due to losing reference clock from NRM mode
 - RESET = immediate reset
- LOCKS
 - 'LK = expecting previous value of LOCKS before entering stop.
 - 0->'LK = current value is 0 until lock is regained which then will be the previous value before entering stop.
 - 0-> = current value will be 0 until lock is regained but lock is never expected to regain.
- LOCS
 - 'LC = expecting previous value of LOCS before entering stop.
 - 1->'LC = current value is 1 until clock is regained which is then will be the previous value before entering stop.
 - 1-> = current value is 1 until clock is regained but clk is never expected to regain.

Clocks

4.7 Programming Model

The Clock programming model consists of the following registers:

- The Synthesizer Control Register (SYNCR) contains bits for defining the clock operation for the system.
- The Synthesizer Status Register (SYNSR) indicates clock status information.
- The Synthesizer Test Register (SYNTR) used for factory test only.
- The Synthesizer Test Register 2 (SYNTR2) used for factory test only.

Table 4-5 Clock Address Map

Offset ¹	31 - 24	23 - 16	15 - 8	7 - 0	Access ²
\$00	SYNCR		SYNSR	SYNTR	Supervisor Only
\$04	SYNTR2				Supervisor Only

NOTES:

1. Absolute address is equal to the Offset plus the base address. The base address is chip dependent.
2. User mode accesses to supervisor only address locations have no effect and result in a cycle termination transfer error.

4.7.1 Synthesizer Control Register (SYNCR)

The Synthesizer Control Register (SYNCR) contains bits for defining the clock operation for the system. This register is read/write always.

SYNCR — Synthesizer Control Register

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	LOLRE	MFD			LOCRE	RFD			LOCEN	DISCLK	FWK-UP	RSVD	STPMD	RSVD	RSVD	
W																
RESET:	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0

Figure 4-3 Synthesizer Control Register

LOLRE — Loss of Lock Reset Enable

The LOLRE bit determines how the system handles a loss of lock indication. See Section 4.3 PLL Lock Detection for additional information.

When operating in normal or 1:1 PLL mode, the PLL must be locked before setting the LOLRE bit. Otherwise reset is immediately asserted.

To prevent an immediate reset, the LOLRE bit must be cleared before writing the MFD bits or entering STOP mode with the PLL disabled.

The LOLRE bit has no affect in external clock mode.

0 = Ignore loss of lock - reset not asserted.

1 = Assert reset on loss of lock.

MFD — Multiplication Factor Divider

The MFD bits control the value of the divider in the PLL feedback loop. The value specified by the MFD bits establishes the multiplication factor applied to the reference frequency. The bit field encoding is shown in row one in Table 4-2 System Frequency Multiplier of the Reference.

When the MFD bits are changed or the PLL is disabled in STOP mode, the PLL loses lock.

In 1:1 PLL mode, the MFD bits are ignored and the multiplication factor is set to 1X.

In external clock mode the MFD bits have no affect.

LOCRES — Loss of Clock Reset Enable

The LOCRES bit determines how the system handles a loss of clock condition when LOCEN=1. LOCRES has no effect when LOCEN=0.

If the LOCS bit in the SYNSR indicates a loss of clock condition, setting the LOCRES bit causes an immediate reset.

To prevent an immediate reset, the LOCRES bit must be cleared before entering STOP mode with the PLL disabled.

In external clock mode LOCRES has no affect.

0 = Ignore loss of clock - reset not asserted.

1 = Assert reset on loss of clock.

RFD — Reduced Frequency Divider

The RFD bits control a divider at the output of the PLL. The value specified by the RFD bits establishes the divisor applied to the PLL frequency. The RFD bit field encoding is shown in column one in Table 4-2 System Frequency Multiplier of the Reference.

Changing the RFD bits does not affect the PLL, hence, no re-lock delay is incurred. Resulting changes in clock frequency are synchronized to the next falling edge of the current system clock. However these bits should only be written when the lock bit (LOCK) is set, to avoid surpassing the allowable system operating frequency.

In external clock mode the RFD bits have no affect.

LOCEN — Loss of Clock Enable

The LOCEN bit determines whether the loss of clock function is operational. See Section 4.4 Loss of Clock Detection for more information.

In external clock mode, this bit has no affect.

LOCEN does not affect the loss of lock circuitry.

0 = Loss of clock disabled.

1 = Loss of clock enabled.

Clocks

DISCLK — Disable CLKOUT

The DISCLK bit determines whether CLKOUT is driven. When CLKOUT is disabled it is held low.

- 0 = CLKOUT enabled.
- 1 = CLKOUT disabled.

FWKUP — Fast WaKeUP

The FWKUP bit determines when the system clocks are enabled during wakeup from the low power STOP mode.

- 0 = The system clocks will not be enabled during wakeup from low power STOP mode until the PLL is locked or operating normally. If the PLL or OSC remained active during STOP mode and was unintentionally lost, then the part would come out in self-clocked mode or reference clock mode depending on the clock that was unintentionally lost.
- 1 = The system clocks will immediately be enabled during wakeup from low power STOP mode regardless of the PLL lock status.

Note: The FWKUP bit has no effect on the wakeup sequence in external clock mode.

STPMD — Stop Modes

The STPMD bits determine how the PLL and CLKOUT operate during the low power STOP mode. See Table 4-6 STOP Mode Operation.

Table 4-6 STOP Mode Operation

STPMD[1]	STPMD[0]	Operation During STOP Mode			
		System Clocks	PLL	OSC	CLKOUT
0	0	Disabled	Enabled	Enabled	Enabled
0	1	Disabled	Enabled	Enabled	Disabled
1	0	Disabled	Disabled	Enabled	Disabled
1	1	Disabled	Disabled	Disabled	Disabled

RSVD — Reserved (Bits 4, 1, 0)

The RSVD bits can be read at any time. Writes to these bits update the register values but have no effect on any other functionality.

4.7.2 Synthesizer Status Register (SYNSR)

The Synthesizer Status Register (SYNSR) indicates clock status information.

This read-only register can be read at any time, but writes to this register have no effect and terminate the cycle normally.

SYNSR — Synthesizer Status Register

	7	6	5	4	3	2	1	0
R	MODE	PLLSEL	PLLREF	LOCKS	LOCK	LOCS	0	0
W								

RESET:

* * * ** ** 0 0 0

* Reset state determined during reset configuration. (See Table 3-6 Configuration During Reset)

** See the LOCKS and LOCK bit descriptions below.

Figure 4-4 Synthesizer Status Register

MODE — Clock Mode

The MODE bit is determined at reset, it indicates which clock mode the system is utilizing as shown in Table 4-7 System Clock Status Per Mode on page 71. See Table 3-6 Configuration During Reset for details on how to configure the system clock mode during reset.

0 = External clock mode.

1 = PLL clock mode.

Table 4-7 System Clock Status Per Mode

MODE	PLLSEL	PLLREF	Clock Mode
0	0	0	External Clock
1	0	0	1:1 PLL Mode
1	1	0	Normal PLL Mode w/external clock reference
1	1	1	Normal PLL Mode w/crystal clock reference

PLLSEL— PLL Mode Select

The PLLSEL bit is determined at reset, it indicates which mode the PLL operates in. See Table 3-4 System Clock Mode. This bit is cleared in 1:1 PLL mode and external clock mode. See Table 3-6 Configuration During Reset for details on how to configure the system clock mode during reset.

0 = 1:1 PLL mode.

1 = Normal PLL mode.

Clocks

PLLREF — PLL Clock Reference Source

The PLLREF is determined at reset, it indicates which reference source has been chosen for normal PLL mode. See Table 3-4 System Clock Mode. This bit is cleared in 1:1 PLL mode and external clock mode. See Table 3-6 Configuration During Reset for details on how to configure the system clock mode during reset.

- 0 = External clock reference chosen.
- 1 = Crystal clock reference chosen.

LOCKS — Sticky PLL Lock Status Bit

The LOCKS bit is a sticky indication of PLL lock status. LOCKS is set by the lock detect circuitry when the PLL acquires lock after: 1) a system reset; or 2) a write to the SYNCR which modifies the MFD bits. Whenever the PLL loses lock, LOCKS is cleared. LOCKS remains cleared even after the PLL relocks, until one of the two previously-stated conditions occurs. In low power STOP mode, if the PLL is intentionally disabled (see Table 4-7 System Clock Status Per Mode), then the LOCKS bit will reflect the value prior to entering STOP. However, if FWKUP is set, then LOCKS will be cleared until the PLL regains lock. Once lock is obtained again, the LOCKS bit will reflect the value prior to entering STOP. Furthermore, if the LOCKS bit is read when the PLL simultaneously loses lock, the bit does not reflect the current loss of lock condition.

If operating in external clock mode, LOCKS remains cleared after reset. In normal and 1:1 PLL mode, LOCKS is set after reset.

- 0 = PLL has lost lock since last system reset or MFD change or is currently not locked due to exit from STOP with FWKUP set.
- 1 = PLL has not lost lock unintentionally since last system reset or MFD change.

LOCK — PLL Lock Status Bit

The LOCK bit indicates whether the PLL has acquired lock. PLL lock occurs when the synthesized frequency matches to within approximately 0.75% of the programmed frequency. See Table 4.2 System Clocks Generation. The PLL loses lock when a frequency deviation of greater than approximately 1.5% occurs. If the LOCK bit is read when the PLL simultaneously loses lock or acquires lock, the bit does not reflect the current condition of the PLL.

The LOCK bit is used by the power-on-reset circuit as a condition for releasing reset. See Section 4.2 System Clocks Generation for additional information.

If operating in external clock mode, LOCK remains cleared after reset.

- 0 = PLL is unlocked.
- 1 = PLL is locked.

LOCS — Sticky Loss Of Clock Status

The LOCS bit is a sticky indication of whether a loss of clock condition has occurred at any time since exiting reset in normal and 1:1 PLL modes. If LOCS=0, the system clocks are operating normally. If LOCS=1, the system clocks have failed due to a reference failure or a PLL failure. If STOP was entered with FWKUP set and the PLL and oscillator were intentionally

disabled (STPMD = 11), upon exit from STOP mode, the PLL will come up in SCM while the oscillator is trying to start up. During this time, LOCS will be temporarily set regardless of LOCEN. It will be cleared once the oscillator comes up and the PLL is attempting to lock. If the read of the LOCS bit and the loss of clock condition occur simultaneously, the bit does not reflect the current loss of clock condition.

A loss of clock condition can only be detected if LOCEN=1 or oscillator has not yet returned from exit of STOP mode with FWKUP=1. See Section 4.4 Loss of Clock Detection. LOCS is always zero in external clock mode.

0 = A loss of clock has not been detected since exiting reset.

1 = A loss of clock has been detected since exiting reset or oscillator has not yet returned from exit of STOP mode with FWKUP=1.

4.7.3 Synthesizer Test Register (SYNTR)

The Synthesizer Test Register (SYNTR) is used for factory testing only.

This register is read only when not in test mode.

SYNTR — Synthesizer Test Register

	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0
W								
RESET:	0	0	0	0	0	0	0	0

Figure 4-5 Synthesizer Test Register

4.7.4 Synthesizer Test Register 2 (SYNTR2)

The Synthesizer Test Register 2 (SYNTR2) is used for factory testing only.

Register bits 31 - 10 are read only and writes have no effect.

SYNTR2 — Synthesizer Test Register 2

	31 - 10	9 - 0
R	0	RSVD
W		
RESET:	0	0

Figure 4-6 Synthesizer Test Register 2

Clocks

RSVD — Reserved (Bits 9 - 0)

The RSVD bits can be read at any time. Writes to these bits update the register values but have no effect on any other functionality.

4.8 PLL Operation

In PLL mode, the system clocks are synthesized by a PLL that is capable of multiplying the reference clock frequency by 2X to 9X, provided the system clock (CLKOUT) frequency remains within the range listed in electrical specifications. For example, if the reference frequency is 2MHz the PLL can synthesize frequencies of 4MHz to 18MHz. In addition, the output of the PLL can be divided down to reduce the system frequency with the RFD. The RFD is not contained in the feedback loop of the PLL, so changing the RFD bits does not affect PLL operation. Figure 4-7 PLL Block Diagram shows the overall block diagram for the PLL. Each of the major blocks shown is discussed briefly below.

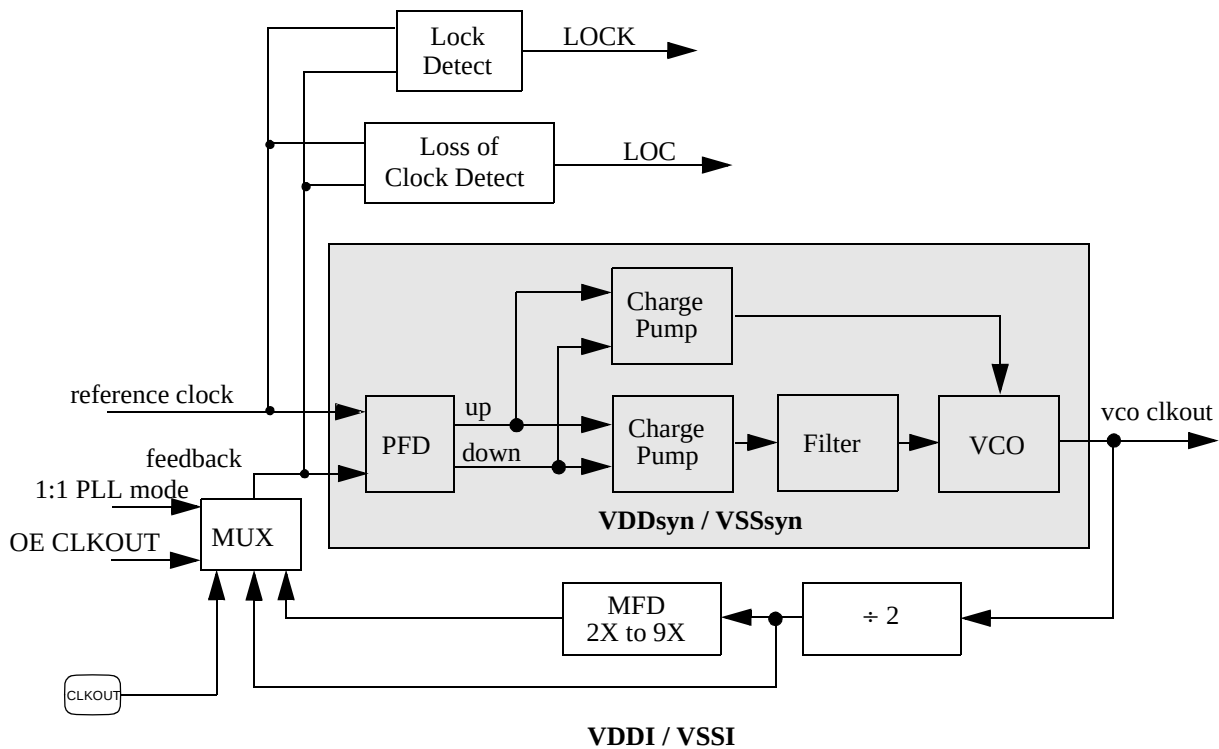


Figure 4-7 PLL Block Diagram

The external support circuitry for the crystal oscillator is shown in Figure 4-8 Crystal Oscillator Network. Example component values are shown as well. Note, the actual circuit should be reviewed with the crystal manufacturer.

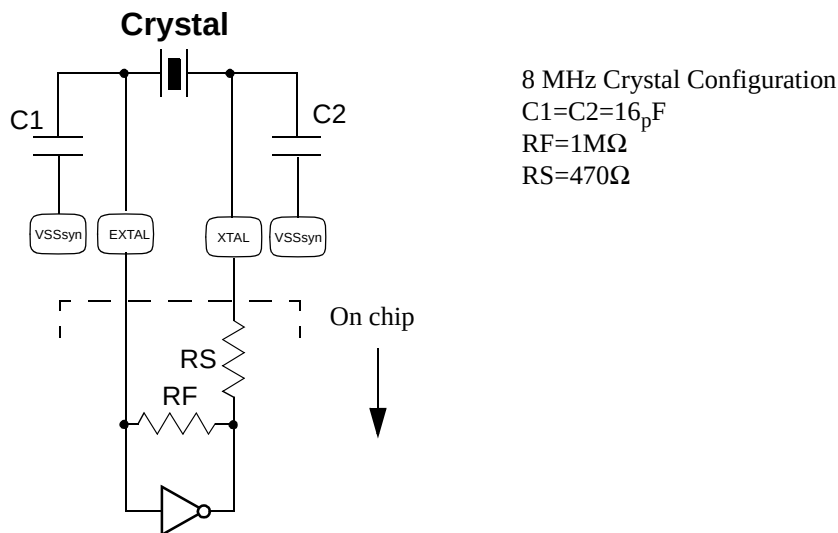


Figure 4-8 Crystal Oscillator Network

4.8.1 Phase/Frequency Detector (PFD)

The PFD is a dual-latch phase-frequency detector. It compares both the phase and frequency of the reference clock and the feedback clock. The reference clock comes from either the crystal oscillator or an external clock source. The feedback clock comes from either CLKOUT in 1:1 PLL mode, a divide by 2 block if CLKOUT is disabled in 1:1 PLL mode, or from the VCO output divided down by the MFD in normal PLL mode.

When the frequency of the feedback clock equals the frequency of the reference clock (i.e. the PLL is frequency locked), the PFD will pulse the UP or DOWN signals depending on the relative phase of the two clocks. If the falling edge of the feedback clock lags the falling edge of the reference clock, then the UP signal is pulsed. If the falling edge of the feedback clock leads the falling edge of the reference clock, then the DOWN signal is pulsed. The width of these pulses relative to the reference clock is dependent on how much the two clocks lead or lag each other. Once phase lock is achieved, the PFD continues to pulse the UP and DOWN signals for a very short duration during each reference clock cycle. These short pulses force the PLL to continually update and prevent a frequency drift phenomena referred to as “dead-banding.”

Charge Pump/Loop Filter

Clocks

The charge pump operates in two modes which are automatically selected dependent upon the PLL mode. In 1:1 mode, a fixed current is utilized. In normal mode the charge pump current magnitude varies with the MFD as shown in Table 4-8 Charge pump vs. MFD in Normal Mode Operation.

Table 4-8 Charge pump vs. MFD in Normal Mode Operation

Ichgpump	MFD
1X	$0 \leq \text{MFD} < 2$
2X	$2 \leq \text{MFD} < 6$
4X	$6 \leq \text{MFD}$

Actual operation of the charge pump is controlled by the UP and DOWN signals from the PFD. They control whether the charge pump applies or removes charge, respectively, from the loop filter. The filter is integrated on chip.

4.8.2 VCO

The voltage formed across the loop filter controls the frequency of the VCO output. The frequency to voltage relationship (or VCO gain) is positive and the output frequency is four times the target system frequency.

4.8.3 MFD

The MFD divides down the output of the VCO and feeds it back (when not in 1:1 PLL mode) to the PFD. The PFD controls the VCO frequency (via the charge pump and loop filter) such that the reference and feedback clocks have the same frequency and phase. Thus, the input to the MFD, which is also the output of the VCO, is at a frequency that is the reference frequency multiplied by the same amount that the MFD divides by. For example, if the MFD divides the VCO frequency by 6, then the PLL will be frequency locked when the VCO frequency is 6 times the reference frequency. The presence of the MFD in the loop allows the PLL to perform frequency multiplication, or synthesis. Table 4-2 System Frequency Multiplier of the Reference shows the possible MFD values. When the PLL is operating in 1:1 PLL mode, the MFD is bypassed and the effective multiplication factor is 1X.

Section 5 Reset

5.1 Overview

The Reset Controller is provided to determine the cause of reset, assert the appropriate reset signals to the system, and then to keep a history of what caused the reset.

The following functionality is provided by the Reset Controller:

- Six sources of reset
 - External
 - Power-on-Reset
 - Watchdog Timer
 - PLL loss of lock
 - PLL loss of clock
 - Software
- Software can assert external $\overline{\text{RSTOUT}}$ pin independent of chip reset state.
- Software readable status flags indicating the cause of the last reset.

Figure 5-1 Reset Block Diagram illustrates the Reset Controller and is explained in the following sections.

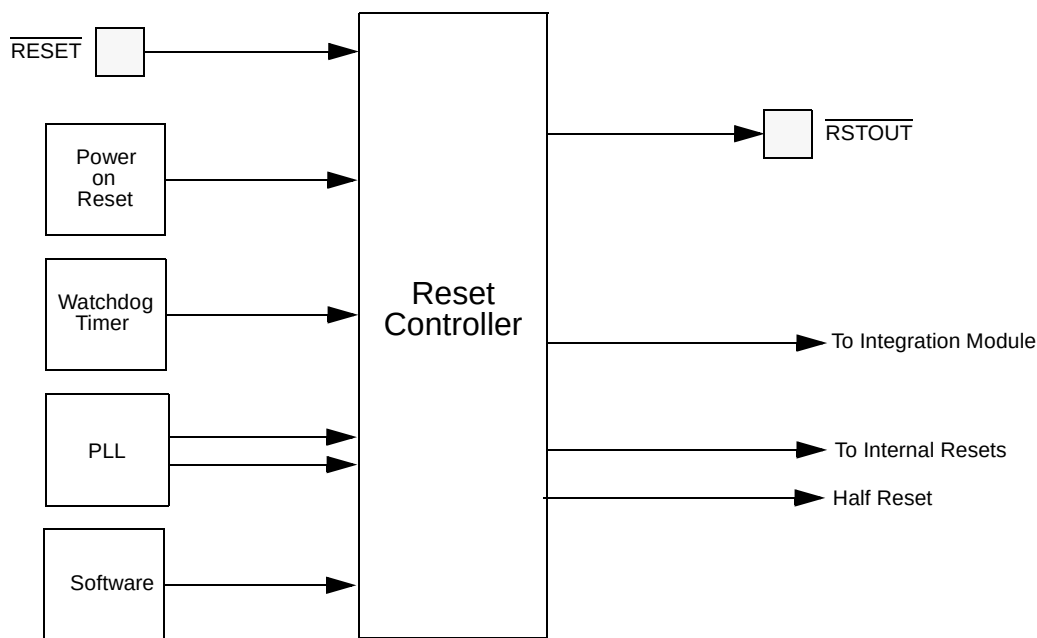


Figure 5-1 Reset Block Diagram

5.2 Sources of Reset

Table 5-1 Reset Source Summary defines the sources of reset and the signals driven by the reset controller.

A synchronous reset source is not acted upon by the reset control logic until the end of the current bus cycle to protect data integrity. Reset is then asserted on the next rising edge of the system clock after the cycle is terminated. Whenever the reset control logic must synchronize reset to the end of the bus cycle, the internal Bus Monitor is automatically enabled regardless of the BME bit setting in the Chip Configuration Register (CCR). Then if the current bus cycle is not terminated normally, the bus monitor terminates the cycle based on the length of time programmed in the BMT field of the CCR.

Internal single byte, half-word, or word writes are guaranteed to complete without data corruption when a synchronous reset occurs. External writes (including word writes to 16-bit ports) are also guaranteed to complete.

Asynchronous reset sources usually indicate a catastrophic failure. Therefore, the reset control logic does not wait for the current bus cycle to complete. Reset is asserted immediately to the system.

Table 5-1 Reset Source Summary

Source	Type
Power On	Asynchronous
External RESET Pin (not STOP mode)	Synchronous
External RESET Pin (during STOP mode)	Asynchronous
Watchdog Timer	Synchronous
Loss of Clock	Asynchronous
Loss of Lock	Asynchronous
Software	Synchronous

5.2.1 Power On Reset

Upon power up, the reset controller asserts $\overline{\text{RSTOUT}}$. $\overline{\text{RSTOUT}}$ continues to be asserted until VDD has reached a minimum acceptable level and, if a PLL clock mode is selected, until the PLL achieves phase-lock. Then after approximately another 512 cycles, $\overline{\text{RSTOUT}}$ is negated and the part begins operation.

5.2.2 External Reset

Asserting the external $\overline{\text{RESET}}$ pin for at least four rising CLKOUT edges causes the external reset request to be recognized and latched. The Bus Monitor is enabled and the current bus cycle

is completed. The reset controller asserts $\overline{\text{RSTOUT}}$ for approximately 512 cycles after the $\overline{\text{RESET}}$ pin is negated and the PLL has acquired a lock. The part then exits reset and resumes operation.

In low power STOP mode, the system clocks are stopped, thus asserting the external $\overline{\text{RESET}}$ pin will asynchronously cause the external reset to be recognized.

5.2.3 Watchdog Timer Reset

A Watchdog timer timeout causes the watchdog timer reset request to be recognized and latched. The Bus Monitor is enabled and the current bus cycle is completed. The reset controller asserts $\overline{\text{RSTOUT}}$ for approximately 512 cycles (if the $\overline{\text{RESET}}$ pin is negated and the PLL has acquired a lock) and then the part exits reset and resumes operation.

5.2.4 Loss of Clock Reset

This reset condition occurs in PLL clock mode when the LOCRE bit in the SYNCR register is set and either the PLL reference or the PLL fails. See Loss of Clock Reset section in the PLL specification for details. The reset controller asserts $\overline{\text{RSTOUT}}$ for approximately 512 cycles after the PLL has acquired a lock. The part then exits reset and resumes operation.

5.2.5 Loss of Lock Reset

This reset condition occurs in PLL clock mode when the LOLRE bit in the SYNCR register is set and the PLL loses lock. See Loss of Lock Reset section in the PLL specification for details. The reset controller asserts $\overline{\text{RSTOUT}}$ for approximately 512 cycles after the PLL has acquired a lock. The part then exits reset and resumes operation.

5.2.6 Software Reset

A Software reset occurs when the $\overline{\text{SOFTRST}}$ bit is set. The reset controller asserts $\overline{\text{RSTOUT}}$ for approximately 512 cycles (if the $\overline{\text{RESET}}$ pin is negated and the PLL has acquired a lock) and then the part exits reset and resumes operation.

5.3 Reset Control Flow

The reset logic control flow is shown in Figure 5-2 Reset Control Flow. In this figure, the control state boxes have been numbered, and these numbers are referred to (within parentheses) in the flow description that follows. All cycle counts given are approximate.

5.3.1 Synchronous Reset Requests

If either the external RESET pin is asserted by an external device for at least four rising CLKOUT edges (3), or the Watchdog timer times out, or software requests a reset, the reset control logic latches the reset request internally and enables the Bus Monitor (5). When the current bus cycle is completed (6), RSTOUT is asserted (7). The reset control logic waits until the RESET pin is negated (8) and for the PLL to attain lock (9, 9A) before waiting 512 CLKOUT cycles (10). The reset control logic may latch the configuration according to the RCON pin level (11, 11A) before negating RSTOUT (12).

If the external RESET pin is asserted by an external device for at least four rising CLKOUT edges during the 512 count (10) or during the wait for PLL lock (9A), the reset flow will switch to (8) and wait for the RESET pin to be negated before continuing.

5.3.2 Internal Reset Request

If reset is asserted by an asynchronous internal reset source, such as loss of clock (1) or loss of lock (2), the reset control logic asserts RSTOUT (4). The reset control logic waits for the PLL to attain lock (9, 9A) before waiting 512 CLKOUT cycles (10). Then the reset control logic may latch the configuration according to the RCON pin level (11, 11A) before negating RSTOUT (12).

If a loss of lock occurs during the 512 count (10), the reset flow will switch to (9A) and wait for the PLL to lock before continuing.

5.3.3 Power-On Reset

When the reset sequence is initiated by power-on reset (0), the same reset sequence is followed as for the other asynchronous reset sources.

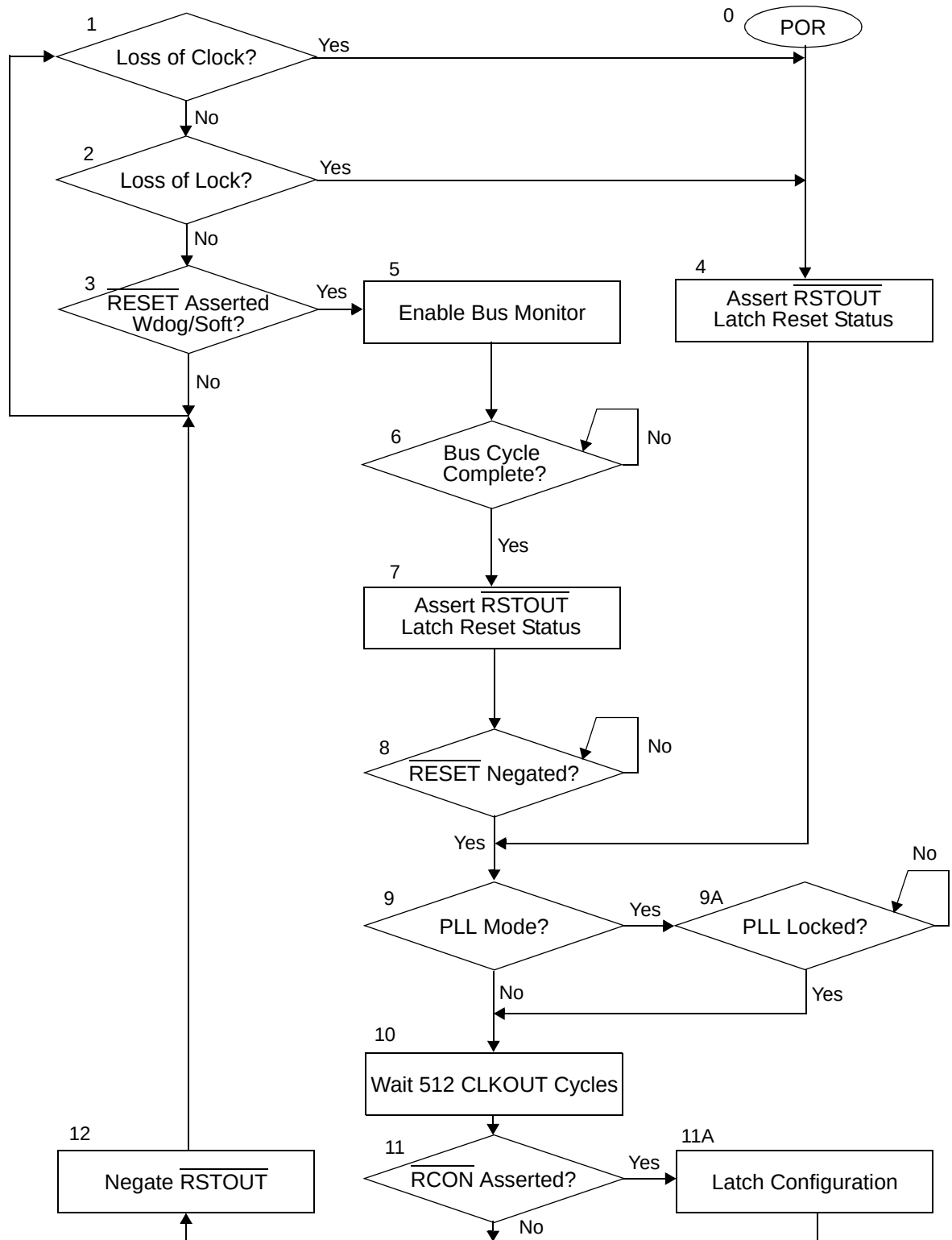


Figure 5-2 Reset Control Flow

5.4 Programming Model

The Reset Controller programming model consists of the following registers:

- The Reset Control Register (RCR) selects different interrupt controller functions.
- The Reset Status Register (RSR) reflects the state of the last reset source.
- The Reset Test Register (RTR) used for factory test only.

Table 5-2 Reset Controller Address Map

Offset ¹	31 - 24	23 - 16	15 - 8	7 - 0	Access
\$00	RCR	RSR	RTR	Reserved ²	Supervisor/User

NOTES:

1. Absolute address is equal to the Offset plus the base address. The base address is chip dependent.
2. Writes to reserved address locations have no effect and reads return zeros.

5.4.1 Reset Control Register (RCR)

The Reset Control Register (RCR) allows software control for requesting a reset or for independently asserting the external RSTOUT pin.

This register is read/write always.

RCR — Reset Control Register

	7	6	5	4	3	2	1	0
R	SOFRST	FRCRSTOUT	0	0	0	0	0	0
W								

RESET:

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

Figure 5-3 Reset Control Register

SOFTRST — Software Reset Request

This allows software to request a reset. Note that the reset caused by setting this bit will clear this bit back to 0.

- 0 = No software reset request.
1 = Request software reset.

FRCRSTOUT — Force RSTOUT Pin

This allows software to assert or negate the external $\overline{\text{RSTOUT}}$ pin.

- 0 = Do not assert the $\overline{\text{RSTOUT}}$ pin.
1 = Assert the $\overline{\text{RSTOUT}}$ pin.

Caution: External logic used to drive reset configuration data during reset needs to be considered when asserting the RSTOUT pin when setting FRCCRSTOUT.

5.4.2 Reset Status Register (RSR)

The Reset Status Register (RSR) contains a status bit for every reset source. When reset is entered, the cause(s) of the reset condition is(are) latched along with a value of zero for the other reset sources that were not pending at the time of the reset condition. These values are then reflected in the RSR. One or more status bits may be set at the same time. The cause of any subsequent reset is also recorded in the register, overwriting status from the previous reset condition. See Section 5.5.2 Reset Status Flags for more details.

This register can be read at any time. Writes to this register have no effect.

RSR — Reset Status Register

	7	6	5	4	3	2	1	0
R	0	0	SOFT	WDR	POR	EXT	LOC	LOL
W								
RESET:	0	0	*	*	*	*	*	*

* Reset Dependent

Figure 5-4 Reset Status Register

SOFT— Software Reset

This bit indicates that the last reset state was caused by software.

0 = No software reset occurred.

1 = Last reset state was caused by software.

WDR — Watchdog Timer Reset

This bit indicates that the last reset state was caused by a Watchdog Timer time out.

0 = No Watchdog Timer time out reset occurred.

1 = Last reset state was caused by a Watchdog Timer time out.

POR — Power On Reset

This bit indicates that the last reset state was caused by a power on reset.

0 = No power on reset occurred.

1 = Last reset state was caused by a power on reset.

EXT — External Reset

This bit indicates that the last reset state was caused by an external device asserting the external RESET pin.

0 = No external reset occurred.

Reset

1 = Last reset state was caused by an external device asserting the external $\overline{\text{RESET}}$ pin.

LOC — Loss of Clock Reset

This bit indicates that the last reset state was caused by a loss of clock as detected by the loss of clock circuit.

- 0 = No loss of clock reset occurred.
- 1 = Last reset state was caused by a loss of clock.

LOL — Loss of Lock Reset

This bit indicates that the last reset state was caused by a loss of lock as detected by the PLL circuit.

- 0 = No loss of lock occurred.
- 1 = Last reset state was caused by a loss of lock.

5.4.3 Reset Test Register (RTR)

The Reset Test Register (RTR) is used for factory testing only.

This register is read-only when not in test mode.

RTR — Reset Test Register

	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0
W								
RESET:	0	0	0	0	0	0	0	0

Figure 5-5 Reset Test Register

5.5 Concurrent Resets

NOTE:

In the descriptions below, the numbers in parenthesis refer to the control state boxes in Figure 5-2 Reset Control Flow.

5.5.1 Reset Flow

If a power-on reset condition is detected during any reset sequence, the power-on reset sequence is immediately started (0).

If the external $\overline{\text{RESET}}$ pin is asserted for at least four rising CLKOUT edges while waiting for PLL lock or the 512 cycles, the external reset is recognized. Reset processing jumps to wait for the external $\overline{\text{RESET}}$ pin to negate (8).

If a loss of clock or loss of lock condition is detected while waiting for the current bus cycle to complete (5,6) for an external reset request, the cycle is terminated. The reset status bits are latched (7) and reset processing waits for the external $\overline{\text{RESET}}$ pin to negate (8).

If a loss of clock or loss of lock condition is detected during the 512 cycle wait, the reset sequence continues after a PLL lock (9,9A).

5.5.2 Reset Status Flags

For a POR reset, the POR bit in the Reset Status Register (RSR) is set, and the SOFT, WDR, EXT, LOC, and LOL bits are cleared to zero even if another type of reset condition is detected during the reset sequence for the POR.

If a loss of clock or loss of lock condition is detected while waiting for the current bus cycle to complete (5,6) for an external reset request, the EXT, SOFT, and/or WDR bits along with the LOC and/or LOL bits are set.

If the RSR bits are latched (7) during the EXT, SOFT, and/or WDR reset sequence with no other reset conditions detected then only the EXT, SOFT, and/or WDR bits are set.

If the RSR bits are latched (4) during the internal reset sequence with the $\overline{\text{RESET}}$ pin not asserted and neither of the SOFT or WDR events, then the LOC and/or LOL bits are the only bits set.

5.6 Half Reset

The half reset signal asserts concurrently with $\overline{\text{RSTOUT}}$ and the internal reset signal. It negates approximately 256 clocks earlier than $\overline{\text{RSTOUT}}$ and the internal reset signal.

Reset

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Section 6 Interrupt Controller

6.1 Overview

The Interrupt Controller is provided to collect requests from multiple interrupt sources and provide an interface to the processor core interrupt logic.

The following functionality is provided by the Interrupt Controller:

- Up to 40 interrupt sources
- 32 unique programmable priority levels for each interrupt source
- Independent enable/disable of pending interrupts based on priority level
- Select normal or fast interrupt request for each priority levelFast interrupt requests always have priority over normal interrupts
- Ability to mask interrupts at and below a defined priority level
- Ability to select between autovectored or vectored interrupt requests
- Vectored interrupts generated based on priority level
- Ability to generate a separate vector number for normal and fast interrupts
- Ability for software to self-schedule interrupts
- Software visibility of pending interrupts and interrupt signals to core
- Asynchronous operation to support wake-up from low power modes

Figure 6-1 illustrates the Interrupt Controller implementation and is explained in the following sections.

Interrupt Controller

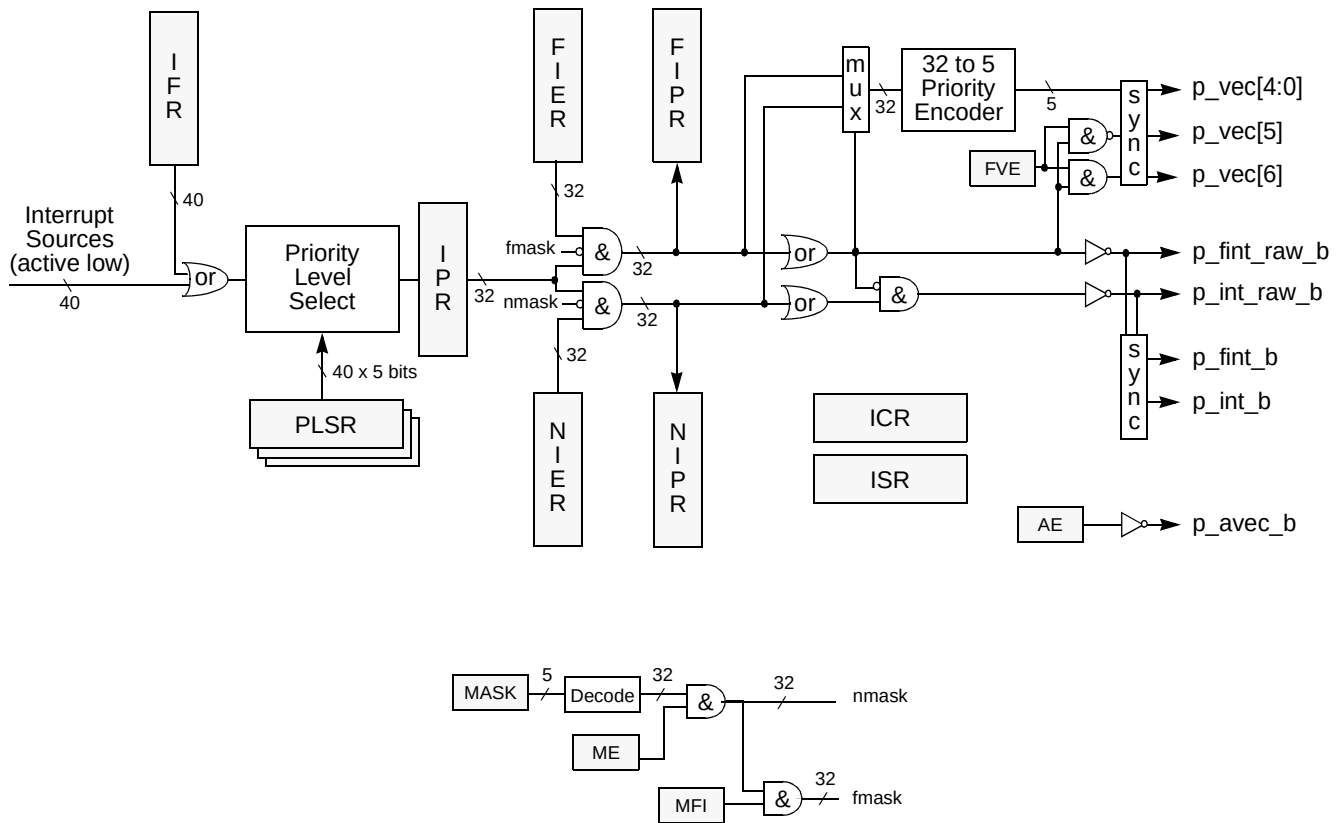


Figure 6-1 Interrupt Controller Block Diagram

6.1.1 Interrupt Sources and Prioritization

Each interrupt source in the system sends a unique signal to the Interrupt Controller. Up to 40 interrupt sources are supported. Each interrupt source can be programmed to one of 32 priority levels using the Priority Level Select Register (PLSR). The highest priority level is 31 and lowest priority level is 0. By default each interrupt source is assigned to the priority level 0. Each interrupt source is associated with a 5 bit priority level select value to allow selecting one of 32 priority levels. The Interrupt Controller uses the priority levels as the basis for the generation of all interrupt signals to the M•CORE processor.

Interrupt requests may be forced by software by writing to the Interrupt Force Registers (IFRH/L). Each bit of the IFR is logically “OR-ed” with the corresponding interrupt source signal before the priority level select logic. To negate the forced interrupt request, the interrupt handler needs to write the appropriate IFR bit to zero. The Interrupt Pending Register (IPR) reflects the state of each priority level.

6.1.2 Fast and Normal Interrupt Requests

The Fast Interrupt Enable Register (FIER) allows individual enabling or masking of the pending interrupt requests. The FIER register is logically "AND-ed" with the IPR to form the contents of the Fast Interrupt Pending Register (FIPR). The FIPR register bits are bit-wise "OR-ed" together (and inverted) to form the fast interrupt signal routed to the M•CORE processor. The FIPR is provided to allow software to quickly determine the highest priority pending fast interrupt. The output of the FIPR also feeds into a 32 to 5 priority encoder to generate the vector number to present to the M•CORE processor if vectored interrupts are required.

The Normal Interrupt Enable Register (NIER) allows individual enabling or masking of the pending interrupt requests. The NIER register is logically "AND-ed" with the IPR to form the contents of the Normal Interrupt Pending Register (NIPR). The NIPR register bits are bit-wise "OR-ed" together (and inverted) to form the normal interrupt signal routed to the M•CORE processor. (The normal interrupt signal will only be asserted if the fast interrupt signal is negated.) The NIPR is provided to allow software to quickly determine the highest priority pending normal interrupt. The output of the NIPR also feeds into a 32 to 5 priority encoder to generate the vector number to present to the M•CORE processor if vectored interrupts are required. (If the fast interrupt signal is asserted, then the vector number will be determined by the highest priority fast interrupt.)

If an interrupt is pending at a given priority level and both the corresponding FIER and NIER bits are set, then both the corresponding FIPR and NIPR bits will be set, assuming these bits are not masked.

Fast interrupt requests always have priority over normal interrupt requests, even if the normal interrupt request is at a higher priority level than the highest fast interrupt request.

The two interrupt lines for fast interrupt and normal interrupt are mutually exclusive. If the fast interrupt signal is asserted when the normal interrupt signal was asserted, then the normal interrupt signal will be negated.

The pending interrupt registers in the Interrupt Controller are read only. To clear a pending interrupt, the interrupt must be cleared at the source (using a special clearing sequence defined by each source) and thus this will be reflected in the IPR. All interrupt sources to the Interrupt Controller are to be held steady until recognized and cleared by the interrupt service routine. The Interrupt Controller will not have any edge detect logic. Edge triggered interrupt sources will be handled at the source module.

Interrupt sources at and below a defined interrupt priority level may be masked as determined by the MASK bits in the Interrupt Control Register (ICR). The MFI bit in the ICR is used to determine whether the mask applies only to normal interrupts or to fast interrupts (with all normal interrupts being masked). Interrupt masking may be enabled or disabled using the ME bit.

The current vector number and the states of the signals to the M•CORE processor are reflected in the Interrupt Status Register (ISR).

Interrupt Controller

The vector number and fast/normal interrupt sources are synchronized before being sent to the M•CORE processor. Thus, the Interrupt Controller adds one clock of latency to the interrupt sequence. The fast and normal interrupt raw sources are not synchronized to allow these signals to be used to wake up the M•CORE processor during STOP mode when all system clocks are stopped.

6.1.3 Autovectored and Vectored Interrupt Requests

The autovector enable (AE) bit in the ICR is use to select between vectored interrupt requests to the M•CORE processor or autovectored interrupt requests. By default, AE is set and all interrupt requests are autovectored and the p_avec_b signal is asserted along with the assertion of the appropriate fast or normal interrupt signal. An interrupt handler may read the FIPR or NIPR registers to determine the priority of the source of the interrupt request. If multiple interrupt sources share the same priority level, then it is up to the interrupt service routine to determine the correct source of the interrupt.

If the AE bit is cleared, then each interrupt request will be presented with a vector number and the p_avec_b signal will not be asserted. The low 5 bits of the vector number (p_vec[4:0]) is determined based on the highest pending priority (with active fast interrupts having priority over active normal interrupts). The remaining two bits (p_vec[6:5]) are determined based on whether the interrupt request is a fast interrupt and the setting of the fast vector enable (FVE) bit. If FVE is set, then a fast interrupt request will have a different vector number from a normal interrupt request. See **Table 6-1** to determine the values of p_vec[6:5].

Table 6-1 Fast Interrupt Vector Number

Fast Interrupt	FVE	p_vec[6:5]
No	X	01
X	0	01
Yes	1	10

If FVE is 0, then both normal and fast interrupts requests assigned to priority levels 0 - 31 will mapped to vector numbers 32 - 63 in the vector table. If FVE is 1, then normal interrupt requests assigned to priority levels 0 - 31 will mapped to vector numbers 32 - 63 in the vector table. If FVE is 1, then fast interrupt requests assigned to priority levels 0 - 31 will mapped to vector numbers 64 - 95 in the vector table.

Table 6-2 Vector Table Mapping

0 - 31	Fixed Exceptions (including autovectors)	p_vec[6:5] = 00
32 - 63	Vectored Interrupts 32 - lowest priority 63 - highest priority	p_vec[6:5] = 01
64 - 95	Vectored Interrupts 64 - lowest priority 95 - highest priority	p_vec[6:5] = 10
96 - 127	Vectored Interrupts (Not Used)	p_vec[6:5] = 11

6.1.4 Low Power Mode Operation

The Interrupt Controller is not affected by any low power modes. All logic between the input sources and generating the interrupt and vector source number to the M•CORE processor will be combinational to allow the ability to wake up the M•CORE processor during low power STOP mode when all system clocks are stopped.

6.2 Programming Model

The Interrupt Controller programming model consists of the following registers:

- The Interrupt Control Register (ICR) selects different interrupt controller functions.
- The Interrupt Status Register (ISR) reflects the state of the outputs to the M•CORE.
- The Interrupt Force Registers (IFRH/L) force interrupt source requests.
- The Interrupt Pending Register (IPR) reflects the pending interrupts.
- The Normal Interrupt Enable Register (NIER) enables individual normal interrupts.
- The Normal Interrupt Pending Register (NIPR) reflects the pending normal interrupts.
- The Fast Interrupt Enable Register (FIER) enables individual fast interrupts.
- The Fast Interrupt Pending Register (FIPR) reflects the pending fast interrupts.
- The Priority Level Select Registers (PLSRx) select the priority level for each interrupt source.

Table 6-3 Interrupt Controller Address Map

Offset ¹	31 - 24	23 - 16	15 - 8	7 - 0	Access ²
\$00	ICR		ISR		Supervisor/User
\$04	IFRH				Supervisor/User
\$08	IFRL				Supervisor/User
\$0C	IPR				Supervisor/User
\$10	NIER				Supervisor/User
\$14	NIPR				Supervisor/User
\$18	FIER				Supervisor/User
\$1C	FIPR				Supervisor/User
\$20 - \$3C	Unimplemented ³				-
\$40	PLSR0	PLSR1	PLSR2	PLSR3	Supervisor Only
\$44	PLSR4	PLSR5	PLSR6	PLSR7	Supervisor Only
\$48	PLSR8	PLSR9	PLSR10	PLSR11	Supervisor Only
\$4C	PLSR12	PLSR13	PLSR14	PLSR15	Supervisor Only
\$50	PLSR16	PLSR17	PLSR18	PLSR19	Supervisor Only
\$54	PLSR20	PLSR21	PLSR22	PLSR23	Supervisor Only
\$58	PLSR24	PLSR25	PLSR26	PLSR27	Supervisor Only
\$5C	PLSR28	PLSR29	PLSR30	PLSR31	Supervisor Only
\$60	PLSR32	PLSR33	PLSR34	PLSR35	Supervisor Only
\$64	PLSR36	PLSR37	PLSR38	PLSR39	Supervisor Only
\$68 - \$7C	Unimplemented ³				-

NOTES:

1. Absolute address is equal to the Offset plus the base address. The base address is chip dependent.
2. User mode accesses to supervisor only address locations have no effect and result in a cycle termination transfer error.
3. Accesses to unimplemented address locations have no effect and result in a cycle termination transfer error.

6.2.1 Interrupt Control Register (ICR)

The 16-bit Interrupt Control Register (ICR) selects whether interrupt requests will be autovectored or vectored and if vectored whether fast interrupts will generate a different vector number than normal interrupts. This register also controls the masking functions.

ICR — Interrupt Control Register

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	AE	FVE	ME	MFI	0	0	0	0	0	0	0	MASK				
W																
RESET:	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 6-2 Interrupt Control Register

AE — Autovector Enable

This bit controls whether the interrupt request to the M•CORE processor will be autovectored or vectored.

0 = All interrupt requests will be vectored.

1 = All interrupt requests will be autovectored.

This bit is read/write always and is set by hardware reset.

FVE — Fast Vector Enable

This bit controls whether fast vectored interrupt requests to the M•CORE processor will have separate vector numbers from normal vectored interrupt requests.

0 = Fast vectored interrupts generate the same vector number as normal vectored interrupts.

1 = Fast vectored interrupts generate an unique vector number.

This bit is read/write always and is cleared by hardware reset.

ME — Mask Enable

This bit controls whether the interrupt mask function is enabled.

0 = Interrupt masking is disabled.

1 = Interrupt masking is enabled.

This bit is read/write always and is cleared by hardware reset.

MFI — Mask Fast Interrupts

This bit determines whether to mask fast interrupts.

0 = Fast interrupt requests are not masked regardless of the MASK value. The MASK only applies to normal interrupts.

1 = Fast interrupt requests are masked based on the MASK value. All normal interrupt requests are masked.

This bit is read/write always and is cleared by hardware reset.

Interrupt Controller

MASK[4:0] — Interrupt Mask Value

These bits determine which interrupt priority levels are masked. All pending interrupts programmed at priority levels at and below the current MASK value are masked if the ME bit is set. To mask all normal interrupts, without masking any fast interrupts, set the MASK value to 31 with the MFI bit cleared. See **Table 6-4** for details.

These bits are read/write always and are cleared by hardware reset.

Table 6-4 MASK Encoding

MASK[4:0]		Masked Priority Levels
Decimal	Binary	
0	00000	0
1	00001	1 - 0
2	00010	2 - 0
3	00011	3 - 0
...	...	...
31	11111	31 - 0

6.2.2 Interrupt Status Register (ISR)

The 16-bit read-only Interrupt Status Register (ISR) reflects the state of the Interrupt Controller outputs to the M•CORE processor. Writes to this register have no effect and are terminated normally.

ISR — Interrupt Status Register

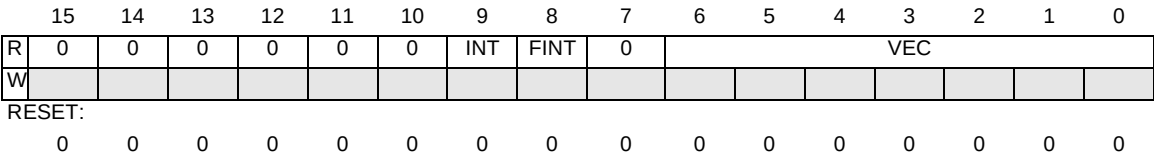


Figure 6-3 Interrupt Status Register

INT — Normal Interrupt Request

This bit reflects whether the normal interrupt request signal to the M•CORE processor is asserted or negated.

- 0 = Normal interrupt request negated.
- 1 = Normal interrupt request asserted.

This bit is read only and will reflect a 0 after reset. Writes have no effect.

FINT — Fast Interrupt Request

This bit reflects whether the fast interrupt request signal to the M•CORE processor is asserted or negated.

0 = Fast interrupt request negated.

1 = Fast interrupt request asserted.

This bit is read only and will reflect a 0 after reset. Writes have no effect.

VEC[6:0] — Interrupt Vector Number

These bit reflect the state of the 7 bit interrupt vector number.

These bits are read only and will reflect all 0s after reset. Writes have no effect.

6.2.3 Interrupt Force Registers (IFRH/L)

The two 32-bit read/write Interrupt Force Registers (IFRH/L) individually force interrupt source requests.

IFRH — Interrupt Force Register High

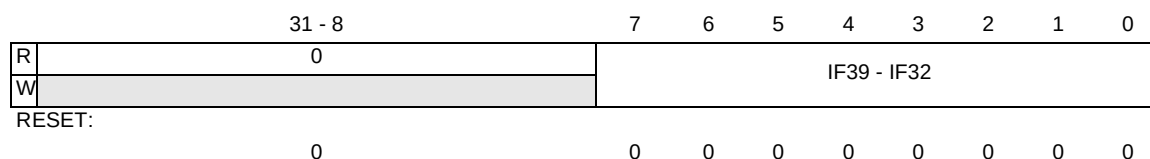


Figure 6-4 Interrupt Force Register High

IFRL — Interrupt Force Register Low

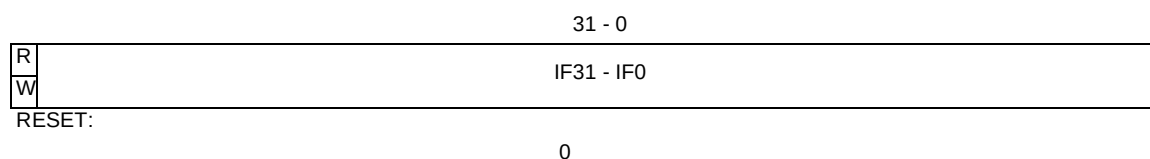


Figure 6-5 Interrupt Force Register Low

IFx — Interrupt Force x

These bits force an interrupt request at the corresponding source number. The Interrupt Force Register allows for software generation of interrupts for each of the possible interrupt sources for functional or debug purposes. Writing a 0 will negate the interrupt request.

0 = Interrupt source not forced.

1 = Force interrupt request.

These bits are cleared at reset.

Interrupt Controller

6.2.4 Interrupt Pending Register (IPR)

This 32-bit Interrupt Pending Register (IPR) reflects any currently pending interrupts which are programmed to each priority level. Writes to this register have no effect and are terminated normally.

IPR — Interrupt Pending Register

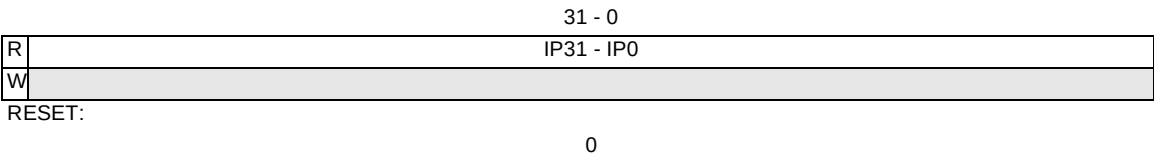


Figure 6-6 Interrupt Pending Register

IPx — Interrupt Pending

These bits reflect the state of the interrupt sources programmed at the corresponding priority level x.

- 0 = All interrupt sources programmed at corresponding priority level x are negated.
- 1 = At least one interrupt source programmed at corresponding priority level x is asserted.

These bits are read only and will reflect all 0s after reset. Writes have no effect.

6.2.5 Normal Interrupt Enable Register (NIER)

This read/write 32-bit Normal Interrupt Enable Register (NIER) individually enables any current pending interrupts which are programmed to each priority level as a normal interrupt source. Enabling an interrupt source which has an asserted request will cause that interrupt to become pending, and a request to the M•CORE processor will be asserted if not already outstanding.

NIER — Normal Interrupt Enable Register

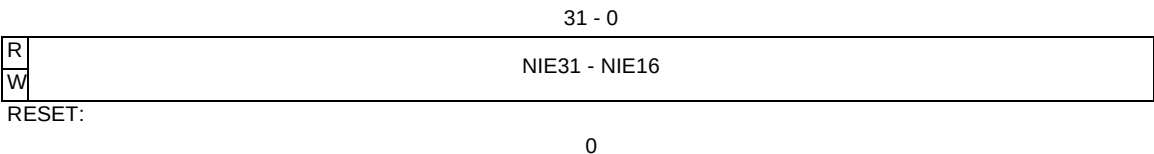


Figure 6-7 Normal Interrupt Enable Register High

NIE_x — Normal Interrupt Enable

These bits enable the interrupt sources programmed at the corresponding priority level *x* as normal interrupts.

0 = Normal interrupt disabled.

1 = Normal interrupt enabled.

These bits are cleared at reset.

6.2.6 Normal Interrupt Pending Register (NIPR)

This 32-bit Normal Interrupt Pending Register (NIPR) reflects any currently pending normal interrupts which are programmed to each priority level. Writes to this register have no effect and are terminated normally.

NIPR — Normal Interrupt Pending Register

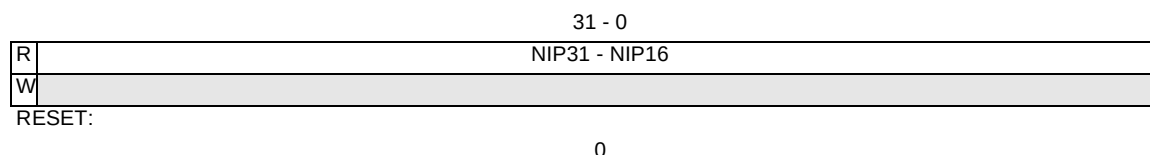


Figure 6-8 Normal Interrupt Pending Register

NIP_x — Normal Interrupt Pending

These bits reflect the state of the interrupt sources programmed at the corresponding priority level *x* which have been enabled as normal interrupts.

0 = All normal interrupt sources programmed at corresponding priority level *x* are negated.

1 = At least one normal interrupt source programmed at corresponding priority level *x* is asserted.

These bits are read only and will reflect all 0s after reset. Writes have no effect.

6.2.7 Fast Interrupt Enable Register (FIER)

This read/write 32-bit Fast Interrupt Enable Register (FIER) individually enables any current pending interrupts which are programmed at each priority level as a fast interrupt source. Enabling an interrupt source which has an asserted request will cause that interrupt to become pending, and a request to the M•CORE processor will be asserted if not already outstanding.

Interrupt Controller

FIER — Fast Interrupt Enable Register

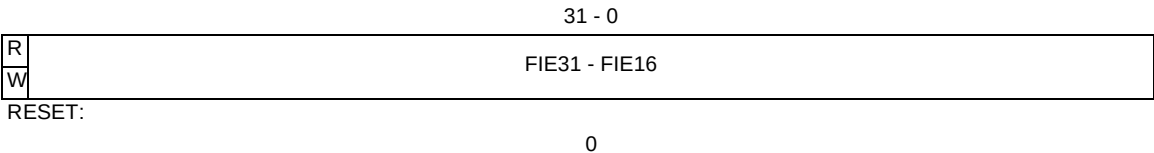


Figure 6-9 Fast Interrupt Enable Register

FIE_x — Fast Interrupt Enable

These bits enable the interrupt sources programmed at the corresponding priority level *x* as fast interrupts.

- 0 = Fast interrupt disabled.
- 1 = Fast interrupt enabled.

These bits are cleared at reset.

6.2.8 Fast Interrupt Pending Register (FIPR)

This 32-bit fast Interrupt Pending Register (FIPR) reflects any currently pending fast interrupts which are programmed to each priority level. Writes to this register have no effect and are terminated normally.

FIPR — Fast Interrupt Pending Register

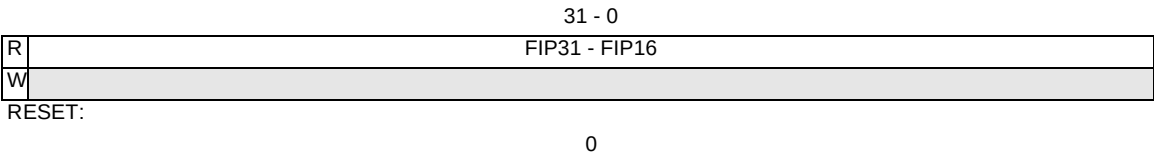


Figure 6-10 Fast Interrupt Pending Register

FIP_x — Fast Interrupt Pending

These bits reflect the state of the interrupt sources programmed at the corresponding priority level *x* which have been enabled as fast interrupts.

- 0 = All fast interrupt sources programmed at corresponding priority level *x* are negated.
- 1 = At least one fast interrupt source programmed at corresponding priority level *x* is asserted.

These bits are read only and will reflect all 0s after reset. Writes have no effect.

6.2.9 Priority Level Select Registers (PLSR39 - PLSR0)

The 40 read/write 8-bit Priority Level Select Registers (PLSR39-PLSR0) individually select the priority levels for each interrupt source x.

PLSRx — Priority Level Select Register x

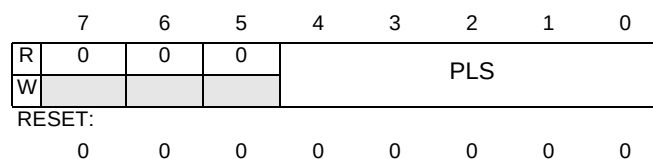


Figure 6-11 Priority Level Select Register x

PLS[4:0] — Priority Level Select

These bits select the priority levels for each interrupt source x.

These bits are cleared at reset.

Table 6-5 Priority Select Encoding

PLS[4:0]	Priority Level	Vector Number ¹ (p_vec[4:0])
00000 (lowest priority)	0	00000
00001 - 11110	1 - 30	00001 - 11110
11111 (highest priority)	31	11111

NOTES:

1. The vector number p_vec[4:0] will be 0b00000 when no interrupts are pending.

6.3 Interrupt Source Assignments

The assignment of each individual interrupt source is shown in **Table 6-6**.

Table 6-6 Interrupt Source Assignment

Source Number	Module	Name	Source Description	Flag Clearing Mechanism
0	TOD	TODAL	TOD Alarm Flag	Write TODFAR
1	DSRC	DSRC1	DSRC	Write 1 to Clear bit
2		DSRC2	DSRC	Write 1 to Clear bit
3		DSRC3	DSRC	Write 1 to Clear bit
4	SPI	MODF	Mode fault	Read/write SPIDR after SPIF read 1.
5		SPIF	Transfer Complete	Write SPICR1 after MODF read 1.
6	SCI1	TDRE	Transmitter: Transmit Data Register Empty	Write SCIDRL after TDRE read 1.
7		TC	Transmitter: Transmit Complete	Write SCIDRL after TC read 1.
8		RDRF	Receiver: Receive Data Register Full	Read SCIDRL after RDRF read 1.
9		OR	Receiver: Overrun	Read SCIDRL after OR read 1.
10		IDLE	Receiver: Idle Line	Read SCIDRL after IDLE read 1.
11	SCI2	TDRE	Transmitter: Transmit Data Register Empty	Write SCIDRL after TDRE read 1.
12		TC	Transmitter: Transmit Complete	Write SCIDRL after TC read 1.
13		RDRF	Receiver: Receive Data Register Full	Read SCIDRL after RDRF read 1.
14		OR	Receiver: Overrun	Read SCIDRL after OR read 1.
15		IDLE	Receiver: Idle Line	Read SCIDRL after IDLE read 1.
16	TIM	C0F	Timer Channel 0	Write C0F 1 or IC/OC access w/TFFCA.
17		C1F	Timer Channel 1	Write C1F 1 or IC/OC access w/TFFCA.
18		C2F	Timer Channel 2	Write C2F 1 or IC/OC access w/TFFCA.
19		C3F	Timer Channel 3	Write C3F 1 or IC/OC access w/TFFCA.
20		TOF	Timer Overflow	Write TOF 1 or timer access w/TFFCA.
21		PAIF	Pulse Accumulator Input	Write PAIF 1 or PAC access w/TFFCA.
22		PAOVF	Pulse Accumulator Overflow	Write PAOVF 1 or PAC access w/TFFCA.
23		N/A	Reserved	
24		N/A	Reserved	
25		N/A	Reserved	
26		N/A	Reserved	
27		N/A	Reserved	
28		N/A	Reserved	
29		N/A	Reserved	
30	PIT1	PIF	PIT Interrupt Flag	Write 1 to PIF or Write PMR
31	PIT2	PIF	PIT Interrupt Flag	Write 1 to PIF or Write PMR

Table 6-6 Interrupt Source Assignment (Continued)

Source Number	Module	Name	Source Description	Flag Clearing Mechanism
32	Edge Port	EPF0	Edge Port Flag 0	Write 1 to EPF0
33		EPF1	Edge Port Flag 1	Write 1 to EPF1
34		EPF2	Edge Port Flag 2	Write 1 to EPF2
35		EPF3	Edge Port Flag 3	Write 1 to EPF3
36		EPF4	Edge Port Flag 4 ¹	Write 1 to EPF4
37		EPF5	Edge Port Flag 5 ¹	Write 1 to EPF5
38		EPF6	Edge Port Flag 6 ¹	Write 1 to EPF6
39		EPF7	Edge Port Flag 7 ¹	Write 1 to EPF7

NOTES:

1. External pin not available

6.4 Interrupt Configuration

After reset, all interrupts are disabled by default. To properly configure the system to handle interrupt requests, configuration must be performed at three levels: M•CORE processor, Interrupt Controller, local interrupt sources. The M•CORE should be configured first, then the Interrupt Controller, then finally the individual modules.

6.4.1 M•CORE Processor Setup

To allow the M•CORE processor to recognize fast interrupts, the FE bit in the M•CORE processor's PLSR register must be set. Likewise, set the IE bit for normal interrupts. Both FE and IE are cleared at reset. Also, set the IC bit if it is desired to cause long latency, multi-cycle instructions to be interrupted before completion.

The M•CORE processor's VBR register is used to define the base address of the exception vector table. If autovectors are to be used, then the INT and FINT autovectors (vector numbers 10 and 11, respectively) need to be set up. If vectored interrupts are to be used, then the vectored interrupts (vector numbers 32-63 and/or 64-95) need to be set up. Whether 32 or 64 vectors are required depends on whether the fast interrupts will share vectors with the normal interrupt sources based on the Interrupt Controller FVE bit.

For each vector number, an interrupt service routine needs to be created to service the interrupt, clear the local interrupt flag, and return from the interrupt routine.

6.4.2 Interrupt Controller

Each interrupt source to the Interrupt Controller is by default assigned a priority level of 0 and disabled. Each interrupt source can be programmed to one of 32 priority levels and enabled as either a fast or normal interrupt source. Also the FVE and AE bits can be programmed to select autovectored/vectored interrupts and also determine if the fast interrupt vector number is to be separate from the normal interrupt vector.

6.4.3 Interrupt Source Setup

Each module that is capable of generating an interrupt request needs to be configured to do so. Each interrupt source has an enable/disable locally in the module that generates it. To allow the interrupt source to be asserted, the local interrupt enable needs to be set.

Once an interrupt is asserted, the module will keep the source asserted until the interrupt service routine performs a special sequence to clear the interrupt flag. Clearing the flag will negate the interrupt request.

Section 7 M210 CPU and OnCE Debug

This section describes the M•CORE CPU and the OnCE debugging system.

7.1 M•CORE Overview

The M•CORE architecture is one of the most compact full 32-bit implementations available. The pipelined RISC execution unit uses 16-bit instructions to achieve maximum speed and code efficiency, while conserving on-chip memory resources. The instruction set is designed to support high-level language implementation. A non-intrusive, JTAG-compliant resident debugging system supports product development and in-situ testing. The M•CORE technology library also encompasses a full complement of on-chip peripheral modules designed specifically for embedded control applications.

Total system power consumption is determined by components other than the processor core. In particular, memory power consumption (both on-chip and external) is a dominant factor in total power consumption of the core plus memory subsystem. With this factor in mind, the M•CORE instruction set architecture trades absolute performance capability for reduced total energy consumption. This is accomplished while maintaining an acceptably high level of performance at a given clock frequency.

The streamlined execution engine uses many of the same performance enhancements and implementation techniques incorporated in desktop RISC processors. A strictly-defined load/store architecture minimizes control complexity. Use of a fixed, 16-bit instruction encoding significantly lowers the memory bandwidth needed to sustain a high rate of instruction execution, and careful selection of the instruction set allows code density and overall memory footprint (efficiency) of the M•CORE architecture to surpass those of CISC architectures.

These factors reduce system energy consumption significantly, and the fully-static M•CORE design uses other techniques to reduce it even more. The core uses dynamic clock management to automatically power-down internal functions that are not in use on a clock-by-clock basis. It also incorporates three power-conservation operating modes, which are invoked via dedicated instructions.

7.2 Features

The main features of the M•CORE are as follows:

- 32-bit load/store RISC architecture

- Fixed 16-bit instruction length
- 16 entry, 32-bit general-purpose register file
- Efficient four-stage execution pipeline, hidden from application software
- Single-cycle execution for most instructions, two-cycle branches and memory accesses
- Support for byte/halfword/word memory access
- Fast interrupt support, with 16-entry user-controlled alternate register file
- Vectored and autovectored interrupt support
- On-chip emulation support
- Full static design for minimal power consumption
- Supported by a comprehensive suite of third-party development tools

7.3 Microarchitecture Summary

Figure 7-1 is a block diagram of the M•CORE processor.

The processor utilizes a four-stage pipeline for instruction execution. The instruction fetch, instruction decode/register file read, execute, and register file writeback stages operate in an overlapped fashion, allowing single clock instruction execution for most instructions.

The execution unit consists of a 32-bit arithmetic/logic unit, a 32-bit barrel shifter, a find-first-one unit, result feed-forward hardware, and miscellaneous support hardware for multiplication, division, and multiple-register loads and stores.

Arithmetic and logical operations are executed in a single cycle. Multiplication is implemented with a 2-bit per clock, overlapped-scan, modified Booth algorithm with early-out capability, to reduce execution time for operations with small multipliers. Divide is implemented with a 1-bit per clock early-in algorithm. The find-first-one unit operates in a single clock cycle.

The program counter unit incorporates a dedicated branch address adder to minimize delays during change of flow operations. Branch target addresses are calculated in parallel with branch instruction decode. Taken branches and jumps require only two clocks; branches which are not taken execute in a single clock.

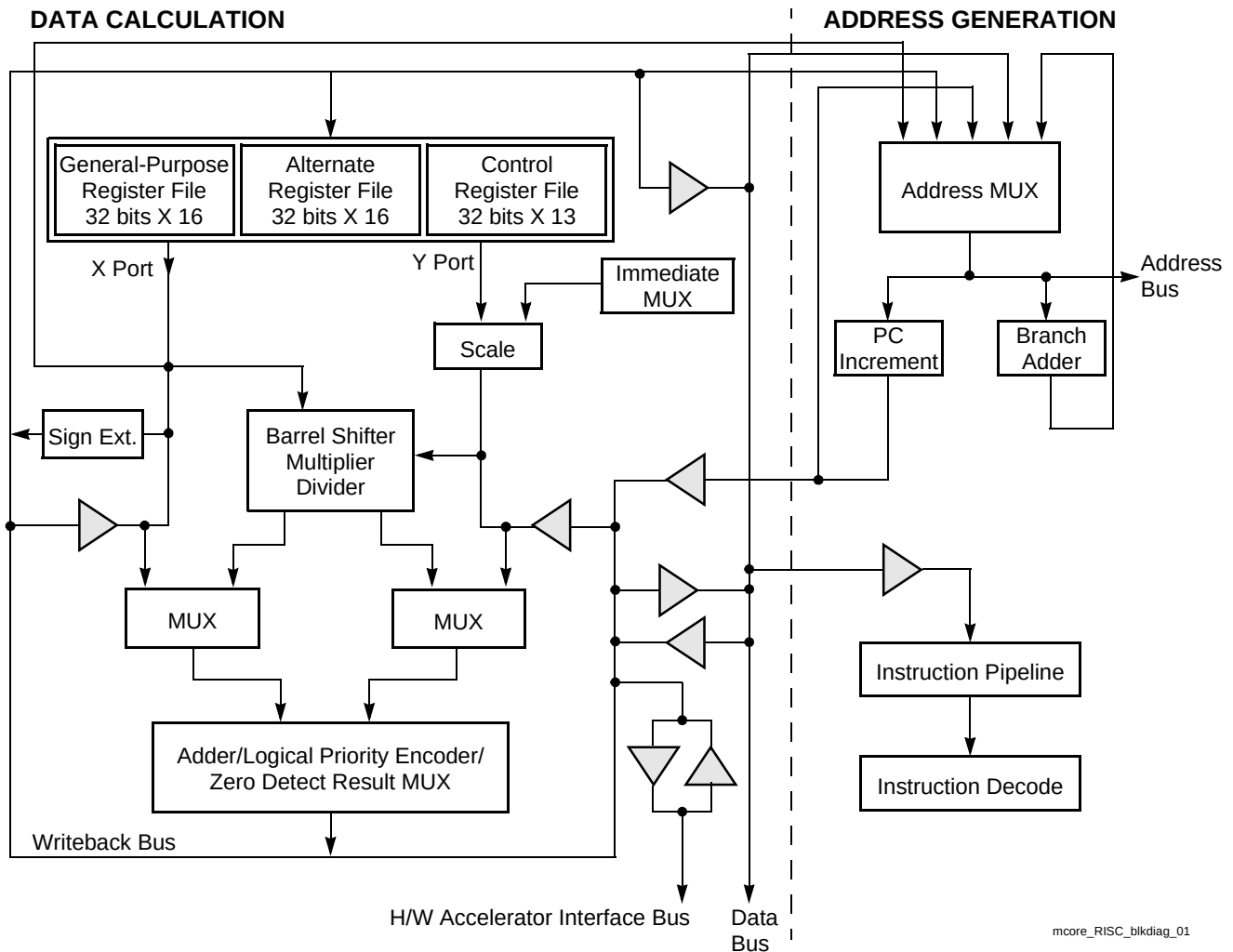


Figure 7-1 M-CORE Processor Block Diagram

Memory load and store operations are provided for 8-bit (byte), 16-bit (halfword), and 32-bit (word) data, with automatic zero extension of byte and halfword load data. These instructions can execute in as few as two clock cycles. Load and store multiple register instructions allow low overhead context save and restore operations. These instructions can execute in (N+1) clock cycles, where N is the number of registers to transfer.

A condition/code carry (C) bit is provided for condition testing and for use in implementing arithmetic and logical operations with operands/results greater than 32 bits. The C bit is typically set by explicit test/comparison operations, not as a side-effect of normal instruction operation. Exceptions to this rule occur for specialized operations where it is desirable to combine condition setting with actual computation.

The processor supports both normal and fast interrupt requests. Fast interrupts take precedence over normal interrupts. Both types have dedicated exception shadow registers. For service requests of either kind, a 7-bit interrupt vector can be supplied when the request is made, or automatic vector generation can be requested by means of an autovector control signal.

7.4 Programming Model

Figure 7-2 shows the M•CORE programming model. The model is defined differently for supervisor and user privilege modes. By convention, in both modes R15 serves as the link register for subroutine calls. R0 is typically used as stack pointer.

The user programming model consists of 16 general-purpose 32-bit registers (R[15:0]), the 32-bit PC and the C bit. The C bit is implemented as bit 0 of the PSR, and is the only portion of the PSR accessible in the user model.

The supervisor programming model consists of the user model plus 16 additional 32-bit general-purpose registers (R[15:0]', or the alternate file), the entire PSR, and a set of status/control registers (CR[12:0]). Setting the S bit in the PSR enables supervisor mode operation.

The alternate file allows very low overhead context switching for real-time event handling. While the alternate file is enabled, general-purpose operands are accessed from it.

The vector base register (VBR) determines the base address of the exception vector table. Exception shadow registers EPC and EPSR are used to save the state of the PSR and the program counter when an exception occurs. Shadow registers FCR and FPSR are used to minimize context switching overhead for fast interrupts.

Scratch registers (SS[4:0]) are used to handle exception events.

The global control (GCR) and status (GSR) registers can be used for a variety of system monitoring tasks.

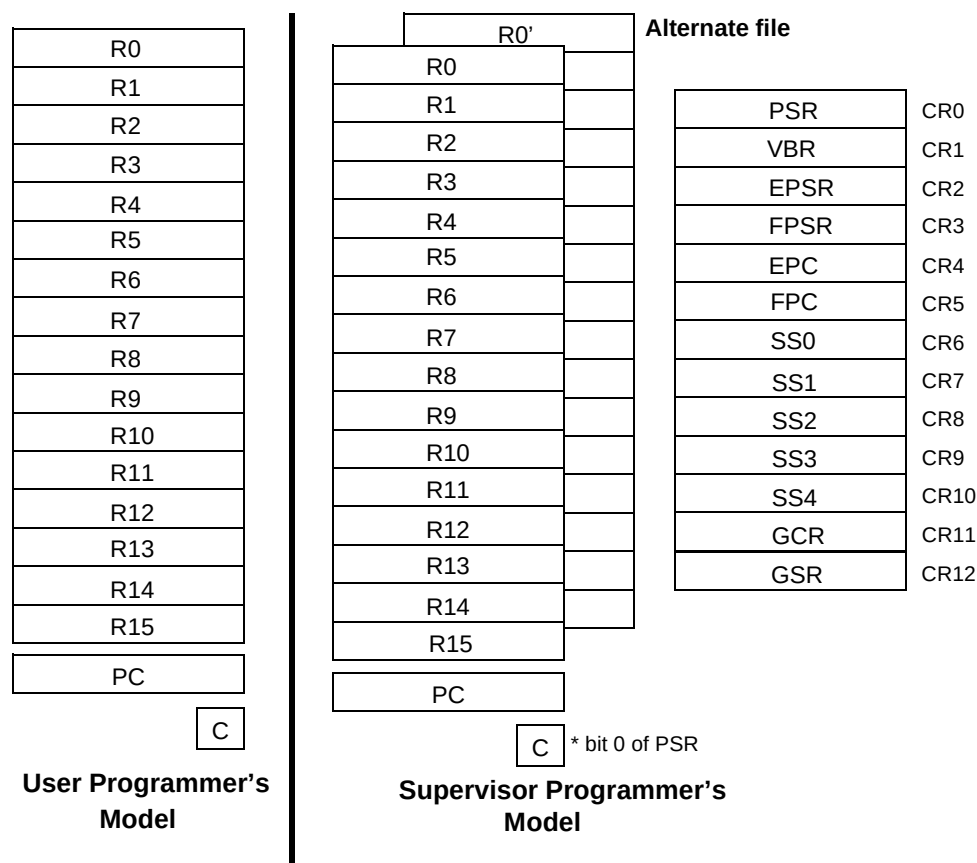


Figure 7-2 Programming Model

The supervisor programming model includes the PSR, which contains operation control and status information. In addition, a set of exception shadow registers is provided to save the state of the PSR and the program counter at the time an exception occurs. A separate set of shadow registers is provided for fast interrupt support to minimize context saving overhead.

Five scratch registers are provided for supervisor software use in handling exception events. A single register is provided to alter the base address of the exception vector table. Two registers are provided for global control and status.

7.5 Data Format Summary

The operand data formats supported by the integer unit are standard two's-complement data formats. The operand size for each instruction is either explicitly encoded in the instruction (load/store instructions) or implicitly defined by the instruction operation (index operations, byte

Memory is viewed from a big-endian byte ordering perspective. The most significant byte (byte 0) of word 0 is located at address 0. Bits are numbered within a word starting with bit 31 as the most significant bit.



M•CORE accesses all memory operands through load and store instructions, transferring data between the general-purpose registers and memory. Register-plus-four-bit scaled displacement addressing mode is used for the load and store instructions to address byte, halfword, or word (32-bit) data.

Load and store multiple instructions allow a subset of the 16 GPRs to be transferred to or from a base address pointed to by register R0 (the default stack pointer by convention).

Load and store register quadrant instructions use register indirect addressing to transfer a register quadrant to or from memory.

7.7 Instruction Set Overview

The instruction set is tailored to support high-level languages and is optimized for those instructions most commonly executed. A standard set of arithmetic and logical instructions is provided, as well as instruction support for bit operations, byte extraction, data movement, control flow modification, and a small set of conditionally executed instructions which can be useful in eliminating short conditional branches.

Table 7-1 is an alphabetized listing of the M•CORE instruction set. Refer to the *M•CORE Reference Manual* (MCORERM/AD) for more details on instruction operation.

Table 7-1 M•CORE Instruction Set

Mnemonic	Description
ABS	Absolute Value
ADDC	Add with C Bit
ADDI	Add Immediate
ADDU	Add Unsigned
AND	Logical AND
ANDI	Logical AND Immediate
ANDN	AND NOT
ASR	Arithmetic Shift Right
ASRC	Arithmetic Shift Right, Update C Bit
BCLRI	Bit Clear Immediate
BF	Branch on Condition False
BGENI	Bit Generate Immediate
BGENR	Bit Generate Register
BKPT	Breakpoint
BMASKI	Bit Mask Immediate
BR	Branch
BREV	Bit Reverse
BSETI	Bit Set Immediate
BSR	Branch to Subroutine
BT	Branch on Condition True
BTSTI	Bit Test Immediate
CLRF	Clear Register on Condition False
CLRT	Clear Register on Condition True
CMPHS	Compare Higher or Same
CMPLT	Compare Less Than
CMPLTI	Compare Less Than Immediate
CMPNE	Compare Not Equal
CMPNEI	Compare Not Equal Immediate
DECF	Decrement on Condition False
DECGT	Decrement Register and Set Condition if Result Greater Than Zero
DECLT	Decrement Register and Set Condition if Result Less Than Zero
DECNE	Decrement Register and Set Condition if Result Not Equal to Zero
DECT	Decrement on Condition True
DIVS	Divide Signed Integer
DIVU	Divide Unsigned Integer
DOZE	Doze
FF1	Find First One
INCF	Increment on Condition False
INCT	Increment on Condition True
IXH	Index Halfword
IXW	Index Word
JMP	Jump
JMPI	Jump Indirect
JSR	Jump to Subroutine
JSRI	Jump to Subroutine Indirect

Table 7-1 M•CORE Instruction Set (Continued)

Mnemonic	Description
LD.[BHW] LDM LDQ LOOP LRW LSL, LSR LSLC, LSRC LSLI, LSRI	Load Load Multiple Registers Load Register Quadrant Decrement with C-Bit Update and Branch if Condition True Load Relative Word Logical Shift Left and Right Logical Shift Left and Right, Update C Bit Logical Shift Left and Right by Immediate
MFCR MOV MOVI MOVF MOVT MTCR MULT MVC MVCV	Move from Control Register Move Move Immediate Move on Condition False Move on Condition True Move to Control Register Multiply Move C Bit to Register Move Inverted C Bit to Register
NOT	Logical Complement
OR	Logical Inclusive-OR
ROTL RSUB RSUBI RTE RFI	Rotate Left by Immediate Reverse Subtract Reverse Subtract Immediate Return from Exception Return from Interrupt
SEXTB SEXTH ST.[BHW] STM STQ STOP SUBC SUBU SUBI SYNC	Sign-Extend Byte Sign-Extend Halfword Store Store Multiple Registers Store Register Quadrant Stop Subtract with C Bit Subtract Subtract Immediate Synchronize
TRAP TST TSTNBZ	Trap Test Operands Test for No Byte Equal Zero
WAIT	Wait
XOR XSR XTRB0 XTRB1 XTRB2 XTRB3	Exclusive OR Extended Shift Right Extract Byte 0 Extract Byte 1 Extract Byte 2 Extract Byte 3
ZEXTB ZEXTH	Zero-Extend Byte Zero-Extend Halfword

7.8 OnCE Debug Module

The on-chip emulation (OnCE™) circuitry provides a simple, inexpensive debugging interface that allows external access to the processor's internal registers and to memory/peripherals. OnCE capabilities are controlled through a serial interface, mapped onto a JTAG test access port (TAP) protocol. **Figure 7-5** shows the components of the OnCE circuitry.

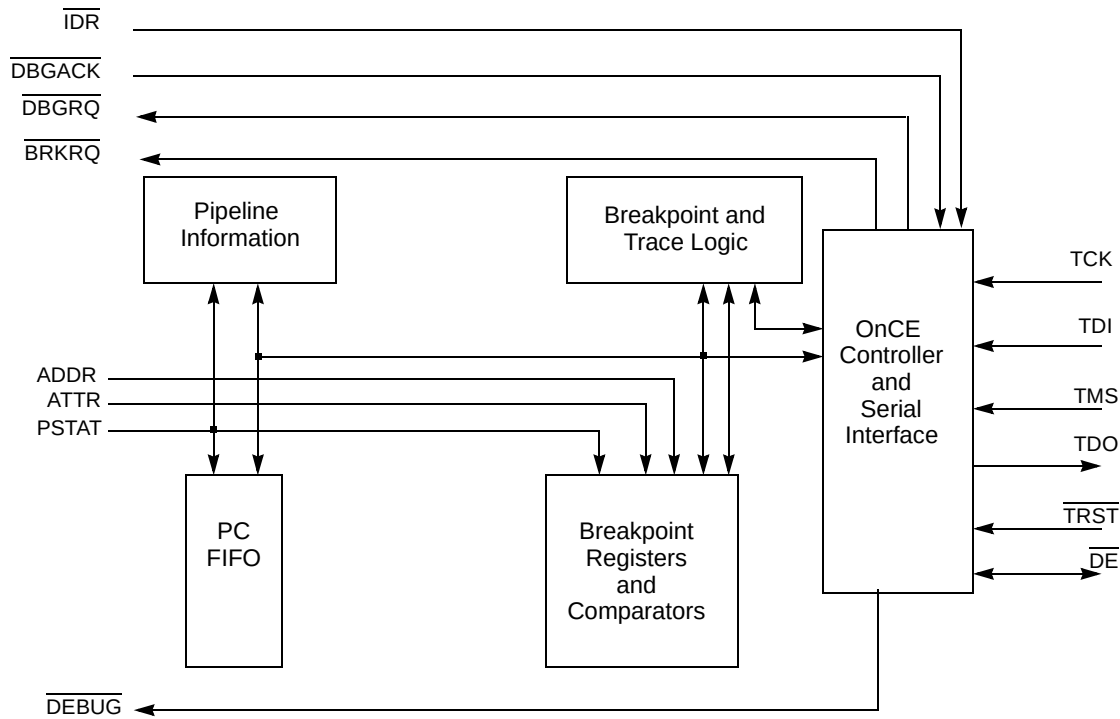


Figure 7-5 OnCE Block Diagram

The interface to the OnCE controller and its resources is based on the TAP defined for JTAG in the IEEE-1149.1a-1993 standard.

7.8.1 Operation

An instruction is scanned into the OnCE module through the serial interface and then decoded. Data may then be scanned in and used to update a register or resource on a write to the resource, or data associated with a resource may be scanned out for a read of the resource.

For accesses to the CPU internal state, the OnCE controller requests the CPU to enter debug mode via the CPU DBGRQ input. Once CPU debug mode has been entered, as indicated by the OnCE status register, the processor state may be accessed through the CPU scan register.

The OnCE controller is implemented as a 16-state FSM, with a one-to-one correspondence to the states defined for the JTAG TAP controller.

CPU registers and the contents of memory locations are accessed by scanning instructions and data into and out of the CPU scan chain. Required data is accessed by executing the scanned instructions. Memory locations may be read by scanning in a load instruction to the CPU that references the desired memory location, executing the load instruction, and then scanning out the result of the load. Other resources are accessed in a similar manner.

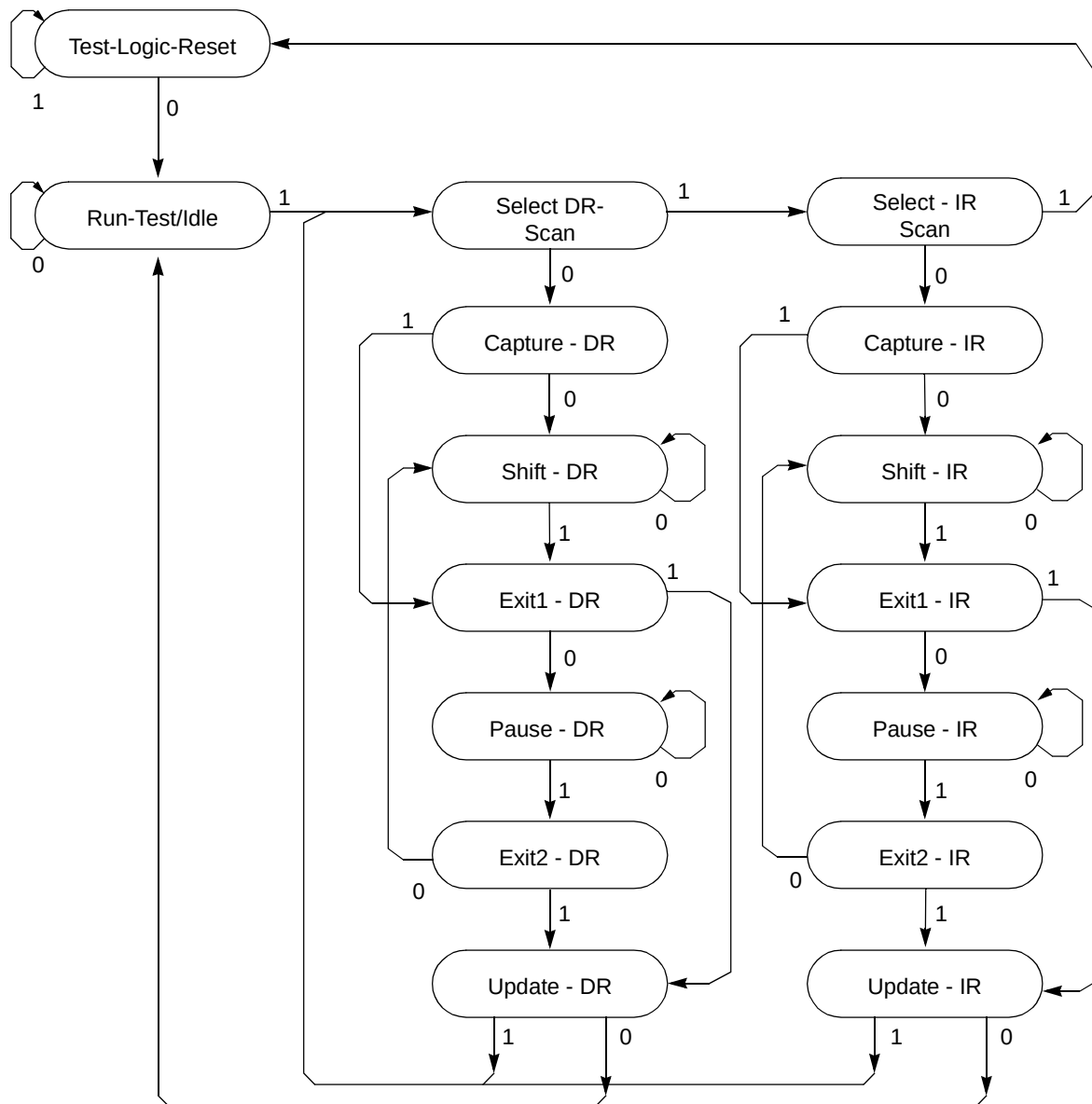


Figure 7-6 OnCE Controller

Resources contained in the OnCE module that do not require the CPU to be halted for access may be controlled while the CPU is executing and do not interfere with normal processor execution. Accesses to certain resources, such as the PC FIFO and the count registers, while not part of the CPU, may require the CPU to be stopped to allow access to avoid synchronization hazards. If it is known that the CPU clock is enabled and running no slower than the TCK input, there is sufficient synchronization performed to allow reads but not writes of these specific resources. Debug firmware may ensure that it is safe to access these resources by reading the OSR to determine the state of the CPU prior to access. All other cases require the CPU to be in the debug state for deterministic operation.

7.8.2 OnCE Pins

The following paragraphs describe the pins associated with the OnCE controller and serial interface component.

The OnCE pin interface is used to transfer OnCE instructions and data to the OnCE control block. Depending on the particular resource being accessed, the CPU may need to be placed in debug mode. For resources outside of the CPU block and contained in the OnCE block, the processor is not disturbed and may continue execution. If a processor resource is required, the OnCE controller may assert a debug request (DBGRQ) to the CPU. This causes the CPU to finish the instruction being executed, save the instruction pipeline information, enter debug mode, and wait for further commands. Asserting DBGRQ causes the device to exit stop, doze, or wait mode.

7.8.2.1 Debug Serial Input (TDI)

Data and commands are provided to the OnCE controller through the TDI pin. Data is latched on the rising edge of the TCK serial clock. Data is shifted into the OnCE serial port least significant bit (LSB) first.

7.8.2.2 Debug Serial Clock (TCK)

The TCK pin supplies the serial clock to the OnCE control block. The serial clock provides pulses required to shift data and commands into and out of the OnCE serial port. (Data is clocked into the OnCE on the rising edge and is clocked out of the OnCE serial port on the falling edge.) The debug serial clock frequency must be no greater than 50% of the processor clock frequency.

7.8.2.3 Debug Serial Output (TDO)

Serial data is read from the OnCE block through the TDO pin. Data is always shifted out the OnCE serial port LSB first. Data is clocked out of the OnCE serial port on the falling edge of TCK. TDO is three-stateable and is actively driven in the shift-IR and shift-DR controller states. TDO changes on the falling edge of TCK.

7.8.2.4 Debug Mode Select (TMS)

The debug mode select input is used to cycle through states in the OnCE debug controller. Toggling the TMS pin while clocking with TCK controls the transitions through the TAP state controller.

7.8.2.5 Test Reset ($\overline{TRST}$)

The test reset input is used to reset the OnCE controller externally by placing the OnCE control logic in a test logic reset state. OnCE operation is disabled in the reset controller and reserved states.

7.8.2.6 Debug Event ($\overline{DE}$)

The debug event ($\overline{DE}$) pin is a bidirectional open drain pin. As an input, $\overline{DE}$ provides a fast means of entering debug mode from an external command controller. As an output, this pin provides a fast means of acknowledging debug mode entry to an external command controller.

The assertion of this pin by a command controller causes the CPU to finish the current instruction being executed, save the instruction pipeline information, enter debug mode, and wait for commands to be entered from the TDI line. If $\overline{DE}$ was used to enter debug mode, then $\overline{DE}$ must be negated after the OnCE responds with an acknowledgment and before sending the first OnCE command.

The assertion of this pin by the CPU acknowledges that it has entered debug mode and is waiting for commands to be entered from the TDI line.

7.8.3 OnCE Controller and Serial Interface

Figure 7-7 is a block diagram of the OnCE controller and serial interface.

M210 CPU and OnCE Debug

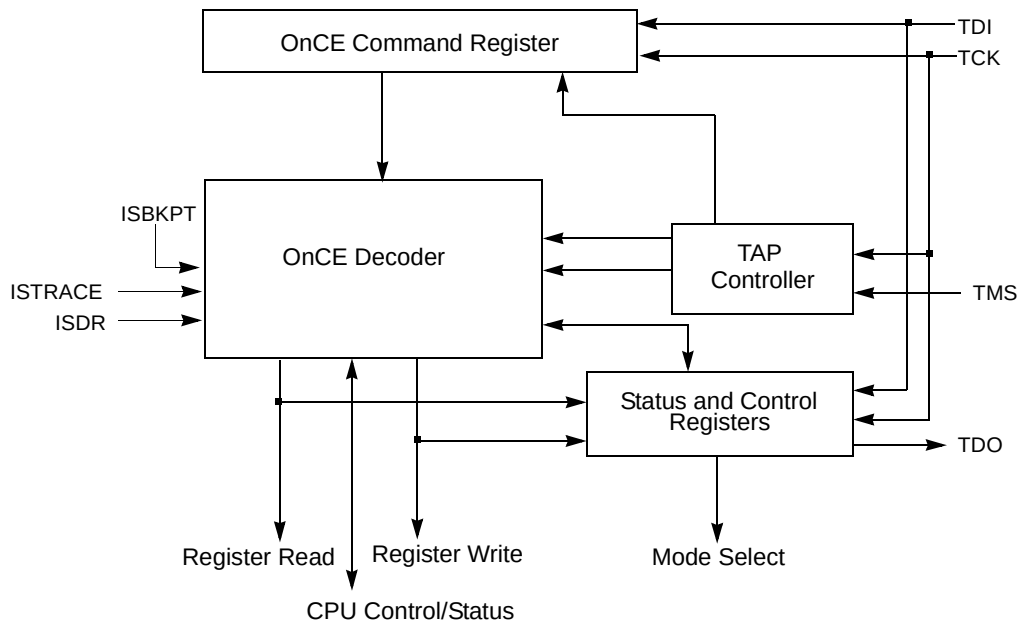


Figure 7-7 OnCE Controller and Serial Interface

The OnCE controller and serial interface contain the following blocks: OnCE TAP controller, the OnCE command register, OnCE decoder, and the OnCE control and status registers.

The OnCE command register acts as the IR for the TAP controller. All other OnCE resources are treated as data registers (DR) by the TAP controller. The command register is loaded by serially shifting in commands during the TAP controller shift-IR state, and is loaded during the update-IR state. The command register selects a OnCE resource to be accessed as a data register (DR) during the TAP controller capture-DR, shift-DR and update-DR states.

7.8.4 OnCE Interface Signals

The following paragraphs describe the OnCE interface signals to other internal blocks associated with the OnCE controller. These signals are not available externally, and descriptions are provided to improve understanding of OnCE operation.

7.8.4.1 Internal Debug Request Input ($\overline{IDR}$)

The internal debug request input is a hardware signal which is used in some implementations to force an immediate debug request to the CPU. If present and enabled, it functions in an identical manner to the control function provided by the DR control bit in the OnCE control register (OCR). This input is maskable by a control bit in OCR.

7.8.4.2 CPU Debug Request ($\overline{\text{DBGRQ}}$)

The $\overline{\text{DBGRQ}}$ signal is asserted by the OnCE control logic to request the CPU to enter the debug state. It may be asserted for a number of different conditions. Assertion of this signal causes the CPU to finish the current instruction being executed, save the instruction pipeline information, enter debug mode, and wait for further commands. Asserting $\overline{\text{DBGRQ}}$ causes the device to exit stop, doze, or wait mode.

7.8.4.3 CPU Debug Acknowledge ($\overline{\text{DBGACK}}$)

The CPU asserts the $\overline{\text{DBGACK}}$ signal upon entering the debug state. This signal is part of the handshake mechanism between the OnCE control logic and the CPU.

7.8.4.4 CPU Breakpoint Request ($\overline{\text{BRKRQ}}$)

The $\overline{\text{BRKRQ}}$ signal is asserted by the OnCE control logic to signal that a breakpoint condition has occurred for the current CPU bus access.

7.8.4.5 CPU Address, Attributes (ADDR, ATTR)

The CPU address and attribute information may be used in the memory breakpoint logic to qualify memory breakpoints with access address and cycle type information.

7.8.4.6 CPU Status (PSTAT)

The trace logic uses the CPU PSTAT signals to qualify trace count decrements with specific CPU activity.

7.8.4.7 OnCE Debug Output ($\overline{\text{DEBUG}}$)

The OnCE debug output ($\overline{\text{DEBUG}}$) is used to indicate to on-chip resources that a debug session is in progress. Peripherals and other units may use this signal to modify normal operation for the duration of a debug session. This may involve the CPU executing a sequence of instructions solely for the purpose of visibility/system control. These instructions are not part of the normal instruction stream the CPU would have executed had it not been placed in debug mode.

This signal is asserted the first time the CPU enters the debug state and remains asserted until the CPU is released by a write to the OnCE command register with the GO and EX bits set, and a register specified as either "No register selected" or the CPUSCR. This signal remains asserted even though the CPU may enter and exit the debug state for each instruction executed under control of the OnCE controller. See **7.8.5.1 OnCE Command Register (OCMR)** for more information on the function of the GO and EX bits.

7.8.5 OnCE Controller Registers

This section describes the OnCE controller registers:

- OnCE Command Register (OCMR)
- OnCE Control Register (OCR)
- OnCE Status Register (OSR)

All OnCE registers are addressed by means of the RS field in the OMCR, as shown in **Table 7-2**.

Other OnCE registers are described in **7.8.7 Memory Breakpoint Logic** and **7.8.8 OnCE Trace Logic**.

7.8.5.1 OnCE Command Register (OCMR)

The OnCE command register (OCMR) is an 8-bit shift register that receives its serial data from the TDI pin. This register corresponds to the JTAG IR, and is loaded when the update-IR TAP controller state is entered. It holds the 8-bit commands shifted in during the shift-IR controller state to be used as input for the OnCE decoder. The OCMR contains fields for controlling access to a OnCE resource, as well as controlling single-step operation, and exit from OnCE mode.

Although the OCMR is updated during the update-IR TAP controller state, the corresponding resource is accessed in the DR scan sequence of the TAP controller, and as such, the update-DR state must be transitioned through in order for an access to occur. In addition, the update-DR state must also be transitioned through in order for the single-step and/or exit functionality to be performed, even though the command appears to have no data resource requirement associated with it.

The command register is shown in **Figure 7-8**.

OCMR — OnCE Command Register

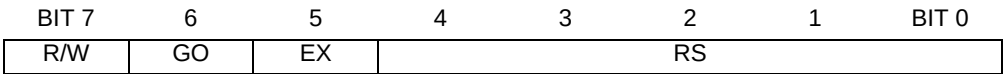


Figure 7-8 OnCE Command Register

R/W — Read/Write Command

- The R/W bit specifies the direction of data transfer.
- 0 = Write the data associated with the command into the register specified by the RS field.
 - 1 = Read the data contained in the register specified by the RS field.

GO — Go Command

When the GO bit is set, the device executes the instruction that resides in the IR register in the CPUSCR. To execute the instruction, the processor leaves debug mode, executes the instruction, and if the EX bit is cleared, returns to debug mode immediately after executing the instruction. The processor resumes normal operation if the EX bit is set. The GO command is executed only if the operation is a read/write to either CPUSCR or "No register selected". Otherwise, the GO bit is ignored. The processor leaves debug mode after the TAP controller update-DR state is entered.

- 0 = Inactive (no action taken)
- 1 = Execute instruction in IR

EX — Exit Command

When the EX bit is set, the processor leaves debug mode and resumes normal operation until another debug request is generated. The exit command is executed only if the Go command is issued, and the operation is a read/write to CPUSCR or read/write to "No register selected". Otherwise the EX bit is ignored. The processor exits debug mode after the TAP controller update-DR state is entered.

- 0 = Remain in debug mode
- 1 = Leave debug mode

RS — Register Select

The register select bits define the source or destination register for the read or write operation, respectively. **Table 7-2** shows OnCE register addresses.

Table 7-2 OnCE Register Addressing

RS	Register Selected
00000	Reserved
00001	Reserved
00010	Reserved
00011	Trace Counter (OTC)
00100	Memory Breakpoint Counter A (MBCA)
00101	Memory Breakpoint Counter B (MBCB)
00110	Program Counter FIFO and Increment Counter
00111	Breakpoint Address Base Register A (BABA)
01000	Breakpoint Address Base Register B (BABB)
01001	Breakpoint Address Mask Register A (BAMA)
01010	Breakpoint Address Mask Register B (BAMB)
01011	CPU Scan Register (CPUSCR)
01100	No Register Selected (Bypass)
01101	OnCE Control Register (OCR)
01110	OnCE Status Register (OSR)
01111	Reserved (Factory Test Control Register — do not access)
10000	Reserved (MEM_BIST, do not access)
10001 – 10110	Reserved (Bypass, do not access)
10111	Reserved (LSRL, do not access)
11000 – 11110	Reserved (Bypass, do not access)
11111	Bypass

7.8.5.2 OnCE Control Register (OCR)

The OnCE control register (OCR) is a 32-bit register used to select the events that will put the device in debug mode and to enable or disable sections of the OnCE logic. The control bits are read/write.

OCR — OnCE Control Register

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	SQC	
W																
RESET:																
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	DR	IDRE	TME	FRZC	RCB	BCB					RCA	BCA				
W																
RESET:																
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 7-9 OnCE Control Register**SQC — Sequential Control**

The SQC field allows memory breakpoint B and trace occurrences to be suspended until a qualifying event occurs. This field is cleared on test logic reset.

Table 7-3 Sequential Control Field Settings

SQC[1:0]	Meaning
00	Disable sequential control operation. Memory breakpoints and trace operation are unaffected by this field.
01	Suspend normal trace counter operation until a breakpoint condition occurs for memory breakpoint B. If this mode is selected, memory breakpoint B occurrences no longer cause a breakpoint request to be generated. Instead, trace counter comparisons are suspended until the first memory breakpoint B occurrence. After the first memory breakpoint B occurrence, trace counter control is released to perform normally (assuming TME is set). This allows a sequence of breakpoint conditions to be specified prior to trace counting.
10	Qualify memory breakpoint B matches with a breakpoint occurrence for memory breakpoint A. If this bit is set, memory breakpoint A occurrences no longer cause a breakpoint request to be generated. Instead, memory breakpoint B comparisons are suspended until the first memory breakpoint A occurrence. After the first memory breakpoint A occurrence, memory breakpoint B is enabled to perform normally. This allows a sequence of breakpoint conditions to be specified.
11	Combine the qualifications specified by the 01 and 10 encodings of this field. In this mode, no breakpoint requests are generated, and trace count operation is enabled (if TME is set) once a memory breakpoint B occurrence follows a memory breakpoint A occurrence.

M210 CPU and OnCE Debug

DR — CPU Debug Request Control

This control bit is used to request the CPU to enter debug mode unconditionally. The CPU indicates that debug mode has been entered via the PM bits in the OnCE status register. Once the CPU enters debug mode, it returns there even with a write to the OCMR with GO and EX set until the DR bit is cleared. This bit is cleared on test logic reset.

IDRE — Internal Debug Request Enable

This control bit is used to enable internally generated debug requests. The internal debug request input to the OnCE control logic ($\overline{\text{IDR}}$) may not be used in all implementations. In some implementations, the $\overline{\text{IDR}}$ control input may be connected and used as an additional hardware debug request. This bit is cleared on test logic reset.

0 = Disable $\overline{\text{IDR}}$ input operation

1 = Enable $\overline{\text{IDR}}$ input operation

TME — Trace Mode Enable

The TME control bit enables the OnCE trace mode operation (see **7.8.8 OnCE Trace Logic**). This bit is cleared on test logic reset. Trace operation is also affected by the SQC field described above.

0 = Disable trace operation

1 = Enable trace operation

FRZC — Freeze Control

This control bit is used in conjunction with memory breakpoint B registers to select between asserting a breakpoint condition when a memory breakpoint B occurs, or freezing the PC FIFO from further updates when memory breakpoint B occurs while allowing the CPU to continue execution. The PC FIFO remains frozen until the FRZO bit in the OSR is cleared.

0 = Memory breakpoint B occurrence causes assertion of a breakpoint condition

1 = Memory breakpoint B occurrence causes a freeze of PC FIFO from further updates and no breakpoint assertion

RCB, RCA — Memory Breakpoint B, A Range Control

These control bits condition enabled memory breakpoints. They condition whether memory breakpoint matches will occur when a memory address falls either within the range defined by memory base address and mask, or outside the range.

0 = Condition breakpoint on access within range

1 = Condition breakpoint on access outside of range

BCB, BCA — Memory Breakpoint B, A Control

These control bits enable memory breakpoints and qualify the access attributes to select whether the breakpoint match will be recognized for read, write, or instruction fetch (program space) accesses. These bits are cleared on test logic reset. See **Table 7-4** for the definition of the BCA and BCB fields.

Table 7-4 Memory Breakpoint Control Field Settings

BC4	BC3	BC2	BC1	BC0	Description
0	0	0	0	0	Breakpoint disabled
0	0	0	0	1	Qualify match with any access
0	0	0	1	0	Qualify match with any instruction access
0	0	0	1	1	Qualify match with any data access
0	0	1	0	0	Qualify match with any change of flow instruction access
0	0	1	0	1	Qualify match with any data write
0	0	1	1	0	Qualify match with any data read
0	0	1	1	1	Reserved
0	1	x	x	x	Reserved
1	0	0	0	0	Reserved
1	0	0	0	1	Qualify match with any user access
1	0	0	1	0	Qualify match with any user instruction access
1	0	0	1	1	Qualify match with any user data access
1	0	1	0	0	Qualify match with any user change of flow access
1	0	1	0	1	Qualify match with any user data write
1	0	1	1	0	Qualify match with any user data read
1	0	1	1	1	Reserved
1	1	0	0	0	Reserved
1	1	0	0	1	Qualify match with any supervisor access
1	1	0	1	0	Qualify match with any supervisor instruction access
1	1	0	1	1	Qualify match with any supervisor data access
1	1	1	0	0	Qualify match with any supervisor change of flow access
1	1	1	0	1	Qualify match with any supervisor data write
1	1	1	1	0	Qualify match with any supervisor data read
1	1	1	1	1	Reserved

7.8.5.3 OnCE Status Register (OSR)

The OnCE status register (OSR) is a 16-bit register used to indicate the reason(s) that debug mode was entered and the current operating mode of the CPU. These status bits are read only.

M210 CPU and OnCE Debug

OSR — OnCE Status Register

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	HDRO	DRO	MBO	SWO	TO	FRZO	SQB	SQA	PM	
W																
RESET:							0	0	0	0	0	0	0	0	0	0

Figure 7-10 OnCE Status Register

HDRO — Hardware Debug Request Occurrence

This read-only status bit is set when the processor enters debug mode as a result of a hardware debug request from the $\overline{\text{IDR}}$ signal or the $\overline{\text{DE}}$ pin. This bit is cleared on test logic reset or when debug mode is exited with the GO and EX bits set.

DRO — Debug Request Occurrence

This read-only status bit is set when the processor enters debug mode and the debug request (DR) control bit in the OnCE control register is set. This bit is cleared on test logic reset or when debug mode is exited with the GO and EX bits set.

MBO — Memory Breakpoint Occurrence

This read-only status bit is set when a memory breakpoint request has been issued to the CPU via the BRKRQ input and the CPU enters debug mode. In some situations involving breakpoint requests on instruction prefetches, the CPU may discard the request along with the prefetch. In this case, this bit may become set due to the CPU entering debug mode for another reason. This bit is cleared on test logic reset or when debug mode is exited with the GO and EX bits set.

SWO — Software Debug Occurrence

This read-only status bit is set when the processor enters debug mode of operation as a result of the execution of the **bkpt** instruction. This bit is cleared on test logic reset or when debug mode is exited with the GO and EX bits set.

TO — Trace Count Occurrence

This read-only status bit is set when the trace counter reaches zero with the trace mode enabled and the CPU enters debug mode. This bit is cleared on test logic reset or when debug mode is exited with the GO and EX bits set.

FRZO — FIFO Freeze Occurrence

This read-only status bit is set when a FIFO freeze occurs. This bit is cleared on test logic reset or when debug mode is exited with the GO and EX bits set.

SQB — Sequential Breakpoint B Arm Occurrence

This read-only status bit is set when sequential operation is enabled and a memory breakpoint B event has occurred to enable trace counter operation. This bit is cleared on test logic reset or when debug mode is exited with the GO and EX bits set.

SQA — Sequential Breakpoint A Arm Occurrence

This read-only status bit is set when sequential operation is enabled and a memory breakpoint A event has occurred to enable memory breakpoint B operation. This bit is cleared on test logic reset or when debug mode is exited with the GO and EX bits set.

PM — Processor Mode

These status bits indicate the processor operating mode. They allow coordination of the OnCE controller with the CPU to synchronize the two.

Table 7-5 Processor Mode Field Settings

PM[1:0]	Meaning
00	Processor in normal mode
01	Processor in stop, doze, or wait mode
10	Processor in debug mode
11	Reserved

7.8.6 OnCE Decoder (ODEC)

The OnCE decoder (ODEC) receives as input the 8-bit command from the OCMR and status signals from the processor. The ODEC generates all the strobes required for reading and writing the selected OnCE registers.

7.8.7 Memory Breakpoint Logic

Memory breakpoints can be set for a particular memory location or on accesses within an address range. The breakpoint logic contains an input latch for addresses, registers that store the base address and address mask, comparators, attribute qualifiers, and a breakpoint counter. **Figure 7-11** illustrates the basic functionality of the OnCE memory breakpoint logic. This logic is duplicated to provide two independent breakpoint resources.

Address comparators can be used to determine where a program may be getting lost or when data is being written to areas which should not be written. They are also useful in halting a program at a specific point to examine or change registers or memory. Using address

comparators to set breakpoints enables the user to set breakpoints in RAM or ROM in any operating mode. Memory accesses are monitored according to the contents of the OCR.

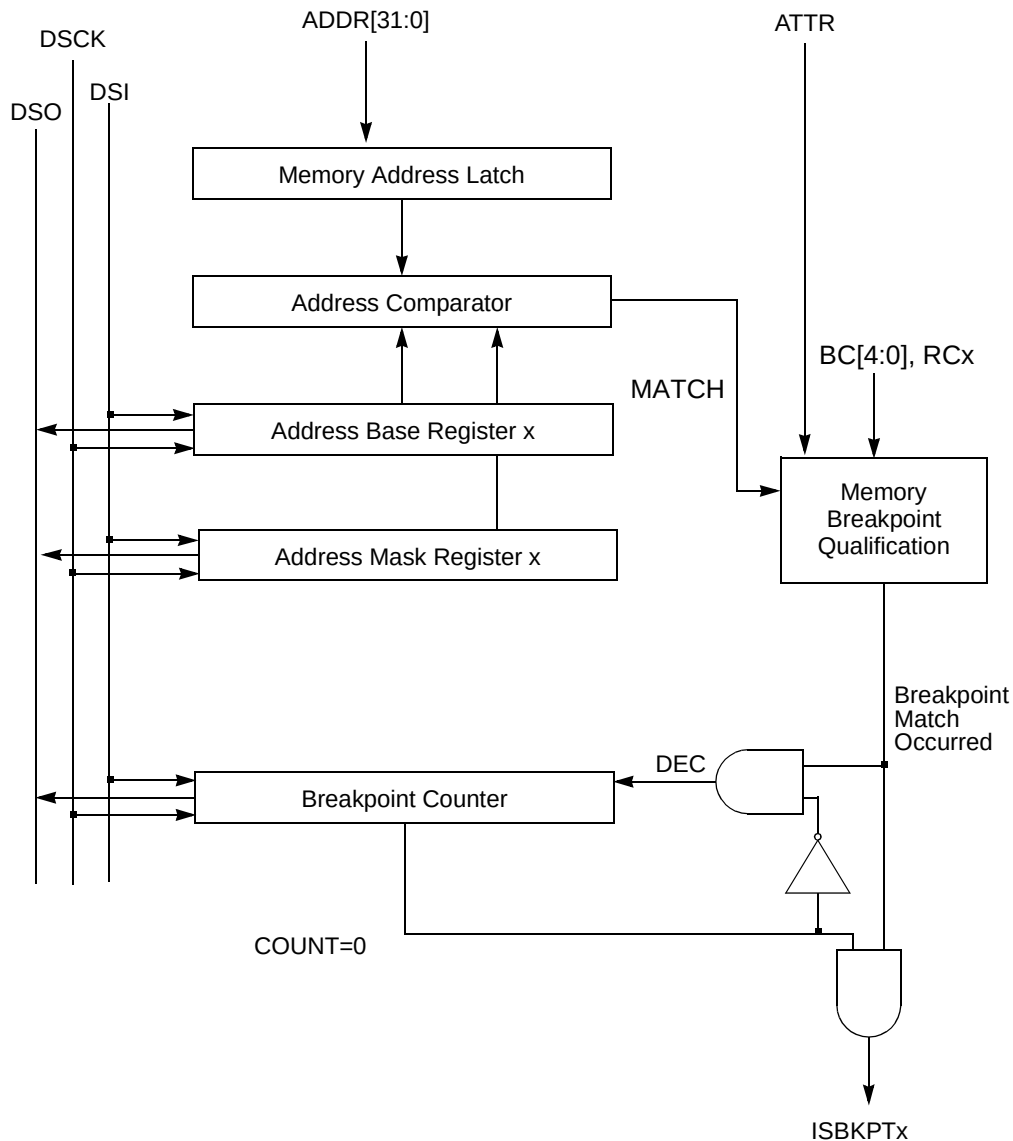


Figure 7-11 OnCE Memory Breakpoint Logic

The address comparator generates a match signal when the address on the bus matches the address stored in the breakpoint address base register, as masked with individual bit masking capability provided by the breakpoint address mask register. The address match signal and the access attributes are further qualified with the RCx and BCx[4:0] control bits. This qualification is used to decrement the breakpoint counter conditionally if its contents are non-zero. If the

contents are zero, the counter is not decremented and the breakpoint event occurs (ISBKPTx asserted).

7.8.7.1 Memory Address Latch (MAL)

The memory address latch (MAL) is a 32-bit register that latches the address bus on every access.

7.8.7.2 Breakpoint Address Base Registers (BABA, BABB)

The 32-bit breakpoint address base registers (BABA, BABB) store memory breakpoint base addresses. BABA and BABB can be read or written through the OnCE serial interface. Before enabling breakpoints, the external command controller should load these registers.

7.8.7.3 Breakpoint Address Mask Registers (BAMA, BAMB)

The 32-bit breakpoint address mask registers (BAMA, BAMB) store memory breakpoint base address masks. BAMA and BAMB can be read or written through the OnCE serial interface. Before enabling breakpoints, the external command controller should load these registers.

7.8.7.4 Breakpoint Address Comparators

The breakpoint address comparators are not externally accessible. Each compares the memory address stored in MAL with the contents of BABx, as masked by BAMx, and signals the control logic when a match occurs.

7.8.7.5 Memory Breakpoint Counters (MBCA, MBCB)

The 16-bit memory breakpoint counter x (MBCx) register is loaded with a value equal to the number of times, minus one, that a memory access event should occur before a memory breakpoint is declared. The memory access event is specified by the RCx and BCx[4:0] bits in the OCR register and by the memory base and mask registers. On each occurrence of the memory access event, the breakpoint counter, if currently non-zero, is decremented. When the counter has reached the value of zero and a new occurrence takes place, the ISBKPTx signal is asserted and causes the CPU's BRKRQ input to be asserted. The MBCx can be read or written through the OnCE serial interface.

Anytime the breakpoint registers are changed, or a different breakpoint event is selected in the OCR, the breakpoint counter must be written afterward. This assures that the OnCE breakpoint logic is reset and that no previous events will affect the new breakpoint event selected.

M210 CPU and OnCE Debug

7.8.8 OnCE Trace Logic

The OnCE trace logic allows the user to execute instructions in single or multiple steps before the device returns to debug mode and awaits OnCE commands from the debug serial port. (The OnCE trace logic is independent of the M•CORE trace facility, which is controlled through the trace mode bits in the M•CORE processor status register). The OnCE trace logic block diagram is shown in **Figure 7-12**.

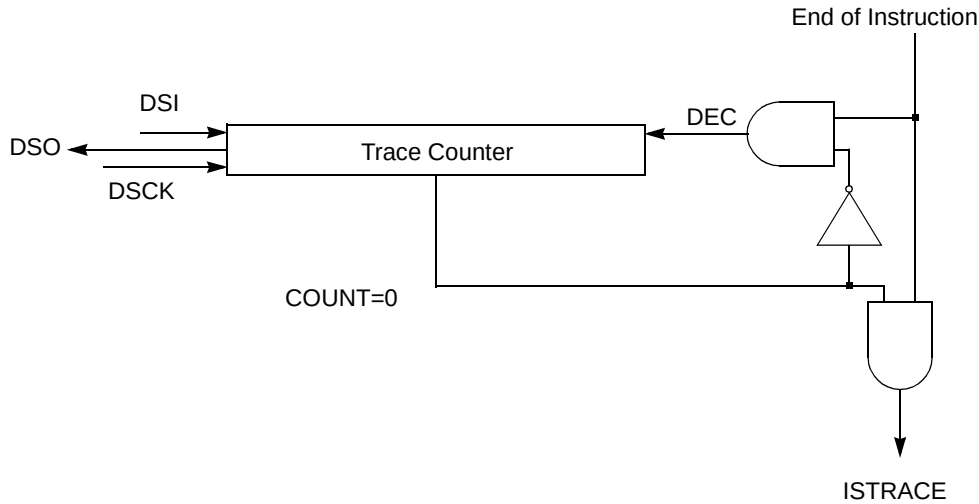


Figure 7-12 OnCE Trace Logic Block Diagram

7.8.8.1 Trace Counter (OTC)

The trace counter (OTC) allows more than one instruction to be executed in real time before the device returns to debug mode. This feature helps the software developer debug sections of code that are time-critical. The trace counter also enables the user to count the number of instructions executed in a code segment.

The OTC is a 16-bit counter that can be read, written, or cleared through the OnCE serial interface. If N instructions are to be executed before entering debug mode, the trace counter should be loaded with N–1. N must not equal zero unless the sequential breakpoint control capability described in **7.8.5.2 OnCE Control Register (OCR)** is being used. In this case a value of zero (indicating a single instruction) is allowed.

The trace counter is cleared by hardware reset.

7.8.8.2 Trace Operation

The following steps initiate trace mode operation:

1. Load the counter with a value. This value must be non-zero, unless the sequential breakpoint control capability described in **7.8.5.2 OnCE Control Register (OCR)** is being used. In this case a value of zero (indicating a single instruction) is allowed.
2. Initialize the program counter and instruction register in the CPUSCR with values corresponding to the start location of the instruction(s) to be executed real-time.
3. Set the TME bit in the OCR.
4. Release the processor from debug mode by executing the appropriate command issued by the external command controller.

When debug mode is exited, the counter is decremented after each execution of an instruction. Interrupts can be serviced, and all instructions executed (including interrupt services) will decrement the trace counter.

When the trace counter decrements to zero, the OnCE control logic requests that the processor re-enter debug mode, and the trace occurrence bit TO in the OSR is set to indicate that debug mode has been requested as a result of the trace count function. The trace counter allows a minimum of two instructions to be specified for execution prior to entering trace (specified by a count value of one), unless the sequential breakpoint control capability described in **7.8.5.2 OnCE Control Register (OCR)** is being used. In this case a value of zero (indicating a single instruction) is allowed.

7.8.9 Methods of Entering Debug Mode

The OSR indicates that the CPU has entered debug mode via the PM status field. The following paragraphs discuss conditions that invoke debug mode.

7.8.9.1 Debug Request During RESET

When the DR bit in the OCR is set, assertion of RESET causes the device to enter debug mode. In this case the device may fetch the reset vector and the first instruction of the reset exception handler but does not execute an instruction before entering debug mode.

7.8.9.2 Debug Request During Normal Activity

Setting the DR bit in the OCR during normal device activity causes the device to finish the execution of the current instruction and then enter debug mode. Note that in this case the device completes the execution of the current instruction and stops after the newly fetched instruction enters the CPU instruction latch. This process is the same for any newly fetched instruction, including instructions fetched by interrupt processing or those that will be aborted by interrupt processing.

7.8.9.3 Debug Request During Stop, Doze, or Wait Mode

Setting the DR bit in the OCR when the device is in stop, doze, or wait mode (i. e., has executed a **stop**, **doze**, or **wait** instruction) causes the device to exit the low-power state and enter the debug mode. Note that in this case, the device completes the execution of the **stop**, **doze**, or **wait** instruction and halts after the next instruction enters the instruction latch.

7.8.9.4 Software Request During Normal Activity

Executing the **bkpt** instruction when the FDB (force debug enable mode) control bit in the control state register is set, causes the CPU to enter debug mode after the instruction following the **bkpt** instruction has entered the instruction latch.

7.8.10 Enabling OnCE Trace Mode

When the OnCE trace mode mechanism is enabled and the trace count is greater than zero, the trace counter is decremented for each instruction executed. Completing execution of an instruction when the trace counter is zero causes the CPU to enter debug mode.

NOTE:

Only instructions actually executed cause the trace counter to decrement, i.e., an aborted instruction does not decrement the trace counter and does not invoke debug mode.

7.8.11 Enabling OnCE Memory Breakpoints

When the OnCE memory breakpoint mechanism is enabled with a breakpoint counter value of zero, the device enters debug mode after completing the execution of the instruction that caused the memory breakpoint to occur. In case of breakpoints on instruction fetches, the breakpoint is acknowledged immediately after the execution of the fetched instruction. In case of breakpoints on data memory addresses, the breakpoint is acknowledged after the completion of the memory access instruction.

7.8.12 Pipeline Information and Write-Back Bus Register

A number of on-chip registers store the CPU pipeline status and are configured in a single scan chain for access by the OnCE controller. The CPUSCR OnCE register contains these processor resources, which are used to restore the pipeline and resume normal device activity upon return from debug mode. These resources also provide a mechanism for the emulator software to access processor and memory contents. **Figure 7-13** shows the block diagram of the pipeline information registers contained in the CPUSCR.

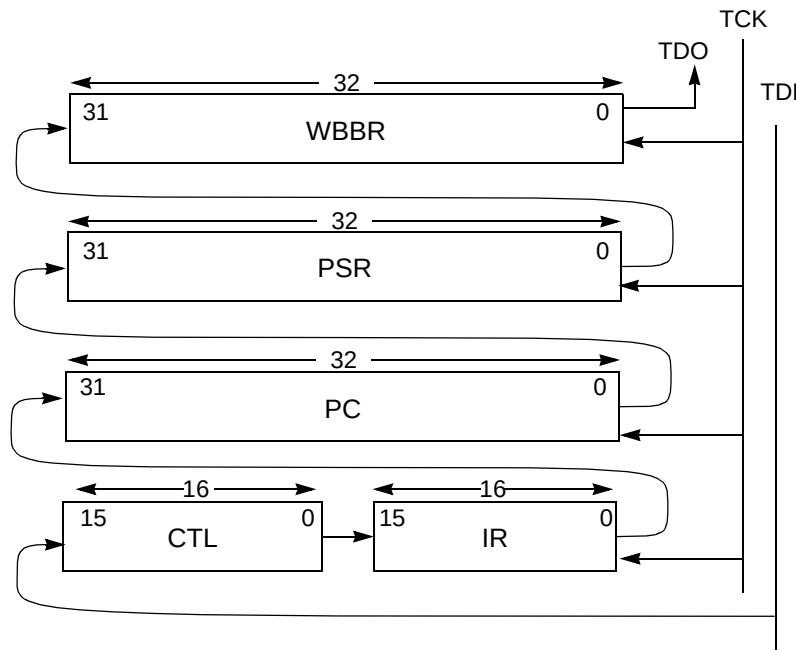


Figure 7-13 CPU Scan Chain Register (CPUSCR)

7.8.12.1 Program Counter Register (PC)

The OnCE program counter register (PC) is a 32-bit latch that stores the value in the CPU program counter when the device enters debug mode. The CPU PC is affected by operations performed during debug mode and must be restored by the external command controller when the CPU returns to normal mode.

7.8.12.2 Instruction Register (IR)

The instruction register (IR) provides a mechanism for controlling the debug session. The IR allows the debug control block to execute selected instructions; the debug control module provides single-step capability.

When scan-out begins, the IR contains the opcode of the next instruction to be executed at the time debug mode was entered. This opcode must be saved in order to resume normal execution at the point debug mode was entered.

On scan-in, the IR can be filled with an opcode selected by debug control software in preparation for exiting debug mode. Selecting appropriate instructions allows a user to examine or change memory locations and processor registers.

M210 CPU and OnCE Debug

Once the debug session is complete and normal processing is to be resumed, the IR can be loaded with the value originally scanned out.

7.8.12.3 Control State Register (CTL)

The control state register (CTL) is used to set control values when debug mode is exited. On scan-in, this register is used to control specific aspects of the CPU. Certain bits reflect internal processor status and should be restored to their original values.

The CTL is a 16-bit latch that stores the value of certain internal CPU state variables before debug mode is entered. This register is affected by the operations performed during the debug session and should be restored by the external command controller when returning to normal mode. In addition to saved internal state variables, the bits are used by emulation firmware to control the debug process.

Reserved bits represent the internal processor state. Restore these bits to their original value after a debug session is completed, i.e., when a OnCE command is issued with the GO and EX bits set and not ignored. Set these bits to ones while instructions are executed during a debug session.

CTL — Control State Register

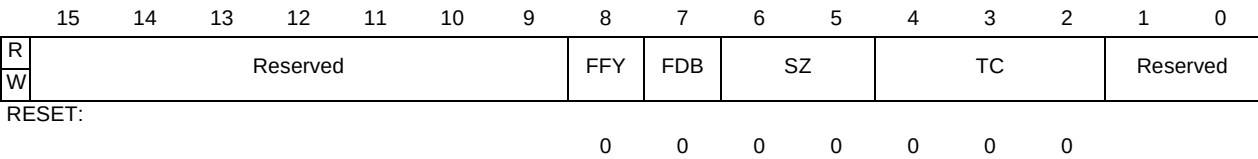


Figure 7-14 Control State Register

FFY — Feed Forward Y Operand

This control bit is used to force the content of the WBBR to be used as the Y operand value of the first instruction to be executed following an update of the CPUSCR. This gives the debug firmware the capability of updating processor registers by initializing the WBBR with the desired value, setting the FFY bit, and executing a **mov** instruction to the desired register.

FDB — Force PSR Debug Enable Mode

Setting this control bit places the processor in debug enable mode. In debug enable mode, execution of the **bkpt** instruction as well as recognition of the **BRKRQ** input causes the processor to enter debug mode, as if the **DBGRQ** input had been asserted.

SZ — Prefetch Size

This control field is used to drive the CPU SIZ[1:0] outputs on the first instruction prefetch caused by issuing a OnCE command with the GO bit set and not ignored. It should be set to indicate a 16-bit size, i.e., 0b10. This field should be restored to its original value after a debug session is completed, i.e., when a OnCE command is issued with the GO and EX bits set and not ignored.

TC — Prefetch Transfer Code

This control field is used to drive the CPU TC[2:0] outputs on the first instruction prefetch caused by issuing a OnCE command with the GO bit set and not ignored. It should typically be set to indicate a supervisor instruction access, i.e., 0b110. This field should be restored to its original value after a debug session is completed, i.e., when a OnCE command is issued with the GO and EX bits set and not ignored.

7.8.12.4 Write-Back Bus Register (WBBR)

The write-back bus register (WBBR) is used as a means of passing operand information between the CPU and the external command controller. Whenever the external command controller needs to read the contents of a register or memory location, it forces the device to execute an instruction that brings that information to WBBR.

For example, to read the content of processor register **r0**, a **mov r0,r0** instruction is executed, and the result value of the instruction is latched into the WBBR. The contents of WBBR can then be delivered serially to the external command controller.

To update a processor resource, this register is initialized with a data value to be written, and a **mov** instruction is executed which uses this value as a write-back data value. The FFY bit in the control state register forces the value of the WBBR to be substituted for the normal source value of a **mov** instruction, thus allowing updates to processor registers to be performed.

7.8.12.5 Processor Status Register (PSR)

The OnCE processor status register (PSR) is a 32-bit latch used to read or write the M•CORE processor status register. Whenever the external command controller needs to save or modify the contents of the M•CORE processor status register, this register is used. This register is affected by the operations performed in debug mode and must be restored by the external command controller when returning to normal mode.

7.8.13 Instruction Address FIFO Buffer (PC FIFO)

To ease debugging activity and keep track of program flow, a first-in-first-out (FIFO) buffer stores the addresses of the last eight instruction change-of-flow prefetches that were issued.

The FIFO is implemented as a circular buffer containing eight 32-bit registers and one 3-bit counter. All the registers have the same address, but any read access to the FIFO address causes the counter to increment and point to the next FIFO register. The registers are serially available to the external command controller through the common FIFO address. **Figure 7-15** shows the block diagram of the PC FIFO.

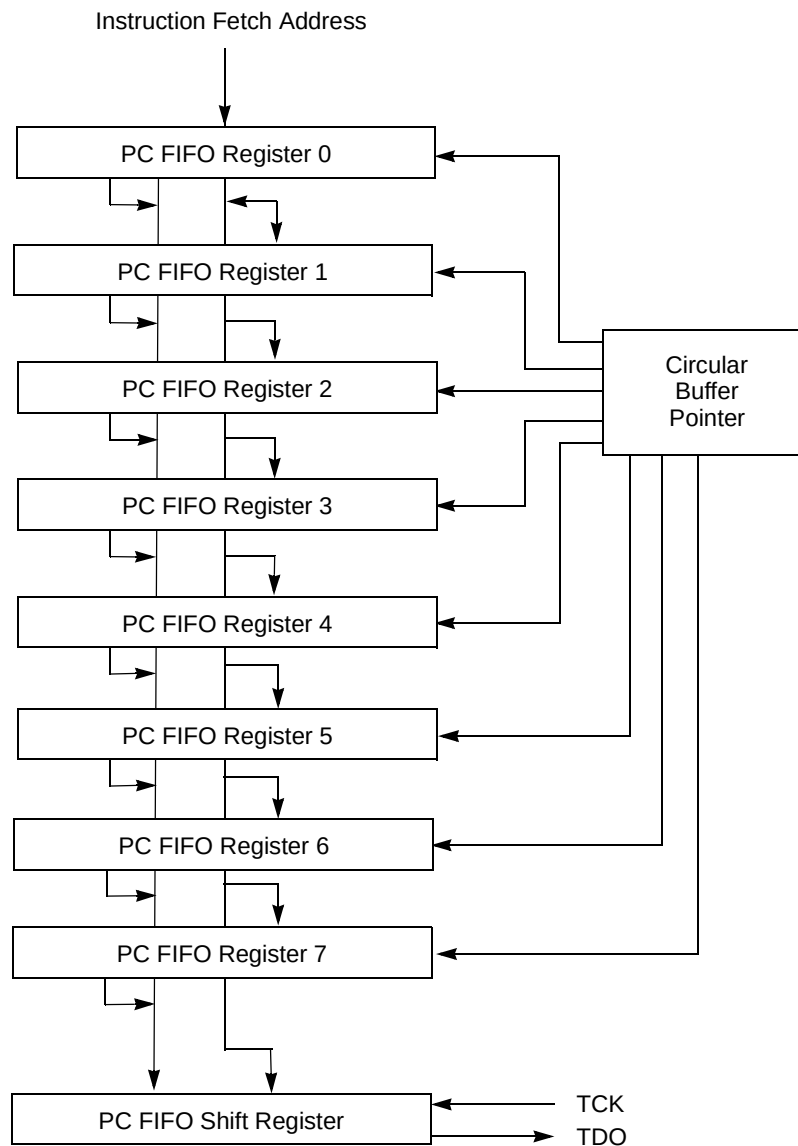


Figure 7-15 OnCE PC FIFO

The FIFO is not affected by operations performed in debug mode, except for incrementing the FIFO pointer when the FIFO is read. When debug mode is entered, the FIFO counter points to

the FIFO register containing the address of the oldest of the eight change-of-flow prefetches. The first FIFO read obtains the oldest address, and the following FIFO reads return the other addresses from the oldest to the newest (in order of execution).

To ensure FIFO coherence, a complete set of eight reads of the FIFO must be performed. Each read increments the FIFO pointer, causing it to point to the next location. After eight reads, the pointer points to the same location as before the start of the read procedure.

7.8.14 Reserved Test Control Registers (MEM_BIST, FTCCR, LSRL)

These registers are reserved for factory testing.

WARNING:

To prevent damage to the device or system, do not access these registers during normal operation.

7.8.15 Serial Protocol

The serial protocol permits an efficient means of communication between the OnCE external command controller and the MCU. Before starting any debugging activity, the external command controller must wait for an acknowledgment that the device has entered debug mode. The external command controller communicates with the device by sending 8-bit commands to the OnCE command register and 16 to 128 bits of data to one of the other OnCE registers. Both commands and data are sent or received LSB first. After sending a command, the external command controller must wait for the processor to acknowledge execution of certain commands before it can properly access another OnCE register.

7.8.16 OnCE Commands

The OnCE commands can be classified as follows:

- Read commands (the device delivers the required data)
- Write commands (the device receives data and writes the data in one of the OnCE registers)
- Commands that do not have data transfers associated with them.

The commands are eight bits long and have the format shown in **Figure 7-8**.

7.8.17 Target Site Debug System Requirements

A typical debug environment consists of a target system in which the MCU resides in the user-defined hardware.

The external command controller acts as the medium between the MCU target system and a host computer. The external command controller circuit acts as a serial debug port driver and host computer command interpreter. The controller issues commands based on the host computer inputs from a user interface program which communicates with the user.

7.8.18 Interface Connector For JTAG/OnCE Serial Port

Figure 7-16 shows the recommended connector pinout and interface requirements for debug controllers that access the JTAG/OnCE port. The connector has two rows of seven pins with 0.1 inch center-to-center spacing between pins in each row and each column.

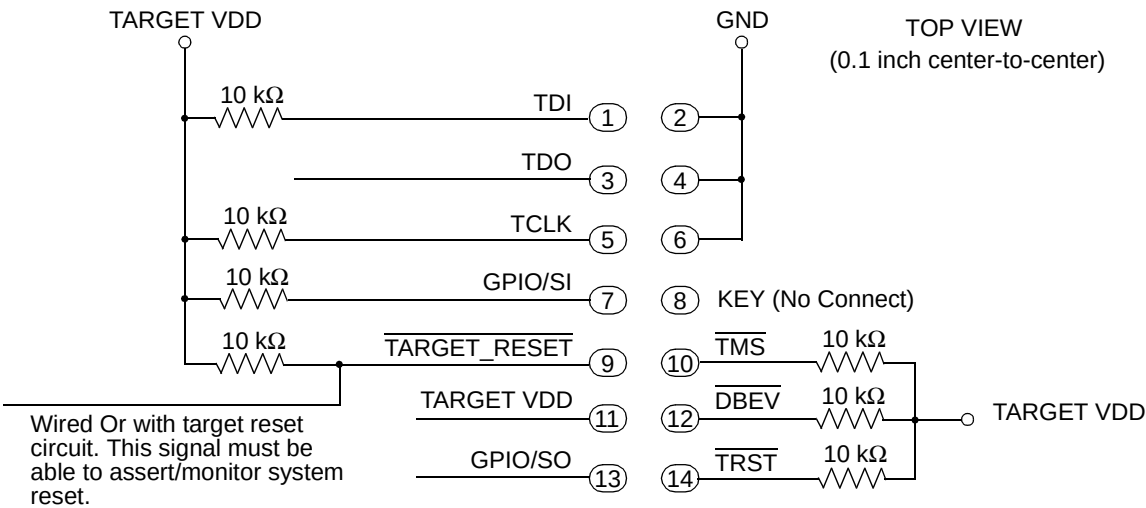


Figure 7-16 Recommended Connector Interface to JTAG/OnCE Port

Section 8 Static Random Access Memory

8.1 Overview

The SRAM is a standard random access memory with the following functionality:

- Fixed address space
- Supports variable array sizes
- Byte, half-word (16-bits), or word (32-bit) read/write accesses
- One clock per access (including bytes, half-words, and words)
- Supervisor or User mode access
- Standby power supply switch to support an external power supply

8.2 Programming Model

The SRAM contains no control registers. The base address is fixed in the bus interface during synthesis (see the micro controller address map for the assigned base address).

Access to the SRAM is not restricted in any way. The array may be accessed in all Supervisor and User modes and may be used to store program or data.

8.3 Read / Write Transactions

All read and write transactions complete in one system clock. Basic back-to-back read/write cycles are illustrated in **Figure 8-1**.

8.4 Reset Operation

The SRAM contents are undefined immediately following a power-on reset. SRAM contents are unaffected by system reset.

If a synchronous reset occurs during a read/write access, then this access will complete normally and any pipelined access that was started will be stopped without corruption of the SRAM contents.

Static Random Access Memory

8.5 Aborted Accesses

If a write transaction is aborted, the SRAM does not store write data into the array. The next transaction can take place on the next low clock phase following the assertion of p_abort_b.

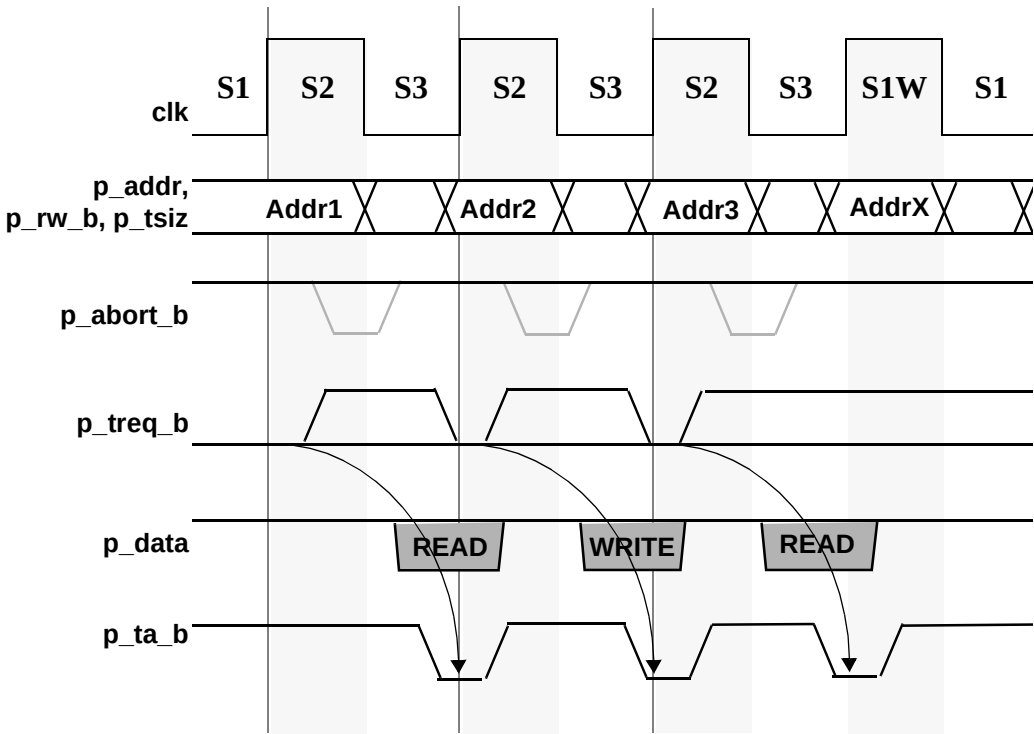


Figure 8-1 Read / Write / Read Sequence

8.6 Stand-by Operation

When the chip is powered down, the contents of the SRAM array are maintained by the standby power supply, VSTBY. If the standby voltage falls below the minimum required voltage, the SRAM contents may be corrupted. The SRAM will automatically switch to standby operation when the voltage on VDD is below the voltage on VSTBY, with no loss of data. In this mode, the SRAM does not respond to any bus cycles. The system may experience unexpected operation when the CPU requests data from the SRAM, since it will not be responding. If standby operation is not desired, then the VSTBY pin should be connected to VDD

The current on VSTBY may exceed its specified maximum value at some time during the transition time that VDD is at or below the voltage switch threshold to a threshold above VSS. If

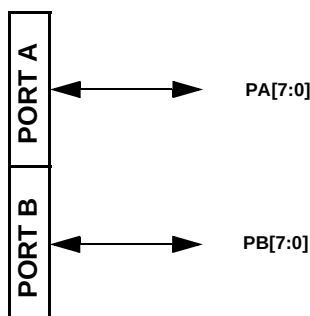
the standby power supply cannot provide enough current to maintain VSTBY above the required minimum value, then a capacitor must be provided from VSTBY to VSS. The value of the capacitor may be calculated as $C = I * t / V$, where 'I' is the difference between the transition current requirement and the power supply's maximum current, 't' is the duration of the VDD transition near the voltage switch threshold, and 'V' is the difference between the minimum available supply voltage and the required minimum VSTBY.

Section 9 Ports

9.1 Overview

General Purpose I/O pins are used as digital I/O pins.

To facilitate the digital I/O function, these pins are grouped into 8-bit ports.



* Optional Pins - Not required for minimum configuration.

Figure 9-1 Block Diagram

9.1.1 Port Operation

All digital I/O pins are programmable as either input or output pins via an associated Data Direction Register. All of these digital I/O pins can be individually defined as input or output.

All ports have both an Output Data Register and a Pin Data Register to monitor and/or control the state of its pins. Writes to an Output Data Register cause data to be stored and then driven to the corresponding pins programmed as outputs. A read of an Output Data Register returns the current state of this data register, regardless of the actual state of its corresponding pins. A read of a Pin Data Register returns the current state of its corresponding pins, regardless of whether the pins are input or output. Writes to a Pin Data Register have no effect.

All ports have set and clear registers which allow for setting or clearing individual bits without affecting the other bit values in the port.

I

9.1.2 Port Digital I/O Timing

Input data on all pins configured as digital I/O will be synchronized to the rising edge of CLKOUT. See Figure 9-2 Digital Input Timing.

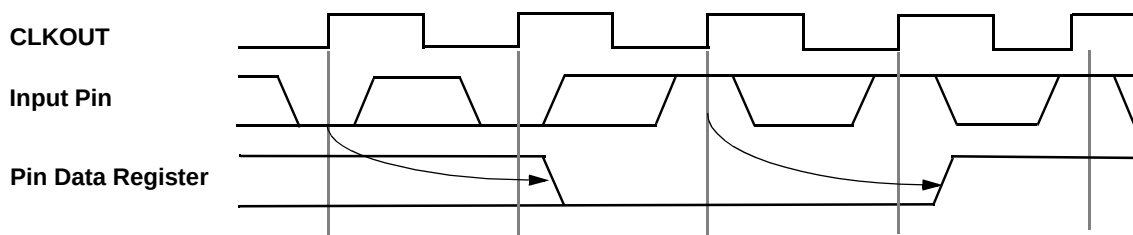


Figure 9-2 Digital Input Timing

Data written to the Output Data Register of any pin configured as digital I/O is immediately driven to its respective pin if the pin is configured as an output. See Figure 9-3 Digital Output Timing.

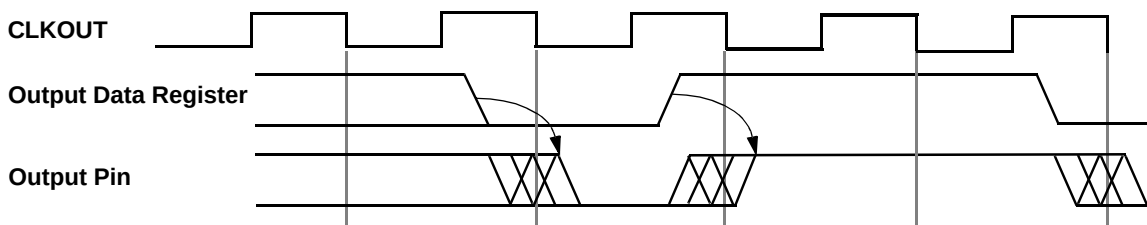


Figure 9-3 Digital Output Timing

9.2 Programming Model

The Ports programming model consists of the following registers:

- The Port A-B Output Data Registers (PORTA-B) store the data to be driven on the corresponding Port output pins when the pins are configured as output.
- The Port A-B Data Direction Registers (DDRA-B) control the direction of the Port pin drivers.
- Port A-B Pin/Set Data Registers (PORTAP-BP/SETA-B) reflect the current state of the Port pins and also allow for setting individual port bits.
- The Port A-B CLR Output Data Registers (CLRA-B) allow for clearing individual port bits.

Ports

All port registers are word, half-word, and byte-accessible, and are grouped to allow coherent access to port data register groups. Writes to reserved bits in the port registers have no effect and reads return zeros.

Table 9-1 Port Address Map

Offset ¹	31 - 24	23 - 16	15 - 8	7 - 0	Access
\$00	PORTA	PORTB	Reserved		Supervisor/User
\$04	Reserved ²				Supervisor/User
\$08	Reserved ³				Supervisor/User
\$0C	DDRA	DDRB	Reserved		Supervisor/User
\$10	Reserved ⁴				Supervisor/User
\$14	Reserved ³				Supervisor/User
\$18	PORTAP/SETA	PORTBP/SETB	Reserved		Supervisor/User
\$1C	Reserved ³				Supervisor/User
\$20	Reserved ³				Supervisor/User
\$24	CLRA	CLRB	Reserved		Supervisor/User
\$28	Reserved ³				Supervisor/User
\$2C	Reserved ³				Supervisor/User
\$30	Reserved ³				Supervisor/User
\$34 - \$3C	Reserved ³				Supervisor/User

NOTES:

1. Absolute address is equal to the Offset plus the base address. The base address is chip dependent.
2. Writes to reserved address locations have no effect and reads return zeros.
3. Writes to reserved address locations have no effect and reads return zeros.
4. Writes to reserved address locations have no effect and reads return zeros.

9.2.1 Port Output Data Registers (PORTA-B)

PORTx is used to store the data to be driven on the corresponding Port x output pins when the pins are configured as output. When read, the current value of the PORTx register, not the Port x pins, is returned.

The PORTx register bits are set when writing a “1” to the corresponding PORTxP/SETx register. The PORTx register bits are cleared when writing a “0” to the corresponding CLRx register.

The Port A - B Output Data Registers are readable/writable. These bits are set by reset.

PORTA - PORTB — Port A - B Output Data Registers

	7	6	5	4	3	2	1	0
R	PORTx7	PORTx6	PORTx5	PORTx4	PORTx3	PORTx2	PORTx1	PORTx0
W								
RESET:	1	1	1	1	1	1	1	1

Figure 9-4 Port Output Data Registers**9.2.2 Port Data Direction Registers (DDRA-B)**

The bits in these registers control the direction of the Port A - B pin drivers. When any bit in these registers is set to 1, the corresponding pin is configured as an output. When any bit in these registers is cleared to 0, the corresponding pin is configured as an input.

The Port A - B Data Direction Registers are readable/writable. These bits are cleared by reset.

DDRA - DDRB — Port A - B Data Direction Registers

	7	6	5	4	3	2	1	0
R	DDRx7	DDRx6	DDRx5	DDRx4	DDRx3	DDRx2	DDRx1	DDRx0
W								
RESET:	0	0	0	0	0	0	0	0

Figure 9-5 Port Data Direction Registers

DDRx[7:0] — Port A - B Data Direction

0 = Input.

1 = Output.

9.2.3 Port Pin/Set Data Registers (PORTAP-BP/SETA-B)

When read, PORTxP/SETx reflects the current state of the Port x pins.

A write of 1 to PORTxP/SETx sets the corresponding bit in the PORTx Output Data Register. A write of 0 has no effect.

The Port A - B Pin Data Registers are readable/writable.

Ports

PORTAP - PORTBP — Port A - B Pin Data Registers

	7	6	5	4	3	2	1	0
R	PORTxP7	PORTxP6	PORTxP5	PORTxP4	PORTxP3	PORTxP2	PORTxP1	PORTxP0
W	SETx7	SETx6	SETx5	SETx4	SETx3	SETx2	SETx1	SETx0
RESET:	*	*	*	*	*	*	*	*

* Read value reflects current state of pin.

Figure 9-6 Port Pin/Set Registers

9.2.4 Port Clear Output Data Registers (CLRA-B)

A write of 0 to CLR_x will clear the corresponding bit in the PORT_x Output Data Register. A write of 1 will not have any effect. Reads to CLR_x will return a “0”.

The Port A - B CLR Output Data Registers are readable/writable.

CLRA-B — Port A - B Clear Output Data Registers

	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0
W	CLR _x 7	CLR _x 6	CLR _x 5	CLR _x 4	CLR _x 3	CLR _x 2	CLR _x 1	CLR _x 0
RESET:	0	0	0	0	0	0	0	0

Figure 9-7 Port Clear Output Data Registers

Section 10 Edge Port

10.1 Overview

The Edge Port has eight external interrupt pins. Each pin may be configured individually as a low level-sensitive interrupt, an edge-detecting interrupt (rising edge, falling edge, or both), or a general-purpose I/O pin.

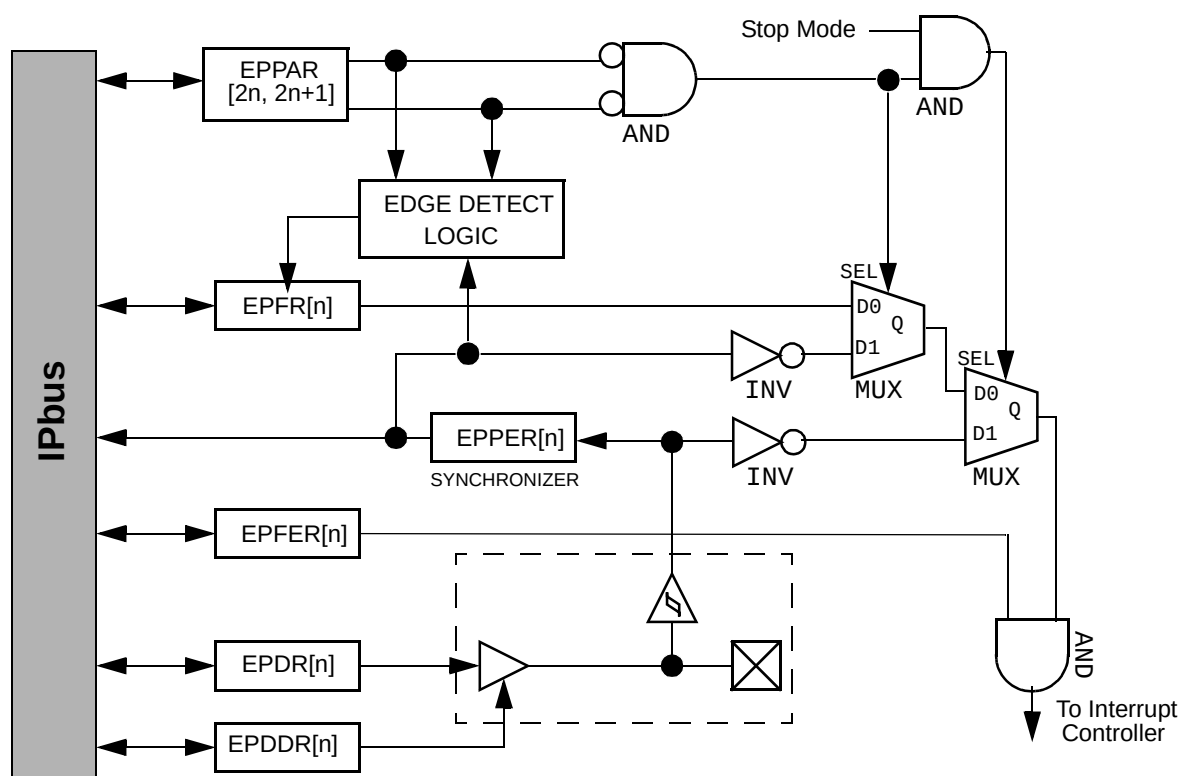


Figure 10-1 Edge Port Block Diagram

10.2 Programming Model

The edge port programming model consists of the following registers:

- The Edge Port Pin Assignment Register (EPPAR) controls the function of each pin individually.
- The Edge Port Data Direction Register (EPDDR) controls the direction of each one of the pins individually.

Edge Port

- The Edge Port Flag Enable Register (EPFER) enables interrupt requests for each pin individually.
- The Edge Port Data Register (EPDR) holds the data to be driven to the pins.
- The Edge Port Pin Data Register (EPPDR) reflects the current state of the pins.
- The Edge Port Flag Register (EPFR) latches the edge event for each one of the pins individually.

Table 10-1 Edge Port Address Map

Offset ¹	15 - 8	7 - 0	Access ²
\$0	EPPAR		Supervisor Only
\$2	EPDDR	EPFER	Supervisor Only
\$4	EPDR	EPPDR	Supervisor/User
\$6	EPFR	Reserved ³	Supervisor/User

NOTES:

1. Absolute address is equal to the Offset plus the base address. The base address is chip dependent.
2. User mode accesses to supervisor only address locations have no effect and result in a cycle termination transfer error.
3. Writes to reserved address locations have no effect and reads return zeros.

10.2.1 Edge Port Pin Assignment Register (EPPAR)

The 16-bit read/write Edge Port Pin Assignment Register (EPPAR) configures each of the interrupt pins as either level-sensitive or edge-triggered. Rising, falling, or both edges can be selected as the active edge. Requests are always generated out of this block but may be masked within the interrupt controller module. The functionality of this register is independent of the programmed pin direction.

EPPAR — Edge Port Pin Assignment Register

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	EPPA7		EPPA6		EPPA5		EPPA4		EPPA3		EPPA2		EPPA1		EPPA0	
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 10-2 Edge Port Pin Assignment Register

EPPAx — Edge Port Pin Assignment Select Field x

Pins configured as level-sensitive are inverted so that a logic low on the external pin represents a valid interrupt request. Level-sensitive interrupt inputs are not latched. To guarantee that a level-sensitive interrupt request is acknowledged, the interrupt source must keep the signal asserted until acknowledged by software.

Pins configured as edge-sensitive interrupts are latched and need not remain asserted for interrupt generation. When the pin is programmed to use the edge detecting circuit, its state is monitored regardless of its configuration as input or output.

These bits are cleared by hardware reset.

Table 10-2 EPPAx Field Settings

Value	Meaning
00	Pin INTx defined as level sensitive
01	Pin INTx defined as rising edge detect
10	Pin INTx defined as falling edge detect
11	Pin INTx defined as both falling and rising edge detect

10.2.2 Edge Port Data Direction Register (EPDDR)

The 8-bit read/write Edge Port Data Direction Register (EPDDR) controls the direction of the port pins. Setting any bit in this register configures the corresponding pin as an output. Clearing any bit in this register configures the corresponding pin as an input. Pin direction is independent of the level/edge mode programmed.

When external sources are desired to generate interrupt requests, then the EPDDR bit must select input. Note that interrupt requests can be generated by programming the Edge Port data output register when the EPDDR selects output.

EPDDR — Edge Port Data Direction Register

	7	6	5	4	3	2	1	0
R	EPDD7	EPDD6	EPDD5	EPDD4	EPDD3	EPDD2	EPDD1	EPDD0
W								
RESET:	0	0	0	0	0	0	0	0

Figure 10-3 Edge Port Data Direction Register

EPDDx — Edge Port Data Direction x

0 = Pin INT x is an input.

1 = Pin INTx is an output.

These bits are cleared by reset.

10.2.3 Edge Port Flag Enable Register (EPFER)

The 8-bit read/write Edge Port Flag Enable Register (EPFER) enables interrupt requests. If a EPFEx bit is set and the corresponding bit in the Edge Port Flag Register is set (or later becomes

Edge Port

set) or if the external pin level is low when the pin is configured for level detect, an interrupt request is generated. If the EPFEx bit is cleared, then the interrupt request negates.

EPFER — Edge Port Flag Enable Register

	7	6	5	4	3	2	1	0
R	EPFE7	EPFE6	EPFE5	EPF4	EPFE3	EPEF2	EPFE1	EPFE0
W								
RESET:	0	0	0	0	0	0	0	0

Figure 10-4 Edge Port Flag Enable Register

EPFEx — Edge Port Flag Enable x
0 = Interrupt disabled for INTx pin.
1 = Interrupt enabled for INTx pin.
These bits are cleared by reset.

10.2.4 Edge Port Data Register (EPDR)

The Edge Port Data Register (EPDR) is a 8-bit read/write register. Writes to EPDR are stored in an internal latch, and if any pin of the port is configured as an output, the data stored for that bit is driven onto the pin. Reads of this register return the value of the data stored in the register.

EPDR — Edge Port Data Register

	7	6	5	4	3	2	1	0
R	EPD7	EPD6	EPD5	EPD4	EPD3	EPD2	EPD1	EPD0
W								
RESET:	1	1	1	1	1	1	1	1

Figure 10-5 Edge Port Data Register

EPDx — Edge Port Data x
See the description above. These bits are set by hardware reset.

10.2.5 Edge Port Pin Data Register (EPPDR)

The Edge Port Pin Data Register (EPPDR) is a 8-bit read-only register, which reflects the current state of the Edge Port pins. Writes to EPPDR have no effect and are terminated normally.

EPPDR — Edge Port Pin Data Register

	7	6	5	4	3	2	1	0
R	EPPD7	EPPD6	EPPD5	EPPD4	EPPD3	EPPD2	EPPD1	EPPD0
W								

RESET:

Figure 10-6 Edge Port Pin Data Register

EPPDx — Edge Port Pin Data x

See the description above. These bits are not affected by hardware reset.

10.2.6 Edge Port Flag Register (EPFR)

The 8-bit read/write Edge Port Flag Register (EPFR) indicates whether the selected edge has been detected on the port pins.

EPFR — Edge Port Flag Register

	7	6	5	4	3	2	1	0
R	EPF7	EPF6	EPF5	EPF4	EPF3	EPF2	EPF1	EPF0
W								

RESET:

0 0 0 0 0 0 0 0

Figure 10-7 Edge Port Flag Register

EPFx — Edge Port Flag x

0 = Selected edge for INTx pin has not been detected.

1 = Selected edge for INTx pin has been detected.

These bits are cleared by reset.

Bits in this register are set when the programmed edge is detected on the corresponding pin. A bit remains set until cleared by writing it to a one. A write of zero has no effect. Pin transitions do not affect this register if the pin is configured as level sensitive (EPPARx=00).

10.3 Interrupt/General-Purpose I/O Pin Descriptions

All pins default to general-purpose input pins at reset. The pin value is synchronized to the rising edge of the Edge Port clock when read from the Edge Port Pin Data Register. The values used in the edge/level detect logic are also synchronized to the rising edge of the Edge Port clock. These pins use Schmitt triggered input buffers which have built in hysteresis designed to

decrease the probability of generating false edge triggered interrupts for slow rising and falling input signals.

10.4 Low-Power Mode Operation

In wait and doze modes, the Edge Port continues to operate normally and may be configured to generate interrupts (either an edge transition or low level on an external pin) to exit the low power modes.

In stop mode, there are no clocks available to perform the edge detect function. Thus, only the level detect logic is active (if configured) to allow any low level on the external interrupt pin to generate an interrupt (if enabled) to exit the stop mode. Note that the input pin synchronizer is bypassed for the level detect logic since there are no clocks available.

10.5 Software Generated Interrupts

The Edge Port may be configured to allow software to generate interrupts. When a pin is configured as a general-purpose output, a level or edge transition can be achieved by the proper write to the EPDR register. This level or edge transition may be used as the source of an interrupt if the appropriate flag enable is set in the EPFER register.

Section 11 Watchdog Timer

11.1 Overview

The watchdog timer is a 16-bit timer used to help software recover from runaway code. The watchdog timer is a free-running down-counter used to generate a reset on underflow. Software must periodically service the watchdog timer in order to restart the count down and prevent a reset.

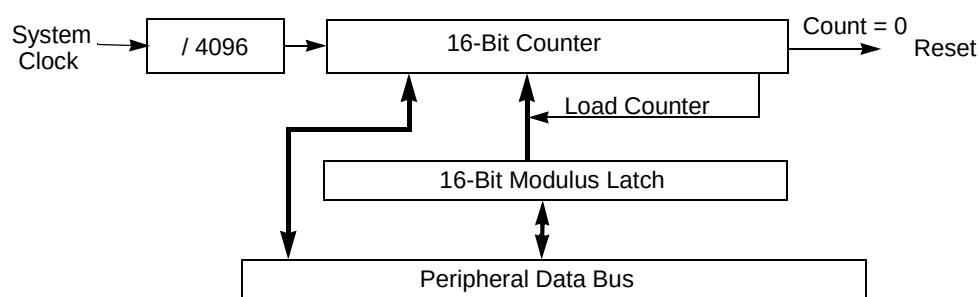


Figure 11-1 Watchdog Timer Block Diagram

11.1.1 Time-out Specifications

The watchdog timer uses a 16 bit counter with a prescaler to support different time-out periods. The prescaler is fixed at a divide by 4096 of the system clock. The time-out period can be selected by writing to the WM bits in the Watchdog Modulus Register (WMR).

$$\text{Time-out} = 4096 * (\text{WM} + 1) \text{ clocks}$$

11.2 Programming Model

The watchdog timer programming model consists of the following registers:

- The Watchdog control Register (WCR) configures the watchdog's operation.
- The Watchdog Modulus Register (WMR) determines the timer modulus reload value.
- The Watchdog Count Register (WCNTR) provides visibility to the counter value.
- The Watchdog Service Register (WSR) requires a service sequence to prevent reset.

Watchdog Timer

Table 11-1 Watchdog Address Map

Offset ¹	15 - 8	7 - 0	Access ²
\$0	WCR		Supervisor Only
\$2	WMR		Supervisor Only
\$4	WCNTR		Supervisor/User
\$6	WSR		Supervisor/User

NOTES:

1. Absolute address is equal to the Offset plus the base address. The base address is chip dependent.
2. User mode accesses to supervisor only address locations have no effect and result in a cycle termination transfer error.

11.2.1 Watchdog Control Register (WCR)

The 16-bit read/write Watchdog Control Register (WCR) configures the timer's operation.

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	0	0	WAIT	DOZE	DBG	EN
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

Figure 11-2 Watchdog Control Register

WAIT - WAIT Mode Control

This bit controls the function of the watchdog timer in WAIT mode.

- 0 = Watchdog timer function is not affected in WAIT mode
- 1 = Watchdog timer function is stopped in WAIT mode

This bit is read always, but once written, further writes have no effect (except during DEBUG or TEST modes).

DOZE - DOZE Mode Control

This bit controls the function of the watchdog timer in DOZE mode.

- 0 = Watchdog timer function is not affected in DOZE mode
- 1 = Watchdog timer function is stopped in DOZE mode

This bit is read always, but once written, further writes have no effect (except during DEBUG or TEST modes).

DBG - DEBUG Mode Control

Watchdog Timer

This bit controls the function of the watchdog timer in DEBUG mode. During DEBUG mode, register read and write accesses function normally. When DEBUG mode is exited, timer operation continues from the state it was prior to entering DEBUG mode, but any updates that occurred in DEBUG mode will remain. A write during DEBUG mode to a write-once register bit that has not previously been written is still writable when DEBUG mode is exited.

- 0 = Watchdog timer function is not affected in DEBUG mode
- 1 = Watchdog timer function is stopped in DEBUG mode

This bit is read always, but once written, further writes have no effect (except during DEBUG or TEST modes).

Note that changing the DBG bit from 1 to 0 during DEBUG mode will unstop the watchdog timer. Likewise, changing the DBG bit from 0 to 1 during DEBUG mode will stop the watchdog timer.

EN - Watchdog Enable

This bit determines whether the watchdog timer is enabled or disabled. When the watchdog timer is disabled, the counter and pre-scalar are held in a stopped state.

- 0 = Watchdog timer is disabled.
- 1 = Watchdog timer is enabled.

This bit is read always, but once written, further writes have no effect (except during DEBUG or TEST modes).

11.2.2 Watchdog Modulus Register (WMR)

The 16-bit read/write Watchdog Modulus Register (WMR) determines the timer modulus that is reloaded into the counter after a proper service sequence.

On a write, the data becomes the new WMR value and this data is immediately loaded into the counter. This value is also used at the next and all subsequent reloads. This value is initialized to the maximum count of 0xFFFF on reset. The pre-scalar counter is reset anytime a new value is loaded into the counter and also during reset.

On a read, the WMR returns the value written in the modulus latch.

These bits are read always, but once written, further writes have no effect (except during DEBUG or TEST modes).

WMR — Watchdog Modulus Register

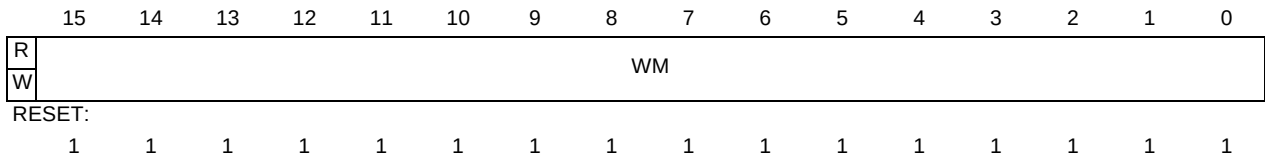


Figure 11-3 Watchdog Modulus Register

Watchdog Timer

11.2.3 Watchdog Count Register (WCNTR)

The Watchdog Count Register is a read-only 16-bit register that provides visibility to the counter value. Reading the 16-bit counter with two 8-bit reads is not guaranteed to be coherent. Writes to this register have no effect and are terminated normally.

WCNTR — Watchdog Count Register

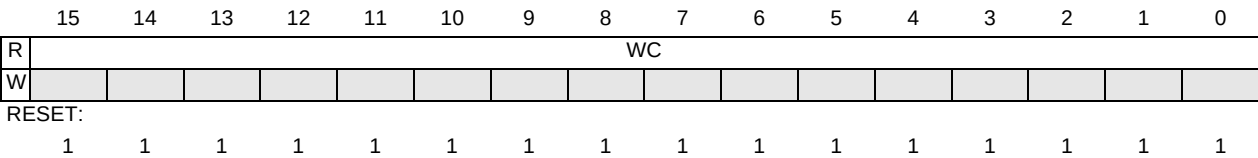


Figure 11-4 Watchdog Count Register

11.2.4 Watchdog Service Register (WSR)

When enabled, the watchdog requires that a service sequence be written to the Watchdog Service Register (WSR). This register controls the restarting of the watchdog counter to keep it from timing out and causing a reset. If this register is not written with 0x5555 followed by 0xAAAA before the selected rate expires, the watchdog sends a signal to the Reset module which sets the WDR bit and asserts a system reset.

Both writes must occur in the order listed prior to the time-out, but any number of instructions can be executed between the two writes. However, a write of any value other than 0x5555 or 0xAAAA to this register resets the servicing sequence, requiring both values to be written to keep the watchdog timer from causing a reset.

WSR — Watchdog Service Register

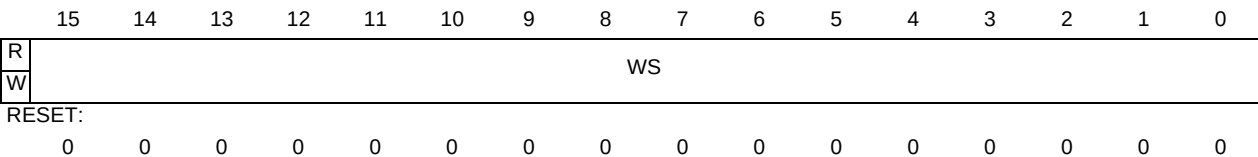


Figure 11-5 Watchdog Service Register

Section 12 Programmable Interval Timer

12.1 Overview

The programmable interval timer (PIT) is a 16-bit timer that provides precise interrupts at regular intervals with minimal processor intervention. The timer can either count down from the value written in the modulus latch, or it can be a free-running down-counter.

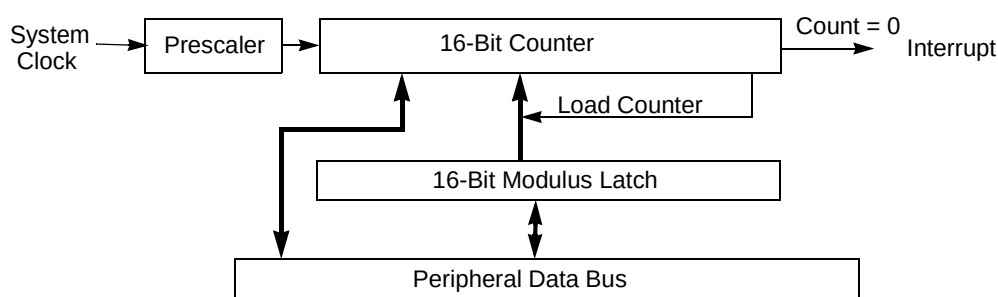


Figure 12-1 PIT Block Diagram

12.1.1 PIT as a “Set-and-Forget” Timer

This mode of operation is selected when the RLD bit in the PIT Control/Status Register (PCSR) is set to a value of one.

When the counter reaches a count of zero, the PIT Interrupt Flag (PIF) is set in the PCSR and the value in the modulus latch is loaded into the counter to decrement towards zero. If the PIT Interrupt Enable (PIE) bit is set in the PCSR, the interrupt flag issues an interrupt to the CPU.

The counter may be directly initialized, without having to wait for the count to reach zero, when the PMR is written with the OVW bit in the PCSR set.

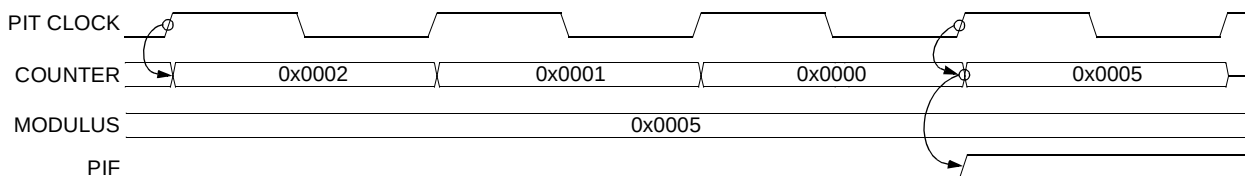


Figure 12-2 Counter Reloading from the Modulus Latch

Programmable Interval Timer

12.1.2 PIT as a “Free-Running” Timer

This mode of operation is selected when the RLD bit in the PCSR is cleared to a value of zero. In this mode, the counter rolls over from 0x0000 to 0xFFFF, without reloading from the modulus latch, and continues to count.

When the counter reaches a count of zero, the PIT interrupt flag (PIF) is set in the PCSR. If the PIT Interrupt Enable (PIE) bit is set in the PCSR, the interrupt flag can issue an interrupt to the CPU.

The counter may be directly initialized, without having to wait for the count to reach zero, when the PMR is written while the OVW bit is set.

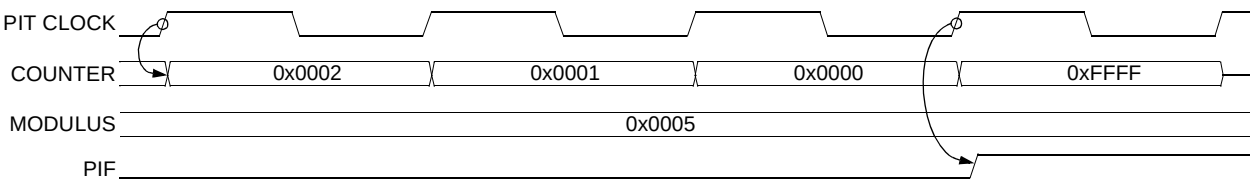


Figure 12-3 Counter in Free-Running Mode

12.1.3 Time-out Specifications

The PIT uses a 16 bit counter with a prescaler to support different time-out periods. The prescaler is divided from the system clock as determined by the PRE bits in the PIT Control/Status Register (PCSR). The time-out period can be selected by writing to the PM bits in the PIT Modulus Register (PMR).

$$\text{Time-out} = \text{Prescaled value} * (\text{PM} + 1) \text{ clocks}$$

12.2 Programming Model

The PIT programming model consists of the following registers:

- The PIT Control/Status Register (PCSR) configures the timer’s operation.
- The PIT Modulus Register (PMR) determines the timer modulus reload value.
- The PIT Count Register (PCNTR) provides visibility to the counter value.

Table 12-1 PIT Address Map

Offset ¹	15 - 8	7 - 0	Access ²
\$0	PCSR		Supervisor Only
\$2	PMR		Supervisor Only

Table 12-1 PIT Address Map

Offset ¹	15 - 8	7 - 0	Access ²
\$4	PCNTR		Supervisor/User
\$6	Unimplemented ³		-

NOTES:

1. Absolute address is equal to the Offset plus the base address. The base address is chip dependent}.
2. User mode accesses to supervisor only address locations have no effect and result in a cycle termination transfer error.
3. Accesses to unimplemented address locations have no effect and result in a cycle termination transfer error.

12.2.1 PIT Control/Status Register (PCSR)

The 16-bit read/write PIT Control/Status Register (PCSR) configures the timer's operation.

PCSR — PIT Control and Status Register

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	PRE				0	DOZE	DBG	OVW	PIE	PIF	RLD	EN
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 12-4 PIT Control and Status Register

PRE[3:0] — Pre-scalar Select

These bits control how the clock for the PIT is generated from the system clock.

Table 12-2 Pre-scalar Select Encoding

PRE[3:0]	Interval Clock Frequency
0000	System Clock
0001	System Clock / 2
0010	System Clock / 4
0011	System Clock / 8
0100	System Clock / 16
0101	System Clock / 32
0110	System Clock / 64
0111	System Clock / 128

Table 12-2 Pre-scalar Select Encoding (Continued)

PRE[3:0]	Interval Clock Frequency
1000	System Clock / 256
1001	System Clock / 512
1010	System Clock / 1024
1011	System Clock / 2048
1100	System Clock / 4096
1101	System Clock / 8192
1110	System Clock / 16384
1111	System Clock / 32768

The PRE bits should only be modified when the enable (EN) bit is negated, to accurately predict the timing of the next count. Any change to the PRE pre-scalar select value will reset the pre-scalar count. The pre-scalar counter is also reset anytime a new value is loaded into the counter and also during reset. Setting the EN bit with the same 16-bit write used to modify the PRE bits is allowed. The pre-scalar counter is stopped when EN is cleared.

DOZE — DOZE Mode Control

This bit controls the function of the PIT in DOZE mode. When DOZE mode is exited, timer operation continues from the state it was prior to entering DOZE mode.

- 0 = PIT function is not affected in DOZE mode
- 1 = PIT function is stopped in DOZE mode.

DBG — DEBUG Mode Control

This bit controls the function of the PIT in DEBUG mode. During DEBUG mode, register read and write accesses function normally. When DEBUG mode is exited, timer operation continues from the state it was prior to entering DEBUG mode, but any updates that occurred in DEBUG mode will remain.

- 0 = PIT function is not affected in DEBUG mode
- 1 = PIT function is stopped in DEBUG mode

Note that changing the DBG bit from 1 to 0 during DEBUG mode will unstop the PIT timer. Likewise, changing the DBG bit from 0 to 1 during DEBUG mode will stop the PIT timer.

OVW — Counter Overwrite Enable

This bit controls what happens to the counter value when the modulus latch is written.

- 0 = Modulus latch is a holding register for values to be loaded into the counter when the count expires to zero.
- 1 = Modulus latch is transparent. All writes to the latch will also overwrite the counter contents.

PIE — PIT Interrupt Enable

This bit controls the PIT interrupt function.

- 0 = PIF is inhibited from reaching the CPU.

1 = PIF is allowed to request an interrupt.

PIF — PIT Interrupt Flag

This bit is the PIT interrupt flag. It is cleared by writing a one to this bit or by writing to the PIT modulus register. A write of zero has no effect.

0 = No PIT interrupt is present

1 = PIT interrupt is present

RLD — Counter Reload Control

This bit controls whether the value contained in the modulus latch is reloaded into the counter when the counter reaches a count of zero or whether the counter rolls over from zero to 0xFFFF

0 = Counter rolls over to 0xFFFF

1 = Counter is reloaded from the modulus latch

EN — PIT Enable

This bit controls whether the PIT is enabled or disabled. When the PIT is disabled, the counter and pre-scalar are held in a stopped state.

0 = PIT is disabled

1 = PIT is enabled

12.2.2 PIT Modulus Register (PMR)

The 16-bit read/write PIT Modulus Register (PMR) determines the timer modulus that is reloaded into the counter when the counter counts down to 0.

On a write, the data becomes the new PMR value. A write to the PMR register will not be loaded into the counter until the counter counts to 0 (with the RLD bit set), except if the OVW bit is set, then the value of the counter is loaded immediately with the new PMR value. This value is initialized to the maximum count of 0xFFFF on reset. The pre-scalar counter is reset anytime a new value is loaded into the counter and also during reset.

On a read, the PMR returns the value written in the modulus latch.

PMR — PIT Modulus Register

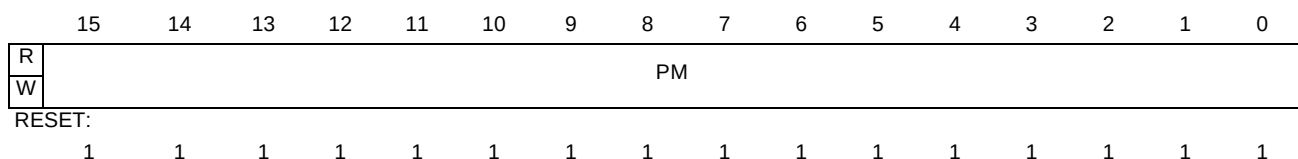


Figure 12-5 PIT Modulus Register

Programmable Interval Timer

12.2.3 PIT Count Register (PCNTR)

The PIT Count Register (PCNTR) is a read-only 16-bit register that provides visibility to the counter value. Reading the 16-bit counter with two 8-bit reads is not guaranteed to be coherent. Writes to this register have no effect and are terminated normally.

PCNTR — PIT Count Register

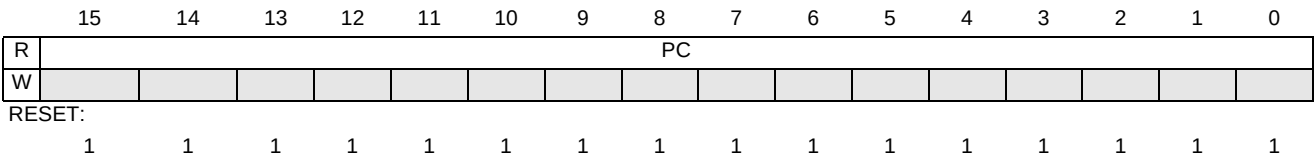


Figure 12-6 PIT Count Register

Section 13 Timer

13.1 Overview

The basic timer consists of a 16-bit, software-programmable counter driven by a seven-stage programmable prescaler. A timer overflow function allows software to extend the timing capability of the system beyond the 16-bit range of the counter. This timer can be used for many purposes, including input waveform measurements while simultaneously generating an output waveform. Pulse widths can vary from microseconds to many seconds. This timer contains 4 complete input capture/output compare channels and one pulse accumulator. The input capture function is used to detect a selected transition edge and record the time. The output compare function is used for generating output signals or for timer software delays. The 16-bit pulse accumulator is used to operate as a simple event counter or a gated time accumulator. The pulse accumulator shares timer channel 3 when in event mode.

13.2 Features

The Timer includes these distinctive features:

- Option for enabling timer port pullups on reset
- 16-bit architecture
- Pulse widths variable from microseconds to seconds
- Prescaler
- Four 16-bit input capture/output compare channels
- 16-bit pulse accumulator
- Toggle on overflow feature for pwm generation

13.3 Modes of Operation

13.3.1 Supervisor Option

- Supervisor Mode

Supervisor mode is indicated by the assertion of the supervisor_mode signal. As required, the Timer uses the supervisor_mode signal to determine if the processor is operating in the user or supervisor privilege mode. Accesses to address locations, which are restricted to supervisor mode accesses, while the supervisor_mode signal is not asserted cause the timer to assert a transfer error.

13.3.2 Low-Power Options

- Run Mode

Clearing the timer enable bit (TE) or the pulse accumulator enable bit (PAE) reduces power consumption in run mode. TE is in timer system control register 1 (TIMSCR1). PAE is in the pulse accumulator control register (TIMPACTL). Timer and pulse accumulator registers are still accessible, but the remaining functions of the timer are disabled.

- Stop Mode

If the CPU enters stop mode, the timer is frozen. Upon exiting stop mode, the timer will resume operation unless stop mode was exited by reset.

- WAIT, DOZE and DEBUG Modes

The timer is unaffected by these low power modes.

13.3.3 Test Option

- Test Mode

Test mode (or special mode) is indicated by the assertion of the test_mode signal. No difference in module behavior results from the assertion of test_mode, other than enabling module test register accesses.

The Timer contains test registers for factory and/or user test. These registers are writable only in test mode (test_mode asserted).

13.4 Block Diagram

Figure 13-1 is a block diagram of the Timer.



Timer

13.5 Timer Signal Description

13.5.1 Overview

This section describes external interface signals that do, or may, connect off chip. It includes a table of signal properties (See **Table 13-1.**), and detailed discussion of signals, including appropriate timing diagrams.

Table 13-1 Signal Properties

Name ¹	Port	Function	Reset State	Pull up ²
ICOC[0]	TIMPORT[0]	Timer Channel 0 Input Capture/Output Compare Pin	Pin State	Active
ICOC[1]	TIMPORT[1]	Timer Channel 1 Input Capture/Output Compare Pin	Pin State	Active
ICOC[2]	TIMPORT[2]	Timer Channel 2 Input Capture/Output Compare Pin	Pin State	Active
ICOC[3]	TIMPORT[3]	Timer Channel 3 Input Capture/Output Compare or Pulse Accumulator Pin	Pin State	Active
sync	---	Clear Counter	0	---

NOTES:

1. The pin signal names may change depending on the user's discretion.
2. Determined at SoC definition.

13.5.2 Detailed Signal Descriptions

13.5.2.1 ICOC[2:0]

The ICOC[2:0] pins are for the timer's input capture and output compare functions. These pins are available for general-purpose I/O when not configured for timer functions.

13.5.2.2 ICOC[3]

The ICOC[3] pin is for the timer's input capture and output compare function or can be used as the pulse accumulator input. This pin is available for general-purpose I/O when not configured for timer functions.

13.5.2.3 sync

The sync signal is used to clear the timer counter at any time. Based on the user's discretion, this signal may or may not be implemented.

13.6 Timer Memory Map and Registers

13.6.1 Overview

This section provides a detailed description of all memory and registers accessible to the end user.

13.6.2 Module Memory Map

The memory map for the timer module is given below in **Table 13-2**. The Address listed for each register is the sum of a base address and an address offset. The base address is defined at the SoC level and the address offset is defined at the module level.

Table 13-2 Module Memory Map

Address Offset	Use	Access
\$__00	Timer IC/OC Select Register (TIMIOS)	Read/Write
\$__01	Compare Force Register (TIMCFORC)	Read/Write ¹
\$__02	Output Compare 3 Mask Register (TIMOC3M)	Read/Write
\$__03	Output Compare 3 Data Register (TIMOC3D)	Read/Write
\$__04	Timer Counter Register High (TIMCNTH)	Read/Write ²
\$__05	Timer Counter Register Low (TIMCNTL)	Read/Write ²
\$__06	Timer System Control Register 1 (TIMSCR1)	Read/Write
\$__07	Reserved	--
\$__08	Timer TOV Register (TIMTOV)	Read/Write
\$__09	Timer Control Register 1 (TIMCTL1)	Read/Write
\$__0A	Reserved	--
\$__0B	Timer Control Register 2 (TIMCTL2)	Read/Write
\$__0C	Timer Interrupt Enable Register (TIMIE)	Read/Write
\$__0D	Timer System Control Register 2 (TIMSCR2)	Read/Write
\$__0E	Timer Flag Register 1 (TIMFLG1)	Read/Write
\$__0F	Timer Flag Register 2 (TIMFLG2)	Read/Write
\$__10	Timer Channel 0 High Register (TIMC0H)	Read/Write ³
\$__11	Timer Channel 0 Low Register (TIMC0L)	Read/Write ³
\$__12	Timer Channel 1 High Register (TIMC1H)	Read/Write ³
\$__13	Timer Channel 1 Low Register (TIMC1L)	Read/Write ³
\$__14	Timer Channel 2 High Register (TIMC2H)	Read/Write ³
\$__15	Timer Channel 2 Low Register (TIMC2L)	Read/Write ³

Timer

Table 13-2 Module Memory Map

\$__16	Timer Channel 3 High Register (TIMC3H)	Read/Write ³
\$__17	Timer Channel 3 Low Register (TIMC3L)	Read/Write ³
\$__18	Pulse Accumulator Control Register (TIMPACTL)	Read/Write
\$__19	Pulse Accumulator Flag Register (TIMPAFLG)	Read/Write
\$__1A	Pulse Accumulator Count High Register (TIMPACNTH)	Read/Write
\$__1B	Pulse Accumulator Count Low Register (TIMPACNTL)	Read/Write
\$__1C	Reserved	--
\$__1D	Timer Port Data Register (TIMPORT)	Read/Write ⁴
\$__1E	Timer Port Data Direction Register (TIMDDR)	Read/Write
\$__1F	Timer Test Register (TIMTST)	Read/Write ²

NOTES:

1. Always reads \$00.
2. Can only write to this register in test (special) mode.
3. Can only write on output compare.
4. Can read pin state when TIMDDR bit is 0; can read pin driver state when TIMDDR bit is 1.

13.6.3 Register Quick Reference

Register		Bit 7	6	5	4	3	2	1	Bit 0	Addr. Offset
TIMIOS	Read:	0	0	0	0	IOS3	IOS2	IOS1	IOS0	\$__00
	Write:									
TIMCFORC	Read:	0	0	0	0	0	0	0	0	\$__01
	Write:					FOC3	FOC2	FOC1	FOC0	
TIMOC3M	Read:	0	0	0	0	OC3M3	OC3M2	OC3M1	OC3M0	\$__02
	Write:									
TIMOC3D	Read:	0	0	0	0	OC3D3	OC3D2	OC3D1	OC3D0	\$__03
	Write:									
TIMCNTH	Read:	Bit 15	14	13	12	11	10	9	Bit 8	\$__04
	Write:									
TIMCNTL	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__05
	Write:									

 = Reserved or unimplemented

Register		Bit 7	6	5	4	3	2	1	Bit 0	Addr. Offset
TIMSCR1	Read: Write:	TE	0	0	TFFCA	0	0	0	0	\$__06
Reserved	Read: Write:	0	0	0	0	0	0	0	0	\$__07
TIMTOV	Read: Write:	0	0	0	0	TOV3	TOV2	TOV1	TOV0	\$__08
TIMCTL1	Read: Write:	OM3	OL3	OM2	OL2	OM1	OL1	OM0	OL0	\$__09
Reserved	Read: Write:	0	0	0	0	0	0	0	0	\$__0A
TIMCTL2	Read: Write:	EDG3B	EDG3A	EDG2B	EDG2A	EDG1B	EDG1A	EDG0B	EDG0A	\$__0B
TIMIE	Read: Write:	0	0	0	0	C3I	C2I	C1I	C0I	\$__0C
TIMSCR2	Read: Write:	TOI	0	PUPT	RDPT	TCRE	PR2	PR1	PR0	\$__0D
TIMFLG1	Read: Write:	0	0	0	0	C3F	C2F	C1F	C0F	\$__0E
TIMFLG2	Read: Write:	TOF	0	0	0	0	0	0	0	\$__0F
TIMC0H	Read: Write:	Bit 15	14	13	12	11	10	9	Bit 8	\$__10
TIMC0L	Read: Write:	Bit 7	6	5	4	3	2	1	Bit 0	\$__11
TIMC1H	Read: Write:	Bit 15	14	13	12	11	10	9	Bit 8	\$__12
TIMC1L	Read: Write:	Bit 7	6	5	4	3	2	1	Bit 0	\$__13
TIMC2H	Read: Write:	Bit 15	14	13	12	11	10	9	Bit 8	\$__14

 = Reserved or unimplemented

Timer

Register		Bit 7	6	5	4	3	2	1	Bit 0	Addr. Offset
TIMC2L	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__15
	Write:									
TIMC3H	Read:	Bit 15	14	13	12	11	10	9	Bit 8	\$__16
	Write:									
TIMC3L	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__17
	Write:									
TIMPACTL	Read:	0	PAE	PAMOD	PEDGE	CLK1	CLK0	PAOVI	PAI	\$__18
	Write:									
TIMPAFLG	Read:	0	0	0	0	0	0	PAOVF	PAIF	\$__19
	Write:									
TIMPACNTH	Read:	Bit 15	14	13	12	11	10	9	Bit 8	\$__1A
	Write:									
TIMPACNTL	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__1B
	Write:									
Reserved	Read:	0	0	0	0	0	0	0	0	\$__1C
	Write:									
TIMPORT	Read:	0	0	0	0	3	2	1	Bit 0	\$__1D
	Write:									
TIMDDR	Read:	0	0	0	0	3	2	1	Bit 0	\$__1E
	Write:									
TIMTST	Read:	0	0	0	0	0	0	TCBYP	PCBYP	\$__1F
	Write:									

= Reserved or unimplemented

13.6.4 Register Descriptions

This section consists of register descriptions in address order. Each description includes a standard register diagram with an associated figure number. Details of register bit and field function follow the register diagrams, in bit order.

13.6.4.1 Timer Input Capture/Output Compare (IC/OC) Select Register

Register address: \$__00

	7	6	5	4	3	2	1	0
R	0	0	0	0	IOS3	IOS2	IOS1	IOS0
W								
RESET:	0	0	0	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-2 Timer IC/OC Select Register (TIMIOS)

Read: anytime

Write: anytime

IOS3 - IOS0 — I/O Select Bits

The IOSx bits enable input capture or output compare operation for the corresponding timer channel.

1 = Output compare enabled

0 = Input capture enabled

13.6.4.2 Compare Force Register

Register address: \$__01

	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0
W					FOC3	FOC2	FOC1	FOC0
RESET:	0	0	0	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-3 Compare Force Register (TIMCFORC)

Read: anytime; always read \$00

Write: anytime

FOC3 - FOC0 — Force Output Compare Bits

Setting an FOCx bit causes an immediate output compare on the corresponding channel. Forcing an output compare does not set the output compare flag.

Timer

1 = Force output compare
0 = No effect

NOTE:

A successful channel 3 output compare overrides any channel 2:0 compares.

13.6.4.3 Output Compare 3 Mask Register

Register address: \$__02

	7	6	5	4	3	2	1	0
R	0	0	0	0	OC3M3	OC3M2	OC3M1	OC3M0
W								
RESET:	0	0	0	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-4 Output Compare 3 Mask Register (TIMOC3M)

Read: anytime

Write: anytime

OC3M3 - OC3M0 — Output Compare 3 Mask Bits

Setting an OC3Mx bit configures the corresponding TIMPORT pin to be an output. OC3Mx makes the timer port pin an output regardless of the data direction bit when the pin is configured for output compare (IOSx = 1). The OC3Mx bits do not change the state of the TIMDDR bits.

1 = Corresponding TIMPORT pin output
0 = No Effect

13.6.4.4 Output Compare 3 Data Register

Register address: \$__03

	7	6	5	4	3	2	1	0
R	0	0	0	0	OC3D3	OC3D2	OC3D1	OC3D0
W								
RESET:	0	0	0	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-5 Output Compare 3 Data Register (TIMOC3D)

Read: anytime

Write: anytime

OC3D3 - OC3D0 — Output Compare 3 Data Bits

When a successful channel 3 output compare occurs, these bits transfer to the timer port data register if the corresponding OC3Mx bits are set.

NOTE:

A successful channel 3 output compare overrides any channel 2:0 compares. For each OC3M bit that is set, the output compare action reflects the corresponding OC3D bit.

13.6.4.5 Timer Counter Registers

Register address: \$__04

	7	6	5	4	3	2	1	0
R	Bit 15	14	13	12	11	10	9	Bit 8
W								
RESET:	0	0	0	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-6 Timer Counter Register High (TIMCNTH)

Register address: \$__05

	7	6	5	4	3	2	1	0
R	Bit 7	6	5	4	3	2	1	Bit 0
W								
RESET:	0	0	0	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-7 Timer Counter Register Low (TIMCNTL)

Read: anytime

Write: only in test (special) mode; has no effect in normal modes

To ensure coherent reading of the timer counter, such that a timer rollover does not occur between two back-to-back 8-bit reads, it is recommended that only half word (16-bit) accesses be used.

Timer

A write to the TIMCNT registers may have an extra cycle on the first count because the write is not synchronized (occurs at least once cycle before the synchronization of the prescaler clock) with the prescaler clock.

13.6.4.6 Timer System Control Register 1

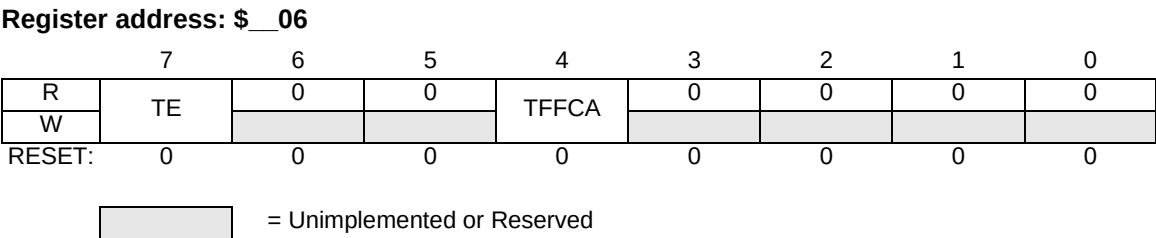


Figure 13-8 Timer System Control Register (TIMSCR1)

Read: anytime

Write: anytime

TE — Timer Enable Bit

TE enables the timer. When the timer is disabled, only the registers are accessible. Clearing TE reduces power consumption.

- 1 = Timer enabled
- 0 = Timer and timer counter disabled

TFFCA — Timer Fast Flag Clear All Bit

TFFCA enables fast clearing of the main timer interrupt flag registers (TIMFLG1 and TIMFLG2) and the PA flag register (TIMPAFLG). TFFCA eliminates the software overhead of a separate clear sequence.

When TFFCA is set:

- An input capture read or a write to an output compare channel clears the corresponding channel flag, CxF.
- Any access of the timer count registers (TIMCNTH/L) clears the TOF flag.
- Any access of the PA counter registers (TIMPACNT) clears both the PAOVF and PAIF flags in the TIMPAFLG register.

Writing logic 1s to the flags clears them only when TFFCA is clear.

- 1 = Fast flag clearing
- 0 = Normal flag clearing

Refer to **Figure 13-9** fast clear flag logic diagram.

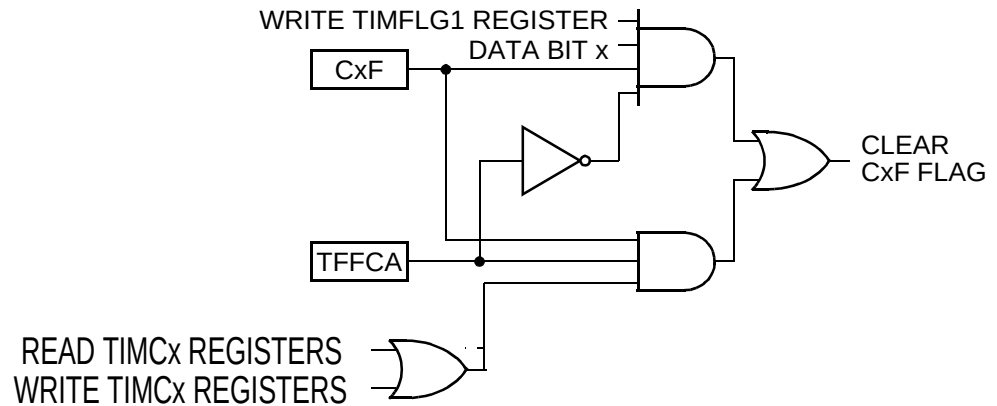


Figure 13-9 Fast Clear Flag Logic

13.6.4.7 Timer Toggle On Overflow (TOV) Register

Register address: \$__08

	7	6	5	4	3	2	1	0
R	0	0	0	0	TOV3	TOV2	TOV1	TOV0
W								
RESET:	0	0	0	0	0	0	0	0

= Unimplemented or Reserved

Figure 13-10 Timer TOV Register (TIMTOV)

Read: anytime

Write: anytime

TOVx — Toggle On Overflow Bits

TOVx toggles output compare pin on overflow. This feature only takes effect when in output compare mode. When set, it takes precedence over forced output compare but not channel 3 override events.

1 = Toggle output compare pin on overflow feature enabled

0 = Toggle output compare pin on overflow feature disabled

Timer

13.6.4.8 Timer Control Register 1

Register address: \$__09

	7	6	5	4	3	2	1	0
R	OM3	OL3	OM2	OL2	OM1	OL1	OM0	OL0
W								
RESET:	0	0	0	0	0	0	0	0

Figure 13-11 Timer Control Register 1 (TIMCTL1)

Read: anytime

Write: anytime

OMx/OLx — Output Mode/Output Level Bits

These bit pairs select the output action to be taken as a result of a successful output compare. When either OMx or OLx is set and the IOSx bit is set, the pin is an output regardless of the state of the corresponding TIMDDR bit.

Table 13-3 Output Compare Action Selection

OMx:OLx	Action on Output Compare
00	Timer disconnected from output pin logic
01	Toggle OCx output line
10	Clear OCx output line
11	Set OCx line

Channel 3 shares a pin with the pulse accumulator input pin. To use the PAI input, clear both the OM3 and OL3 bits and clear the OC3M3 bit in the output compare 3 mask register.

13.6.4.9 Timer Control Register 2

Register address: \$__0B

	7	6	5	4	3	2	1	0
R	EDG3B	EDG3A	EDG2B	EDG2A	EDG1B	EDG1A	EDG0B	EDG0A
W								
RESET:	0	0	0	0	0	0	0	0

Figure 13-12 Timer Control Register 2 (TIMCTL2)

Read: anytime

Write: anytime

EDGxB/EDGxA — Input Capture Edge Control Bits

These eight bit pairs configure the input capture edge detector circuits.

Table 13-4 Input Capture Edge Selection

EDGxB:EDGxA	Edge Selection
00	Input capture disabled
01	Input capture on rising edges only
10	Input capture on falling edges only
11	Input capture on any edge (rising or falling)

13.6.4.10 Timer Interrupt Enable Register

Register address: \$__0C

	7	6	5	4	3	2	1	0
R	0	0	0	0	C3I	C2I	C1I	C0I
W								
RESET:	0	0	0	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-13 Timer Interrupt Enable Register (TIMIE)

Read: anytime

Write: anytime

C3I-C0I — Channel Interrupt Enable Bits

These bits enable the flags in timer flag register 1

1 = Corresponding channel interrupt requests enabled

0 = Corresponding channel interrupt requests disabled

Timer

13.6.4.11 Timer System Control Register 2

Register address: \$__0D

	7	6	5	4	3	2	1	0
R	TOI	0	PUPT	RDPT	TCRE	PR2	PR1	PR0
W								
RESET:	0	0	0 or 1	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-14 Timer System Control Register 2 (TIMSCR2)

Read: anytime

Write: anytime

TOI — Timer Overflow Interrupt Enable Bit

TOI enables timer overflow interrupt requests

1 = Overflow interrupt requests enabled

0 = Overflow interrupt requests disabled

PUPT — Timer Pullup Enable Bit

PUPT enables pullup resistors on the timer ports when the ports are configured as inputs

1 = Pullup resistors enabled

0 = Pullup resistors disabled

RDPT — Timer Drive Reduction Bit

RDPT reduces the output driver size

1 = Output drive reduction enabled

0 = Output drive reduction disabled

TCRE — Timer Counter Reset Enable Bit

TCRE allow the counter to be reset after a channel 3 compare

1 = Counter reset enabled

0 = Counter reset disabled

NOTE:

When the timer channel 3 registers contain \$0000 and TCRE is set, the timer counter registers remain at \$0000 all the time.

When the timer channel 3 registers contain \$FFFF and TCRE is set, TOF never gets set even though the timer counter registers go from \$FFFF to \$0000.

PR[2:0] — Prescaler Bits

These bits select the prescaler divisor for the timer counter.

Table 13-5 Prescaler Selection

PR[2:0]	Prescaler Divisor
000	1
001	2
010	4
011	8
100	16
101	32
110	64
111	128

NOTE:

The newly selected prescaled clock does not take effect until the next synchronized edge of all prescaled clock (i.e. clock count transitions to \$0000).

13.6.4.12 Timer Flag Register 1

Register address: \$__0E

	7	6	5	4	3	2	1	0
R	0	0	0	0	C3F	C2F	C1F	C0F
W								
RESET:	0	0	0	0	0	0	0	0

 = Unimplemented or Reserved

Figure 13-15 Timer Flag Register 1 (TIMFLG1)

Read: anytime

Write: anytime; writing 1 clears flag; writing 0 has no effect

C3F-C0F — Channel Flags

These flags are set when an input capture or output compare event occurs. Clear a channel flag by writing a 1 to it.

NOTE:

When the fast flag clear all bit, TFFCA, is set, an input capture read or an output compare write clears the corresponding channel flag. TFFCA is in timer system control register 1 (TIMSCR1).

Channel Flags, when set, do not inhibit future output compares or input captures.

Timer

13.6.4.13 Timer Flag Register 2

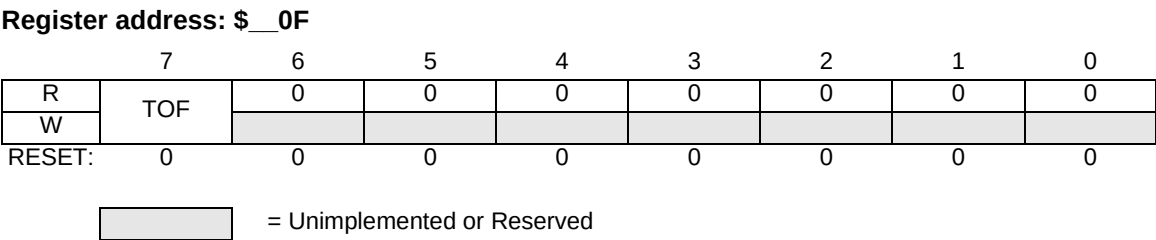


Figure 13-16 Timer Flag Register 2 (TIMFLG2)

Read: anytime

Write: anytime; writing 1 clears flag; writing 0 has no effect

TOF — Timer Overflow Flag

TOF is set when the timer counter rolls over from \$FFFF to \$0000. Clear TOF by writing a 1 to it.

- 1 = Timer overflow
- 0 = No timer overflow

NOTE:

When the timer channel 3 registers contain \$FFFF and TCRE is set, TOF never gets set even though the timer counter registers go from \$FFFF to \$0000.

NOTE:

When the fast flag clear all bit, TFFCA, is set, any access to the timer counter registers clears timer flag register 2. The TFFCA bit is in timer system control register 1 (TIMSCR1).

TOF, when set, does not inhibit future overflow events.

13.6.4.14 Timer Channel Registers

Register addresses: \$ __10(CH0), \$ __12(CH1), \$ __14(CH2), \$ __16(CH3)

	7	6	5	4	3	2	1	0
R	Bit 15	14	13	12	11	10	9	Bit 8
W								
RESET:	0	0	0	0	0	0	0	0

Figure 13-17 Timer Channel [0-3] High Register (TIMCxH)

Register addresses: \$ __11(CH0), \$ __13(CH1), \$ __15(CH2), \$ __17(CH3)

	7	6	5	4	3	2	1	0
R	Bit 7	6	5	4	3	2	1	Bit 0
W								
RESET:	0	0	0	0	0	0	0	0

Figure 13-18 Timer Channel [0-3] Low Register (TIMCxL)

Read: anytime

Write: output compare channel, anytime; input capture channel, no effect

When a channel is configured for input capture (IOSx = 0), the timer channel registers latch the value of the free-running counter when a defined transition occurs on the corresponding input capture pin.

When a channel is configured for output compare (IOSx = 1), the timer channel registers contain the output compare value.

To ensure coherent reading of the timer counter, such that a timer rollover does not occur between back-to-back 8-bit reads, it is recommended that only half word (16-bit) accesses be used.

Timer

13.6.4.15 Pulse Accumulator Control Register

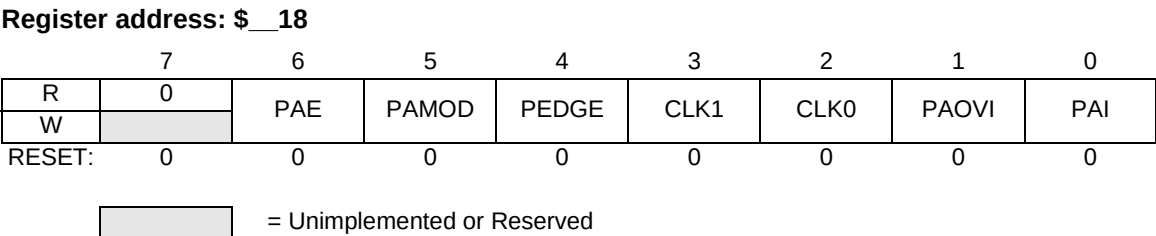


Figure 13-19 Pulse Accumulator Control Register (TIMPACTL)

Read: anytime

Write: anytime

PAE — Pulse Accumulator Enable Bit

- PAE enables the pulse accumulator.
- 1 = Pulse accumulator enabled
- 0 = Pulse accumulator disabled

NOTE:

The pulse accumulator can operate in event mode even when the timer enable bit, TE, is clear.

PAMOD — Pulse Accumulator Mode Bit.

- PAMOD selects event counter mode or gated time accumulation mode.
- 1 = Gated time accumulation mode
- 0 = Event counter mode

PEDGE — Pulse Accumulator Edge Bit

- PEDGE selects falling or rising edges on the PAI pin to increment the counter.
- In event counter mode (PAMOD = 0):
 - 1 = Rising PAI edge increments counter
 - 0 = Falling PAI edge increments counter
- In gated time accumulation mode (PAMOD = 1):
 - 1 = Low PAI input enables divided-by-64 clock to pulse accumulator and trailing rising edge on PAI sets the PAIF flag
 - 0 = High PAI input enables divided-by-64 clock to pulse accumulator and trailing falling PAI edge sets PAIF flag

NOTE:

The timer prescaler generates the divided-by-64 clock. If the timer is not active, there is no divided-by-64 clock.

To operate in gated time accumulation mode:

1. Apply logic 0 to the RESET pin.
2. Initialize registers for pulse accumulator mode test.
3. Apply appropriate level on PAI pin.
4. Enable the timer.

CLK1 and CLK0 — Clock Select Bits

CLK1 and CLK0 select the timer counter input clock as shown in **Table 13-6**.

Table 13-6 Clock Selection

CLK[1:0]	Timer Counter Clock ¹
00	Timer prescaler clock ²
01	PACLK
10	PACLK/256
11	PACLK/65536

NOTES:

1. Changing the CLKx bits causes an immediate change in the timer counter clock input.
2. When PAE = 0, the timer prescaler clock is always the timer counter clock.

PAOVI — Pulse Accumulator Overflow Interrupt Enable Bit

PAOVI enables the pulse accumulator overflow flag, PAOVF, to generate interrupt requests.

- 1 = PAOVF interrupt requests enabled
- 0 = PAOVF interrupt requests disabled

PAI — Pulse Accumulator Input Interrupt Enable Bit

PAI enables the pulse accumulator input flag, PAIF, to generate interrupt requests.

- 1 = PAIF interrupt requests enabled
- 0 = PAIF interrupt requests disabled

Timer

13.6.4.16 Pulse Accumulator Flag Register

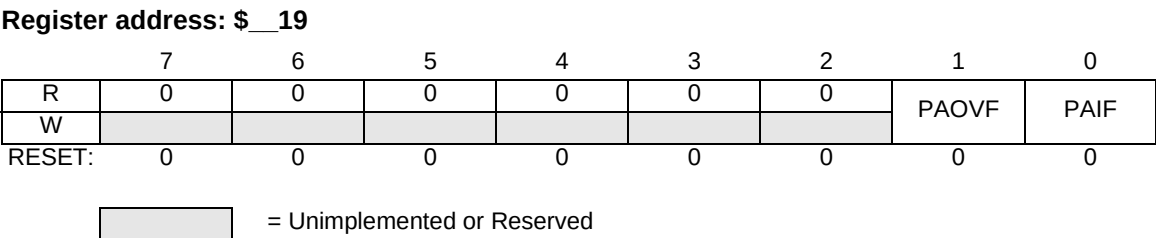


Figure 13-20 Pulse Accumulator Flag Register (TIMPAFLG)

Read: anytime

Write: anytime; writing 1 clears the flag; writing 0 has no effect

PAOVF — Pulse Accumulator Overflow Flag

PAOVF is set when the 16-bit pulse accumulator rolls over from \$FFFF to \$0000. Clear PAOVF by writing to the pulse accumulator flag register with PAOVF set.

1 = 1 = Pulse accumulator overflow

0 = 0 = No pulse accumulator overflow

PAIF — Pulse Accumulator Input Flag

PAIF is set when the selected edge is detected at the PAI pin. In event counter mode, the event edge sets PAIF. In gated time accumulation mode, the trailing edge of the gate signal at the PAI pin sets PAIF. Clear PAIF by writing to the pulse accumulator flag register with PAIF set.

1 = 1 = Active PAI input

0 = 0 = No active PAI input

NOTE:

When the fast flag clear all enable bit, TFFCA, is set, any access to the pulse accumulator counter registers clears all the flags in the TIMPAFLG register.

13.6.4.17 Pulse Accumulator Counter Registers

Register address: \$__1A

	7	6	5	4	3	2	1	0
R	Bit 15	14	13	12	11	10	9	Bit 8
W								
RESET:	0	0	0	0	0	0	0	0

Figure 13-21 Pulse Accumulator Counter High Register (TIMPACNTH)

Register address: \$__1B

	7	6	5	4	3	2	1	0
R	Bit 7	6	5	4	3	2	1	Bit 0
W								
RESET:	0	0	0	0	0	0	0	0

Figure 13-22 Pulse Accumulator Counter Low Register (TIMPACNTL)

Read: anytime

Write: anytime

These registers contain the number of active input edges on the PAI pin since the last reset.

NOTE:

Reading the pulse accumulator counter registers immediately after an active edge on the PAI pin may miss the last count since the input has to be synchronized with the bus clock first.

To ensure coherent reading of the PA counter, such that the counter does not increment between back-to-back 8-bit reads, it is recommended that only half word (16-bit) accesses be used.

Timer

13.6.4.18 Timer Port Data Register

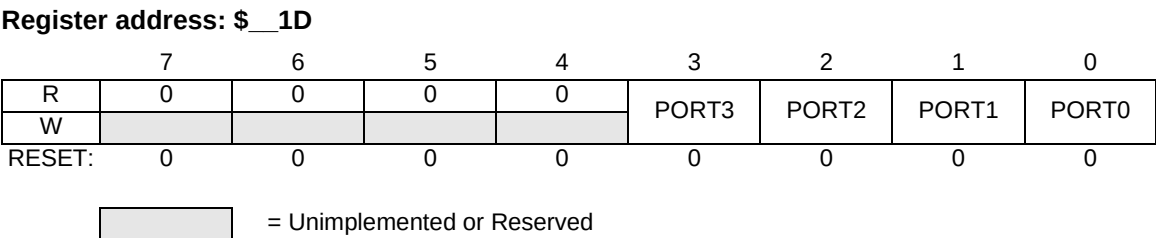


Figure 13-23 Timer Port Data Register (TIMPORT)

Read: anytime; read pin state when the respected TIMDDR bit is 0; read pin driver state when the respected TIMDDR bit is 1.

Write: anytime

Bits 3:0 — Timer Port Input Capture/Output Compare Data Bits

Bit 3 — Pulse Accumulator Input bit

Data written to TIMPORT is buffered and drives the pins only when they are configured as general-purpose outputs.

Reading an input (data direction bit = 0) reads the pin state; reading an output (data direction bit = 1) reads the latch.

Writing to a pin configured as a timer output does not change the pin state.

13.6.4.19 Timer Port Data Direction Register

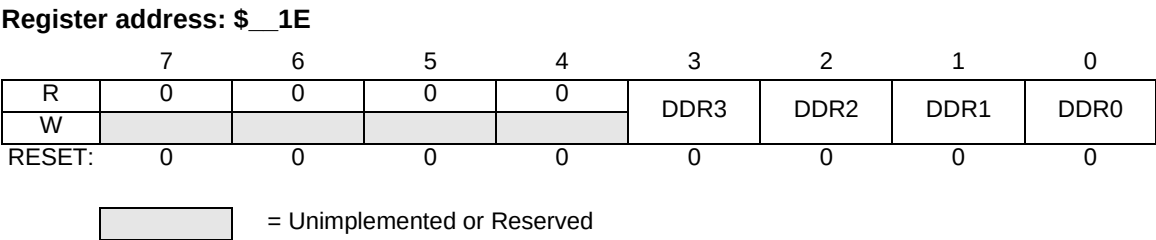


Figure 13-24 Timer Port Data Direction Register (TIMDDR)

Read: anytime

Write: anytime

Bits 3:0 — TIMPORT Data Direction Bits

These bits control the port logic of TIMPORT. Reset clears the timer port data direction register, configuring all timer port pins as inputs.

- 1 = Corresponding pin configured as output
- 0 = Corresponding pin configured as input

13.6.4.20 Timer Test Register

Register address: \$__1F

	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	TCBYP	PCBYP
W								
RESET:	0	0	0	0	0	0	0	0

= Unimplemented or Reserved

Figure 13-25 Timer Test Register (TIMTST)

Read: anytime

Write: only in test (special) mode

TCBYP — Timer divider chain bypass bit

TCBYP divides the 16-bit free-running timer counter (TIMCNTH/L) into two independent 8-bit counters. The clock drives both counters directly and bypasses the timer prescaler

- 1 = Timer counter divided in half and prescaler bypassed
- 0 = Normal operation

PCBYP — Pulse accumulator divider chain bypass bit

PCBYP divides the 16-bit PA counter (TIMPACNTH/L) into two independent 8-bit counters. The clock drives both counters directly and bypasses the timer prescaler if you are in Gated Time Accumulation and the PAI pin is enabled. The pin drives both counters directly and bypasses the timer prescaler if you are in Event Counter Mode.

- 1 = PA counter divided in half and prescaler bypassed
- 0 = Normal Operation

13.7 Timer Functional Description

13.7.1 General

This section provides a complete functional description of the Timer block, detailing the operation of the design from the end user perspective in a number of subsections.

Refer to the timer block diagram in **Figure 13-1** as necessary.

13.7.2 Prescaler

The prescaler divides the module clock by 1, 2, 4, 8, 16, 32, 64, or 128. The prescaler select bits, PR[2:1:0], select the prescaler divisor. PR[2:0] are in timer system control register 2 (TIMSCR2).

13.7.3 Input Capture

Clearing the I/O (input/output) select bit, IOSx, configures channel x as an input capture channel. The input capture function captures the time at which an external event occurs. When an active edge occurs on the pin of an input capture channel, the timer transfers the value in the timer counter into the timer channel registers, TIMCxH and TIMCxL.

If using the Timer in 8-bit MCUs (HC05, HC08, etc.), the low byte of the timer channel register (TIMCxL) is held for one bus cycle after the high byte (TIMCxH) is read. This allows coherent reading of the timer channel such that an input capture won't occur between two back-to-back 8-bit reads. To read the 16-bit timer channel register, use a double-byte read instruction such as LDD or LDX.

The minimum pulse width for the input capture input is greater than two module clocks.

The input capture function does not force data direction. The timer port data direction register controls the data direction of an input capture pin. Pin conditions such as rising or falling edges can trigger an input capture on a pin configured only as an input.

An input capture on channel x sets the CxF flag. The CxI bit enables the CxF flag to generate interrupt requests.

13.7.4 Output Compare

Setting the I/O select bit, IOSx, configures channel x as an output compare channel. The output compare function can generate a periodic pulse with a programmable polarity, duration, and frequency. When the timer counter reaches the value in the channel registers of an output

compare channel, the timer can set, clear, or toggle the channel pin. An output compare on channel x sets the CxF flag. The CxI bit enables the CxF flag to generate interrupt requests.

The output mode and level bits, OMx and OLx, select set, clear, or toggle on output compare. Clearing both OMx and OLx disconnects the pin from the output logic.

Setting a force output compare bit, FOCx, causes an output compare on channel x. A forced output compare does not set the channel flag.

A successful output compare on channel 3 overrides output compares on all other output compare channels. A channel 3 output compare can cause any unmasked bits in the output compare 3 data register to transfer to the timer port data register. The output compare 3 mask register masks the bits in the output compare 3 data register. The timer counter reset enable bit, TCRE, enables channel 3 output compares to reset the timer counter. A channel 3 output compare can reset the timer counter even if the OC3/PAI pin is being used as the pulse accumulator input.

An output compare overrides the data direction bit of the output compare pin but does not change the state of the data direction bit.

Writing to the timer port bit of an output compare pin does not affect the pin state. The value written is stored in an internal latch. When the pin becomes available for general-purpose output, the last value written to the bit appears at the pin.

13.7.5 Pulse Accumulator

The pulse accumulator (PA) is a 16-bit counter that can operate in two modes:

- Event counter mode — Counting edges of selected polarity on the pulse accumulator input pin, PAI
- Gated time accumulation mode — Counting pulses from a divide-by-64 clock

The PA mode bit, PAMOD, selects the mode of operation.

The minimum pulse width for the PAI input is greater than two module clocks.

13.7.5.1 Event Counter Mode

Clearing the PAMOD bit configures the PA for event counter operation. An active edge on the PAI pin increments the PA. The PA edge bit, PEDGE, selects falling edges or rising edges to increment the PA.

Timer

An active edge on the PAI pin sets the PA input flag, PAIF. The PA input interrupt enable bit, PAI, enables the PAIF flag to generate interrupt requests.

NOTE:

The PAI input and timer channel 3 use the same pin. To use the PAI input, disconnect it from the output logic by clearing the channel 3 output mode and output level bits, OM3 and OL3. Also clear the channel 3 output compare 3 mask bit, OC3M3.

The PA counter registers, TIMPACNTH/L reflect the number of active input edges on the PAI pin since the last reset.

The PA overflow flag, PAOVF, is set when the PA rolls over from \$FFFF to \$0000. The PA overflow interrupt enable bit, PAOVI, enables the PAOVF flag to generate interrupt requests.

NOTE:

The PA can operate in event counter mode even when the timer enable bit, TE, is clear.

13.7.5.2 Gated Time Accumulation Mode

Setting the PAMOD bit configures the PA for gated time accumulation operation. An active level on the PAI pin enables a divided-by-64 clock to drive the PA. The PA edge bit, PEDGE, selects low levels or high levels to enable the divided-by-64 clock.

The trailing edge of the active level at the PAI pin sets the PA input flag, PAIF. The PA input interrupt enable bit, PAI, enables the PAIF flag to generate interrupt requests.

NOTE:

The PAI input and timer channel 3 use the same pin. To use the PAI input, disconnect it from the output logic by clearing the channel 3 output mode and output level bits, OM3 and OL3. Also clear the channel 3 output compare mask bit, OC3M3.

The PA counter registers, TIMPACNTH/L reflect the number of pulses from the divided-by-64 clock since the last reset.

NOTE:

The timer prescaler generates the divided-by-64 clock. If the timer is not active, there is no divided-by-64 clock.

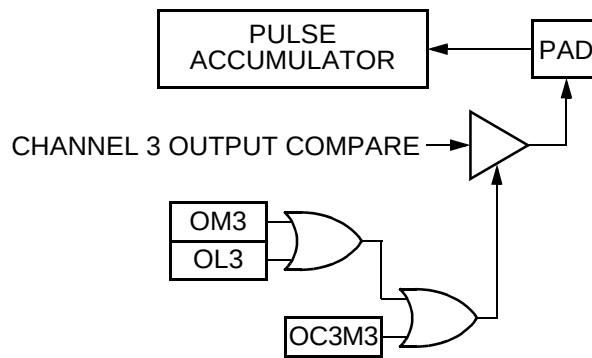


Figure 13-26 Channel 3 Output Compare/Pulse Accumulator Logic

13.7.6 General-Purpose I/O Ports

An I/O pin used by the timer defaults to general-purpose I/O unless an internal function which uses that pin is enabled.

13.7.6.1 Timer Port

The timer pins can be configured for either an input capture function or an output compare function. The IOSx bits in the timer IC/OC select register configure the timer port pins as either input capture or output compare pins.

The timer port data direction register controls the data direction of an input capture pin. External pin conditions trigger input captures on input capture pins configured as inputs.

To configure a pin for input capture:

- set TIMIOS register to 0 for respective channel
- set TIMDDR register to 0 for respective channel
- select input edge to detect (TIMCTL2 register)

The timer port data direction register does not affect the data direction of an output compare pin. The output compare function overrides the data direction register but does not affect the state of the data direction register.

To configure a pin for output capture:

- set TIMIOS register to 1 for respective channel
- initialize TIMCxH/L registers to the output compare value
- set TIMDDR register to 0 for respective channel

Timer

- select the output option (TIMCTL1 register)

Refer to **Table 13-7**, **Figure 13-27**, and **Figure 13-28**.

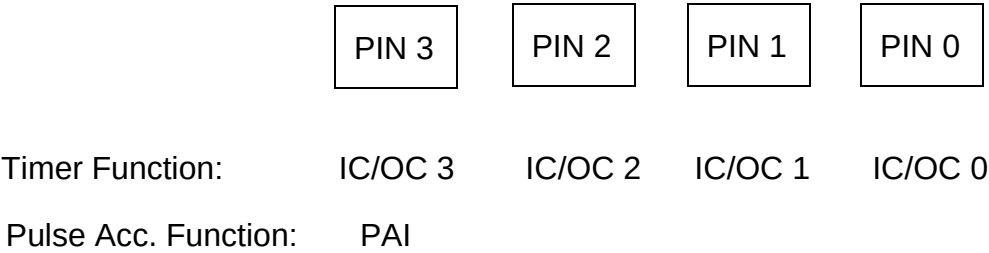


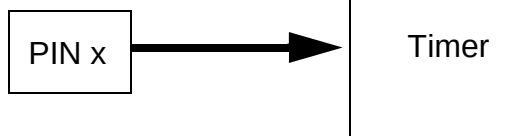
Figure 13-27 Timer Pin Functions

Table 13-7 TIMPORT I/O Function

In		Out	
Data Direction Register	Output Compare Action	Reading at Data Bus	Reading at Pin
0	0	Pin	Pin
0	1	Pin	Output compare action
1	1	Port data register	Output compare action
1	0	Port data register	Port data register

TIMPORT Data Direction Bit

DDR_x = 0



DDR_x = 1

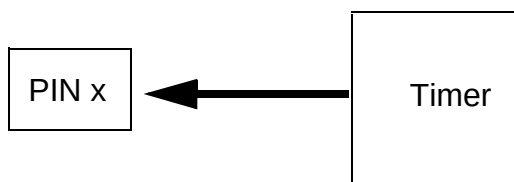


Figure 13-28 Pin Data Direction

13.8 Timer Reset

13.8.1 General

The reset values of registers and signals are provided as described in the Memory Map and Registers, **13.6**, and Signal Descriptions, **13.5**, of this document.

13.8.2 sync Signal Reset

The sync signal can be used to reset the counter at any time when asserted. When the sync signal is deasserted, the counter will begin counting on the next clock cycle.

Timer

13.9 Timer Interrupts

13.9.1 General

This section describes interrupts originated by the timer block. The MCU must service the interrupt requests. **Table 13-8** lists the interrupts generated by the timer to communicate with the MCU.

Table 13-8 Interrupts

Interrupt	Offset ¹	Vector ¹	Priority ¹	Source	Description
tiimch3int_b	-	-	-	Timer Channel 3	Active low timer channel interrupt 3
timch2int_b	-	-	-	Timer Channel 2	Active low timer channel interrupt 2
timch1int_b	-	-	-	Timer Channel 1	Active low timer channel interrupt 1
timch0int_b	-	-	-	Timer Channel 0	Active low timer channel interrupt 0
timpaovint_b	-	-	-	Pulse Accumulator Overflow	Pulse accumulator overflow interrupt
timpaint_b	-	-	-	Pulse Accumulator Input	Pulse accumulator event interrupt
timovint_b	-	-	-	Timer Overflow	Timer overflow interrupt

NOTES:

1. Chip Dependent

13.9.2 Description of Interrupt Operation

The timer only originates interrupt requests. The following is a description of how the timer makes a request and how the MCU should acknowledge that request. The interrupt vector offset and interrupt number and priority are chip dependent.

13.9.2.1 *timchxint_b*

If the timer channel x is configured as an input capture, an input capture on channel x sets the CxF flag. The CxI bit enables the CxF flag to generate interrupt requests.

If the timer channel x is configured as an output compare, an output compare on channel x sets the CxF flag. The CxI bit enables the CxF flag to generate interrupt requests.

Refer to **Table 13-9** for additional interrupt source related information.

13.9.2.2 Channel Flag Clearing

The timer provides two modes for clearing the timer interrupt flag registers: Normal Flag Clearing and Fast Flag Clearing.

- Normal Flag Clearing

Writing logic 1's to the flags clears them only when the TFFCA bit in the Timer System Control Register (TIMSCR1) is clear.

- Fast Flag Clearing

If the TFFCA bit in the Timer System Control Register (TIMSCR1) is set, an input capture read or a write to an output compare channel clears the corresponding channel flag, CxF. Fast Flag Clearing enables fast clearing of the main timer interrupt flag register thus eliminating the software overhead of a separate clear sequence.

NOTE:

Channel flags when set, do not inhibit future output compares or input captures

Refer to **13.6**, Memory Map and Registers, of this document as necessary.

13.9.2.3 timpaovint_b

The pulse accumulator overflow flag, PAOVF, is set when the pulse accumulator rolls over from \$FFFF to \$0000. The pulse accumulator overflow interrupt enable bit, PAOVI, enables the PAOVF flag to generate interrupt requests.

Refer to **Table 13-9** for additional interrupt source related information.

13.9.2.4 Pulse Accumulator Overflow Flag Clearing

The timer provides two modes for clearing the timer interrupt flag registers: Normal Flag Clearing and Fast Flag Clearing.

- Normal Flag Clearing

Writing logic 1's to the flags clears them only when the TFFCA bit in the Timer System Control Register (TIMSCR1) is clear.

- Fast Flag Clearing

Timer

If the TFFCA bit in the Timer System Control Register (TIMSCR1) is set, any access of the pulse accumulator counter registers (TIMPACNT) clears both the PAOVF and PAIF flags in the TIMPAFLG register. Fast Flag Clearing enables fast clearing of the main timer interrupt flag register thus eliminating the software overhead of a separate clear sequence.

Refer to **13.6**, Memory Map and Registers, of this document as necessary.

13.9.2.5 *timpaint_b*

An active edge on the PAI pin sets the pulse accumulator input flag, PAIF. The pulse accumulator input interrupt enable bit, PAI, enables the PAIF flag to generate interrupt requests.

Refer to **Table 13-9** for additional interrupt source related information.

13.9.2.6 *Pulse Accumulator Input Flag Clearing*

The timer provides two modes for clearing the timer interrupt flag registers: Normal Flag Clearing and Fast Flag Clearing.

- Normal Flag Clearing

Writing logic 1's to the flags clears them only when the TFFCA bit in the Timer System Control Register (TIMSCR1) is clear.

- Fast Flag Clearing

If the TFFCA bit in the Timer System Control Register (TIMSCR1) is set, any access of the pulse accumulator counter registers (TIMPACNT) clears both the PAOVF and PAIF flags in the TIMPAFLG register. Fast Flag Clearing enables fast clearing of the main timer interrupt flag register thus eliminating the software overhead of a separate clear sequence.

Refer to **13.6**, Memory Map and Registers, of this document as necessary.

13.9.2.7 *timovint_b*

The timer overflow flag, TOF, is set when the timer rolls over from \$FFFF to \$0000. The timer overflow interrupt enable bit, TOI, enables the TOF flag to generate interrupt requests.

Refer to **Table 13-9** for additional interrupt source related information.

13.9.2.8 Timer Overflow Flag Clearing

The timer provides two modes for clearing the timer interrupt flag registers: Normal Flag Clearing and Fast Flag Clearing.

- Normal Flag Clearing

Writing logic 1's to the flags clears them only when the TFFCA bit in the Timer System Control Register (TIMSCR1) is clear.

- Fast Flag Clearing

If the TFFCA bit in the Timer System Control Register (TIMSCR1) is set, any access of the timer count registers (TIMCNTH/L) clears the TOF flag. Fast Flag Clearing enables fast clearing of the main timer interrupt flag register thus eliminating the software overhead of a separate clear sequence.

Refer to **13.6**, Memory Map and Registers, of this document as necessary.

NOTE:

When the timer channel 3 registers contain \$FFFF and TCRE is set, TOF never gets set even though the timer counter registers go from \$FFFF to \$0000.

NOTE:

When the fast flag clear all bit, TFFCA, is set, any access to the timer counter registers clears timer flag register 2. The TFFCA bit is in timer system control register 1 (TIMSCR1).

TOF, when set, does not inhibit future overflow events.

Table 13-9 Timer Interrupt Sources

Interrupt Source	Flag	Local Enable	Global (CCR) Mask
Timer Channel 0	C0F	C0I	I bit
Timer Channel 1	C1F	C1I	I bit
Timer Channel 2	C2F	C2I	I bit
Timer Channel 3	C3F	C3I	I bit
Pulse Accumulator Overflow	PAOVF	PAOVI	I bit
Pulse Accumulator input	PAIF	PAI	I bit
Timer Overflow	TOF	TOI	I bit

Section 14 Time Of Day timer

14.1 Time-of-Day Timer

The time-of-day timer (TOD) is a free-running timer that is clocked by the crystal oscillator at a frequency of 256 Hz (LOW_REFCLK/128). It is comprised of a 32-bit register which counts seconds, and another 32-bit register which holds an 8-bit fraction of a second, where bit 31 is 1/2 of a second and bit 24 is 1/256 of a second.

The TOD has an alarm function which compares the seconds and fraction registers with the seconds alarm and fraction alarm register and issues an interrupt if there is a match.

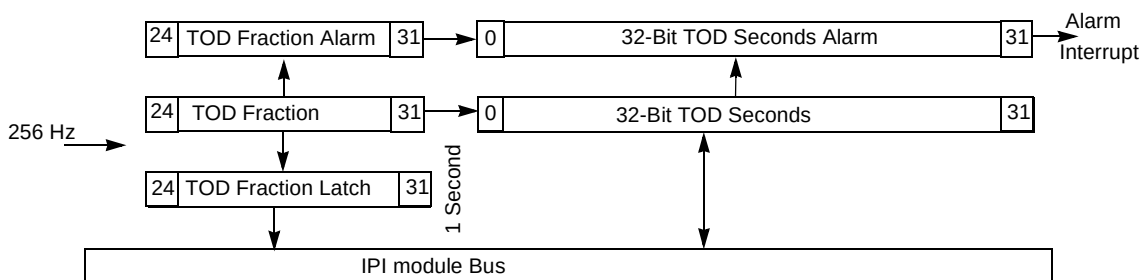


Figure 14-1 TOD Block Diagram

14.1.1 TOD Operation

The TOD seconds register (TODSR) is readable and writable at any time. The TOD fraction register (TODFR) is always cleared to zero when the TODSR is written. Writes to the TODFR cause the fraction register to be set to all ones, regardless of the data written. For read operations, always read the TODSR before the TODFR. This latches the TODFR, which prevents any loss of carry information from the fraction to the seconds registers.

The TOD value is matched every 1/256 second to the alarm register if the alarm function is enabled. When a match occurs, the alarm interrupt flag (AIF) is set, and when the alarm interrupt enable bit (AIE) is set, an interrupt is issued to the CPU. A write to the TOD fraction alarm register (TODFAR) clears the AIF. Writes to the TOD seconds alarm register (TODSAR) do not affect the clearing of the AIF.

The alarm match function is temporarily disabled after a write to the TODSAR until the TODFAR is written. This prevents an alarm match from occurring before the entire 40-bit alarm value is written.

Time Of Day timer

The TOD fraction and seconds counters are undefined after a POR operation and are unaffected by any other reset source. The TOD alarm is disabled by any reset source.

14.1.2 TOD in Low-Power Modes

The TOD is unaffected by the low-power modes. An alarm interrupt may be used to exit from a low power state.

14.1.3 Time-of-Day Control/Status Register (TODCSR)

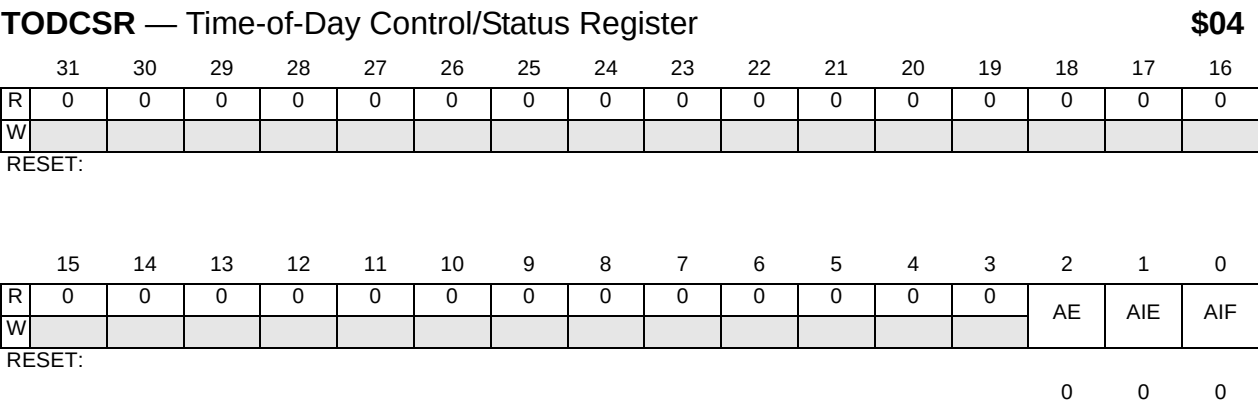


Figure 14-2 TOD Control/Status Register

Access this register with 32-bit loads and stores only.

AE — Alarm Enable

- This bit controls the function of the TOD alarm
- 0 = Alarm function is off to save power
- 1 = Alarm function is on

AIE — Alarm Interrupt Enable

- This bit controls the alarm interrupt function
- 0 = AIF is inhibited from reaching the CPU
- 1 = AIF is allowed to request an interrupt

AIF — Alarm Interrupt Flag

- This read-only bit is the alarm interrupt flag. It is cleared by writing to the TOD alarm fraction register (TODFAR).
- 0 = No alarm interrupt is present
- 1 = Alarm interrupt is present

14.1.4 TOD Seconds Register (TODSR)

The time-of-day seconds register is a 32-bit read/write register that holds the number of elapsed seconds. It is clocked by a 1-Hz signal generated as a carry from the TOD fraction register. When TODSR is read, the content of the fraction counter is latched into a holding buffer to be read later. This prevents a fraction rollover between reads of the two registers from causing incorrect data to be read. When TODSR is written, the TODFR is cleared to all zeros. TODSR is not affected by the watchdog reset or by a reset initiated by the external reset signal, but is undefined after a POR.

NOTE:

Actual write to TODSR is occurred right after the positive edge of the 32KHz clock to avoid timing confliction between CPU write and count up of the TODSR counter.

Access this register with 32-bit loads and stores only.

TODSR — Time-of-Day Seconds Register

\$08



Figure 14-3 TOD Seconds Register

14.1.5 TOD Fraction Register (TODFR)

The 32-bit time-of-day fraction register holds eight bits of data that represent the binary fraction of a second. It is clocked by the 32.768-kHz LOW_REFCLK divided by 128 (256 Hz). Reads of this register return the value latched when the TOD seconds register was previously read. Continuous reads return the same value until the TODSR is read and new data is latched from the fraction counter. These eight bits are cleared whenever the TODSR is written but are not affected by either reset pin or the watchdog reset conditions. The fraction counter is undefined after a POR. Writes to this register cause all eight bits to be set.

NOTE:

Actual set or reset of the TODFR is occurred right after the positive edge of 32KHz clock to avoid confliction between set/reset and count up of TODFR.

Access this register with 32-bit accesses only.

Time Of Day timer

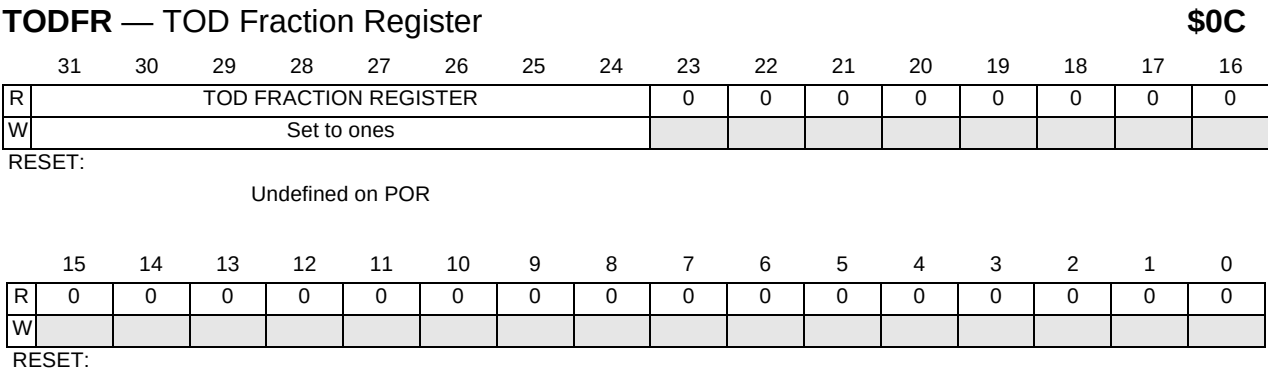


Figure 14-4 TOD Fraction Register

14.1.6 TOD Seconds Alarm Register (TODSAR)

The time-of-day seconds alarm register is a 32-bit read/write register which holds data (in seconds) to be compared to the TOD seconds register. The comparison is made every 1/256 of a second if the alarm function is enabled by the AE bit in the TODCSR. Writes to this register inhibit alarm compares until the TODFAR is written. For proper alarm operation, the fraction alarm register must be (re)written after a write to this register. This register is not affected by any of the reset conditions.

Access this register with 32-bit loads and stores only.

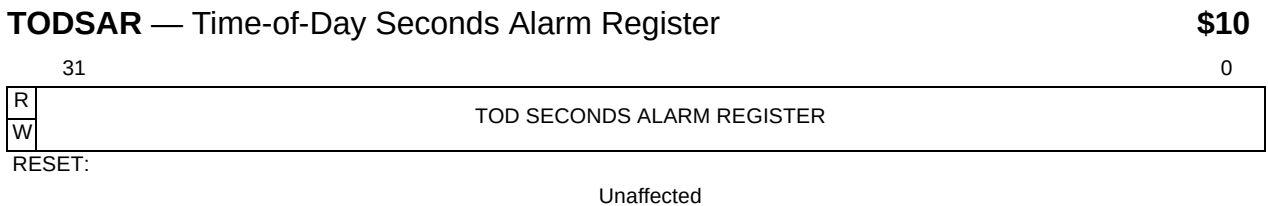


Figure 14-5 TOD Seconds Alarm Register

14.1.7 TOD Fraction Alarm Register (TODFAR)

The time-of-day fraction alarm register is a 32-bit read/write register which holds eight bits of data to be compared to the TOD fraction register. The comparison is made every 1/256 of a second if the alarm function is enabled by the AE bit in the TODCSR. This register is not affected by any of the reset conditions.

Access this register with 32-bit loads and stores only.

TODFAR — TOD Fraction Alarm Register **\$14**

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	TOD FRACTION ALARM REGISTER								0	0	0	0	0	0	0	0
W																
RESET:																
Undefined on POR																
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W																
RESET:																

Figure 14-6 TOD Fraction Alarm Register

Section 15 Serial Communications Interface

15.1 Overview

The SCI allows asynchronous serial communications with peripheral devices and other MCUs.

15.2 Features

- Full-duplex operation
- Standard mark/space non-return-to-zero (NRZ) format
- 13-bit baud rate selection
- Programmable 8-bit or 9-bit data format
- Separately enabled transmitter and receiver
- Separate receiver and transmitter CPU interrupt requests
- Programmable transmitter output polarity
- Two receiver wakeup methods:
 - Idle line wakeup
 - Address mark wakeup
- Interrupt-driven operation with eight flags:
 - Transmitter empty
 - Transmission complete
 - Receiver full
 - Idle receiver input
 - Receiver overrun
 - Noise error
 - Framing error
 - Parity error
- Receiver framing error detection
- Hardware parity checking
- 1/16 bit-time noise detection
- General-purpose, 8-bit I/O (input/output) port

Serial Communications Interface

15.3 Block Diagram

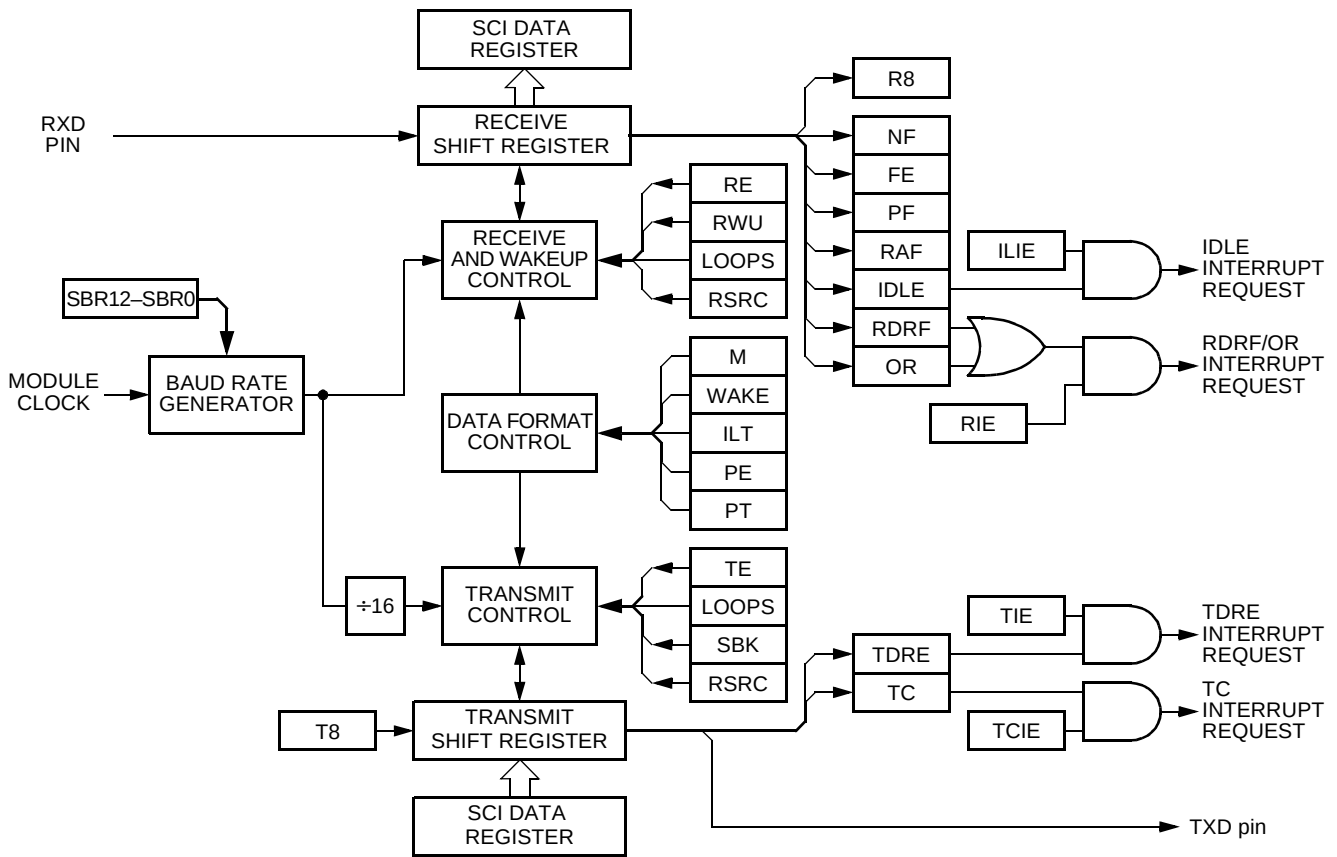


Figure 15-1 . SCI Block Diagram

15.4 Register Map

NOTE:

The address of a register is the sum of a base address and an address offset. The base address is defined at the MCU level and the address offset is defined at the module level.

Register name		Bit 7	6	5	4	3	2	1	Bit 0	Addr. offset
SCIBDH	Read:	0	0	0	SBR12	SBR11	SBR10	SBR9	SBR8	\$__0
	Write:									
SCIBDL	Read:	SBR7	SBR6	SBR5	SBR4	SBR3	SBR2	SBR1	SBR0	\$__1
	Write:									
SCICR1	Read:	LOOPS	WOMS	RSRC	M	WAKE	ILT	PE	PT	\$__2
	Write:									
SCICR2	Read:	TIE	TCIE	RIE	ILIE	TE	RE	RWU	SBK	\$__3
	Write:									
SCISR1	Read:	TDRE	TC	RDRF	IDLE	OR	NF	FE	PF	\$__4
	Write:									
SCISR2	Read:	0	0	0	0	0	0	0	RAF	\$__5
	Write:									
SCIDRH	Read:	R8	T8	0	0	0	0	0	0	\$__6
	Write:									
SCIDRL	Read:	R7T7	R6T6	R5T5	R4T4	R3T3	R2T2	R1T1	R0T0	\$__7
	Write:									
SCIPURD	Read:	SCISWAI	0	RDPSCI1	RDPSCI0	0	0	PUPSCI1	PUPSCI0	\$__8
	Write:									
SCIPORT	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__9
	Write:									
SCIDDR	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__A
	Write:									
Reserved	Read:	0	0	0	0	0	0	0	0	\$__B
	Write:									

 = Reserved or unimplemented

Figure 15-2 . SCI I/O Register Summary

Serial Communications Interface

NOTE:
In normal modes, writing to a reserved bit has no effect and reading returns logic 0.
In any mode, writing to an unimplemented bit has no effect and reading returns a logic 0.

15.5 Functional Description

Figure 15-1 shows the structure of the SCI module. The SCI allows full duplex, asynchronous, NRZ serial communication between the MCU and remote devices, including other MCUs. The SCI transmitter and receiver operate independently, although they use the same baud rate generator. The CPU monitors the status of the SCI, writes the data to be transmitted, and processes received data.

15.5.1 Data Format

The SCI uses the standard NRZ mark/space data format illustrated in **Figure 15-3**.

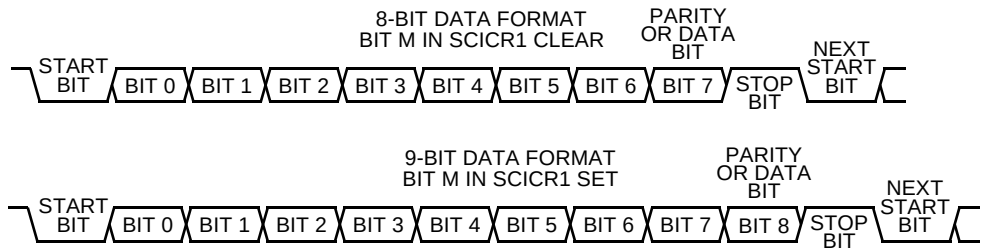


Figure 15-3 . SCI Data Formats

Each data character is contained in a frame that includes a start bit, eight or nine data bits, and a stop bit. Clearing the M bit in SCI control register 1 configures the SCI for 8-bit data characters. A frame with eight data bits has a total of 10 bits. Setting the M bit configures the SCI for nine-bit data characters. A frame with nine data bits has a total of 11 bits.

Table 15-1 . Example 8-Bit Data Formats

Start Bit	Data Bits	Address Bit	Parity Bit	Stop Bit
1	8	0	0	1
1	7	0		2
1	7	0	1	1
1	7	1 ¹		1

NOTES:

1. The address bit identifies the frame as an address character. See **15.5.4.6 Receiver Wakeup**.

Setting the M bit configures the SCI for 9-bit data characters. The ninth data bit is the T8 bit in SCI data register high (SCIDRH). It remains unchanged after transmission and can be used repeatedly without rewriting it. A frame with nine data bits has a total of 11 bits.

Table 15-2 . Example 9-Bit Data Formats

Start Bit	Data Bits	Address Bit	Parity Bit	Stop Bit
1	9	0	0	1
1	8	0		2
1	8	0	1	1
1	8	1 ¹		1

NOTES:

1. The address bit identifies the frame as an address character. See **15.5.4.6 Receiver Wakeup**.

15.5.2 Baud Rate Generation

A 13-bit modulus counter in the baud rate generator derives the baud rate for both the receiver and the transmitter. The value from 0 to 8191 written to the SBR12–SBR0 bits determines the module clock divisor. The SBR bits are in the SCI baud rate registers (SCIBRH and SCIBRL). The baud rate clock is synchronized with the bus clock and drives the receiver. The baud rate clock divided by 16 drives the transmitter. The receiver has an acquisition rate of 16 samples per bit time.

Baud rate generation is subject to two sources of error:

- Integer division of the module clock may not give the exact target frequency.
- Synchronization with the bus clock can cause phase shift.

Table 15-3 lists some examples of achieving target baud rates with a module clock frequency of 10.2 MHz.

Table 15-3 . Baud Rates (Module Clock = 10.2 Mhz)

Bits SBR[12–0]	Receiver Clock (Hz)	Transmitter Clock (Hz)	Target Baud Rate	Error (%)
17	600,000.0	37,500.0	38,400	2.3
33	309,090.9	19,318.2	19,200	0.62
66	154,545.5	9659.1	9600	0.62
133	76,691.7	4793.2	4800	0.14
266	38,345.9	2396.6	2400	0.14
531	19,209.0	1200.6	1200	0.11
1062	9604.5	600.3	600	0.05
2125	4800.0	300.0	300	0.00
4250	2400.0	150.0	150	0.00
5795	1760.1	110.0	110	0.00

15.5.3 Transmitter

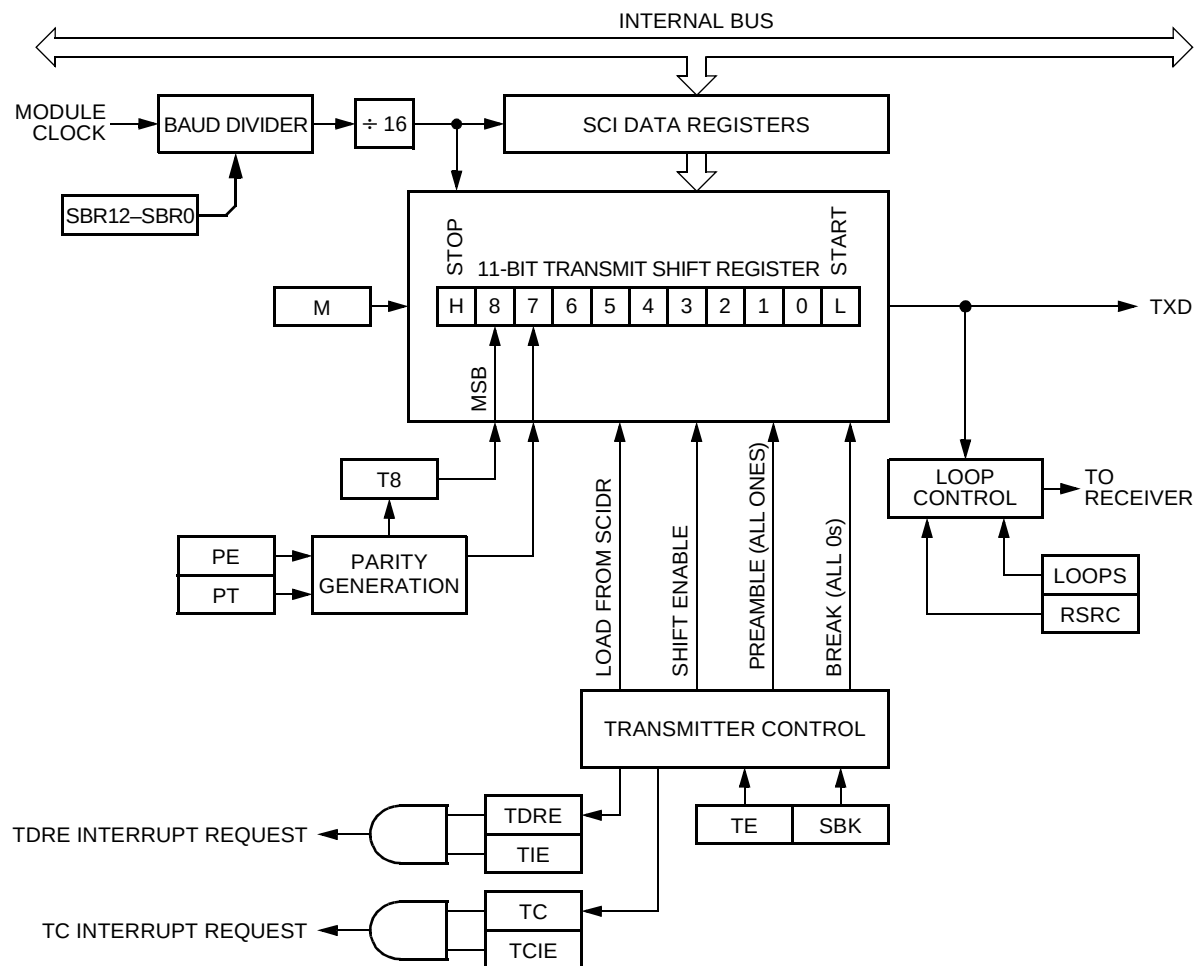


Figure 15-4 . SCI Transmitter Block Diagram

15.5.3.1 Character Length

The SCI transmitter can accommodate either 8-bit or 9-bit data characters. The state of the M bit in SCI control register 1 (SCICR1) determines the length of data characters. When transmitting 9-bit data, bit T8 in SCI data register high (SCIDRH) is the ninth bit (bit 8).

15.5.3.2 Character Transmission

During an SCI transmission, the transmit shift register shifts a frame out to the TXD pin. The SCI data registers (SCIDRH and SCIDRL) are the write-only buffers between the internal data bus and the transmit shift register.

Serial Communications Interface

To initiate an SCI transmission:

1. Enable the transmitter by writing a logic 1 to the transmitter enable bit, TE, in SCI control register 2 (SCICR2).
2. Clear the transmit data register empty flag, TDRE, by first reading SCI status register 1 (SCISR1) and then writing to SCI data register low (SCIDRL). In 9-data-bit format, write the ninth bit to the T8 bit in SCI data register high (SCIDRH).
3. Repeat step 2 for each subsequent transmission.

Writing the TE bit from 0 to a 1 automatically loads the transmit shift register with a preamble of 10 logic 1s (if M = 0) or 11 logic 1s (if M = 1). After the preamble shifts out, control logic transfers the data from the SCI data register into the transmit shift register. A logic 0 start bit automatically goes into the least significant bit position of the transmit shift register. A logic 1 stop bit goes into the most significant bit position.

Hardware supports odd or even parity. When parity is enabled, the most significant bit (msb) of the data character is the parity bit.

The transmit data register empty flag, TDRE, in SCI status register 1 (SCISR1) becomes set when the SCI data register transfers a byte to the transmit shift register. The TDRE flag indicates that the SCI data register can accept new data from the internal data bus. If the transmit interrupt enable bit, TIE, in SCI control register 2 (SCICR2) is also set, the TDRE flag generates a transmitter interrupt request.

When the transmit shift register is not transmitting a frame, the TXD pin goes to the idle condition, logic 1. If at any time software clears the TE bit in SCI control register 2 (SCICR2), the transmitter and receiver relinquish control of the port I/O pins.

If software clears TE while a transmission is in progress (TC = 0), the frame in the transmit shift register continues to shift out. Then the TXD pin reverts to being a general-purpose I/O pin even if there is data pending in the SCI data register. To avoid accidentally cutting off the last frame in a message, always wait for TDRE to go high after the last frame before clearing TE.

To separate messages with preambles with minimum idle line time, use this sequence between messages:

1. Write the last byte of the first message to SCIDRH/L.
2. Wait for the TDRE flag to go high, indicating the transfer of the last frame to the transmit shift register.
3. Queue a preamble by clearing and then setting the TE bit.
4. Write the first byte of the second message to SCIDRH/L.

When the SCI relinquishes the TXD pin, the SCI_{PORT} and SCI_{DDR} registers control the TXD pin.

To force TXD high when turning off the transmitter, set bit 1 of the SCI port register (SCI_{PORT}) and bit 1 of the SCI data direction register (SCID_{DDR}). The TXD pin goes high as soon as the SCI relinquishes it.

15.5.3.3 Break Characters

Writing a logic 1 to the send break bit, SBK, in SCI control register 2 (SCIC_{R2}) loads the transmit shift register with a break character. A break character contains all logic 0s and has no start, stop, or parity bit. Break character length depends on the M bit in SCI control register 1 (SCIC_{R1}). As long as SBK is at logic 1, transmitter logic continuously loads break characters into the transmit shift register. After software clears the SBK bit, the shift register finishes transmitting the last break character and then transmits at least one logic 1. The automatic logic 1 at the end of a break character guarantees the recognition of the start bit of the next frame.

The SCI recognizes a break character when a start bit is followed by eight or nine logic 0 data bits and a logic 0 where the stop bit should be. Receiving a break character has these effects on SCI registers:

- Sets the framing error flag, FE
- Sets the receive data register full flag, RDRF
- Clears the SCI data registers (SCID_{RH/L})
- May set the overrun flag, OR, noise flag, NF, parity error flag, PE, or the receiver active flag, RAF (see **15.6.4 SCI Status Register 1**)

15.5.3.4 Idle Characters

An idle character contains all logic 1s and has no start, stop, or parity bit. Idle character length depends on the M bit in SCI control register 1 (SCIC_{R1}). The preamble is a synchronizing idle character that begins the first transmission initiated after writing the TE bit from 0 to 1.

If the TE bit is cleared during a transmission, the TXD pin becomes idle after completion of the transmission in progress. Clearing and then setting the TE bit during a transmission queues an idle character to be sent after the frame currently being transmitted.

NOTE:

When queueing an idle character, return the TE bit to logic 1 before the stop bit of the current frame shifts out to the TXD pin. Setting TE after the stop bit appears on TXD causes data previously written to the SCI data register to be lost.

Toggle the TE bit for a queued idle character when the TDRE flag becomes set and immediately before writing the next byte to the SCI data register.

Serial Communications Interface

15.5.4 Receiver

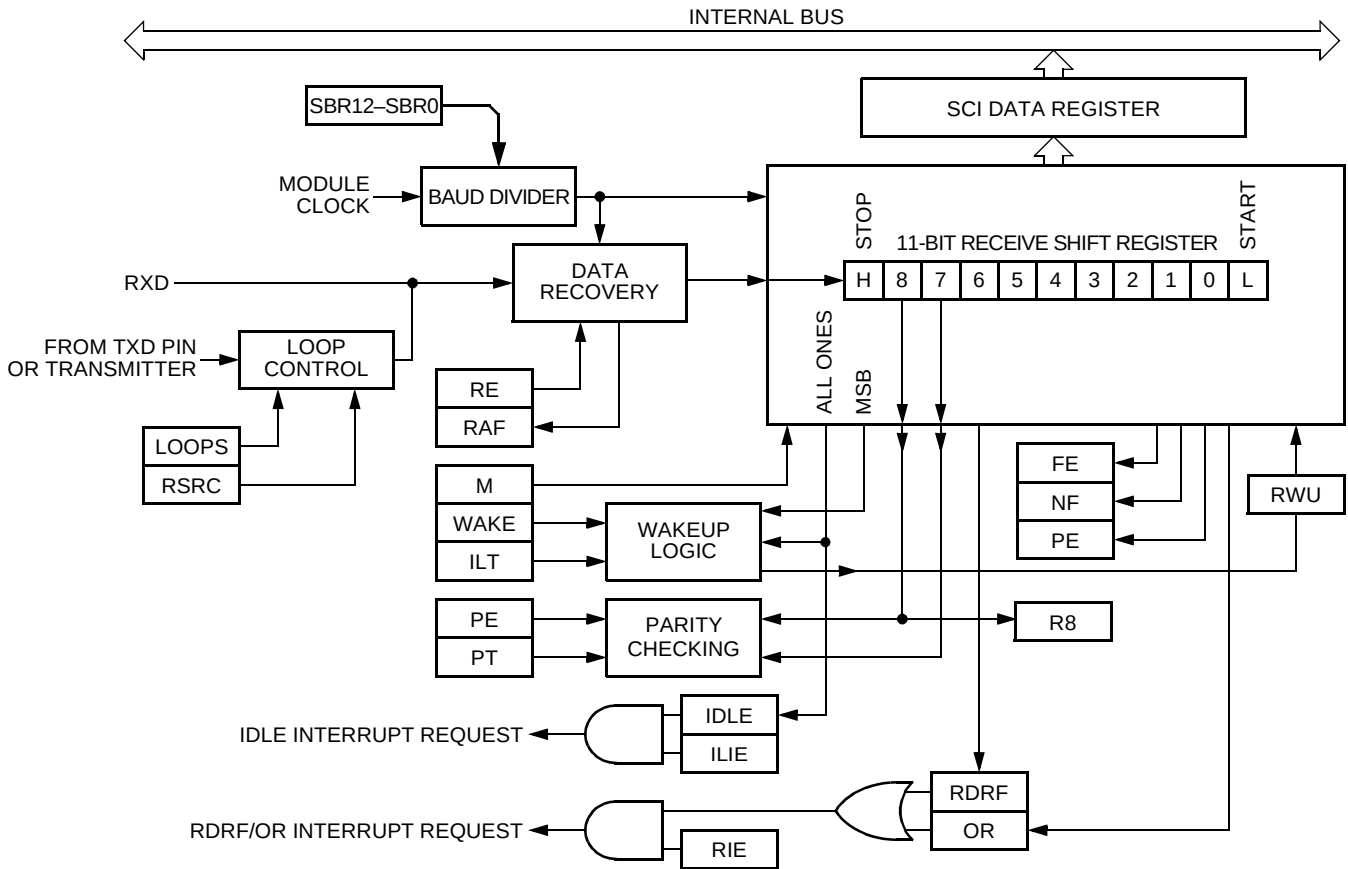


Figure 15-5 . SCI Receiver Block Diagram

15.5.4.1 Character Length

The SCI receiver can accommodate either 8-bit or 9-bit data characters. The state of the **M** bit in SCI control register 1 (SCICR1) determines the length of data characters. When receiving 9-bit data, bit **R8** in SCI data register high (SCIDRH) is the ninth bit (bit 8).

15.5.4.2 Character Reception

During an SCI reception, the receive shift register shifts a frame in from the **RXD** pin. The SCI data register is the read-only buffer between the internal data bus and the receive shift register.

After a complete frame shifts into the receive shift register, the data portion of the frame transfers to the SCI data register. The receive data register full flag, **RDRF**, in SCI status register 1 (SCISR1) becomes set, indicating that the received byte can be read. If the receive interrupt

enable bit, RIE, in SCI control register 2 (SCICR2) is also set, the RDRF flag generates an RDRF interrupt request.

15.5.4.3 Data Sampling

The receiver samples the RXD pin at the RT clock rate. The RT clock is an internal signal with a frequency 16 times the baud rate. To adjust for baud rate mismatch, the RT clock (see **Figure 15-6**) is resynchronized:

- After every start bit
- After the receiver detects a data bit change from logic 1 to logic 0 (after the majority of data bit samples at RT8, RT9, and RT10 returns a valid logic 1 and the majority of the next RT8, RT9, and RT10 samples returns a valid logic 0)

To locate the start bit, data recovery logic does an asynchronous search for a logic 0 preceded by three logic 1s. When the falling edge of a possible start bit occurs, the RT clock begins to count to 16.

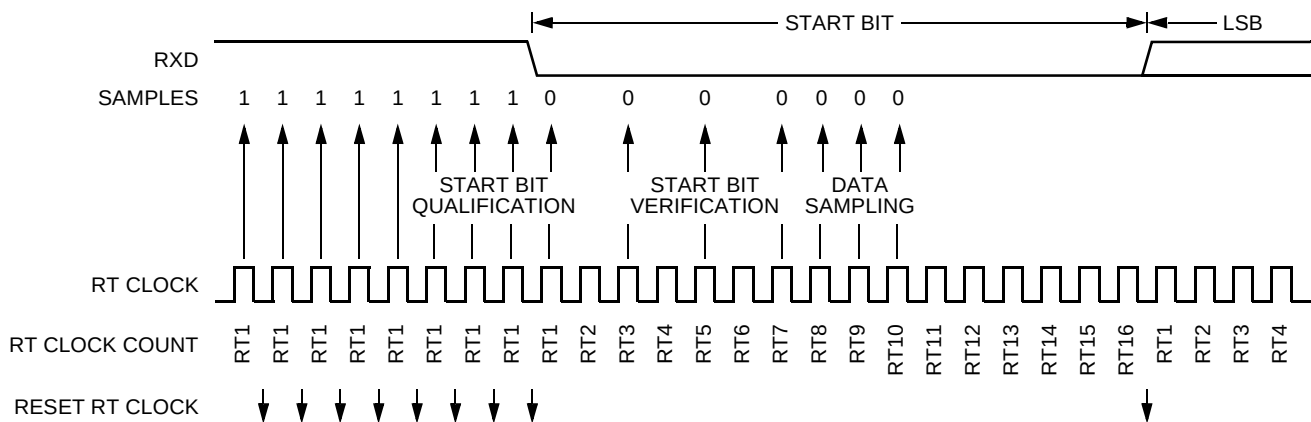


Figure 15-6 . Receiver Data Sampling

To verify the start bit and to detect noise, data recovery logic takes samples at RT3, RT5, and RT7. **Table 15-4** summarizes the results of the start bit verification samples.

Table 15-4 . Start Bit Verification

RT3, RT5, and RT7 Samples	Start Bit Verification	Noise Flag
000	Yes	0
001	Yes	1
010	Yes	1
011	No	0
100	Yes	1
101	No	0
110	No	0
111	No	0

If start bit verification is not successful, the RT clock is reset and a new search for a start bit begins.

To determine the value of a data bit and to detect noise, recovery logic takes samples at RT8, RT9, and RT10. **Table 15-5** summarizes the results of the data bit samples.

Table 15-5 . Data Bit Recovery

RT8, RT9, and RT10 Samples	Data Bit Determination	Noise Flag
000	0	0
001	0	1
010	0	1
011	1	1
100	0	1
101	1	1
110	1	1
111	1	0

NOTE:

The RT8, RT9, and RT10 samples do not affect start bit verification. If any or all of the RT8, RT9, and RT10 start bit samples are logic 1s following a successful start bit verification, the noise flag (NF) is set and the receiver assumes that the bit is a start bit (logic 0).

To verify a stop bit and to detect noise, recovery logic takes samples at RT8, RT9, and RT10. **Table 15-6** summarizes the results of the stop bit samples.

Table 15-6 . Stop Bit Recovery

RT8, RT9, and RT10 Samples	Framing Error Flag	Noise Flag
000	1	0
001	1	1
010	1	1
011	0	1
100	1	1
101	0	1
110	0	1
111	0	0

In **Figure 15-7** the verification samples RT3 and RT5 determine that the first low detected was noise and not the beginning of a start bit. The RT clock is reset and the start bit search begins again. The noise flag is not set because the noise occurred before the start bit was found.

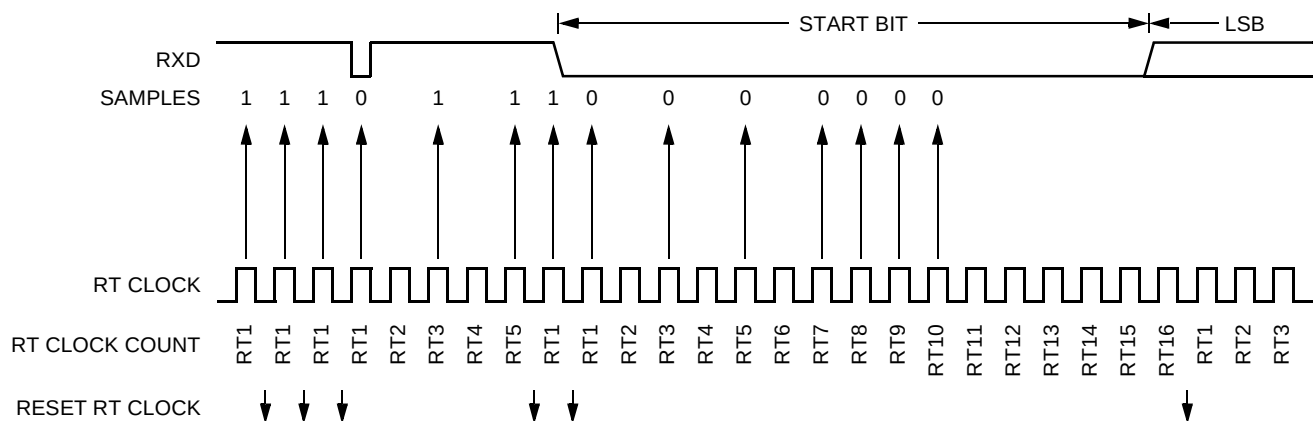


Figure 15-7 . Start Bit Search Example 1

In **Figure 15-8** noise is perceived as the beginning of a start bit although the verification sample at RT3 is high. The RT3 sample sets the noise flag. Although the perceived bit time is misaligned, the data samples RT8, RT9, and RT10 are within the bit time and data recovery is successful.

Serial Communications Interface

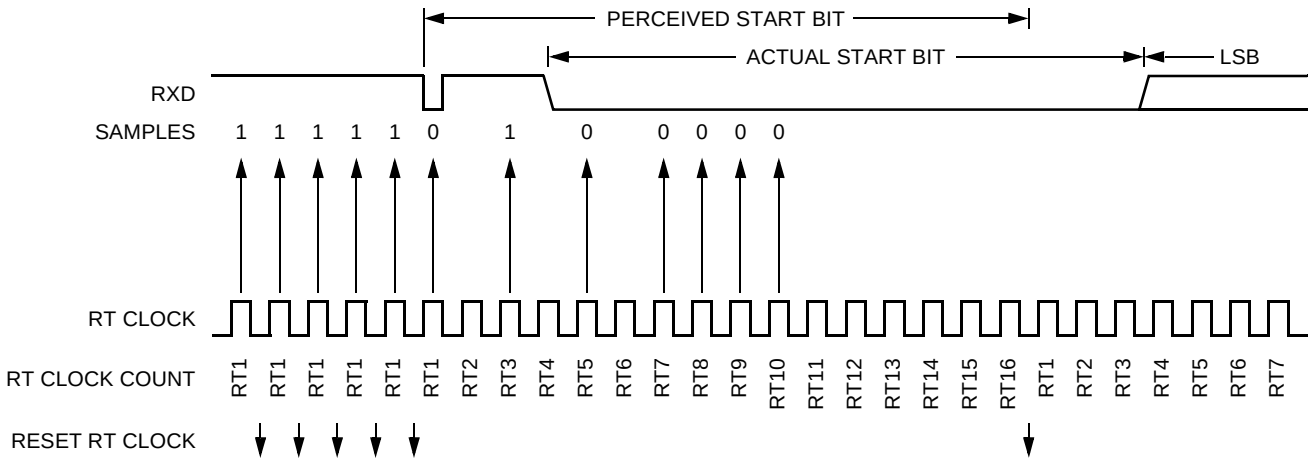


Figure 15-8 . Start Bit Search Example 2

In **Figure 15-9** a large burst of noise is perceived as the beginning of a start bit, although the test sample at RT5 is high. The RT5 sample sets the noise flag. Although this is a worst-case misalignment of perceived bit time, the data samples RT8, RT9, and RT10 are within the bit time and data recovery is successful.

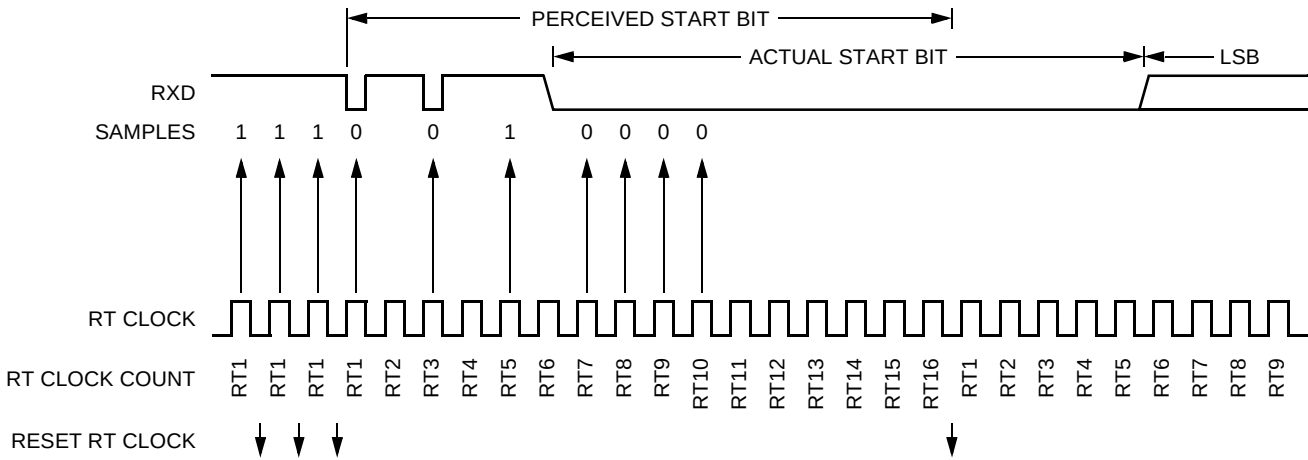


Figure 15-9 . Start Bit Search Example 3

Figure 15-10 shows the effect of noise early in the start bit time. Although this noise does not affect proper synchronization with the start bit time, it does set the noise flag.

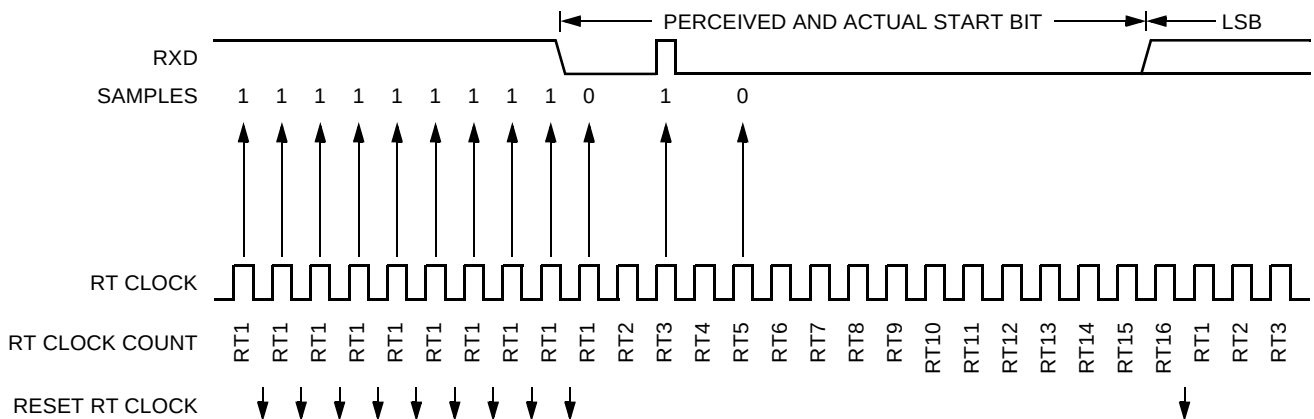


Figure 15-10 . Start Bit Search Example 4

Figure 15-11 shows a burst of noise near the beginning of the start bit that resets the RT clock. The sample after the reset is low but is not preceded by three high samples that would qualify as a falling edge. Depending on the timing of the start bit search and on the data, the frame may be missed entirely or it may set the framing error flag.

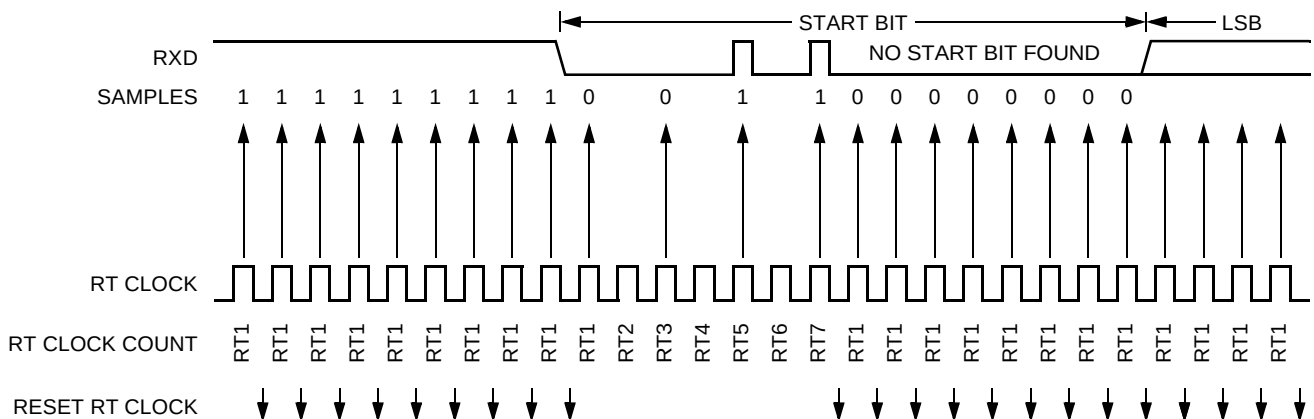


Figure 15-11 . Start Bit Search Example 5

In **Figure 15-12** a noise burst makes the majority of data samples RT8, RT9, and RT10 high. This sets the noise flag but does not reset the RT clock. In start bits only, the RT8, RT9, and RT10 data samples are ignored.

Serial Communications Interface

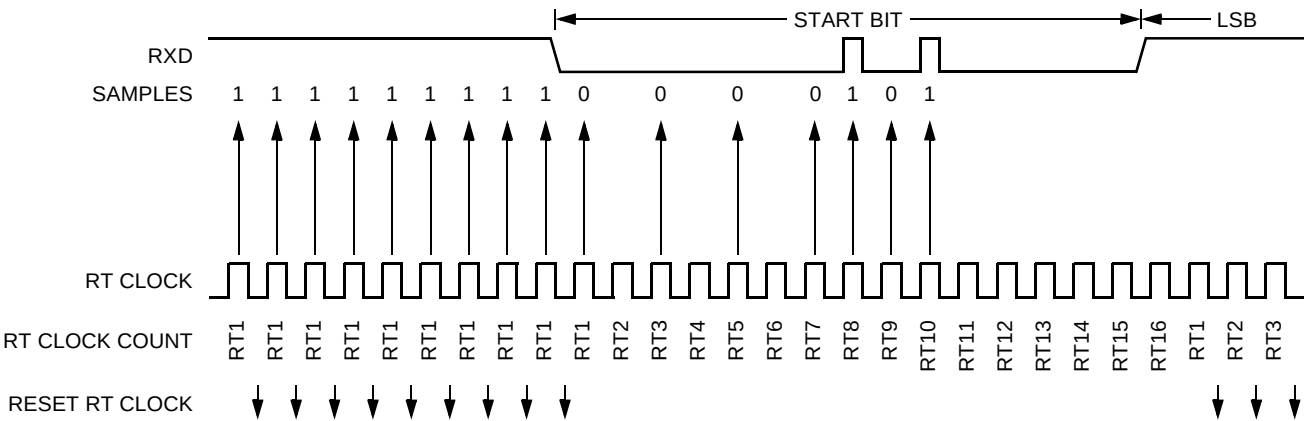


Figure 15-12 . Start Bit Search Example 6

15.5.4.4 Framing Errors

If the data recovery logic does not detect a logic 1 where the stop bit should be in an incoming frame, it sets the framing error flag, FE, in SCI status register 1 (SCISR1). A break character also sets the FE flag because a break character has no stop bit. The FE flag is set at the same time that the RDRF flag is set.

15.5.4.5 Baud Rate Tolerance

A transmitting device may be operating at a baud rate below or above the receiver baud rate. Accumulated bit time misalignment can cause one of the three stop bit data samples to fall outside the actual stop bit. Then a noise error occurs. If more than one of the samples is outside the stop bit, a framing error occurs. In most applications, the baud rate tolerance is much more than the degree of misalignment that is likely to occur.

As the receiver samples an incoming frame, it resynchronizes the RT clock on any valid falling edge within the frame. Resynchronization within frames corrects misalignments between transmitter bit times and receiver bit times.

Slow Data Tolerance

Figure 15-13 shows how much a slow received frame can be misaligned without causing a noise error or a framing error. The slow stop bit begins at RT8 instead of RT1 but arrives in time for the stop bit data samples at RT8, RT9, and RT10.

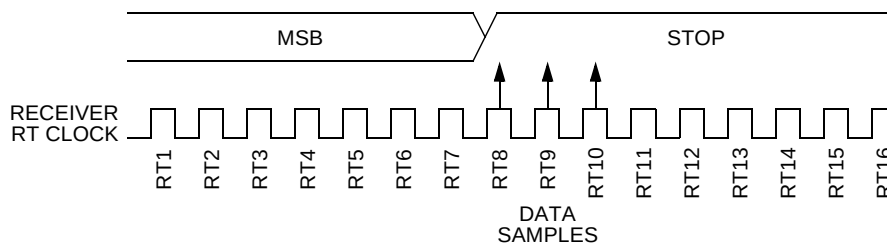


Figure 15-13 . Slow Data

For an 8-bit data character, data sampling of the stop bit takes the receiver 9 bit times $\times$ 16 RT cycles + 10 RT cycles = 154 RT cycles.

With the misaligned character shown in **Figure 15-13**, the receiver counts 154 RT cycles at the point when the count of the transmitting device is 9 bit times $\times$ 16 RT cycles + 3 RT cycles = 147 RT cycles.

The maximum percent difference between the receiver count and the transmitter count of a slow 8-bit data character with no errors is:

$$\left| \frac{154 - 147}{154} \right| \times 100 = 4.54\%$$

For a 9-bit data character, data sampling of the stop bit takes the receiver 10 bit times $\times$ 16 RT cycles + 10 RT cycles = 170 RT cycles.

With the misaligned character shown in **Figure 15-13**, the receiver counts 170 RT cycles at the point when the count of the transmitting device is 10 bit times $\times$ 16 RT cycles + 3 RT cycles = 163 RT cycles.

The maximum percent difference between the receiver count and the transmitter count of a slow 9-bit character with no errors is:

$$\left| \frac{170 - 163}{170} \right| \times 100 = 4.12\%$$

Fast Data Tolerance

Figure 15-14 shows how much a fast received frame can be misaligned without causing a noise error or a framing error. The fast stop bit ends at RT10 instead of RT16 but is still sampled at RT8, RT9, and RT10.

Serial Communications Interface

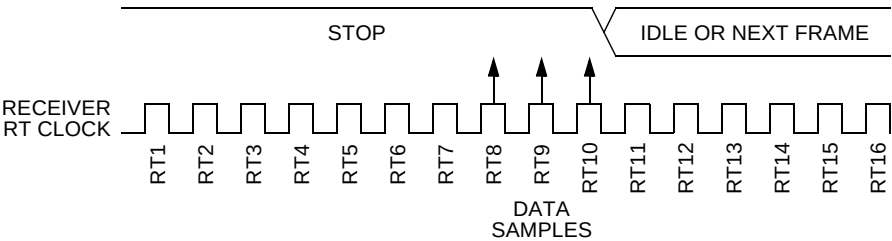


Figure 15-14 . Fast Data

For an 8-bit data character, data sampling of the stop bit takes the receiver 9 bit times $\times$ 16 RT cycles + 10 RT cycles = 154 RT cycles.

With the misaligned character shown in **Figure 15-14**, the receiver counts 154 RT cycles at the point when the count of the transmitting device is 10 bit times $\times$ 16 RT cycles = 160 RT cycles.

The maximum percent difference between the receiver count and the transmitter count of a fast 8-bit character with no errors is:

$$\left| \frac{154 - 160}{154} \right| \times 100 = 3.90\%$$

For a 9-bit data character, data sampling of the stop bit takes the receiver 10 bit times $\times$ 16 RT cycles + 10 RT cycles = 170 RT cycles.

With the misaligned character shown in **Figure 15-14**, the receiver counts 170 RT cycles at the point when the count of the transmitting device is 11 bit times $\times$ 16 RT cycles = 176 RT cycles.

The maximum percent difference between the receiver count and the transmitter count of a fast 9-bit character with no errors is:

$$\left| \frac{170 - 176}{170} \right| \times 100 = 3.53\%$$

15.5.4.6 Receiver Wakeup

So that the SCI can ignore transmissions intended only for other receivers in multiple-receiver systems, the receiver can be put into a standby state. Setting the receiver wakeup bit, RWU, in SCI control register 2 (SCICR2) puts the receiver into a standby state during which receiver interrupts are disabled.

The transmitting device can address messages to selected receivers by including addressing information in the initial frame or frames of each message.

The WAKE bit in SCI control register 1 (SCICR1) determines how the SCI is brought out of the standby state to process an incoming message. The WAKE bit enables either idle line wakeup or address mark wakeup:

- Idle input line wakeup (WAKE = 0) — In this wakeup method, an idle condition on the RXD pin clears the RWU bit and wakes up the SCI. The initial frame or frames of every message contain addressing information. All receivers evaluate the addressing information, and receivers for which the message is addressed process the frames that follow. Any receiver for which a message is not addressed can set its RWU bit and return to the standby state. The RWU bit remains set and the receiver remains on standby until another idle character appears on the RXD pin.

Idle line wakeup requires that messages be separated by at least one idle character and that no message contains idle characters.

The idle character that wakes a receiver does not set the receiver idle bit, IDLE, or the receive data register full flag, RDRF.

The idle line type bit, ILT, determines whether the receiver begins counting logic 1s as idle character bits after the start bit or after the stop bit. ILT is in SCI control register 1 (SCICR1).

- Address mark wakeup (WAKE = 1) — In this wakeup method, a logic 1 in the most significant bit (msb) position of a frame clears the RWU bit and wakes up the SCI. The logic 1 in the msb position marks a frame as an address frame that contains addressing information. All receivers evaluate the addressing information, and the receivers for which the message is addressed process the frames that follow. Any receiver for which a message is not addressed can set its RWU bit and return to the standby state. The RWU bit remains set and the receiver remains on standby until another address frame appears on the RXD pin.

The logic 1 msb of an address frame clears the receiver's RWU bit before the stop bit is received and sets the RDRF flag.

Address mark wakeup allows messages to contain idle characters but requires that the msb be reserved for use in address frames.

NOTE:

With the WAKE bit clear, setting the RWU bit after the RXD pin has been idle can cause the receiver to wake up immediately.

15.5.5 Single-Wire Operation

Normally, the SCI uses two pins for transmitting and receiving. In single-wire operation, the RXD pin is disconnected from the SCI and is available as a general-purpose I/O pin. The SCI uses the TXD pin for both receiving and transmitting.

Serial Communications Interface

Setting the data direction bit for the TXD pin configures TXD as the output for transmitted data. Clearing the data direction bit configures TXD as the input for received data.

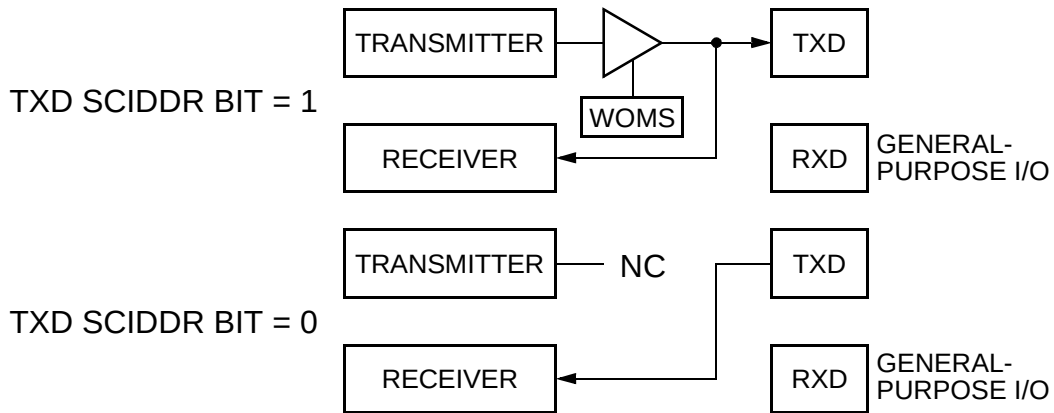


Figure 15-15 . Single-Wire Operation (LOOPS = 1, RSRC = 1)

Enable single-wire operation by setting the LOOPS bit and the receiver source bit, RSRC, in SCI control register 1 (SCICR1). Setting the LOOPS bit disables the path from the RXD pin to the receiver. Setting the RSRC bit connects the receiver input to the output of the TXD pin driver. Both the transmitter and receiver must be enabled (TE = 1 and RE = 1).

The wired-OR mode select bit, WOMS, configures the TXD pin for full CMOS drive or for open-drain drive. WOMS controls the TXD pin in both normal operation and in single-wire operation. When WOMS is set, the DDR bit for the TXD pin does not have to be cleared for TXD to receive data.

15.5.6 Loop Operation

In loop operation the transmitter output goes to the receiver input. The RXD pin is disconnected from the SCI and is available as a general-purpose I/O pin.

Setting the data direction bit for the TXD pin connects the transmitter output to the TXD pin. Clearing the data direction bit disconnects the transmitter output from the TXD pin.

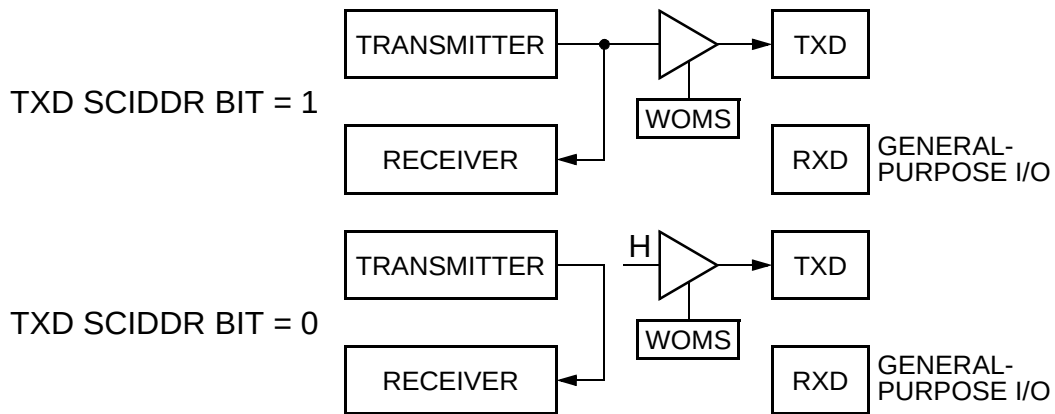


Figure 15-16 . Loop Operation (LOOPS = 1, RSRC = 0)

Enable loop operation by setting the LOOPS bit and clearing the RSRC bit in SCI control register 1 (SCICR1). Setting the LOOPS bit disables the path from the RXD pin to the receiver. Clearing the RSRC bit connects the transmitter output to the receiver input. Both the transmitter and receiver must be enabled (TE = 1 and RE = 1).

The wired-OR mode select bit, WOMS, configures the TXD pin for full CMOS drive or for open-drain drive. WOMS controls the TXD pin in both normal operation and in loop operation.

15.6 Register Descriptions

15.6.1 SCI Baud Rate Registers

Address: \$__0							
	Bit 15	14	13	12	11	10	9 Bit 8
Read:	0	0	0	SBR12	SBR11	SBR10	SBR9 SBR8
Write:							
Reset:	0	0	0	0	0	0	0 0

Address: \$__1							
	Bit 7	6	5	4	3	2	1 Bit 0
Read:	SBR7	SBR6	SBR5	SBR4	SBR3	SBR2	SBR1 SBR0
Write:							

Figure 15-17 . SCI Baud Rate Registers (SCBDH/L)

Serial Communications Interface

	Bit 7	6	5	4	3	2	1	Bit 0
Reset:	0	0	0	0	0	1	0	0
	= Reserved or unimplemented							

Figure 15-17 . SCI Baud Rate Registers (SCBDH/L)

Read: anytime
 Write: bits 15–13 only in special modes; bits 12–0 any time; writing to SCIBDH has no effect without writing to SCIBDL
 The count in this register determines the baud rate of the SCI. The formula for calculating baud rate is:

$$\text{SCI baud rate} = \frac{\text{SCI module clock}}{16 \times \text{BR}}$$

BR = contents of baud rate registers, a value from 1 to 8191

NOTE:
The baud rate generator is disabled until the TE bit or the RE bit is set for the first time after reset. The baud rate generator is disabled when BR = 0.

15.6.2 SCI Control Register 1

Address: \$__2

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	LOOPS	WOMS	RSRC	M	WAKE	ILT	PE	PT
Write:								
Reset:	0	0	0	0	0	0	0	0

Figure 15-18 . SCI Control Register 1 (SCICR1)

Read: anytime
 Write: anytime
 LOOPS — Loop Select Bit
 LOOPS enables loop operation. In loop operation the RXD pin is disconnected from the SCI, and the transmitter output goes into the receiver input. Both the transmitter and the receiver must be enabled to use the loop function.
 1 = Loop operation enabled
 0 = Normal operation enabled

The receiver input is determined by the RSRC bit. The transmitter output is controlled by the associated SCIDDR bit.

If the data direction bit for the TXD pin is set and LOOPS = 1, the transmitter output appears on the TXD pin. If the data direction bit is clear and LOOPS = 1, the TXD pin is idle (high) if RSRC = 0 and high-impedance if RSRC = 1. See **Table 15-7**.

Table 15-7 . Loop Functions

LOOPS	RSRC	SCIDDR Bit 1	WOMS	Function
0	X	X	X	Normal operation
1	0	0	X	Loop mode with high-impedance TXD output
1	0	1	0	Loop mode with TXD CMOS output
1	0	1	1	Loop mode with open-drain TXD output
1	1	1	0	Single-wire mode with TXD output fed back to RXD
1	1	1	1	Open-drain single-wire operation

WOMS — Wired-OR Mode Select Bit

WOMS configures the TXD and RXD pins for open-drain operation. WOMS allows TXD pins to be tied together in a multiple-transmitter system. Then the TXD pins of nonactive transmitters follow the logic level of an active one. WOMS also affects the TXD and RXD pins when they are general-purpose outputs. External pullup resistors are necessary on open-drain outputs.

- 1 = TXD and RXD pins — open-drain when outputs
- 0 = TXD and RXD pins — full CMOS drive capability

RSRC — Receiver Source Bit

When LOOPS = 1, the RSRC bit determines the internal feedback path for the receiver.

- 1 = Receiver input connected to TXD pin
- 0 = Receiver input internally connected to transmitter output

M — Data Format Mode Bit

MODE determines whether data characters are eight or nine bits long.

- 1 = One start bit, nine data bits, one stop bit
- 0 = One start bit, eight data bits, one stop bit

WAKE — Wakeup Condition Bit

WAKE determines which condition wakes up the SCI: a logic 1 (address mark) in the most significant bit position of a received data character or an idle condition on the RXD pin.

- 1 = Address mark wakeup

Serial Communications Interface

0 = Idle line wakeup

ILT — Idle Line Type Bit

ILT determines when the receiver starts counting logic 1s as idle character bits. The counting begins either after the start bit or after the stop bit. If the count begins after the start bit, then a string of logic 1s preceding the stop bit may cause false recognition of an idle character. Beginning the count after the stop bit avoids false idle character recognition, but requires properly synchronized transmissions.

- 1 = Idle character bit count begins after stop bit
- 0 = Idle character bit count begins after start bit

PE — Parity Enable Bit

PE enables the parity function. When enabled, the parity function inserts a parity bit in the most significant bit position.

- 1 = Parity function enabled
- 0 = Parity function disabled

PT — Parity Type Bit

PT determines whether the SCI generates and checks for even parity or odd parity. With even parity, an even number of 1s clears the parity bit and an odd number of 1s sets the parity bit. With odd parity, an odd number of 1s clears the parity bit and an even number of 1s sets the parity bit.

- 1 = Odd parity
- 0 = Even parity

15.6.3 SCI Control Register 2

Address: \$___3

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	TIE	TCIE	RIE	ILIE	TE	RE	RWU	SBK
Write:								
Reset:	0	0	0	0	0	0	0	0

Figure 15-19 . SCI Control Register 2 (SCICR2)

Read: anytime
Write: anytime

TIE — Transmitter Interrupt Enable Bit

TIE enables the transmit data register empty flag, TDRE, to generate interrupt requests.

- 1 = TDRE interrupt requests enabled
- 0 = TDRE interrupt requests disabled

TCIE — Transmission Complete Interrupt Enable Bit

TCIE enables the transmission complete flag, TC, to generate interrupt requests.

- 1 = TC interrupt requests enabled
- 0 = TC interrupt requests disabled

RIE — Receiver Full Interrupt Enable Bit

RIE enables the receive data register full flag, RDRF, or the overrun flag, OR, to generate interrupt requests.

- 1 = RDRF and OR interrupt requests enabled
- 0 = RDRF and OR interrupt requests disabled

ILIE — Idle Line Interrupt Enable Bit

ILIE enables the idle line flag, IDLE, to generate interrupt requests.

- 1 = IDLE interrupt requests enabled
- 0 = IDLE interrupt requests disabled

TE — Transmitter Enable Bit

TE enables the SCI transmitter and configures the TXD pin as the SCI transmitter output. The TE bit can be used to queue an idle preamble.

- 1 = Transmitter enabled
- 0 = Transmitter disabled

RE — Receiver Enable Bit

RE enables the SCI receiver.

- 1 = Receiver enabled
- 0 = Receiver disabled

RWU — Receiver Wakeup Bit

- 1 = Standby state
- 0 = Normal operation

RWU enables the wakeup function and inhibits further receiver interrupt requests. Normally, hardware wakes the receiver by automatically clearing RWU.

SBK — Send Break Bit

Toggling SBK sends one break character (10 or 11 logic 0s). As long as SBK is set, the transmitter sends logic 0s.

- 1 = Transmit break characters
- 0 = No break characters

Serial Communications Interface

15.6.4 SCI Status Register 1



Figure 15-20 . SCI Status Register 1 (SCISR1)

Read: anytime
 Write: has no meaning or effect

TDRE — Transmit Data Register Empty Flag

TDRE is set when the transmit shift register receives a byte from the SCI data register. Clear TDRE by reading SCI status register 1 with TDRE set and then writing to SCI data register low (SCIDRL).

- 1 = Byte transferred to transmit shift register; transmit data register empty
- 0 = No byte transferred to transmit shift register

TC — Transmit Complete Flag

TC is set when the TDRE flag is set and no data, preamble, or break character is being transmitted. When TC is set, the TXD pin becomes idle (logic 1). Clear TC by reading SCI status register 1 (SCISR1) with TC set and then writing to SCI data register low (SCIDRL). TC is cleared automatically when data, preamble, or break is queued and ready to be sent.

- 1 = No transmission in progress
- 0 = Transmission in progress

RDRF — Receive Data Register Full Flag

RDRF is set when the data in the receive shift register transfers to the SCI data register. Clear RDRF by reading SCI status register 1 (SCISR1) with RDRF set and then reading SCI data register low (SCIDRL).

- 1 = Received data available in SCI data register
- 0 = Data not available in SCI data register

IDLE — Idle Line Flag

IDLE is set when 10 consecutive logic 1s (if M = 0) or 11 consecutive logic 1s (if M = 1) appear on the receiver input. Once the IDLE flag is cleared, a valid frame must again set the RDRF flag before an idle condition can set the IDLE flag.

- 1 = Receiver input has become idle

0 = Receiver input is either active now or has never become active since the IDLE flag was last cleared

NOTE:

When the receiver wakeup bit (RWU) is set, an idle line condition does not set the IDLE flag.

OR — Overrun Flag

OR is set when software fails to read the SCI data register before the receive shift register receives the next frame. The data in the shift register is lost, but the data already in the SCI data registers is not affected. Clear OR by reading SCI status register 1 (SCISR1) with OR set and then reading SCI data register low (SCIDRL).

1 = Overrun

0 = No overrun

NF — Noise Flag

NF is set when the SCI detects noise on the receiver input. NF bit is set during the same cycle as the RDRF flag but does not get set in the case of an overrun. Clear NF by reading SCI status register 1 and then reading SCI data register low (SCIDRL).

1 = Noise

0 = No noise

FE — Framing Error Flag

FE is set when a logic 0 is accepted as the stop bit. FE bit is set during the same cycle as the RDRF flag but does not get set in the case of an overrun. FE inhibits further data reception until it is cleared. Clear FE by reading SCI status register 1 (SCISR1) with FE set and then reading the SCI data register low (SCIDRL).

1 = Framing error

0 = No framing error

PF — Parity Error Flag

PF is set when the parity enable bit, PE, is set and the parity of the received data does not match its parity bit. Clear PF by reading SCI status register 1 and then reading SCI data register low (SCIDRL).

1 = Parity error

0 = No parity error

Serial Communications Interface

15.6.5 SCI Status Register 2

Address: \$__5

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	0	0	0	0	0	0	0	RAF
Write:								
Reset:	0	0	0	0	0	0	0	0

 = Reserved or unimplemented

Figure 15-21 . SCI Status Register 2 (SCISR2)

Read: anytime
 Write: has no meaning or effect

RAF — Receiver Active Flag

RAF is set when the receiver detects a logic 0 during the RT1 time period of the start bit search. RAF is cleared when the receiver detects false start bits (usually from noise or baud rate mismatch) or when the receiver detects an idle character.

1 = Reception in progress
 0 = No reception in progress

15.6.6 SCI Data Registers

Address: \$__6

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	R8	T8	0	0	0	0	0	0
Write:								
Reset:	Unaffected by reset							

Address: \$__7

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	R7	R6	R5	R4	R3	R2	R1	R0
Write:	T7	T6	T5	T4	T3	T2	T1	T0
Reset:	Unaffected by reset							

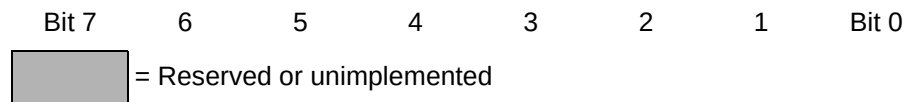


Figure 15-22 . SCI Data Registers (SCIDRH/L)

Read: anytime; reading accesses SCI receive data register

Write: anytime; writing accesses SCI transmit data register; writing to R8 has no effect

R8 — Received Bit 8

R8 is the ninth data bit received when the SCI is configured for 9-bit data format (M = 1).

T8 — Transmitted Bit 8

T8 is the ninth data bit transmitted when the SCI is configured for 9-bit data format (M = 1).

NOTE:

If the value of T8 is the same as in the previous transmission, T8 does not have to be rewritten. The same value is transmitted until T8 is rewritten.

NOTE:

In 8-bit data format, only SCI data register low (SCIDRL) needs to be accessed.

NOTE:

When transmitting in 9-bit data format and using 8-bit write instructions, write first to SCI data register high (SCIDRH).

15.7 External Pin Descriptions

15.7.1 TXD Pin

TXD is the SCI transmitter pin. TXD is available for general-purpose I/O when it is not configured for transmitter operation.

15.7.2 RXD Pin

RXD is the SCI receiver pin. RXD is available for general-purpose I/O when it is not configured for receiver operation.

15.8 Reset Initialization

See 15.6 Register Descriptions.

15.9 Modes of Operation

The SCI functions the same in normal, special, and emulation modes.

15.10 Low-Power Options

15.10.1 Run Mode

Clearing the transmitter enable or receiver enable bits (TE or RE) in SCI control register 2 (SCICR2) reduces power consumption in run mode. SCI registers are still accessible when TE or RE is cleared, but clocks to the core of the SCI are disabled.

15.10.2 Wait Mode

SCI operation in wait mode depends on the state of the SCISWAI bit in the SCI pullup and reduced drive register (SCIPURD).

- If SCISWAI is clear, the SCI operates normally when the CPU is in wait mode.
- If SCISWAI is set, SCI clock generation ceases and the SCI module enters a power-conservation state when the CPU is in wait mode. In this condition, SCI registers are not accessible. Setting SCISWAI does not affect the state of the receiver enable bit, RE, or the transmitter enable bit, TE.

If SCISWAI is set, any transmission or reception in progress stops at wait mode entry. The transmission or reception resumes when either an internal or external interrupt brings the MCU out of wait mode. Exiting wait mode by reset aborts any transmission or reception in progress and resets the SCI.

15.10.3 Stop Mode

The SCI is inactive in stop mode for reduced power consumption. The STOP instruction does not affect SCI register states. SCI operation resumes after an external interrupt brings the CPU out of stop mode. Exiting stop mode by reset aborts any transmission or reception in progress and resets the SCI.

15.11 Interrupt Operation

15.11.1 Interrupt Sources

Table 15-8 . SCI Interrupt Sources

Interrupt Source	Flag	Local Enable	Global (CCR) Mask
Transmitter	TDRE	TIE	I bit
Transmitter	TC	TCIE	I bit
Receiver	RDRF	RIE	I bit
	OR		
Receiver	IDLE	ILIE	I bit

15.11.2 Recovery from Wait Mode

Any enabled SCI interrupt request can bring the MCU out of wait mode.

15.12 General-Purpose I/O Ports

15.12.1 SCI Port Data Register

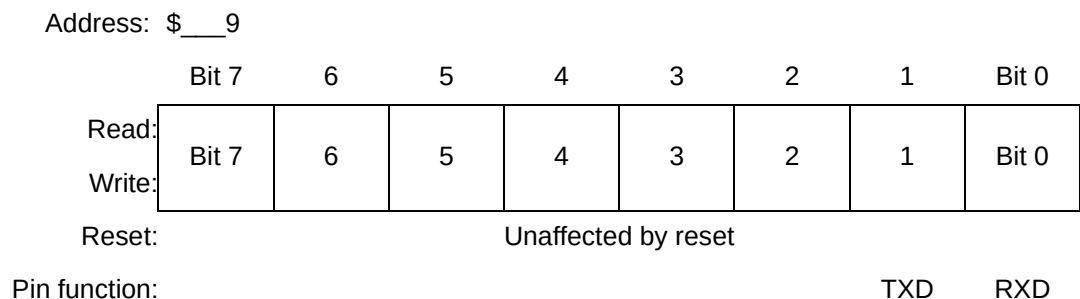


Figure 15-23 . SCI Port Data Register (SCI PORT)

Read: anytime; when SCIDDR bit = 0, reads return pin level; when SCIDDR bit = 1, reads return pin driver input level

Write: data stored in internal latch (drives pin only if SCIDDR bit = 1)

Serial Communications Interface

NOTE:
Writes do not change the pin state when the pin is configured for SCI output.

CAUTION:
To ensure that you read the value present on the SCI_{PORT} pins, always wait at least one cycle after writing to the SCID_{DR} register before reading from the SCI_{PORT} register.

15.12.2 SCI Data Direction Register

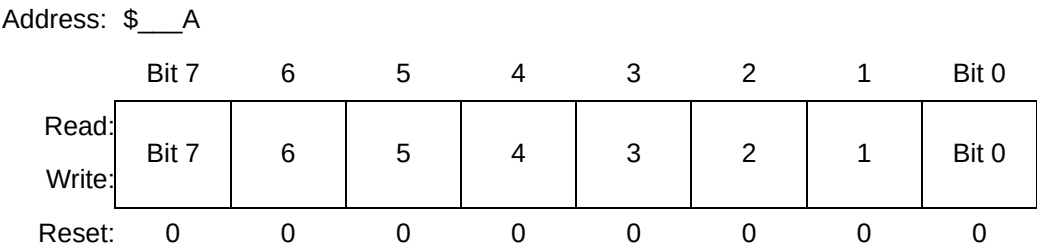


Figure 15-24 . SCI Data Direction Register (SCID_{DR})

Read: anytime
Write: anytime

Bits 7–0 — SCI_{PORT} Data Direction Bits

These bits control the port logic of SCI_{PORT}. Reset clears the SCI data direction register, configuring all SCI port pins as inputs.
1 = Corresponding pin configured as output
0 = Corresponding pin configured as input

NOTE:
When the *LOOPS* bit is clear, the *RXD* pin is an input and the *TXD* pin is an output regardless of the state of their SCID_{DR} bits.

15.12.3 SCI Pullup and Reduced Drive Register

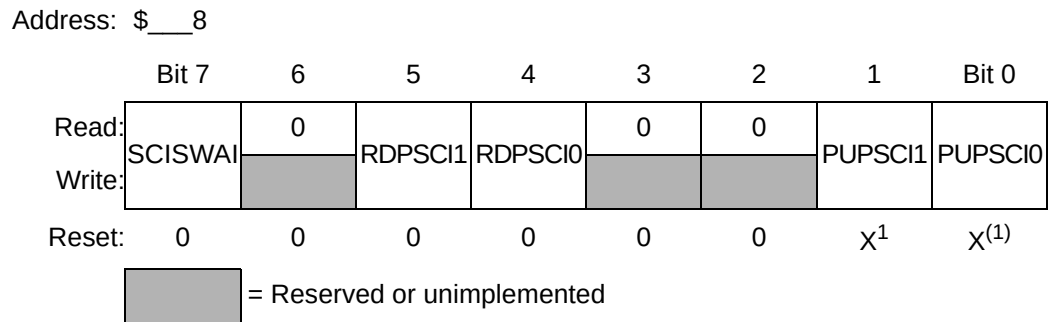


Figure 15-25 . SCI Pullup and Reduced Drive Register (SCIPURD)

NOTES:

1. X means determined at MCU definition.

Read: anytime

Write: anytime

SCISWAI — SCI Stop in Wait Mode Bit

SCISWAI disables the SCI in wait mode.

1 = SCI disabled in wait mode

0 = SCI enabled in wait mode

RDPSCI1 — Reduced Drive Bit 1

RDPSCI1 controls the drive capability of pins 7–2.

1 = Reduced drive

0 = Full drive

RDPSCI0 — Reduced Drive Bit 0

RDPSCI0 controls the drive capability of pins 1 and 0.

1 = Reduced drive

0 = Full drive

PUPSCI1 — Pullup Enable Bit 1

PUPSCI1 enables the pullups on pins 7–2. If a pin is programmed as an output, the pullup is disabled.

1 = Pullup enabled

0 = Pullup disabled

PUPSCI0 — Pullup Enable Bit 0

PUPSCI0 enables the pullups on pins 1 and 0. If a pin is programmed as an output, the pullup is disabled.

1 = Pullup enabled

0 = Pullup disabled

Table 15-9 . SCI Port Control Summary

Pullup Enable Control			Reduced Drive Control			Wired-OR Mode Control		
Register	Bit	Reset State	Register	Bit	Reset State	Register	Bit	Reset State
SCIPURD	PUPSCI0, PUPSCI1	X ¹	SCIPURD	RDPSCI0, RDPSCI1	Full drive	SCICR1	WOMS	Normal

NOTES:

1. X means determined at MCU definition.

Section 16 Serial Peripheral Interface Module

16.1 Introduction

The SPI allows full-duplex, synchronous, serial communications with peripheral devices.

16.2 Features

- Master mode and slave mode
- Wired-OR mode
- Slave select output
- Bidirectional serial data pin mode
- Mode fault error flag with CPU interrupt capability
- Double-buffered operation
- Serial clock with programmable polarity and phase
- Control of SPI operation during wait mode
- Reduced drive control for lower power consumption

Serial Peripheral Interface Module

16.3 Block Diagram

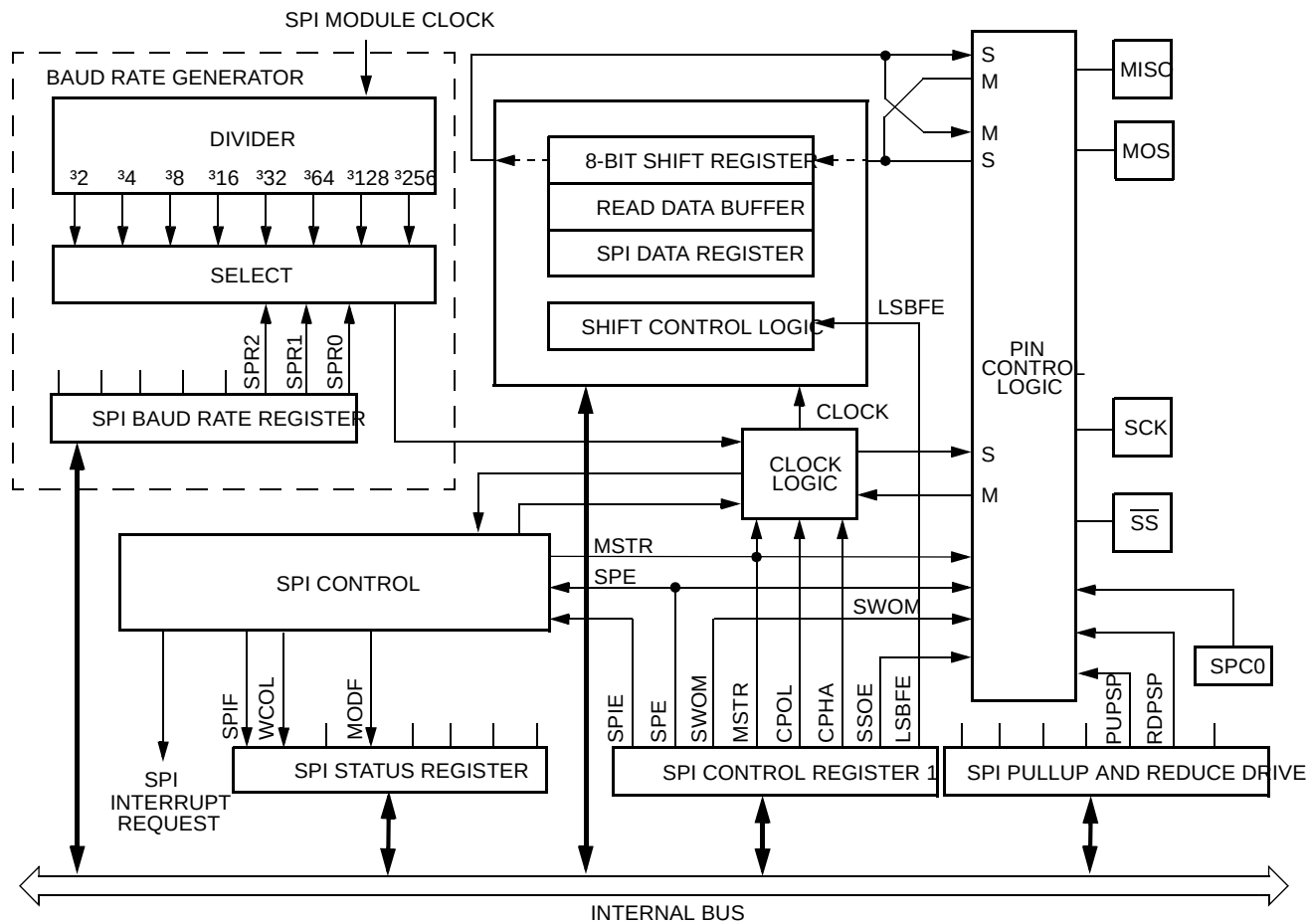


Figure 16-1 . SPI Block Diagram

16.4 Register Map

NOTE:
The address of a register is the sum of a base address and an address offset. The base address is defined at the MCU level and the address offset is defined at the module level.

Design Specification — MMC2110_Avalanche

Register Name		Bit 7	6	5	4	3	2	1	Bit 0	Addr. Offset
SPICR1	Read:	SPIE	SPE	SWOM	MSTR	CPOL	CPHA	SSOE	LSBFE	\$__0
	Write:									
SPICR2	Read:	0	0	0	0	0	0	SPISWAI	SPC0	\$__1
	Write:									
SPIBR	Read:	0	0	0	0	0	SPR2	SPR1	SPR0	\$__2
	Write:									
SPISR	Read:	SPIF	WCOL	0	MODF	0	0	0	0	\$__3
	Write:									
SPI reserved	Read:	0	0	0	0	0	0	0	0	\$__4
	Write:									
SPIDR	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__5
	Write:									
SPIPURD	Read:	0	0	RDPSP1	RDPSP0	0	0	PUPSP1	PUPSP0	\$__6
	Write:									
SPIPORT	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__7
	Write:									
SPIDDR	Read:	Bit 7	6	5	4	3	2	1	Bit 0	\$__8
	Write:									
SPI reserved	Read:	0	0	0	0	0	0	0	0	\$__9
	Write:									
SPI reserved	Read:	0	0	0	0	0	0	0	0	\$__A
	Write:									
SPI reserved	Read:	0	0	0	0	0	0	0	0	\$__B
	Write:									

 = Unimplemented

Figure 16-2 . I/O Register Summary

16.5 Functional Description

The SPI module allows full-duplex, synchronous, serial communication between the MCU and peripheral devices. Software can poll the SPI status flags or SPI operation can be interrupt driven.

The SPI system is enabled by setting the SPI enable (SPE) bit in SPI control register 1. While SPE is set, the four associated SPI port pins are dedicated to the SPI function as:

- Slave select ($\overline{SS}$)
- Serial clock (SCK)
- Master out/slave in (MOSI)
- Master in/slave out (MISO)

While SPE is clear, SPI port pins 3, 2, 1, and 0 are general-purpose I/O (input/output) pins controlled by the SPI port data direction register.

The main element of the SPI system is the SPI data register. The 8-bit data register in the master and the 8-bit data register in the slave are linked by the MOSI and MISO pins to form a distributed 16-bit register. When a data transfer operation is performed, this 16-bit register is serially shifted eight bit positions by the SCK clock from the master; data is exchanged between the master and the slave. Data written to the master SPI data register becomes the output data for the slave, and data read from the master SPI data register after a transfer operation is the input data from the slave.

A write to the SPI data register puts data into a serial shifter. When a transfer is complete, received data is moved into a receive data register. Data may be read from this double-buffered system any time before the next transfer is complete. This 8-bit data register acts as the SPI receive data register for reads and as the SPI transmit data register for writes. A single MCU register address is used for reading data from the read data buffer and for writing data to the shifter.

The clock phase control bit (CPHA) and a clock polarity control bit (CPOL) in the SPI control register 1 select one of four possible clock formats to be used by the SPI system. The CPOL bit simply selects a non-inverted or inverted clock. The CPHA bit is used to accommodate two fundamentally different protocols by shifting the clock by a half cycle or by not shifting the clock (see **16.5.3 Transmission Formats**).

The SPI can be configured to operate as a master or as a slave. When MSTR in SPI control register 1 is set, the master mode is selected; when the MSTR bit is clear, the slave mode is selected.

16.5.1 Master Mode

The SPI operates in master mode when the MSTR bit is set. Only a master SPI module can initiate transmissions. A transmission begins by writing to the master SPI data register. If the shift register is empty, the byte immediately transfers to the shift register. The byte begins shifting out on the MOSI pin under the control of the serial clock.

The SPR2, SPR1, and SPR0 bits in the SPI baud rate register control the baud rate generator and determine the speed of the shift register. The SCK pin is the SPI clock output. Through the SCK pin, the baud rate generator of the master controls the shift register of the slave peripheral.

In master mode, the function of the serial data output pin (MOSI) and the serial data input pin (MISO) is determined by the SPC0 and MSTR control bits.

The $\overline{SS}$ pin is normally an input which should remain in the inactive high state. However, in the master mode, if the associated data direction bit (SPIDDR bit 3) is set, then the $\overline{SS}$ pin is a general-purpose output or the slave select output depending on the state of the SSOE bit.

General-purpose output (SSOE = 0) or slave select output (SSOE = 1) is specified by the SSOE bit in SPI control register 1. When this pin is being used as the output pin with SPIDDR bit 3 set, the mode error function for the master is disabled (MODF in the SPI status register) and SPIPORT bit 3 has no effect on the SPI system.

The $\overline{SS}$ output becomes low during each transmission and is high when the SPI is in the idling state. If the $\overline{SS}$ input becomes low while the MCU is configured as a master, it indicates a mode fault error where more than one master may be trying to drive the MOSI and SCK lines simultaneously. In this case, the MCU immediately clears the data direction bits associated with the MISO, MOSI (or MOMI), and SCK pins so that these pins become inputs. This mode fault error also clears the MSTR control bit and sets the mode fault (MODF) flag in the SPI status register. If the SPI interrupt enable bit (SPIE) is set when the MODF bit gets set, then an SPI interrupt sequence is requested also.

When a write to the SPI data register in the master occurs, there is a half SCK-cycle delay. After the delay, SCK is started within the master. The rest of the transfer operation differs slightly, depending on the clock format specified by the SPI clock phase bit, CPHA, in SPI control register 1 (see **16.5.3 Transmission Formats**).

16.5.2 Slave Mode

The SPI operates in slave mode when the MSTR bit in SPI control register 1 is clear. In slave mode, SCK is the SPI clock input from the master, and $\overline{SS}$ is the slave select input. Before a data transmission occurs, the $\overline{SS}$ pin of the slave SPI must be at logic 0. $\overline{SS}$ must remain low until the transmission is complete.

Serial Peripheral Interface Module

In slave mode, the function of the serial data output pin (MISO) and serial data input pin (MOSI) is determined by the SPC0 bit in SPI control register 2 and the MSTR control bit. While in slave mode, the $\overline{SS}$ input controls the serial data output pin; if $\overline{SS}$ is high (not selected), the serial data output pin is high impedance, and, if $\overline{SS}$ is low the msb (most significant bit) in the SPI data register is driven out of the serial data output pin. Also, if the slave is not selected ($\overline{SS}$ is high), then the SCK input is ignored and no internal shifting of the SPI shift register takes place.

Although the SPI is capable of full-duplex operation, some SPI peripherals are capable of only receiving SPI data in a slave mode. For these simpler devices, there is no serial data out pin.

NOTE:

When peripherals with full-duplex capability are used, take care not to simultaneously enable two receivers whose serial outputs drive the same system slave's serial data output line.

As long as no more than one slave device drives the system slave's serial data output line, it is possible for several slaves to receive the same transmission from a master, although the master would not receive return information from all of the receiving slaves.

If the CPHA bit in SPI control register 1 is clear, odd numbered edges on the SCK input cause the data at the serial data input pin to be latched. Even numbered edges cause the value previously latched from the serial data input pin to shift into the LSB of the SPI shifter.

If the CPHA bit is set, even numbered edges on the SCK input cause the data at the serial data input pin to be latched. Odd numbered edges cause the value previously latched from the serial data input pin to shift into the LSB of the SPI shifter.

When CPHA is set, the first edge is used to get the most significant data bit onto the serial data output pin. When CPHA is clear and the $\overline{SS}$ input is low (slave selected), the msb of the SPI data is driven out of the serial data input pin. After the eighth shift, the transfer is considered complete and the received data is transferred into the SPI data register. To indicate transfer is complete, the SPIF flag in the SPI status register is set.

16.5.3 Transmission Formats

During an SPI transmission, data is transmitted (shifted out serially) and received (shifted in serially) simultaneously. The serial clock (SCK) synchronizes shifting and sampling of the information on the two serial data lines. A slave select line allows selection of an individual slave SPI device; slave devices that are not selected do not interfere with SPI bus activities. Optionally, on a master SPI device, the slave select line can be used to indicate multiple-master bus contention.

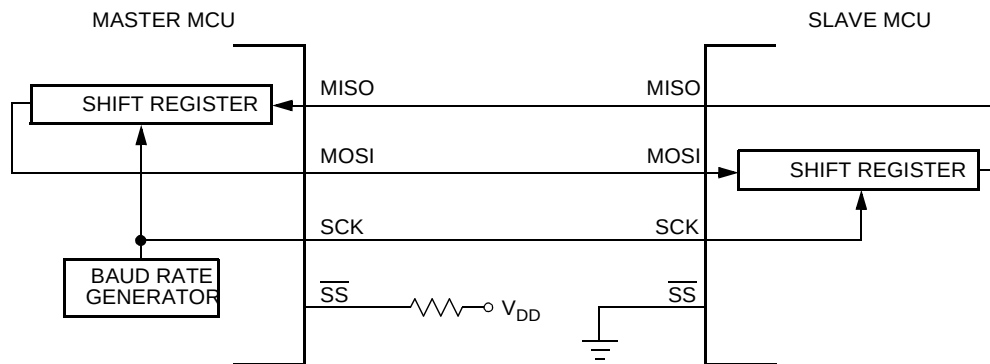


Figure 16-3 . Master/Slave Transfer Block Diagram

16.5.3.1 Clock Phase and Polarity Controls

Using two bits in the SPI control register 1, software selects one of four combinations of serial clock phase and polarity.

The CPOL clock polarity control bit specifies an active high or low clock and has no significant effect on the transmission format.

The CPHA clock phase control bit selects one of two fundamentally different transmission formats.

Clock phase and polarity should be identical for the master SPI device and the communicating slave device. In some cases, the phase and polarity are changed between transmissions to allow a master device to communicate with peripheral slaves having different requirements.

16.5.3.2 CPHA = 0 Transfer Format

The first edge on the SCK line is used to clock the slave msb into the master and the master msb into the slave. In some peripherals, the msb of the slave's data is available at the slave data out pin as soon as the slave is selected. In this format, the first SCK edge is not issued until a half cycle into the 8-cycle transfer operation. The first edge of SCK is delayed a half cycle by clearing the CPHA bit.

The SCK output from the master remains in the inactive state for a half SCK period before the first edge appears. A half SCK cycle later, the second edge appears on the SCK line. When this second edge occurs, the value previously latched from the serial data input pin is shifted into the LSB of the shifter.

After this second edge, the next bit of the SPI master data is transmitted out of the serial data output pin of the master to the serial input pin on the slave. This process continues for a total of

Serial Peripheral Interface Module

16 edges on the SCK line, with data being latched on odd numbered edges and shifted on even numbered edges.

Data reception is double buffered. Data is shifted serially into the SPI shift register during the transfer and is transferred to the parallel SPI data register after the last bit is shifted in.

After the 16th (last) SCK edge:

- Data that was previously in the master SPI data register should now be in the slave data register and the data that was in the slave data register should be in the master.
- The SPIF flag in the SPI status register is set and the clock is stopped, indicating that the transfer is complete.

Figure 16-4 is a timing diagram of an SPI transfer where CPHA = 0. SCK waveforms are shown for CPOL = 0 and CPOL = 1. The diagram may be interpreted as a master or slave timing diagram since the SCK, MISO, and MOSI pins are connected directly between the master and the slave. The MISO signal is the output from the slave and the MOSI signal is the output from the master. The SS pin of the master must be either high or reconfigured as a general-purpose output not affecting the SPI.

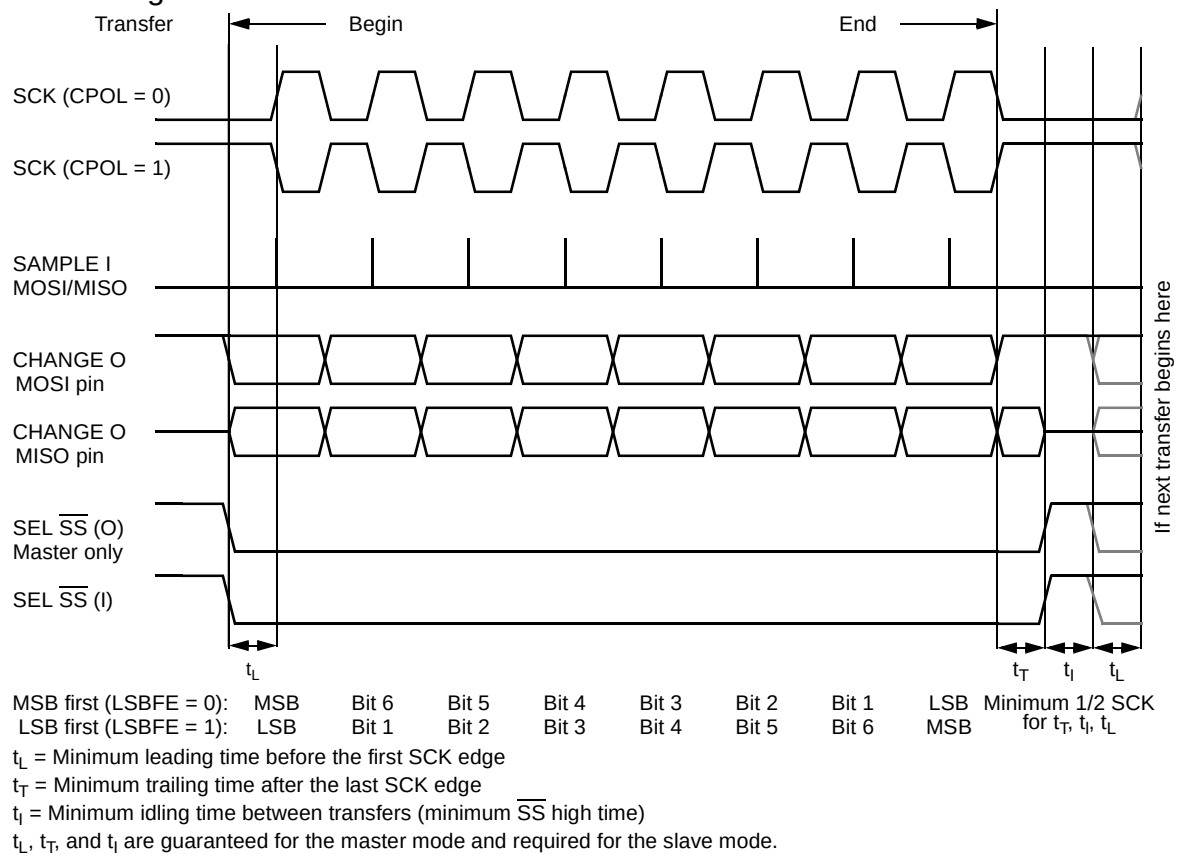


Figure 16-4 . SPI Clock Format 0 (CPHA = 0)

When $CPHA = 0$, the $\overline{SS}$ line must be negated and reasserted between each successive serial byte. Also, if the slave writes data to the SPI data register (SPIDR) while $\overline{SS}$ is active low, a write-collision error results.

16.5.3.3 $CPHA = 1$ Transfer Format

Some peripherals require the first SCK edge before the msb becomes available at the data out pin; the second edge clocks data into the system. In this format, the first SCK edge is issued by setting the CPHA bit at the beginning of the 8-cycle transfer operation.

The first edge of SCK occurs immediately after the half SCK clock cycle synchronization delay. This first edge commands the slave to transfer its most significant data bit to the serial data input pin of the master.

A half SCK cycle later, the second edge appears on the SCK pin. This is the latching edge for both the master and slave.

When the third edge occurs, the value previously latched from the serial data input pin is shifted into the LSB of the SPI shifter. After this edge, the next bit of the master data is coupled out of the serial data output pin of the master to the serial input pins on the slave.

This process continues for a total of 16 edges on the SCK line with data being latched on even numbered edges and shifting taking place on odd numbered edges.

Data reception is double buffered; data is serially shifted into the SPI shift register during the transfer and is transferred to the parallel SPI data register after the last bit is shifted in.

After the 16th SCK edge:

- Data that was previously in the SPI data register of the master is now in the data register of the slave, and data that was in the data register of the slave is in the master.
- The SPIF flag bit in SPISR is set and the clock is stopped, indicating that the transfer is complete.

Figure 16-5 shows two clocking variations for $CPHA = 1$. The diagram may be interpreted as a master or slave timing diagram since the SCK, MISO, and MOSI pins are connected directly between the master and the slave. The $\overline{MISO}$ signal is the output from the slave, and the $\overline{MOSI}$ signal is the output from the master. The $\overline{SS}$ line is the slave select input to the slave. The $\overline{SS}$

Serial Peripheral Interface Module

pin of the master must be either high or reconfigured as a general-purpose output not affecting the SPI.

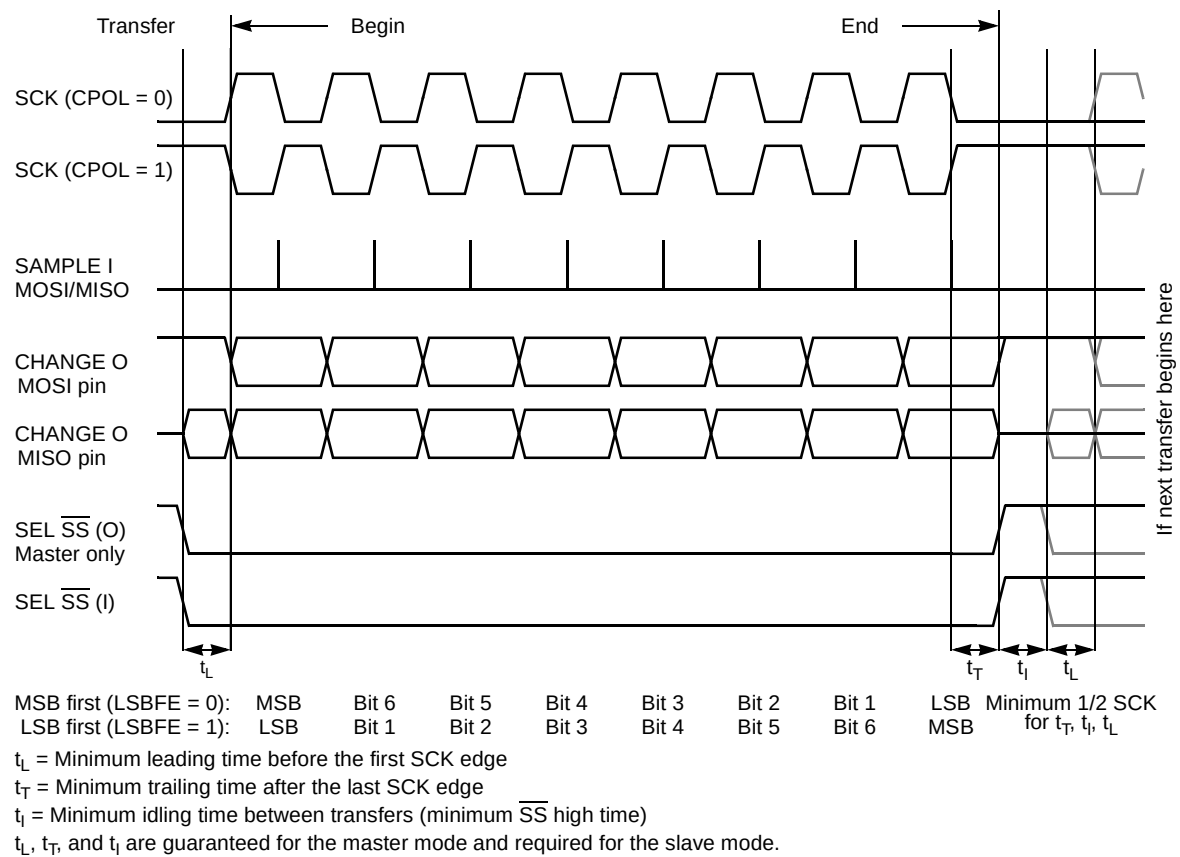


Figure 16-5 . SPI Clock Format 1 (CPHA = 1)

When CPHA = 1, the $\overline{SS}$ line can remain active low between successive transfers (can be tied low at all times). This format is sometimes preferred in systems having a single fixed master and a single slave that drive the MISO data line.

The SPI interrupt request flag (SPIF) is common to both the master and slave modes. SPIF gets set after the last SCK cycle in a data transfer operation to indicate that the transfer is complete. SPIF is cleared automatically when the SPI status register is read (with SPIF set) followed by a read or write to the SPI data register. If the SPIE bit is set when the SPIF flag is set, a hardware interrupt is requested.

A warning flag (WCOL) is set if a write to the SPI data register is attempted while a transfer is in progress. This is a conflict since the write would erroneously overwrite the current contents of the SPI serial shift register. If this situation arises, the write to the SPI data register is inhibited so as not to disturb the transfer in progress, and the WCOL flag is set to indicate the error. No interrupt

is generated by WCOL because an interrupt comes at the end of the transfer that was in progress at the time of the error.

16.5.4 SPI Baud Rate Generation

Baud rate generation consists of a series of divider stages. Three bits in the SPI baud rate register (SPR2, SPR1, and SPR0) control the SPI clock rate. The SPI system clock is the input to this divider series and the resulting SPI clock rate is selected as the SPI module clock divided by 2, 4, 8, 16, 32, 64, 128, or 256. (See **Table 16-4**.)

The baud rate generator is activated only when the SPI is in the master mode and a serial transfer is taking place. In the other cases, the divider is disabled to decrease I_{DD} current.

16.5.5 Special Features

16.5.5.1 $\overline{SS}$ Output

The $\overline{SS}$ output feature automatically drives the $\overline{SS}$ pin low during transmission to select external devices and drives it high during idle to deselect external devices. When $\overline{SS}$ output is selected, the $\overline{SS}$ output pin is connected to the $\overline{SS}$ input pin of the external device.

The $\overline{SS}$ output is available only in master mode during normal SPI operation by asserting SSOE and SPIDDR bit 3 as shown in **Table 16-2**.

The mode fault feature is disabled while $\overline{SS}$ output is enabled.

NOTE:

Care must be taken when using the $\overline{SS}$ output feature in a multimaster system since the mode fault feature is not available for detecting system errors between masters.

16.5.5.2 Bidirectional Mode (MOMI or SISO)

The bidirectional mode is selected when the SPC0 bit is set in SPI control register 2. In this mode, the SPI uses only one serial data pin for the interface with external device(s). The MSTR bit decides which pin to use. The MOSI pin becomes the serial data I/O (MOMI) pin for the master mode, and the MISO pin becomes serial data I/O (SISO) pin for the slave mode. The MISO pin in the master mode and MOSI pin in the slave mode become general-purpose I/O.

The direction of each serial I/O pin depends on the corresponding data direction register bit. If the pin is configured as an output, serial data from the shift register is driven out on the pin. The same pin is also the serial input to the shift register.

Serial Peripheral Interface Module

If the pin is configured as an input, serial data from the shift register is discarded, but the external serial data through the pin is the serial input to the shift register.

The SCK is output for the master mode and input for the slave mode.

The $\overline{SS}$ is the input or output for the master mode, and it is always the input for the slave mode.

The bidirectional mode does not affect SCK and $\overline{SS}$ functions; however, the SPIDDR bit 0 is not cleared by the mode fault error in the bidirectional mode.

Table 16-1 . Normal Mode and Bidirectional Mode

When SPE = 1	Master Mode MSTR = 1	Slave Mode MSTR = 0
Normal Mode SPC0 = 0	SWOM enables open drain output.	SWOM enables open drain output.
Bidirectional Mode SPC0 = 1	SWOM enables open drain output. SPI port pin 0 becomes general-purpose I/O.	SWOM enables open drain output. SPI port pin 1 becomes general-purpose I/O.

16.5.6 Error Conditions

The SPI has two error conditions:

- Write collision error
- Mode fault error

16.5.6.1 Write Collision Error

The WCOL status flag in the SPI status register indicates that a serial transfer was in progress when the MCU tried to write new data into the SPI data register. The MCU write is disabled to avoid writing over the data being transmitted. No interrupt is generated because the error status flag can be read upon completion of the transfer that was in progress at the time of the error. This

flag is cleared automatically by a read of the SPI status register (with WCOL set) followed by a read or write access to the SPI data register.

16.5.6.2 Mode Fault Error

If the $\overline{SS}$ input becomes low while the MCU is configured as a master, it indicates a system error where more than one master may be trying to drive the MOSI and SCK lines simultaneously. This condition is not permitted in normal operation; the MODF bit in the SPI status register is set automatically.

In the special case where SPIDDR bit 3 is set, the $\overline{SS}$ pin is either a general-purpose output pin or $\overline{SS}$ output pin rather than being dedicated as the $\overline{SS}$ input for the SPI system. In this special case, the mode error function is inhibited and MODF remains cleared.

When a mode fault error occurs, the MSTR bit is cleared and data direction bits controlling the output enable for the SCK, MISO, and MOSI (or MOMI) pins are cleared. This forces those pins to be high impedance inputs to avoid any possibility of conflict with another output driver.

If the mode fault error occurs in the bidirectional mode, the data direction bit associated with MISO (SISO) is not affected, since this bit is dedicated for general purpose.

This flag is cleared automatically by a read of the SPI status register (with MODF set) followed by a write to SPI control register 1.

16.6 Register Descriptions

16.6.1 SPI Control Register 1

Address: \$___0

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	SPIE	SPE	SWOM	MSTR	CPOL	CPHA	SSOE	LSBFE
Write:								
Reset:	0	0	0	0	0	1	0	0

Figure 16-6 . SPI Control Register 1 (SPICR1)

Read: anytime

Write: anytime

Serial Peripheral Interface Module

SPIE — SPI Interrupt Enable Bit

This bit enables SPI interrupts each time the SPIF or MODF status flag is set.

- 1 = SPI interrupts enabled
- 0 = SPI interrupts disabled

SPE — SPI System Enable Bit

This bit enables the SPI system and dedicates SPI port pins 3–0 to SPI functions. When SPE is clear, the SPI system is initialized but in a low-power disabled state.

- 1 = SPI port pins 3–0 are dedicated to SPI functions.
- 0 = SPI system is in a low-power, disabled state.

SWOM — SPI Wired-OR Mode Bit

This bit controls output pins 3–0, independent of function. SWOM does not affect pins 7–4.

- 1 = General-purpose and/or SPI port pins 3–0 output buffers behave as open-drain outputs.
- 0 = General-purpose and/or SPI port pins 3–0 output buffers behave normally.

MSTR — SPI Master/Slave Mode Select Bit

- 1 = Master mode
- 0 = Slave mode

CPOL — SPI Clock Polarity Bit

This bit selects an inverted or non-inverted SPI clock. To transmit data between SPI modules, the SPI modules must have identical CPOL values.

- 1 = Active-low clocks selected; SCK idles high
- 0 = Active-high clocks selected; SCK idles low

CPHA — SPI Clock Phase Bit

This bit is used to shift the SCK serial clock.

- 1 = The first SCK edge is issued at the beginning of the 8-cycle transfer operation.
- 0 = The first SCK edge is issued one-half cycle into the 8-cycle transfer operation.

SSOE — Slave Select Output Enable

The $\overline{SS}$ output feature is enabled only in the master mode by asserting the SSOE and SPIDDR bit 3 as shown in **Table 16-2**.

Table 16-2 . $\overline{SS}$ Output Selection

SPIDDR Bit 3	SSOE	Master Mode	Slave Mode
0	0	$\overline{SS}$ input with MODF feature	$\overline{SS}$ input
0	1	General-purpose input	$\overline{SS}$ input
1	0	General-purpose output	$\overline{SS}$ input

Table 16-2 . $\overline{SS}$ Output Selection

SPIDDR Bit 3	SSOE	Master Mode	Slave Mode
1	1	$\overline{SS}$ output	$\overline{SS}$ input

LSBFE — SPI LSB-First Enable

This bit does not affect the position of the msb and lsb in the data register. Reads and writes of the data register always have the msb in bit 7.

1 = Data is transferred least significant bit first.

0 = Data is transferred most significant bit first.

16.6.2 SPI Control Register 2

Address: \$__1

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	0	0	0	0	0	0	SPISWA I	SPC0
Write:								
Reset:	0	0	0	0	0	0	0	0

 = Reserved or unimplemented

Figure 16-7 . SPI Control Register 2 (SPICR2)

Read: anytime

Write: anytime; writes to unimplemented bits have no effect.

SPISWAI — SPI Stop in Wait Mode Bit

This bit is used for power conservation while in wait mode.

1 = Stop SPI clock generation when in wait mode

0 = SPI clock operates normally in wait mode.

SPC0 — Serial Pin Control Bit 0

With the MSTR control bit, this bit enables bidirectional pin configurations as shown in **Table 16-3**.

Serial Peripheral Interface Module

Table 16-3 . Bidirectional Pin Configurations

Pin Mode		SPC0	MSTR	MISO ¹	MOSI ²	SCK ³	SS ⁴
A	Normal	0	0	Slave out	Slave in	SCK in	SS in
B			1	Master in	Master out	SCK out	SS I/O
C	Bidirectional	1	0	Slave I/O	GP ⁵ I/O	SCK in	SS in
D			1	GP I/O	Master I/O	SCK out	SS I/O

- NOTES:
1. Slave output is enabled if SPIDDR bit 0 = 1, SS = 0, and MSTR = 0 (A, C)
 2. Master output is enabled if SPIDDR bit 1 = 1 and MSTR = 1 (B, D)
 3. SCK output is enabled if SPIDDR bit 2 = 1 and MSTR = 1 (B, D)
 4. SS output is enabled if SPIDDR bit 3 = 1, SSOE = 1, and MSTR =1 (B, D)
 5. GP = general purpose

16.6.3 SPI Baud Rate Register

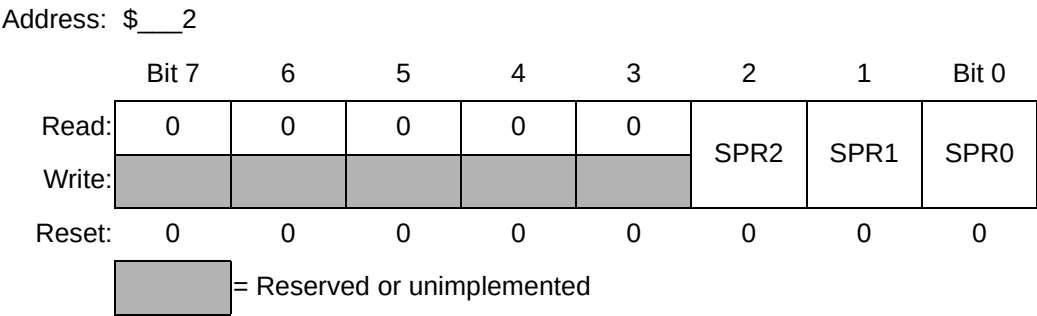


Figure 16-8 . SPI Baud Rate Register (SPIBR)

Read: anytime
 Write: anytime; writes to unimplemented bits have no effect.

NOTE:
Writing to this register during data transfers may cause spurious results.

SPR2–SPR0 — SPI Baud Rate Selection Bits

These bits specify the SPI baud rate as shown in **Table 16-4**.

Table 16-4 . SPI Baud Rate Selection

SPR2	SPR1	SPR0	SPI Module Clock Divisor	Baud Rate SPI Module Clock = 8 MHz
0	0	0	2	4 MHz
0	0	1	4	2 MHz
0	1	0	8	1 MHz
0	1	1	16	500 MHz
1	0	0	32	250 kHz
1	0	1	64	125 kHz
1	1	0	128	62.5 kHz
1	1	1	256	31.3 kHz

16.6.4 SPI Status Register

Address: \$__3

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	SPIF	WCOL	0	MODF	0	0	0	0
Write:								
Reset:	0	0	0	0	0	0	0	0

 = Reserved or unimplemented

Figure 16-9 . SPI Status Register (SPISR)

Read: anytime

Write: has no meaning or effect

SPIF — SPI Interrupt Flag

This bit is set after the eighth SCK cycle in a data transfer and is cleared by reading the SPISR register (with SPIF set) followed by a read or write access to the SPI data register.

WCOL — Write Collision Flag

This error status flag indicates that a serial transfer was in progress when the MCU tried to write new data into the SPI data register. The flag is cleared automatically by a read of the SPI status register (with WCOL set) followed by an read or write access to the SPI data register.

1 = Write collision has occurred.

Serial Peripheral Interface Module

0 = Write collision has not occurred.

MODF — Mode Fault Flag

This bit is set if the $\overline{SS}$ input becomes low while the MCU is configured as a master. The flag is cleared automatically by a read of the SPI status register (with MODF set) followed by a write to the SPI control register 1.

- 1 = Mode fault has occurred.
- 0 = Mode fault has not occurred.

16.6.5 SPI Data Register

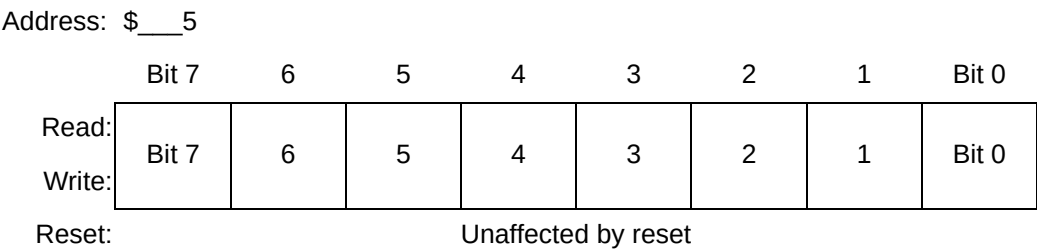


Figure 16-10 . SPI Data Register (SPIDR)

Read: anytime; normally read only after SPIF is set
 Write: anytime; see WCOL

The SPI data register is both the input and output register for SPI data. Attempts to write to this register while data transfers are in progress set the WCOL flag and disable the attempted write. Review the WCOL bit description in **16.6.4 SPI Status Register** for more information.

16.7 External Pin Descriptions

The SPI interface consists of four SPI port pins:

- MISO — Master data in, slave data out
- MOSI — Master data out, slave data in
- SCK — Serial clock
- $\overline{SS}$ — Slave select

16.7.1 MISO (Master In/Slave Out)

MISO is one of the two SPI module pins that transmit serial data. MISO is an input when the SPI is configured as master and an output when the SPI is configured as a slave.

If the bidirectional serial pin mode is selected in the slave, MISO becomes SISO (slave in/slave out) and the direction is controlled by the associated bit in the SPI port data direction register.

In a multiple-master system, all MISO pins are tied together.

16.7.2 MOSI (Master Out/Slave In)

MOSI is one of the two SPI module pins that transmit serial data. MOSI is an output when the SPI is configured as a master and an input when the SPI is configured as a slave.

If the bidirectional serial pin mode is selected in the master, MOSI becomes MOMI (master out/master in) and the direction is controlled by the associated bit in the SPI port data direction register.

In a multiple-master system, all MOSI pins are tied together.

16.7.3 SCK (Serial Clock)

The serial clock synchronizes data transmissions between master and slave devices. SCK is an output if the SPI is configured as a master and SCK is an input if the SPI is configured as a slave.

In a multiple-master system, all SCK pins are tied together.

16.7.4 $\overline{SS}$ (Slave Select)

The slave select output or input provides a means of selectively enabling slaves so several may coexist in one system. $\overline{SS}$ is a general-purpose output (SSOE = 0) or the slave select output (SSOE = 1) when the SPI is in master mode and the associated data direction bit is set.

The $\overline{SS}$ pin is the mode fault input when the SPI is in master mode and the associated data direction bit is clear.

$\overline{SS}$ is always an input when the SPI is in slave mode, regardless of the state of the data direction bit for that pin. When the SPI is configured as a slave, the MISO (or SISO) output driver is three-stated until enabled by the slave select input (low true) so that many slaves may be wire-ORed to the same MISO (or SISO) line.

The directions of the MOSI and MISO pins are also determined by the serial pin control (SPC0) bit.

16.8 Reset Initialization

See **16.6 Register Descriptions**.

Serial Peripheral Interface Module

16.9 Modes of Operation

This module functions the same in normal, special, and emulation modes.

16.10 Low-Power Options

The wait and stop modes put the MCU in a low power-consumption, standby state.

16.10.1 SPI in Run Mode

In run mode with the SPI system enable (SPE) bit in the SPI control register 1 clear, the SPI system is in a low-power, disabled state. SPI registers can still be accessed, but clocks to the core of this module are disabled.

16.10.2 SPI in Wait Mode

SPI operation in wait mode depends upon the state of the SPISWAI bit in SPI control register 2.

- If SPISWAI is clear, the SPI operates normally when the CPU is in wait mode.
- If SPISWAI is set, SPI clock generation ceases and the SPI module enters a power conservation state when the CPU is in wait mode. In this condition, SPI registers cannot be accessed.

16.10.3 SPI in Stop Mode

If the SPI is in master mode and exchanging data when the MCU enters stop mode, the transmission is frozen until the MCU exits stop mode. After stop, data to and from the external SPI is exchanged correctly.

If the SPI is in slave mode when the MCU enters stop mode, no data exchange with the external SPI occurs.

16.11 Interrupt Operation

These SPI sources can generate CPU interrupt requests:

- Mode fault flag (MODF) — MODF is set when a logic 0 occurs on the $\overline{SS}$ pin of a master SPI.
- SPI interrupt flag (SPIF) — SPIF is set after the last SCK cycle in a data transfer operation to indicate that the transfer is complete.

SPIF is cleared automatically when the SPI status register is read (with SPIF set) followed by either a read or write to the SPI data register. If the SPIE bit is set when the SPIF flag is set, a hardware interrupt is requested.

Table 16-5 . SPI Interrupt Vectors

Interrupt Source	Interrupt Flag	Local Enable	Global (CCR) Mask	Interrupt Vector
Mode fault	MODF	SPIE	I bit	1
Transfer complete	SPIF	SPIE	I bit	

NOTES:

1. Determined at MCU definition

16.12 General-Purpose I/O Ports

There is one general-purpose I/O port with two associated registers:

- SPI port reduced drive and pullup select register
- SPI port data register
- SPI port data direction register

16.12.1 SPI Port Reduced Drive and Pullup Select Register

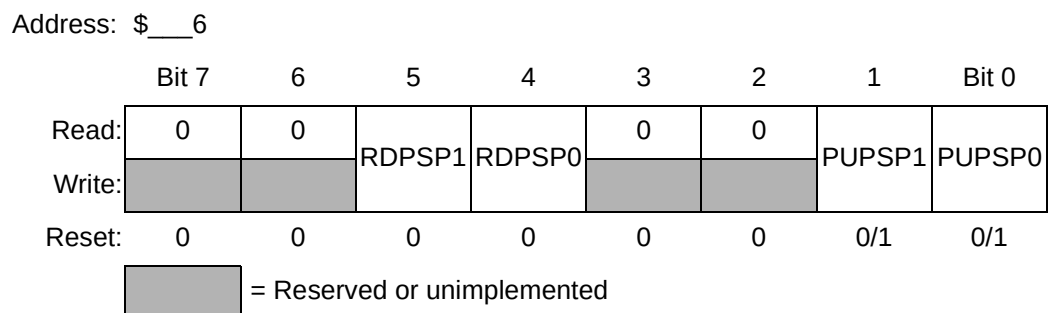


Figure 16-11 . SPI Port Reduced Drive and Pullup Select (SPIPURD)

Read: anytime

Write: anytime; writes to unimplemented bits have no effect.

Serial Peripheral Interface Module

NOTE:
The reset state of bits 1–0 is determined at MCU definition.

RDPSP1 — SPI Port Reduced Drive Control Bit 1
 1 = Reduced drive capability on pins 7–4
 0 = Full drive enabled on pins 7–4

RDPSP0 — SPI Port Reduced Drive Control Bit 0
 1 = Reduced drive capability on pins 3–0
 0 = Full drive enabled on pins 3–0

PUPSP1 — SPI Port Pullup Enable Bit 1
 1 = Pullup devices enabled for pins 7–4
 0 = Pullup devices disabled for pins 7–4

PUPSP0 — SPI Port Pullup Enable Bit 0
 1 = Pullup devices enabled for pins 3–0
 0 = Pullup devices disabled for pins 3–0

NOTE:
If a pin is programmed as output, the pullup device becomes inactive.

16.12.2 SPI Port Data Register

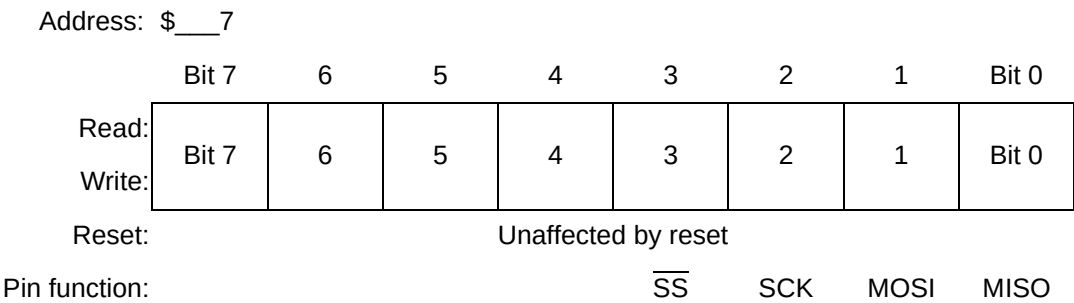


Figure 16-12 . SPI Port Data Register (SPIPORT)

Read: anytime
 Write: anytime

Bits 7–0 — SPI Port Data Bits

Data written to SPIPORT drives pins only when they are configured as general-purpose outputs.
 Reading an input (data direction bit is clear) returns the pin level; reading an output (data direction bit is set) returns the pin driver input level.

Writes do not change the state of pins 3–0 when the pin is configured for SPI output.

SPIPORT I/O function depends upon the state of the SPE bit in SPI control register 1 and the state of each associated data direction bit in SPIDDR.

CAUTION:

To ensure that you read the value present on the SPIPORT pins, always wait at least one cycle after writing to the SPIDDR register before reading from the SPIPORT register.

Table 16-6 . SPI Port Summary

Pullup Enable Control			Reduced Drive Control			Wired-OR mode control		
Register Name	Bit Name	Reset State	Register Name	Bit Name	Reset State	Register Name	Bit Name	Reset State
SPIPURD	PUPSP[1:0]	1	SPIPURD	RDPSP[1:0]	Full drive	SPICR1	SWOM	Normal

NOTES:

1. Determined at MCU definition

16.12.3 SPI Port Data Direction Register

Address: \$__8

	Bit 7	6	5	4	3	2	1	Bit 0
Read:	Bit 7	6	5	4	3	2	1	Bit 0
Write:	Bit 7	6	5	4	3	2	1	Bit 0
Reset:	0	0	0	0	0	0	0	0

Figure 16-13 . SPI Port Data Direction Register (SPIDDR)

Read: anytime

Write: anytime

In SPI slave mode, SPIDDR bit 3 has no meaning or effect. In SPI master mode, SPIDDR bit 3 determines if SPI port pin 3 is an error-detect input to the SPI, a general-purpose output, or a slave select output line.

NOTE:

When the SPI is enabled, MISO, MOSI, and SCK:

Serial Peripheral Interface Module

- *Are inputs if they are expected to be inputs regardless of the state of the associated data direction bit*
- *Are outputs if they are expected to be outputs only if the associated data direction bit is set*

SPIDDR Bits 7–0 — SPI Port Data Direction Control Bits

1 = Associated pin is an output.

0 = Associated pin is an input.

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Section 17 DSRC

17.1 DSRC

The Dedicated Short Range Communication(DSRC) module is provided to capable data modulation/demodulation to communicate with RF module on out side of the chip.

The DSRC has 2 input and 3 output to RF module. Rxd is provided as data input from RF module. WAKE is provided as input from RF module to wake this module up. Txd is provide as data output to RF module. Txrnask is provided as transmit disable control to RF module. Rxmask is provided as receive disable control signal to RF module. Communication speed is 1024K BPS and data is coded as modified mancheser coding. Both Tx and Rx have internal data buffer. Each of their size are 512byte for Tx adn 256byte for Rx.

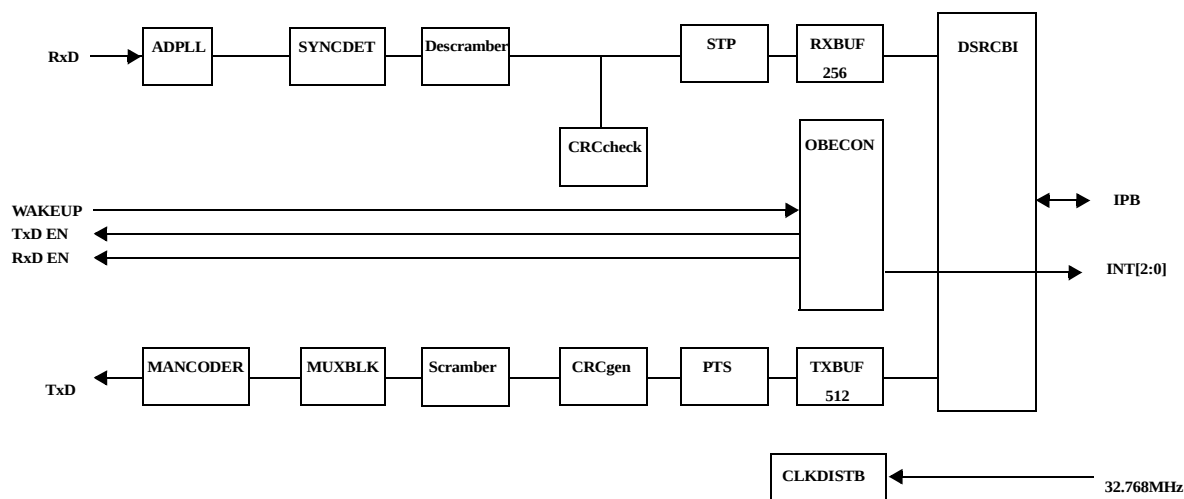


Figure 17-1 DSRC Block Diagram

17.1.1 DSRC Operation

The operation of DSRC will not be described in the this version of specification due to the module is designed by customer and IP is belonging to customer.

17.1.2 DSRC in Low-Power Modes

The DSRC is stopped at STOP mode. In the WAIT mode, DSRC can operate as nomally and may create interrupt to wake up CPU.

17.1.3 DSRC MyLIDH register(MyLIDH)

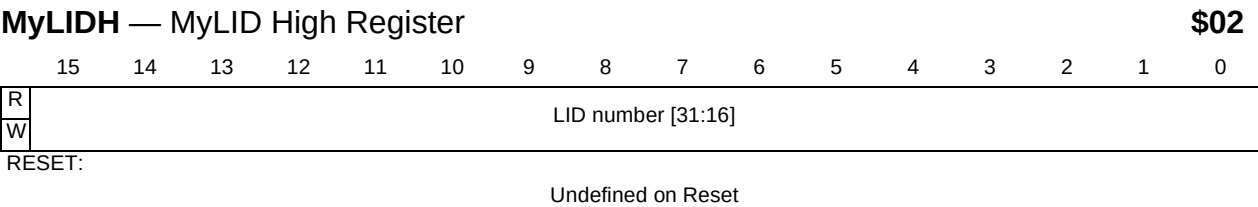


Figure 17-2 MyLID high Register

Access this register with 16-bit loads and stores only.
 This register is not affrected by reset.

17.1.4 DSRC MyLIDL register(MyLIDL)

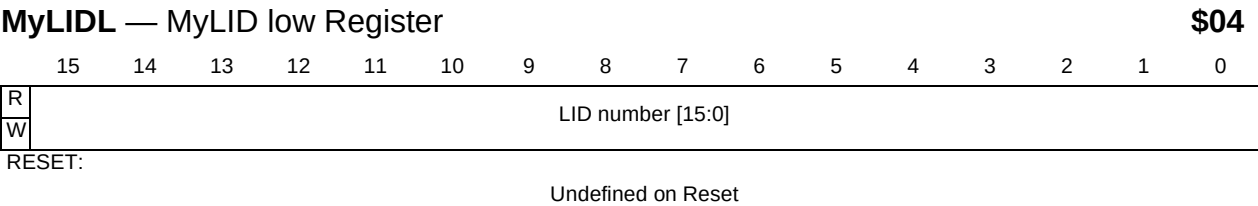


Figure 17-3 MyLID low Register

Access this register with 16-bit loads and stores only.
 This register is not affrected by reset.

17.1.5 DSRC GID register(GIDR)

GIDR — GID Register															\$06	
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	GID number [7:0]							
W	-	-	-	-	-	-	-	-								
RESET:																
	0	0	0	0	0	0	0	0	Undefined on Reset							

Figure 17-4 GID Register

Access this register with 16-bit loads and stores only.

This register is not affected by reset.

17.1.6 DSRC ACTC register(ACTCR)

ACTCR — ACTC Register															\$08	
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	TXDE	ACTS				ACTC		
W	-	-	-	-	-	-	-	-								
RESET:																
	0	0	0	0	0	0	0	0	0		0				0	

Figure 17-5 ACTC Register

Access this register with 16-bit loads and stores only.

TXDE - TxD Enable

This bit is cleared by reset.

0 = TxD is not enabled.

1 = TxD is enabled in ACTS slot

ACTS - Slot number of ACTS

These bits are cleared by reset.

ACTC - Slot number of ACTC in ACTS

These bits are cleared by reset.

DSRC

17.1.7 DSRC MyKey register(MyKeyR)

MyKeyR — MyKey Register

\$0A

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Scramble Key[15:0]															
W																
RESET:	Undefined on Reset															

Figure 17-6 MyKey Register

Access this register with 16-bit loads and stores only.

This register is not affected by reset.

17.1.8 DSRC BrSLT register(BrSLTR)

BrSLTR — Multi Broad Casting Register

\$0C

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	BrSLT[7:0]							
W	-	-	-	-	-	-	-	-								
RESET:	0	0	0	0	0	0	0	0	Undefined on Reset							

Figure 17-7 BrSLT Register

Access this register with 16-bit loads and stores only.

Upper byte is always read 0 and no meaning to write upper byte of the register.

17.1.9 DSRC INTF register(INTFR)

INTFR — Interrupt Flag Register

\$10

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	0	0	0	INT2	INT1	INT0
W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 17-8 INT Flag egister

Access this register with 16-bit loads and stores only.

Upper 13 bits are always read 0 and no meaning to write.

17.1.10 DSRC INTE register(INTER)

INTER — Interrupt enable Register

\$12

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	0	0	0	INTE2	INTE1	INTE0
W	-	-	-	-	-	-	-	-	-	-	-	-	-			
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 17-9 INT Enable egister

Access this register with 16-bit loads and stores only.

Upper 13 bits are always read 0 and no meaning to write.

17.1.11 DSRC INTC register(INTEC)

INTER — Interrupt flag Clear Register

\$14

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W	-	-	-	-	-	-	-	-	-	-	-	-	-	INTC2	INTC1	INTC0
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 17-10 INTF Clear register

Access this register with 16-bit loads and stores only.

Upper 13 bits are always read 0 and no meaning to write.

17.1.12 DSRC DRV register(DRVR)

DRVR — MDS Direction Register

\$16

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	DR_Vec[7:0]							
W	-	-	-	-	-	-	-	-	-							
RESET:	0	0	0	0	0	0	0	0	Undefined on Reset							

Figure 17-11 BrSLT Register

DSRC

Access this register with 16-bit loads and stores only.

Upper byte is always read 0 and no meaning to write upper byte of the register.

17.1.13 DSRC BMG register(BMGR)

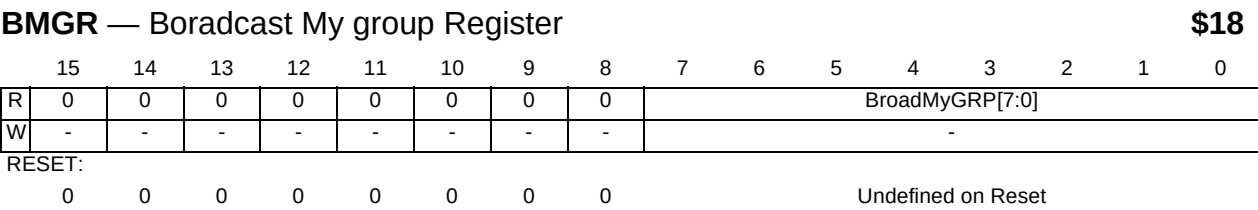


Figure 17-12 BrSLT Register

Access this register with 16-bit loads and stores only.

Upper byte is always read 0 and no meaning to write upper byte of the register.

17.1.14 DSRC Status register(STATR)

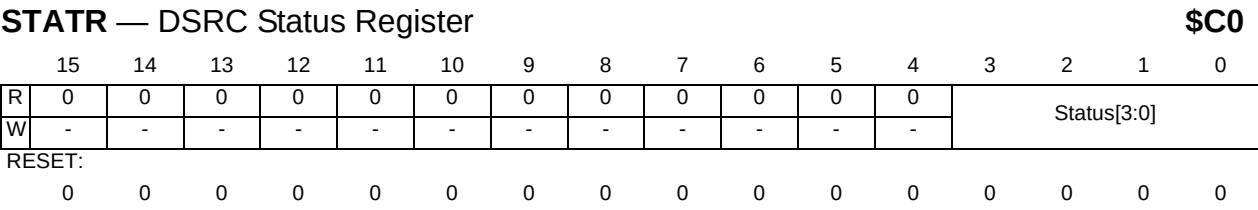


Figure 17-13 INT Flag egister

Access this register with 16-bit loads and stores only.

Upper 12 bits are always read 0 and no meaning to write.

Table 17-1Status code

Status[3:0]	Slot	FSW/CSW Detect	CRC
'0000'	-	-	-
'0001'	MDS(Down link) (Broadcaasting/groupcasting)	Detected(CSW)	passed
'0010'	MDS(Down link) (Broadcaasting/groupcasting)	Detected(CSW)	failed

Table 17-1Status code

Status[3:0]	Slot	FSW/CSW Detect	CRC
'0011'	MDS(Down link) (Broadcaasting/groupcasting)	failed(CSW)	-
'0100'	-	-	-
'0101'	MDS(Up link) (Broadcaasting/groupcasting)	Detected(CSW)	passed
'0110'	MDS(Up link) (Broadcaasting/groupcasting)	Detected(CSW)	failed
'0111'	MDS(Up link) (Broadcaasting/groupcasting)	failed(CSW)	-
'1000'	MDS(Down link)	Detected(CSW)	passed
'1001'	MDS(Down link)	Detected(CSW)	failed
'1010'	MDS(Up link)	Detected(CSW)	passed
'1011'	MDS(Up link)	Detected(CSW)	failed
'1100'	FCMS	Detected(FSW)	passed
'1101'	FCMS	Detected(FSW)	failed
'1110'	MDS(Down link)	failed(CSW)	-
'1111'	MDS(Up link)	failed(CSW)	-

17.1.15 DSRC Soft Reset(SWRR)

SWRR — Software Reset Register

\$E0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W	Software Reset															

RESET:

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Figure 17-14 BrSLT Register

Access this register with 16-bit loads and stores only.

Any data write to this register, module is reset to initial conditions.

DSRC

17.1.16 DSRC MDCE(MDCER)

MDCER — MDC enable Register

\$EC

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W	MDC Ready															
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 17-15 BrSLT Register

Access this register with 16-bit loads and stores only.

Any data write to this register, MDC gets ready.

17.1.17 DSRC ACTE(ACTER)

ACTER — ACT enable Register

\$EE

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
W	ACT Ready															
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 17-16 BrSLT Register

Access this register with 16-bit loads and stores only.

Any data write to this register, ACTC gets ready.

17.1.18 TxBUF

This DSRC module has 512byte of the transmitt data buffer. CPU can write data to this buffer directly. The address of this buffer is \$400 - \$5FF.

17.1.19 RxBUF

This DSRC module has 256byte of the recive data buffer. CPU can write data to this buffer directly. The address of this buffer is \$800 - \$8FF.

Section 18 Chip Selects

18.1 Overview

Typical systems require some external hardware to provide chip selects for external peripherals. Many of these devices do not have all the signals that are required for an asynchronous bus interface. Additionally, higher levels of integration can provide cost, speed, and reliability advantages over external logic. The chip selects described here provide the means to minimize this external logic.

The primary function of the chip selects is to provide the chip enables ($\overline{CE}$) for external memory and peripheral devices. The chip selects may also be programmed to terminate bus cycles, eliminating the external glue logic required to terminate the cycle.

There are up to eight asynchronous chip select signals available. All chip select pins are optional and may not be included on a given MCU implementation.

The chip selects have the following features:

- Reduced system complexity.
 - No external glue logic required for typical systems, if the chip selects are used.
- Eight programmable asynchronous active-low chip selects ($\overline{CS7}$ - $\overline{CS0}$).
 - Chip selects may be independently programmed with various features.
- Control for external boot device.
 - $\overline{CS0}$ enabled at reset to optionally select an external boot device.
- Fixed base addresses with 8M block sizes.
- Support for emulating internal memory space.
 - By setting the EMINT bit in the CCR register, $\overline{CS1}$ will only match addresses in the internal memory space.
- Support for 16-bit and 32-bit external devices.
 - The external port size can be programmed to be 16 or 32 bits.
- Programmable write protection.
 - Each chip select address range can be designated for read access only.
- Programmable access protection.
 - Each chip select address range can be designated for supervisor access only.
- Write Enable Selection.

Chip Selects

- The Enable Byte ($\overline{EB}[3:0]$) pins may be configured as byte enables (assert on both external read and write accesses) or write enables (only assert on external write accesses).
- Bus cycle termination.
 - This option allows the chip select logic to terminate the bus cycle.
- Programmable wait states.
 - Up to 7 wait states can be programmed, before the access is terminated, to interface with various devices.
- Programmable extra wait state for write accesses.
 - One wait state can be added to write accesses to allow writing to memories that require additional data setup time.

Figure 18-1 Chip Select Functional Block Diagram shows a programmable asynchronous chip select. All asynchronous chip selects have the same structure. All signals used to generate chip select signals are taken from the internal bus. Each chip select has a Chip Select Control Register, which contains the programmable characteristics of each chip select.

There is only one cycle termination generator in the chip select submodule, and it is shared by all of the chip selects. The active chip select for a particular bus cycle determines the number of wait states produced by the cycle termination generator before the cycle is terminated.

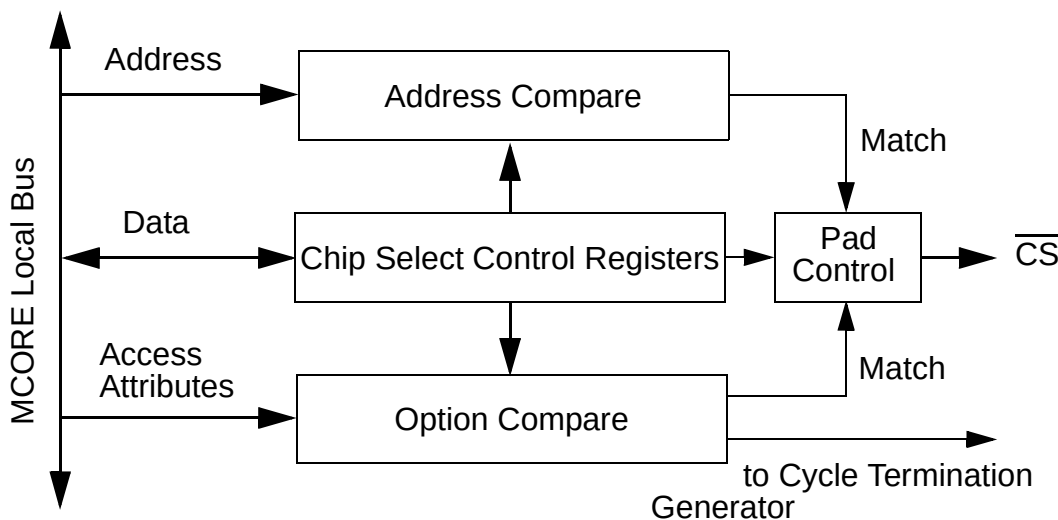


Figure 18-1 Chip Select Functional Block Diagram

18.1.1 Operation

Each chip select can provide a chip enable signal for an external device and also assert the internal bus cycle termination signal.

To enable the chip select to provide an external chip enable, the chip select function must be enabled by setting the CSEN bit in the Chip Select Control Register. To enable the internal generation of a termination signal to terminate a bus cycle, both the CSEN and TAEN bits must be set in the Chip Select Control Register. Both the chip select pin assertion and the bus cycle termination function depend on an initial address/option match for activation.

During the matching process, the fixed base addresses for each chip select are compared against the corresponding address for the bus cycle to determine whether an address match has occurred. This match is further qualified by a comparison of the internal read/write indication and access type with the programmed values in each chip select's control register. When the address and option information match the current cycle, the chip select is activated. If no chip select matches the bus cycle information for the current access, the chip select logic does not respond in any way.

Only one chip select can be active for a bus cycle. The active chip select configuration is used for the access as determined by the wait state (WS/WWS) field, the port size (PS) field, and the write enable (WE) field. (Note: WWS and WS are valid only if TAEN is enabled for the active chip select.)

When no chip select pin is available, the active chip select can still terminate the bus cycle. If both the CSEN and TAEN bits are set and the address/options match the chip select configuration, then the internal termination signal is asserted by the chip select logic to terminate the bus cycle after the programmed number of wait states expires. If the external TA or TEA pins are asserted before cycle termination signal is asserted internally by the chip select logic, then the bus cycle is terminated early.

The internal/external address space is defined by the internal address bit 31. If bit 31 is low, then the access is internal. If bit 31 is high, then the access is external. (Internal address bits 30 to 25 are not decoded by the chip select logic.)

Table 18-1 Chip Select Address Range Encoding

Chip Select	Block Size (bytes)	Address Bits Compared (A31-A24) ¹
CS0	16M	1xxx_x000
CS1	16M	1xxx_x001 ²
CS2	16M	1xxx_x010
CS3	16M	1xxx_x011
CS4	16M	1xxx_x100

Chip Selects

Table 18-1 Chip Select Address Range Encoding

Chip Select	Block Size (bytes)	Address Bits Compared (A31-A24) ¹
CS5	16M	1xxx_x101
CS6	16M	1xxx_x110
CS7	16M	1xxx_x111

NOTES:

1. A30-A27 are not decoded by the chip selects. Thus, the total 128M block size is repeated/mirrored in the external memory space.
2. If the EMINT bit the CCR register is enabled, then CS1 will only match internal accesses to the 16M block starting at address 0 to support emulation of internal memory. Thus, A31-A24 will match 0xxx_x000.

18.2 Programming Model

The Chip Select programming model consists of the following registers:

- The Chip Select Control Register 0 - 7 (CSCR0-7) selects different chip select functions.

Each chip select is configured by a Chip Select Control Register. There are eight Chip Select Control Registers (CSCR0 - CSCR7), one for each chip select (CS0 - CS7), respectively. These registers are read/write always.

Table 18-2 Chip Select Address Map

Offset ¹	31 - 16	15 - 0	Access ²
\$00	CSCR0	CSCR1	Supervisor Only
\$04	CSCR2	CSCR3	Supervisor Only
\$08	CSCR4	CSCR5	Supervisor Only
\$0C	CSCR6	CSCR7	Supervisor Only

NOTES:

1. Absolute address is equal to the Offset plus the base address. The base address is chip dependent.
2. User mode accesses to supervisor only address locations have no effect and result in a cycle termination transfer error.

18.2.1 Chip Select Control Registers (CSCR0 - CSCR7)

The Chip Select Control Registers define the conditions for asserting the chip select signals.

All the Chip Select Control Registers are the same except for the reset state of the CSEN and PS bits in CSCR0 and the CSEN bit in CSCR1. This allows CS0 to be enabled at reset with either a 16-bit or 32-bit port size to be used to select an external boot device and CS1 to be used to emulate internal memory.

CSCR0 — Chip Select Control Register 0

15	14	13	12	11	10 - 8	7 - 2	1	0
SO	RO	PS	WWS	WE	WS	0	TAEN	CSEN
RESET:								
0	0	*	1	1	111	0	1	*

* Reset state determined during reset configuration. (See Section 3.3.2 Boot Device Selection for details)

CSCR1 — Chip Select Control Register 1

15	14	13	12	11	10 - 8	7 - 2	1	0
SO	RO	PS	WWS	WE	WS	0	TAEN	CSEN
RESET:								
0	0	1	1	1	111	0	1	*

* Reset state determined during reset configuration. (See Section 3.3.2 Boot Device Selection for details)

CSCR2 - CSCR7 — Chip Select Control Registers 2 - 7

15	14	13	12	11	10 - 8	7 - 2	1	0
SO	RO	PS	WWS	WE	WS	0	TAEN	CSEN
RESET:								
0	0	1	1	1	111	0	1	0

SO — Supervisor Only

The SO bit is used to restrict user level accesses to the address range defined by the corresponding chip select. If the SO bit is set, access is permitted at the supervisor privilege level only. If the SO bit is cleared, both supervisor and user level accesses are permitted.

When an access is made to a memory space assigned to the chip select, the chip select logic compares the SO bit with the internal transfer code bit 2, which indicates whether the access is at the supervisor (tc[2] = 1) or user (tc[2] = 0) privilege level. If the chip select logic detects a protection violation (SO = 1 with tc[2] = 0), the access is ignored.

0 = Supervisor and user mode accesses are allowed.

1 = Only supervisor mode accesses are allowed. User mode accesses are ignored by the chip select logic.

RO — Read Only

Chip Selects

The RO bit is used to restrict write accesses to the address range defined by the corresponding chip select. If the RO bit is set, only read access is permitted. If the RO bit is cleared, both read and write accesses are permitted.

When an access is made to a memory space assigned to the chip select, the chip select logic compares the RO bit with the internal read/write signal, which indicates whether the access is a read (read/write = 1) or a write (read/write = 0). If the chip select logic detects a violation (RO = 1 with read/write = 0), the access is ignored.

0 = Read and write accesses are allowed.

1 = Only read accesses are allowed. Write accesses are ignored by the chip select logic.

PS — Port Size

The PS bit defines the width of the external data port supported by the chip select as either 16-bit or 32-bit. When a chip select is programmed as an 16-bit port, the external device must be connected to D[31:16]. For 32-bit accesses to 16-bit ports, the External Memory Interface initiates two bus cycles and provides multiplexing of data as needed to satisfy the data transfer.

0 = 16 bit port.

1 = 32 bit port.

WWS — Write Wait State

The WWS bit is used to determine if an additional wait state is required for write cycles. Read cycles are not affected by this bit.

0 = No additional wait state added for write cycles.

1 = One additional wait state is added for write cycles.

WE — Write Enable

The WE bit defines when the Enable Byte ($\overline{EB[3:0]}$) output pins are asserted. When WE is zero, $\overline{EB[3:0]}$ are configured as byte enables and assert for both external read and write accesses. When WE is set, $\overline{EB[3:0]}$ are configured as write enables and only assert for external write accesses.

0 = $\overline{EB[3:0]}$ configured as byte enables.

1 = $\overline{EB[3:0]}$ configured as write enables.

NOTE:

The WE bit has no effect on the $\overline{EB[3:0]}$ pin function if the chip select is not active. If the chip select is not active, the $\overline{EB[3:0]}$ pin function is byte enable by default.

WS — Wait States

The WS field determines the number of wait states for the chip select logic to insert before asserting the internal cycle termination signal. One wait state is equal to one system clock cycle. If WS is configured for zero wait states, then the internal cycle termination signal is asserted in the clock cycle following the start of the cycle access, resulting in one clock transfers. A WS configured for one wait state means that the internal cycle termination signal is asserted two clocks cycles after the start of the cycle access.

Since the internal cycle termination signal is asserted internally after the programmed number of wait states expires, the user can adjust the bus timing to accommodate the access speed of the external device. With up to 7 wait states selectable, even slow devices can be interfaced with the MCU.

Table 18-3 Chip Select Wait States Encoding

WS[2]	WS[1]	WS[0]	Number of Wait States			
			WWS = 0		WWS = 1	
			Read Access	Write Access	Read Access	Write Access
0	0	0	0	0	0	1
0	0	1	1	1	1	2
0	1	0	2	2	2	3
0	1	1	3	3	3	4
1	0	0	4	4	4	5
1	0	1	5	5	5	6
1	1	0	6	6	6	7
1	1	1	7	7	7	8

TAEN — Transfer Acknowledge Enable

The TAEN bit determines whether the internal cycle termination signal is asserted by the chip select logic when accesses occur to the address range defined by the corresponding chip select. When TAEN is cleared, an external device is responsible for asserting the external TA pin to terminate the bus access. When TAEN is set, the chip select logic asserts the internal cycle termination signal after a time determined by the programmed number of wait states. When TAEN is set, external logic can still terminate the access before the internal cycle termination signal is asserted by asserting the external TA pin.

0 = Internal cycle termination signal is asserted by external logic.

1 = Internal cycle termination signal is asserted by the chip select logic.

CSEN — Chip Select Enable

Chip Selects

The CSEN bit controls the operation of the chip select logic. When the chip select function is disabled, the $\overline{CS}$ signal is negated high.

0 = Chip select function disabled.

1 = Chip select function enabled.

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Section 19 External Bus Interface

The External Bus Interface (EBI) is responsible for controlling the transfer of information between the internal M-CORE local bus and external address space.

19.1 Overview

In Master Mode, the EBI functions as a bus master and allows internal bus cycles to access external resources. In Factory Access Slave Test (FAST) Mode, the EBI allows an external master to access internal modules for testing purposes. The EBI is active in Single Chip Mode, however the external data bus is not available and no external data or termination signals are transferred to the internal bus.

The EBI supports data transfers to both 32-bit and 16-bit ports. Chip select channels are programmed to define the port size for specific address ranges. When no chip select is active during an external data transfer, the port size is assumed to be 32 bits.

The EBI supports a variable length external bus cycle to accommodate the access speed of any device. During an external data transfer, the EBI drives the Address, Byte and Output Enables, Size, and Read/Write pins. Wait states are inserted until the bus cycle is terminated by the assertion of the internal Transfer Acknowledge signal by a chip select channel or by the assertion of the external TA or TEA pins. The minimum external bus cycle is one clock.

The EBI also drives the Address, Size, and Read/Write pins during internal data transfers, however the Output and Byte Enables are not asserted. To see the internal data bus on the external pins, Show Cycles must be enabled. See Section 19.2.4 Show Strobe (SHS) on page 288.

All internal data transfers are terminated only by internal sources, and cannot be terminated by a chip select channel, external TA assertion, or external TEA assertion.

19.2 Bus Master Operation

19.2.1 Operand Transfer Overview

From the internal M-CORE bus, the possible operand accesses are: byte, aligned upper half-word, aligned lower half-word, and aligned word. No misaligned transfers are supported. The EBI controls the byte, half-word, or word operand transfers between the M-CORE bus and a 16-bit or 32-bit port. "Port" refers to the width of the data path that an external device utilizes during a data transfer. Each port is assigned to particular bits of the data bus. A 16-bit port is assigned to pins D[31:16] and a 32-bit port is assigned to pins D[31:0].

External Bus Interface

For the case of a word operand size to a 16-bit port size, the EBI runs two external bus cycles to complete the transfer. During the first external bus cycle, the A[1:0] pins are driven low and the TSIZ[1:0] pins are driven to indicate word size. During the second cycle, A[1] is driven high to increment the external address by 2 bytes, A[0] is still driven low, and the TSIZ[1:0] pins are driven to indicate half-word size.

During any word-size transfer, the p_addr[1:0] signals are undefined. As a result, the EBI will always drive the A[1:0] pins low during a word transfer (except on the second cycle of a word to half-word port transfer in which A[1] is incremented).

Table 19-1 Data Transfer Cases lists the combination of p_tsiz[1:0], p_addr[1:0], and port size that are used for each possible transfer size, alignment, and port width. The data bytes shown in the table represent external data pins. This data is muxed and driven to the internal or external data bus as shown. The bytes labeled with a dash are not required; the RIM may ignore them on read transfers, and they may be driven with undefined data on write transfers.

Table 19-1 Data Transfer Cases

Transfer Size	Internal M-CORE Bus		Port Width	External Pins				Data Bus Transfer			
	p_tsiz[1:0]	p_addr[1:0]		TSIZ1	TSIZ0	A1	A0	p_data [31:24]	p_data [23:16]	p_data [15:8]	p_data [7:0]
Byte	0 1	0 0	16	0	1	0	0	D[31:24]	---	---	---
			32					D[31:24]	---	---	---
		0 1	16			0	1	---	D[23:16]	---	---
			32					---	D[23:16]	---	---
		1 0	16			1	0	---	---	D[31:24]	---
			32					---	---	D[15:8]	---
		1 1	16			1	1	---	---	---	D[23:16]
			32					---	---	---	D[7:0]
Halfword	1 0	0 X	16	1	0	0	0	D[31:24]	D[23:16]	---	---
			32					D[31:24]	D[23:16]	---	---
		1 X	16			1	0	---	---	D[31:24]	D[23:16]
			32					---	---	D[15:8]	D[7:0]
Word	0 0	X X	16 ¹	0	0	0	0	D[31:24]	D[23:16]	---	---
				1		1	0	---	---	D[31:24]	D[23:16]
			32	0		0	0	D[31:24]	D[23:16]	D[15:8]	D[7:0]

NOTES:

1. The EBI runs two cycles for word accesses to 16-bit ports. The table shows the data placement for both bus cycles.

19.2.1.1 Byte Enable Pins ($\overline{EB[3:0]}$)

The EB[3:0] pins are provided as Byte Enables for read and write cycles, or optionally as Write Enables only. The default function is Byte Enable unless there is an active chip select match with the WE bit set. In all external cycles when one or more EB pins are asserted, the encoding

corresponds to the external data pins to be utilized for the transfer as outlined in Table 9-1 Data transfer cases. The EB encoding is shown in Table 19-2 EB[3:0] Assertion Encoding.

Table 19-2 $\overline{\text{EB}}[3:0]$ Assertion Encoding

$\overline{\text{EB}}$ PIN	External Data Pins
EB[0]	D[31:24]
EB[1]	D[23:16]
EB[2]	D[15:8]
EB[3]	D[7:0]

19.2.2 Bus Master Cycles

In this section, each EBI bus cycle type is defined in terms of actions associated with a succession of internal states. These internal states are only for reference and may not correspond to any implemented machine states.

Read or write operations may require multiple bus cycles to complete based on the operand size and target port size. Refer to Section 19.2.1 Operand Transfer Overview on page 279 for more information. In the discussion that follows, it is assumed that only a single bus cycle is required for a transfer.

In the waveform diagrams, data transfers are related to clock cycles, independent of the clock frequency. The external bus states are also noted.

19.2.2.1 Read Cycles

During a read cycle, the EBI receives data from an external memory or peripheral device. A flowchart of a typical read cycle operation is shown in Figure 19-1 Read Cycle Flowchart on page 283. Figure 19-3 Master Mode - 1 Clock Read and Write Cycle on page 286 and Figure 19-4 Master Mode - 2 Clock Read and Write Cycle on page 287 are waveform diagrams of external bus master cycles with and without wait states. The waveform diagrams also show M-CORE bus activity to assist in understanding.

During external read cycles, the $\overline{\text{OE}}$ pin is asserted regardless of operand size.

State 1 (X1)

The EBI drives the address bus. $\overline{\text{R}/\text{W}}$ is driven high indicating a read cycle. The TSIZ[1:0] pins are driven to indicate the number of bytes in the transfer. TC[2:0] pins are driven to indicate the type of access. $\overline{\text{CS}}$ may be asserted to drive a device CE

External Bus Interface

Later in State 1, $\overline{OE}$ is asserted. One or more $\overline{EB}$ pins are also asserted, depending on the size and position of the data to be transferred only if the $\overline{EB}$ pins are not configured as write enables for this cycle.

If either the external $\overline{TA}$ pin or internal Chip Select Transfer Acknowledge signal is asserted before the end of State 1, the EBI proceeds to State 2.

Optional Wait States (X2W)

Wait states¹ are inserted until the slave asserts the $\overline{TA}$ pin or the internal Chip Select Transfer Acknowledge signal is asserted.

State 2 (X2)

One-half clock later in State 2, the selected device places its information on the data bus (D[31:16] and/or D[15:0]). One or both half-words of the external data bus are driven to the internal data bus.

The address bus, $R/\overline{W}$, $\overline{CS}$, $\overline{OE}$, $\overline{EB}$, TC, and TSIZ pins remain valid through State 2 to allow for static memory operation and signal skew.

The slave device asserts data until it detects the negation of $\overline{OE}$ and must remove its data within approximately one-half of a state after recognizing the negation of $\overline{OE}$. Note that the data bus may not become free until State 1.

NOTES:

1. Wait states are counted in full clocks.



External Bus Interface

19.2.2.2 Write Cycles

On a write cycle, the EBI transfers data to an external memory or peripheral device. Figure 19-2 Write Cycle Flowchart describes write cycle operation. Figure 19-3 Master Mode - 1 Clock Read and Write Cycle on page 286 and Figure 19-4 Master Mode - 2 Clock Read and Write Cycle on page 287 are timing diagrams of external bus master cycles with and without wait states. The timing diagrams also show M-CORE bus activity to assist in understanding.

State 1 (X1)

The EBI drives the address bus. The TSIZ[1:0] pins are driven to indicate the number of bytes in the transfer. TC[2:0] pins are driven to indicate the type of access. $\overline{CS}$ may be asserted to drive a device CE OE is negated.

Later in State 1, $R/\overline{W}$ is driven low indicating a write cycle. One or more $\overline{EB}$ pins are asserted, depending on the size and position of the data to be transferred.

If either the external $\overline{TA}$ pin or internal Chip Select Transfer Acknowledge signal is asserted before the end of State 1, the EBI proceeds to State 2.

Optional Wait States (X2W)

Wait states¹ are inserted until the slave asserts the $\overline{TA}$ pin or the internal Chip Select Transfer Acknowledge signal is asserted. The EBI drives its data onto the data bus (D[31:16] and/or D[15:0]) on the first optional wait state.

State 2 (X2)

The EBI drives its data onto the data bus (D[31:16] and/or D[15:0]) in State 2 (if not already driven during optional wait states).

$\overline{EB}$ is negated by the end of State 2. The address bus, data bus, $R/\overline{W}$, $\overline{CS}$, TC[2:0], and TSIZ[1:0] pins remain valid through State 2 to allow for static memory operation and signal skew.

NOTES:

1. Wait states are counted in full clocks.

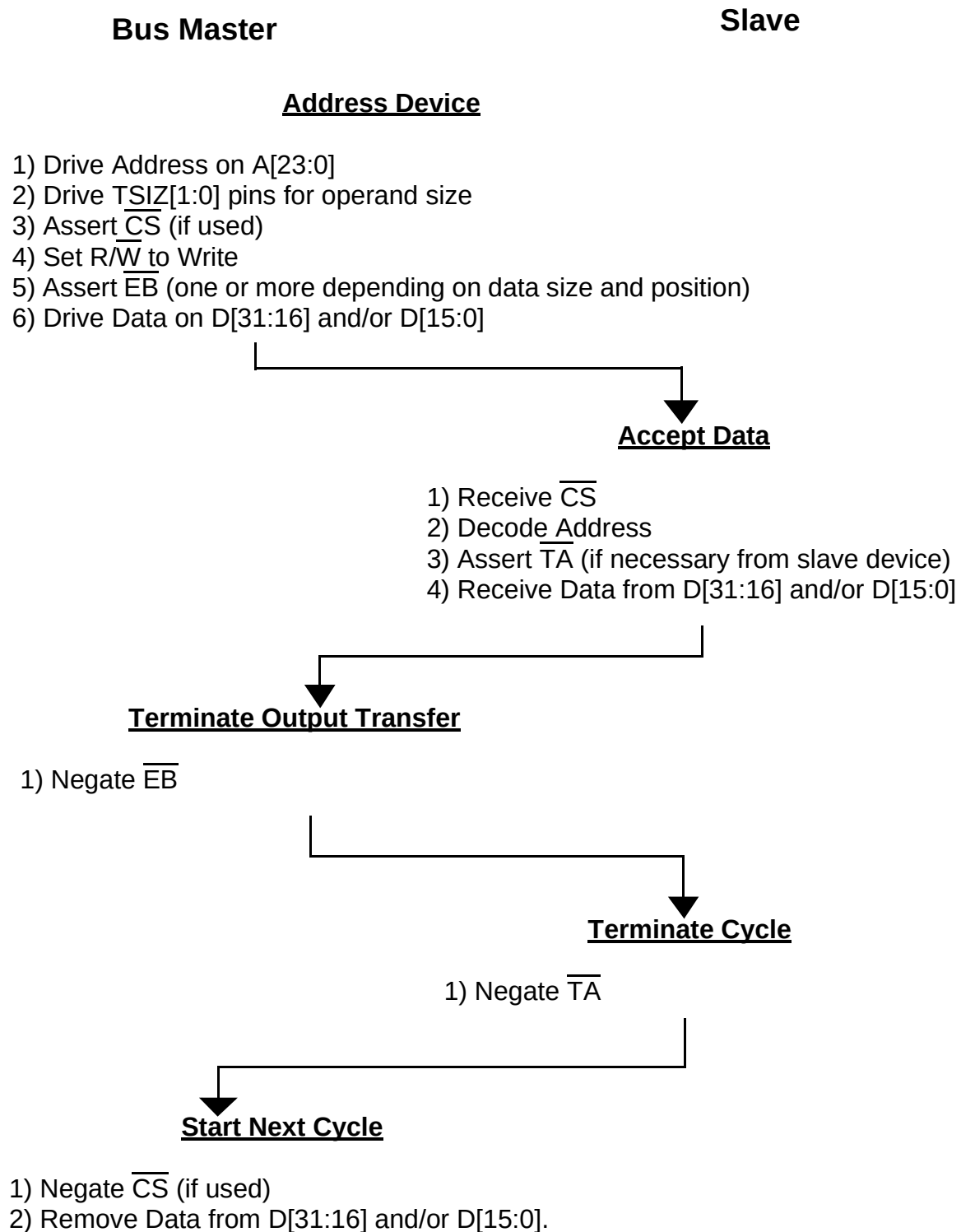
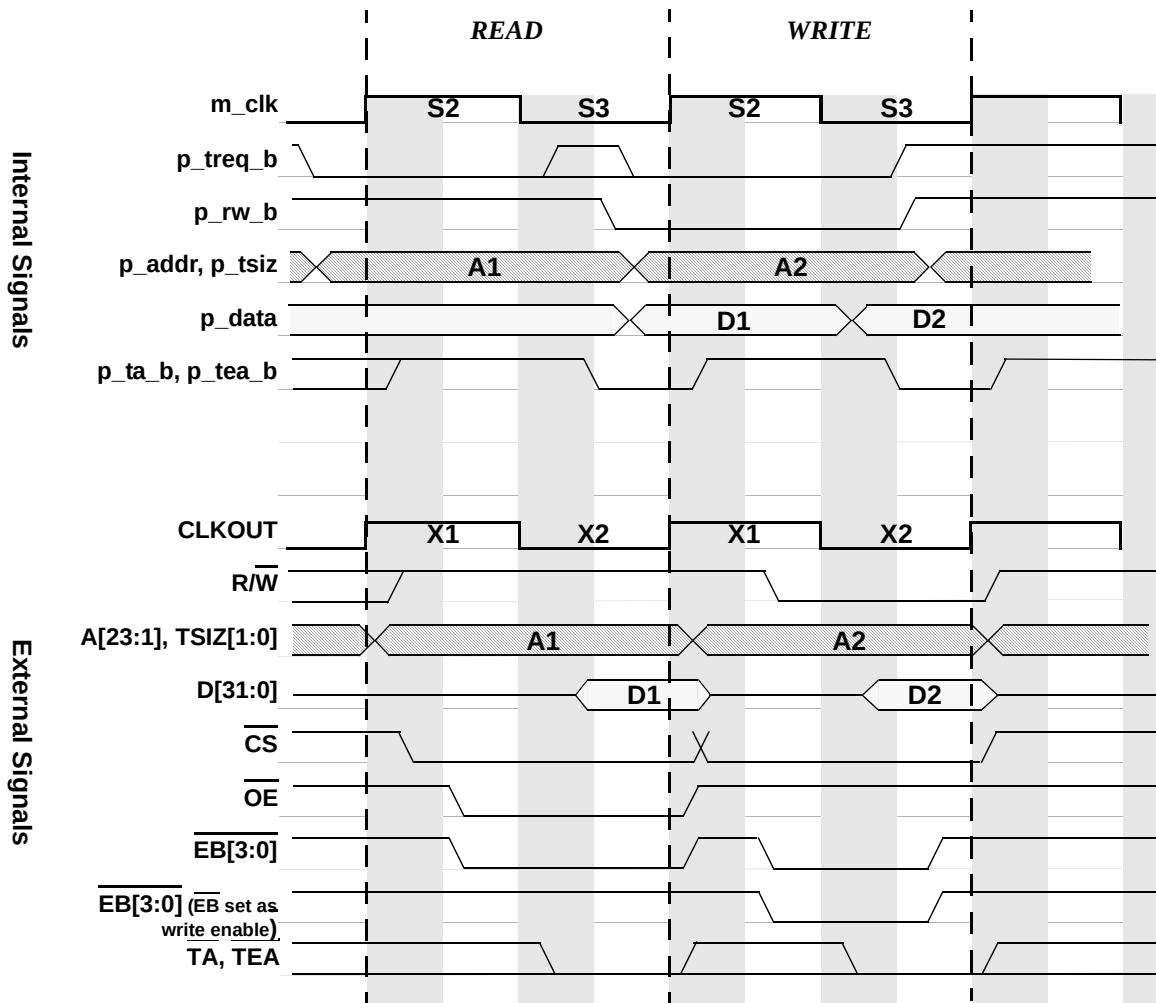


Figure 19-2 Write Cycle Flowchart



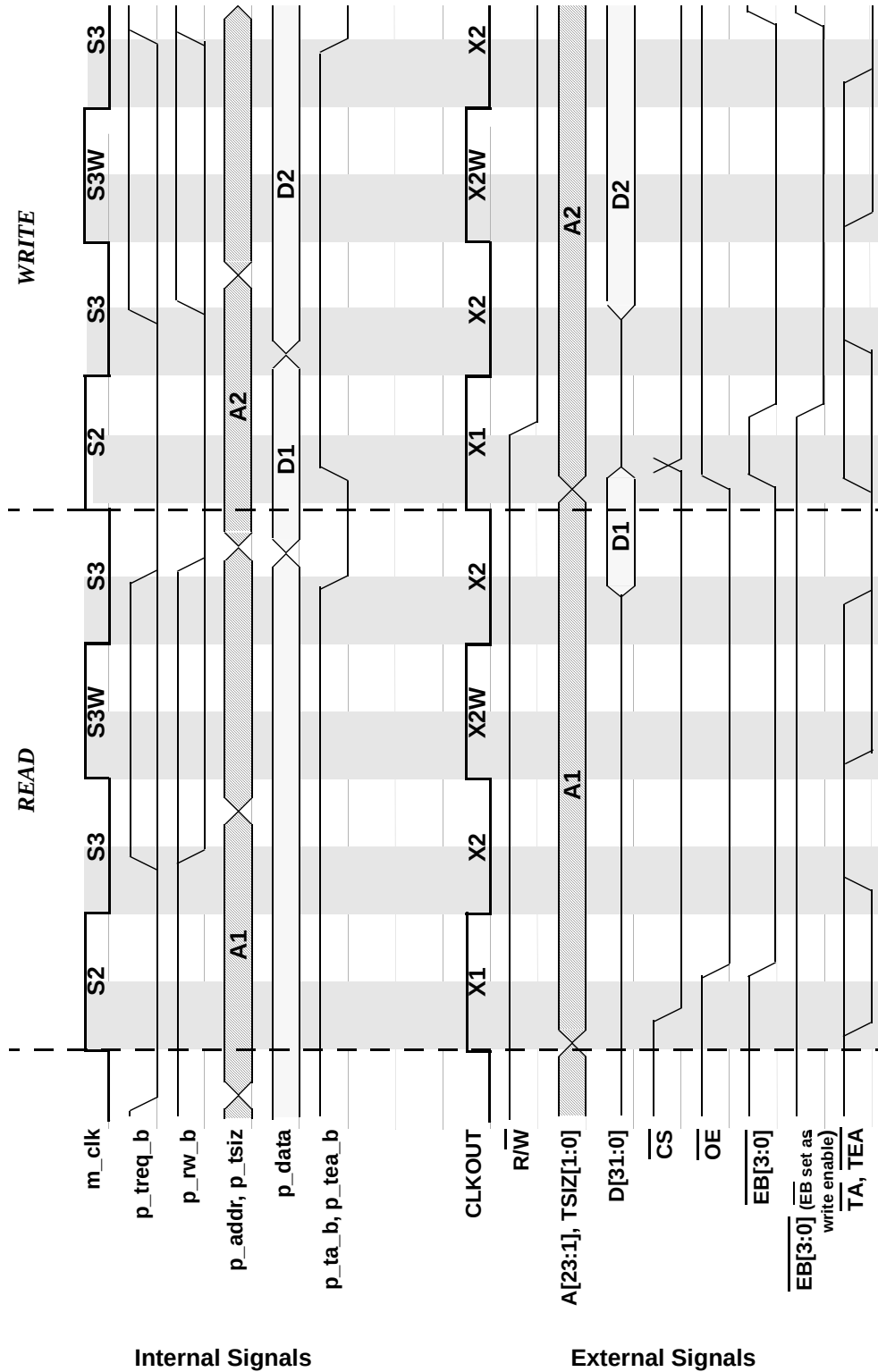


Figure 19-4 Master Mode - 2 Clock Read and Write Cycle

External Bus Interface

19.2.3 Bus Exception Operation

19.2.3.1 Transfer Error Termination

Normal bus cycle termination requires the assertion of the $\overline{\text{TA}}$ pin or the internal Transfer Acknowledge signal. Minimal bus exception support is provided by Transfer Error cycle termination. For Transfer Error cycle termination, the external $\overline{\text{TEA}}$ pin or the internal Transfer Error Acknowledge signal is asserted. Transfer Error cycle termination takes precedence over normal cycle termination, provided $\overline{\text{TEA}}$ assertion meets its timing constraints.

The RIM provides an internal bus monitor which optionally asserts the internal Transfer Error Acknowledge signal when $\overline{\text{TA}}$ response time is too long.

19.2.3.2 Transfer Abort Termination

External bus cycles which are aborted by the master using the p_abort_b signal, still have the address, $\text{R}/\overline{\text{W}}$, $\text{TC}[2:0]$, $\text{TSIZ}[1:0]$, $\overline{\text{CS}}$ (if used), $\overline{\text{OE}}$ (reads only), and $\overline{\text{SHS}}$ (if used) driven to the external pins.

19.2.4 Show Strobe ($\overline{\text{SHS}}$)

The Show Strobe ($\overline{\text{SHS}}$) pin provides an indication to an external device (emulator or logic analyzer) when to latch Address, $\text{TC}[2:0]$, $\text{TSIZ}[1:0]$, $\text{R}/\overline{\text{W}}$, PSTAT , and Data from the external pins. In Master Mode, the $\overline{\text{SHS}}$ pin may be enabled by writing to the PEPAR register.

For any external cycle or Show Cycle, the $\overline{\text{SHS}}$ pin is driven low to indicate valid Address, TC , TSIZ , $\text{R}/\overline{\text{W}}$, and PSTAT are present at the pins, and driven back high to indicate valid Data. The following waveforms describe the Show Strobe. The $\overline{\text{SHS}}$ pin is driven low and back high only once per bus cycle.

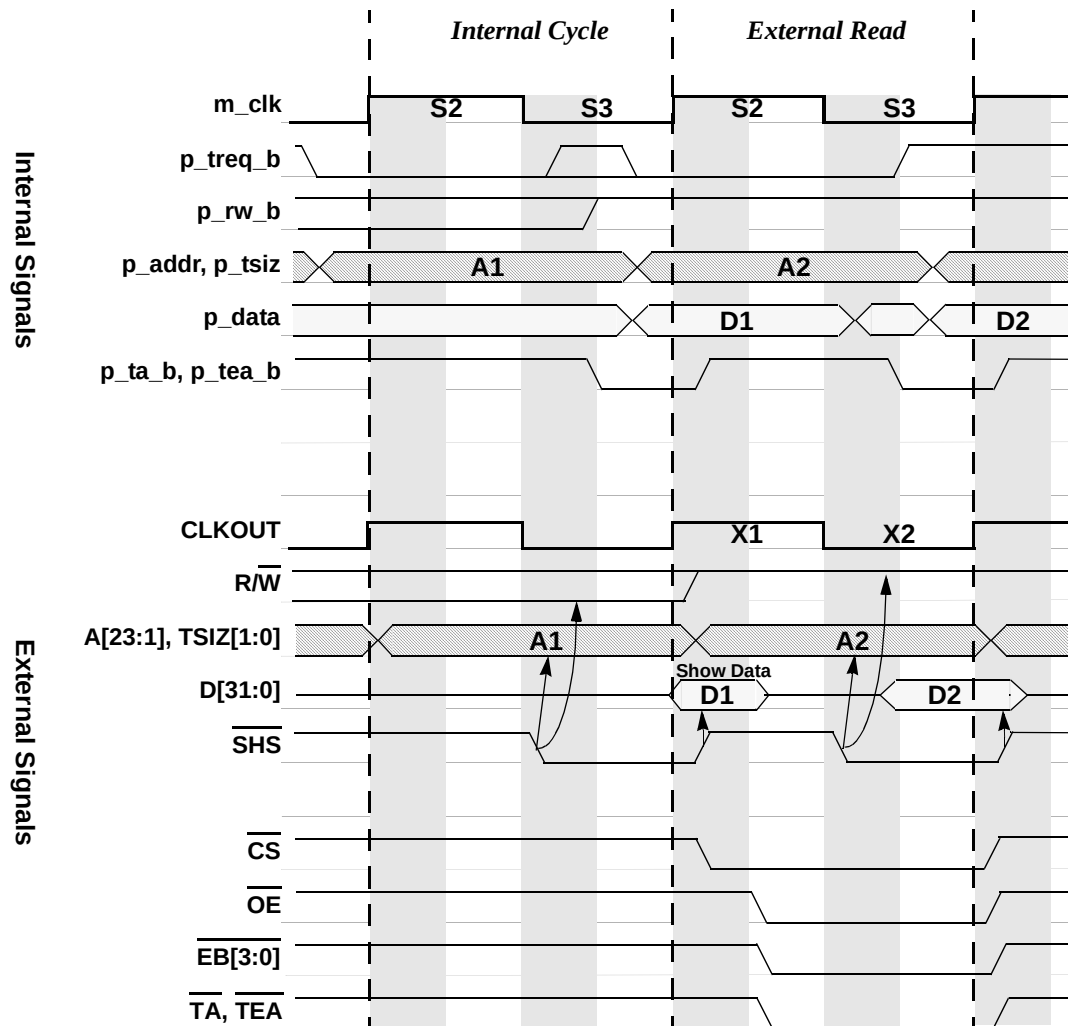


Figure 19-5 Internal (Show) Cycle followed by External 1 - Clock Read

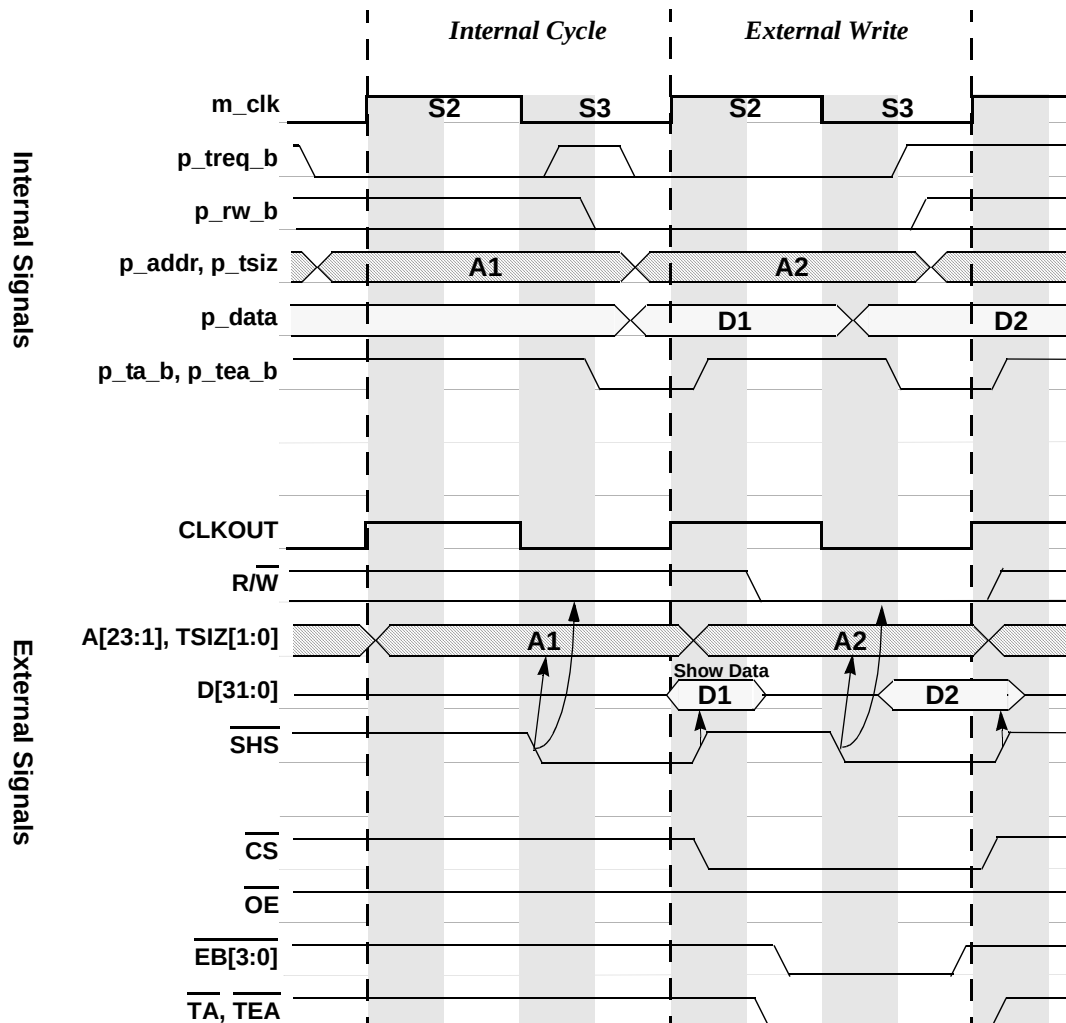


Figure 19-6 Internal (Show) Cycle followed by External 1 - Clock Write

19.3 Monitors

19.3.1 Bus Monitor

Typical bus systems require a bus monitor to detect excessively long bus access termination response times. Whenever an un-decoded address is accessed or a peripheral is inoperative, the access is not terminated and the bus is locked up while it waits for the required response.

The Bus Monitor monitors the cycle termination response times during a bus cycle. If the cycle termination response time exceeds a programmed count, the Bus Monitor asserts an internal bus error (p_tea_b).

The Bus Monitor monitors the cycle termination response time (in system clock cycles) by using a programmable maximum allowable response period. There are four selectable response time periods for the Bus Monitor, selectable among 8, 16, 32, and 64 system clock cycles. The periods are selectable with the BMT field in the Chip Configuration Register. The programmability of the time-out allows for varying external peripheral response times. The timing mechanism is derived from taps off a six stage divider chain clocked by the system clock. The taps are taken at the “÷8”, “÷16”, “÷32”, and “÷64” stages. The divider chain is cleared and restarted on all bus accesses. If the cycle is not terminated within the selected response time, a time-out occurs and the Bus Monitor asserts the internal `p_tea_b` to terminate the bus cycle.

The Bus Monitor can be configured with the BME bit in the Chip Configuration Register to monitor only internal bus accesses or both internal and external bus accesses. Also, the Bus Monitor can be disabled during debug mode for both internal and external accesses.

For internal bus cycles, the `p_ta_b` and `p_tea_b` signals are used to determine whether the cycle has been terminated. For external bus cycles, the Bus Monitor does not monitor the internal `p_ta_b` and `p_tea_b` signals but monitors the termination signals from the external TA and TEA pins and the termination signal from the chip selects.

Two external bus cycles are required for a single 32-bit access to a 16-bit port. If the Bus Monitor is enabled to monitor external accesses, then the Bus Monitor views the 32 bit access as two separate external bus cycles and not as one internal bus cycle.

19.3.2 Abort Monitor

Any bus access in progress can be aborted by the master in the first cycle of the access by asserting the `p_abort_b` signal. For aborted accesses, modules on the bus must not perform any write actions during write cycles. The next internal bus access may start after the next falling edge of `m_clk`.

Although the master has aborted the bus access, it still expects a termination signal (`p_tea_b`) to be asserted on the next rising `m_clk` after the `p_abort_b` signal is asserted. The Abort Monitor is responsible for monitoring the `p_abort_b` signal and is always enabled. If `p_abort_b` is asserted during a bus access, then the Abort Monitor asserts `p_tea_b` to terminate the aborted bus access.

NOTE:

The `p_tea_b` assertion to terminate an aborted access is not considered an exception by the CPU.

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Section 20 JTAG

20.1 IEEE 1149.1 Test Access Port

Avalanche provides a dedicated user-accessible test access port (TAP) that is fully compatible with the *IEEE 1149.1 Standard Test Access Port and Boundary Scan Architecture*. Problems associated with testing high density circuit boards have led to development of this proposed standard under the sponsorship of the Test Technology Committee of IEEE and the Joint Test Action Group (JTAG). The Avalanche implementation supports circuit-board test strategies based on this standard.

The TAP consists of five dedicated signal pins, a 16-state TAP controller, an instruction register, and three data registers. A boundary scan register links all device signal pins into a single shift register. The test logic, implemented utilizing static logic design, is independent of the device system logic. The Avalanche implementation provides the capability to:

1. Perform boundary scan operations to test circuit-board electrical continuity.
2. Bypass the Avalanche for a given circuit-board test by effectively reducing the boundary scan register to a single cell.
3. Sample the Avalanche system pins during operation and transparently shift out the result in the boundary scan register.
4. Disable the output drive to pins during circuit-board testing.
5. Set the Avalanche output drive pins to fixed logic values while reducing the shift register path to a single cell (bypass register).

NOTE: *Certain precautions must be observed to ensure that the JTAG module does not interfere with nontest operation. See section 20.9 Non-SCAN Chain Operation for details.*

20.2 Overview

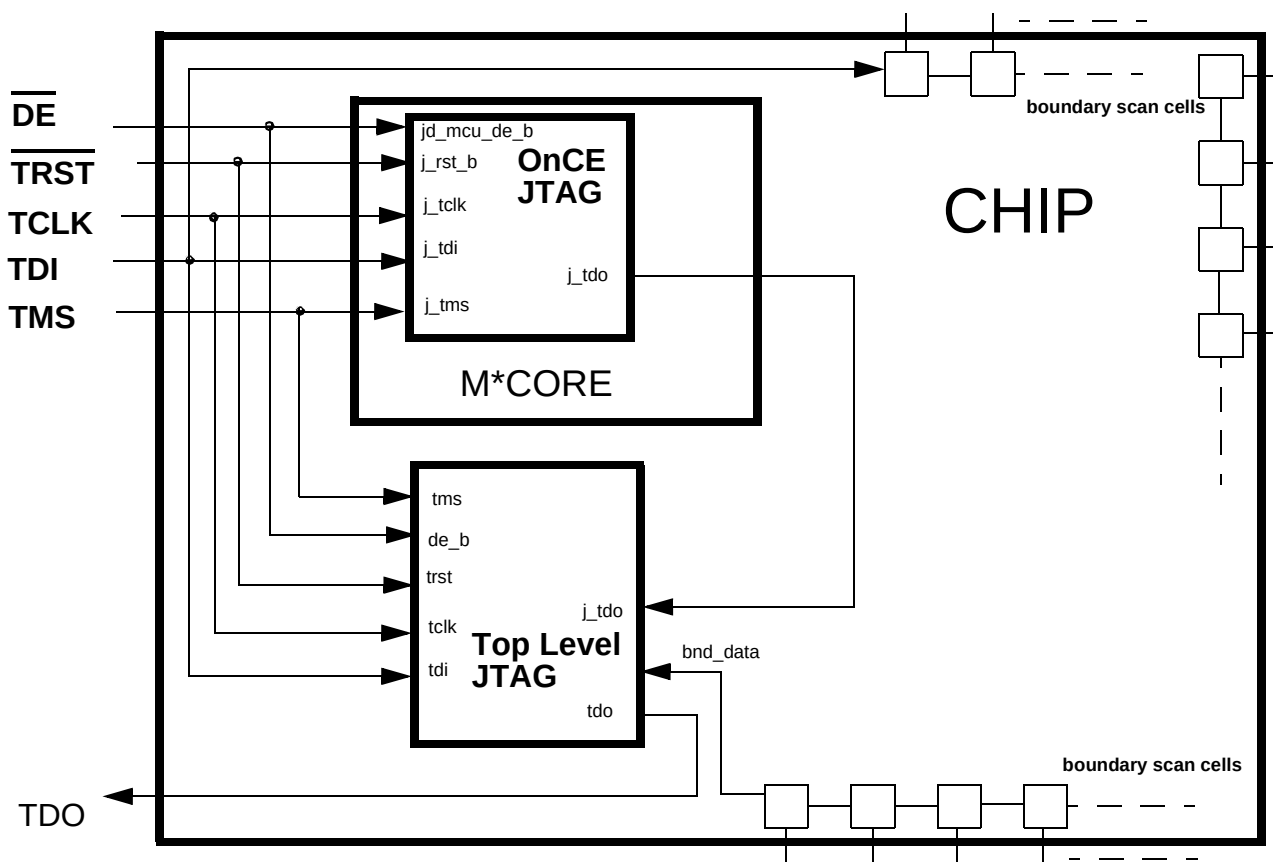
On Avalanche there exists two JTAG TAP controllers: a top level controller which controls the boundary scan register, and a controller inside the M*CORE which controls the operation of the M*CORE's on-chip emulation (OnCE) circuitry. The top level TAP controller will be visible at power-up and will function as a singular entity (JTAG compliant) until the M*CORE TAP controller is enabled. The OnCE TAP controller in the M*CORE is disabled on power-up. To enable the OnCE TAP controller, an enable sequence must be issued. This enable sequence is interpreted by the top level JTAG. After the OnCE enable sequence has been issued, the top level JTAG connects the internal OnCE TDO (j_tdo) to the pin TDO, and remains in the run-test/idle state. It will remain in this state until $\overline{\text{TRST}}$ is asserted low. While the OnCE TAP controller is enabled, the top level JTAG remains transparent.

The OnCE TAP controller can be enabled in either of two ways:

1. Assertion of the $\overline{\text{DE}}$ input (logic 0) for two TCLK periods while $\overline{\text{TRST}}$ is negated (logic 1) and the TAP controller is in the test-logic-reset state.
2. Shifting the ENABLE_MCU_ONCE command (4'b0011) into the instruction register, entering the update_IR state, and returning to the run-test-idle state. At this point, the OnCE TAP controller is enabled.

This section describes the top level JTAG module. For further information regarding OnCE and its JTAG TAP controller, refer to the M*CORE section of the Avalanche specification.

Figure 20-1 Avalanche's JTAG Connections

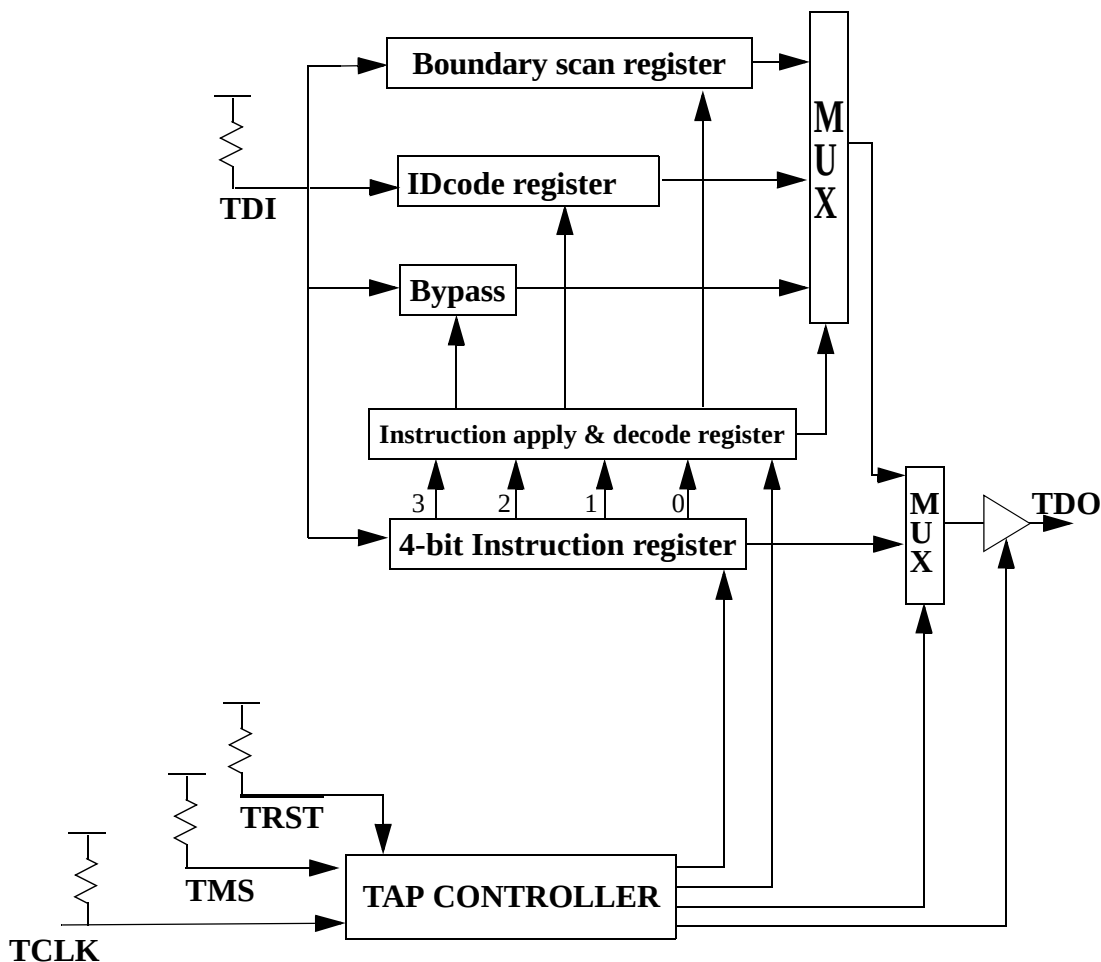


Avalanche's top level JTAG module includes a TAP controller, a 4-bit instruction register, and three test data registers (a 1-bit bypass register, boundary scan register, and a 32-bit IDcode register). This implementation includes a dedicated TAP consisting of the following signals:

- **TCLK**—a test clock input to synchronize the test logic. TCLK is independent of the Avalanche processor clock. It includes an internal pullup resistor.
- **TMS**—a test mode select input (with an internal pullup resistor) that is sampled on the rising edge of TCLK to sequence the TAP controller's state machine.
- **TDI**—a serial test data input (with an internal pullup resistor) that is sampled on the rising edge of TCLK.
- **TDO**—a three-stateable test data output that is actively driven in the shift-IR and shift-DR controller states. TDO changes on the falling edge of TCLK.

- $\overline{\text{TRST}}$ —an active low asynchronous reset with an internal pullup resistor that forces the TAP controller into the test-logic-reset state.

Figure 20-2 Avalanche's Top Level JTAG Block Diagram

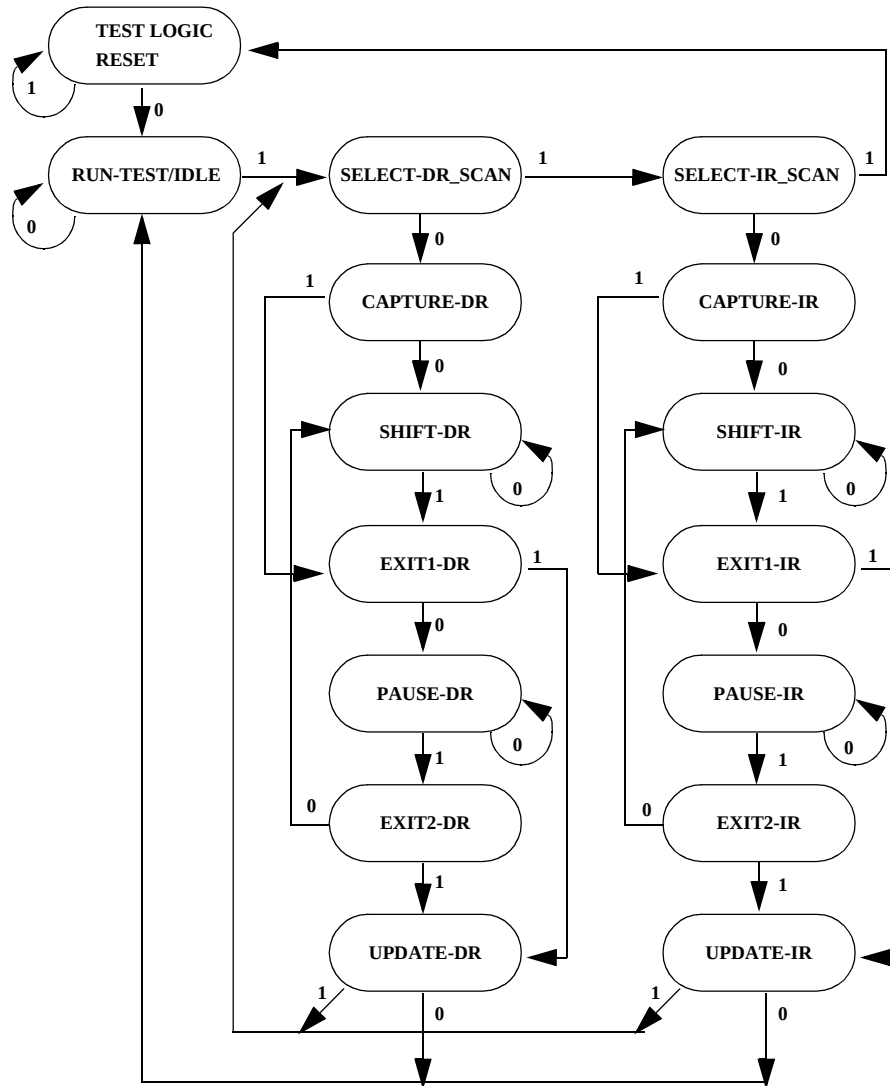


20.3 TAP Controller

The TAP controller is responsible for interpreting the sequence of logical values on the TMS signal. It is a synchronous state machine that controls the operation of the JTAG logic. The state machine is shown in Figure 20-3. The value shown adjacent to each arc represents the value of the TMS signal sampled on the rising edge of the TCLK signal.

A JTAG reset will cause the TAP controller state machine to enter the test-logic-reset state. An asynchronous JTAG reset is accomplished by asserting TRST. Holding TMS high while clocking TCLK through at least five rising edges will also reset JTAG.

Figure 20-3 TAP Controller State Machine



20.4 Instruction Shift Register

The Avalanche JTAG implementation uses a 4-bit instruction shift register without parity. This register transfers its value to a parallel hold register and applies an instruction on the falling edge of TCLK when the TAP state machine is in the update-IR state. To load the instructions into the shift portion of the register, place the serial data on the TDI pin prior to each rising edge of TCLK.

The MSB of the instruction shift register is the bit closest to the TDI pin and the LSB is the bit closest to the TDO pin.

Table 20-1 lists the instructions supported along with their encoding. The IEEE 1149.1 Standard requires the EXTEST, SAMPLE/PRELOAD, and BYPASS instructions. IDCODE, CLAMP, and HIGHZ are optional standard instructions that the Avalanche implementation supports and are described in the IEEE Standard 1149.1. There exists one public instruction, ENABLE_MCU_ONCE, to enable M*CORE's on-chip emulation (OnCE). In addition, there exists three private instructions which are intended for manufacturing purposes and should be avoided during circuit board testing.

Unused opcodes currently decode to BYPASS but Motorola reserves the right to change them in the future.

Table 20-1 JTAG Instructions

INSTRUCTION	ABBR	CLASS	IR[3:0]	INSTRUCTION SUMMARY
EXTEST	EXT	Public	0000	Selects BS register while applying fixed values to output pins and asserting functional reset
IDCODE	IDC	Public	0001	Selects IDcode register for shift
SAMPLE/PRELOAD	SMP	Public	0010	Selects BS register for shift, sample, and preload without disturbing functional operation
ENABLE_MCU_ONCE	EMO	Public	0011	Instruction to enable the M*CORE TAP controller
ENTER_SCAN	SCN	Private	0100	Instruction for chip manufacturing purposes only.
TEST_LEAKAGE	LKG	Private	0101	Instruction for chip manufacturing purposes only. ¹
EXTEST_SPC	SPC	Private	0110	Instruction for chip manufacturing purposes only.
RESERVED	-	-	0111 - 1000	Decoded to select BYPASS register ²
HIGHZ	HIZ	Public	1001	Selects the bypass register while three-stating all output pins and asserting functional reset
RESERVED	-	-	1010 - 1011	Decoded to select BYPASS register ²
CLAMP	CMP	Public	1100	Selects bypass while applying fixed values to output pins and asserting functional reset
RESERVED	-	-	1101 - 1110	Decoded to select BYPASS register ²
BYPASS	BYP	Public	1111	Selects the bypass register for data operations

NOTES:

1. To exit the TEST_LEAKAGE instruction, the $\overline{\text{TRST}}$ pin must be asserted.
2. Motorola reserves the right to change the decoding of the unused opcodes in the future.

20.4.1 EXTEST Instruction

The external test instruction (EXTEST) selects the boundary-scan register. The EXTEST instruction forces all output pins and bidirectional pins configured as outputs to the preloaded fixed values (with the SAMPLE/PRELOAD instruction) and held in the boundary-scan update registers. The EXTEST instruction can also configure the direction of bidirectional pins and establish high-impedance states on some pins. EXTEST also asserts internal reset for the Avalanche system logic to force a predictable internal state while performing external boundary scan operations.

20.4.2 IDCODE Instruction

The IDCODE instruction selects the 32-bit IDcode register for connection as a shift path between the TDI pin and the TDO pin. This instruction lets you interrogate the Avalanche to determine its version number and other part identification data. The IDcode register has been implemented in accordance with IEEE 1149.1 so that the least significant bit of the shift register stage is set to logic 1 on the rising edge of TCLK following entry into the capture-DR state. Therefore, the first bit to be shifted out after selecting the IDcode register is always a logic 1. The remaining 31-bits are also set to fixed values on the rising edge of TCLK following entry into the capture-DR state.

The IDCODE instruction is the default value placed in the instruction register when JTAG resets. Thus, after a JTAG reset, the IDcode register can be shifted out without having to shift in the IDCODE instruction.

20.4.3 SAMPLE/PRELOAD Instruction

The SAMPLE/PRELOAD instruction provides two separate functions. First, it obtains a sample of the system data and control signals present at the Avalanche input pins and just prior to the boundary scan cell at the output pins. This sampling occurs on the rising edge of TCLK in the capture-DR state when an instruction encoding of hex 2 is resident in the instruction register. You can observe this sampled data by shifting it through the boundary scan register to the output TDO by using the shift-DR state. Both the data capture and the shift operation are transparent to system operation.

NOTE: *You are responsible for providing some form of external synchronization to achieve meaningful results because there is no internal synchronization between TCLK and the system clock.*

The second function of the SAMPLE/PRELOAD instruction is to initialize the boundary scan register update cells before selecting EXTEST or CLAMP. This is achieved by ignoring the data being shifted out of the TDO pin while shifting in initialization data. The update-DR state in conjunction with the falling edge of TCLK can then transfer this data to the update cells. This data will be applied to the external output pins when EXTEST or CLAMP instruction is applied.

JTAG

20.4.4 ENABLE_MCU_ONCE Instruction

The ENABLE_MCU_ONCE is a public instruction to enable the M*CORE OnCE TAP controller. When the OnCE TAP controller is enabled, the top level JTAG connects the internal OnCE TDO (j_tdo) to the pin TDO, and remains in the run-test/idle state. It will remain in this state until TRST is asserted. While the OnCE TAP controller is enabled, the top level JTAG remains transparent.

20.4.5 HIGHZ Instruction

The HIGHZ instruction is provided as a manufacturer's optional public instruction to prevent having to backdrive the output pins during circuit-board testing. When HIGHZ is invoked, all output drivers, including the two-state drivers, are turned off (i.e., high impedance). The instruction selects the bypass register. HIGHZ also asserts internal reset for the Avalanche system logic to force a predictable internal state.

20.4.6 CLAMP Instruction

The CLAMP instruction selects the bypass register and asserts internal reset while simultaneously forcing all output pins and bidirectional pins configured as outputs to the fixed values that are preloaded and held in the boundary scan update register. This instruction enhances test efficiency by reducing the overall shift path to a single bit (the bypass register) while conducting an EXTEST type of instruction through the boundary scan register.

20.4.7 BYPASS Instruction

The BYPASS instruction selects the single-bit bypass register, creating a single-bit shift register path from the TDI pin to the bypass register to the TDO pin. This instruction enhances test efficiency by reducing the overall shift path when a device other than the Avalanche processor becomes the device under test on a board design with multiple chips on the overall 1149.1 defined boundary scan chain. The bypass register has been implemented in accordance with 1149.1 so that the shift register state is set to logic zero on the rising edge of TCLK following entry into the capture-DR state. Therefore, the first bit to be shifted out after selecting the bypass register is always a logic zero (to differentiate a part that supports an IDcode register from a part that supports only the bypass register).

20.5 IDcode Register

An IEEE 1149.1 compliant JTAG identification register has been included on Avalanche.

Figure 20-4 IDcode Register Bit Specification

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRN				0	1	0	1	1	1	0	0	PIN			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				0	0	0	0	0	0	0	1	1	1	0	1

- Bits 31-28: Version Number - Part Revision Number. This is equivalent to the lower four bits of the PRN of the Chip Identification Register located in the Chip Configuration Module.
- Bits 27-22: Design Center - Indicates the Motorola Microcontroller Division.
- Bits 21-12: Device Number - Part Identification Number. Bits 19-12 are equivalent to the PIN of the Chip Identification Register located in the Chip Configuration Module.
- Bits 11-1: JEDEC ID - Indicates the reduced JEDEC ID for Motorola (JEDEC refers to the Joint Electron Device Engineering Council. Refer to JEDEC publication 106-A and chapter 11 of the IEEE 1149.1 Standard for further information on this field).
- Bit 0: Differentiates this register as the JTAG IDcode register (as opposed to the bypass register) according to the IEEE 1149.1 Standard.

20.6 Bypass Register

Avalanche includes an IEEE 1149.1-compliant bypass register, which creates a single bit shift register path from TDI to the bypass register to TDO when the BYPASS instruction is selected.

20.7 Boundary SCAN Register

Avalanche includes an IEEE 1149.1-compliant boundary scan register. The boundary scan register is connected between TDI and TDO when the EXTEST or SAMPLE/PRELOAD instructions are selected. This register captures signal pin data on the input pins, forces fixed values on the output signal pins, and selects the direction and drive characteristics (a logic value or high impedance) of the bidirectional and three-state signal pins.

20.8 Restrictions

The test logic is implemented using static logic design, and TCLK can be stopped in either a high or low state without loss of data. The system logic, however, operates on a different system clock which is not synchronized to TCLK internally. Any mixed operation requiring the use of the 1149.1 test logic in conjunction with system functional logic that uses both clocks, must have coordination and synchronization of these clocks done externally.

The control afforded by the output enable signals using the boundary scan register and the EXTEST instruction requires a compatible circuit-board test environment to avoid device-destructive configurations. The user must avoid situations in which Avalanche output drivers are enabled into actively driven networks.

Avalanche features a low-power stop mode. The interaction of the scan chain interface with low-power stop mode is as follows:

1. The TAP controller must be in the test-logic-reset state to either enter or remain in the low-power stop mode. Leaving the test-logic-reset state negates the ability to achieve low-power, but does not otherwise affect device functionality.
2. The TCLK input is not blocked in low-power stop mode. To consume minimal power, the TCLK input should be externally connected to vdd.
3. The TMS, TDI, $\overline{\text{TRST}}$ pins include on-chip pullup resistors. In low-power stop mode, these three pins should remain either unconnected or connected to vdd to achieve minimal power consumption.

20.9 Non-SCAN Chain Operation

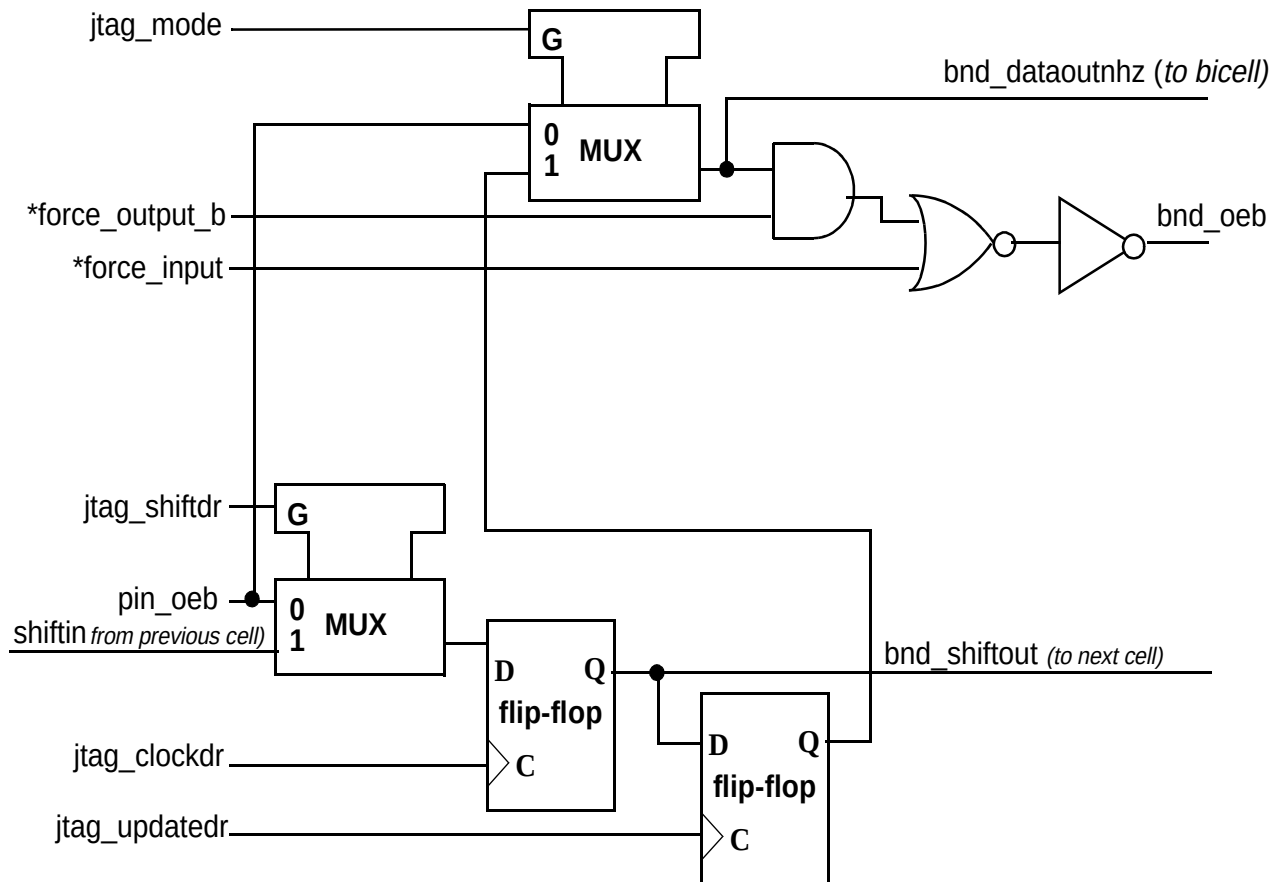
Keeping the TAP controller in the test-logic-reset state will ensure that the scan chain test logic is kept transparent to the system logic. It is recommended that TMS, TDI, TCLK, and $\overline{\text{TRST}}$ be pulled up. $\overline{\text{TRST}}$ could be connected to ground. However, since there is a pullup on $\overline{\text{TRST}}$, some amount of current will result. JTAG will be initialized to the test-logic-reset state on power up without $\overline{\text{TRST}}$ asserted low due to the JTAG power-on-reset internal input. The JTAG/OnCE module in the M*CORE also has the power-on-reset input.

20.10 Boundary Scan

The Avalanche boundary scan register consist of 200-bits. This register can be connected between TDI and TDO when EXTEST or SAMPLE/PRELOAD instructions are selected. This register is used for capturing signal pin data on the input pins, forcing fixed values on the output signal pins, and selecting the direction and drive characteristics (a logic value or high impedance)

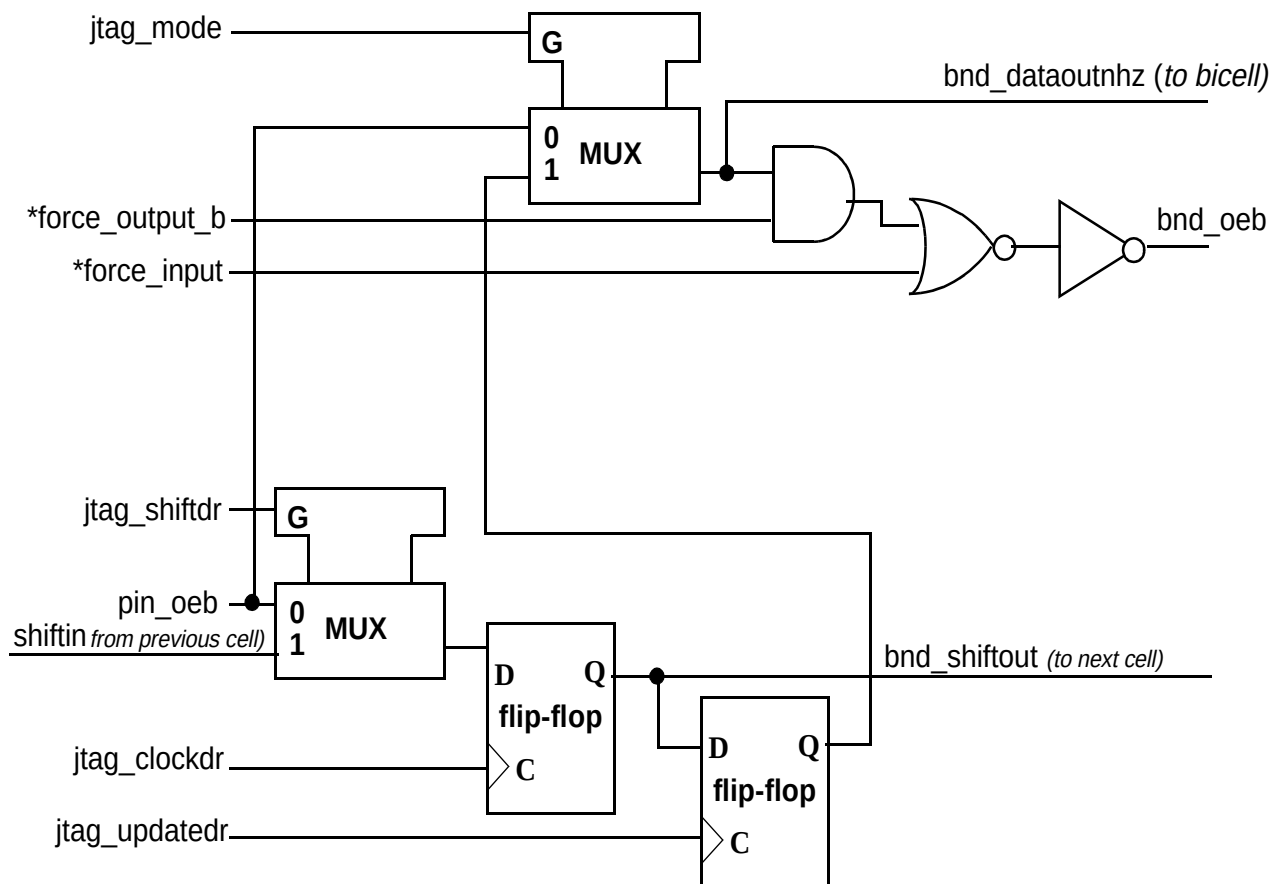
of the bidirectional and three-state signal pins. Figures 22-5 through 22-6 depict the various cell types.

Figure 20-5 Bidirectional Pin Cell (bnd_bicell)



**NOTE: See Avalanche Spec Appendix E (Test Controller Pin Direction)*

Figure 20-6 Control Cell for Bidi Pin Cell (bnd_dicell)



**NOTE: See Avalanche Spec Appendix E (Test Controller Pin Direction)*

This IEEE-1149.1 compliant boundary scan register contains bits for bonded-out and non-bonded signals excluding JTAG signals, analog signals, power supplies, compliance enable pins, and clock signals. To maintain JTAG compliance, TEST should be held to logic zero and DE should be held to logic one. These non-scanned pins are shown in Table 22-3, "List of Pins Not Scanned in JTAG mode".

Table 20-2 List of Pins Not Scanned in JTAG Mode

Pin Name	Pin Type
extal	clock/analog
xtal	clock/analog
exosc	clock/analog
xosc	clock/analog
vddsyn	supply
vsssyn	supply
trst	JTAG
tclk	JTAG
tms	JTAG
tdi	JTAG
tdo	JTAG
de	JTAG compliance enable
test	JTAG compliance enable
vddosc	supply
vssosc	supply
vstby	supply
vdd	supply
vss	supply

Below in Table 22-4, "Boundary Scan Bit Definition", is the bit ordering of the boundary scan register. The first column in the table defines the bit's ordinal position in the boundary scan register. The shift register cell nearest TDO (i.e., first to be shifted in) is defined as bit 0.

The second column references one of the Avalanche cell types.

The third column lists the pin name for all pin-related cells or lists an (*) to indicate that cell is a bidirectional control register bit (as represented in the BSDL file).

The fourth column lists the pin type for convenience.

The last column indicates the associated boundary scan register control cell bit number for bidirectional output pins(as indicated in the BSDL file).

Table 20-3 Boundary Scan Bit Definition

(NOTE: shaded region indicates optional pins)

Bit	Cell Type	Pin/ Control Cell Name	Pin Type	Control Cell (bit number)
0	bnd_bicell	d31	inout	1
1	bnd_dicell	*	-	-
2	bnd_bicell	a12	inout	3
3	bnd_dicell	*	-	-
4	bnd_bicell	a13	inout	5
5	bnd_dicell	*	-	-
6	bnd_bicell	a14	inout	7
7	bnd_dicell	*	-	-
8	bnd_bicell	a15	inout	9
9	bnd_dicell	*	-	-
10	bnd_bicell	a16	inout	11
11	bnd_dicell	*	-	-
12	bnd_bicell	a17	inout	13
13	bnd_dicell	*	-	-
14	bnd_bicell	clkout	inout	15
15	bnd_dicell	*	-	-
16	bnd_bicell	a18	inout	17
17	bnd_dicell	*	-	-
18	bnd_bicell	a19	inout	19
19	bnd_dicell	*	-	-
20	bnd_bicell	rstout	inout	21
21	bnd_dicell	*	-	-
22	bnd_bicell	a20	inout	23
23	bnd_dicell	*	-	-
24	bnd_bicell	reset	inout	25
25	bnd_dicell	*	-	-
26	bnd_bicell	a21	inout	27

Table 20-3 Boundary Scan Bit Definition

(NOTE: shaded region indicates optional pins)

Bit	Cell Type	Pin/ Control Cell Name	Pin Type	Control Cell (bit number)
27	bnd_dicell	*	-	-
28	bnd_bicell	a22	inout	29
29	bnd_dicell	*	-	-
30	bnd_bicell	tea	inout	31
31	bnd_dicell	*	-	-
32	bnd_bicell	eb0	inout	33
33	bnd_dicell	*	-	-
34	bnd_bicell	eb1	inout	35
35	bnd_dicell	*	-	-
36	bnd_bicell	ta	inout	37
37	bnd_dicell	*	-	-
38	bnd_bicell	eb2	inout	39
39	bnd_dicell	*	-	-
40	bnd_bicell	shs	inout	41
41	bnd_dicell	*	-	-
42	bnd_bicell	eb3	inout	43
43	bnd_dicell	*	-	-
44	bnd_bicell	oe	inout	45
45	bnd_dicell	*	-	-
46	bnd_bicell	ss	inout	47
47	bnd_dicell	*	-	-
48	bnd_bicell	sck	inout	49
49	bnd_dicell	*	-	-
50	bnd_bicell	miso	inout	51
51	bnd_dicell	*	-	-
52	bnd_bicell	mosi	inout	53
53	bnd_dicell	*	-	-
54	bnd_bicell	int7	inout	55
55	bnd_dicell	*	-	-
56	bnd_bicell	int6	inout	57
57	bnd_dicell	*	-	-
58	bnd_bicell	cs0	inout	59
59	bnd_dicell	*	-	-
60	bnd_bicell	cs1	inout	61
61	bnd_dicell	*	-	-
62	bnd_bicell	int5	inout	63
63	bnd_dicell	*	-	-

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Table 20-3 Boundary Scan Bit Definition*(NOTE: shaded region indicates optional pins)*

Bit	Cell Type	Pin/ Control Cell Name	Pin Type	Control Cell (bit number)
64	bnd_bicell	cs2	inout	65
65	bnd_dicell	*	-	-
66	bnd_bicell	int4	inout	67
67	bnd_dicell	*	-	-
68	bnd_bicell	cs3	inout	69
69	bnd_dicell	*	-	-
70	bnd_bicell	tc0	inout	71
71	bnd_dicell	*	-	-
72	bnd_bicell	int3	inout	73
73	bnd_dicell	*	-	-
74	bnd_bicell	tc1	inout	75
75	bnd_dicell	*	-	-
76	bnd_bicell	int2	inout	77
77	bnd_dicell	*	-	-
78	bnd_bicell	int1	inout	79
79	bnd_dicell	*	-	-
80	bnd_bicell	int0	inout	81
81	bnd_dicell	*	-	-
82	bnd_bicell	rx1	inout	83
83	bnd_dicell	*	-	-
84	bnd_bicell	tx1	inout	85
85	bnd_dicell	*	-	-
86	bnd_bicell	rx2	inout	87
87	bnd_dicell	*	-	-
88	bnd_bicell	tc2	inout	89
89	bnd_dicell	*	-	-
90	bnd_bicell	tx2	inout	91
91	bnd_dicell	*	-	-
92	bnd_bicell	cse0	inout	93
93	bnd_dicell	*	-	-
94	bnd_bicell	icoc1_0	inout	95
95	bnd_dicell	*	-	-
96	bnd_bicell	cse1	inout	97
97	bnd_dicell	*	-	-
98	bnd_bicell	rw	inout	99
99	bnd_dicell	*	-	-
100	bnd_bicell	icoc1_1	inout	101

Table 20-3 Boundary Scan Bit Definition*(NOTE: shaded region indicates optional pins)*

Bit	Cell Type	Pin/ Control Cell Name	Pin Type	Control Cell (bit number)
101	bnd_dicell	*	-	-
102	bnd_bicell	icoc1_2	inout	103
103	bnd_dicell	*	-	-
104	bnd_bicell	icoc1_3	inout	105
105	bnd_dicell	*	-	-
106	bnd_bicell	icoc2_0	inout	107
107	bnd_dicell	*	-	-
108	bnd_bicell	icoc2_1	inout	109
109	bnd_dicell	*	-	-
110	bnd_bicell	icoc2_2	inout	111
111	bnd_dicell	*	-	-
112	bnd_bicell	icoc2_3	inout	113
113	bnd_dicell	*	-	-
114	bnd_bicell	d0	inout	115
115	bnd_dicell	*	-	-
116	bnd_bicell	a0	inout	117
117	bnd_dicell	*	-	-
118	bnd_bicell	a1	inout	119
119	bnd_dicell	*	-	-
120	bnd_bicell	d1	inout	121
121	bnd_dicell	*	-	-
122	bnd_bicell	a2	inout	123
123	bnd_dicell	*	-	-
124	bnd_bicell	d2	inout	125
125	bnd_dicell	*	-	-
126	bnd_bicell	d3	inout	127
127	bnd_dicell	*	-	-
128	bnd_bicell	d4	inout	129
129	bnd_dicell	*	-	-
130	bnd_bicell	d5	inout	131
131	bnd_dicell	*	-	-
132	bnd_bicell	d6	inout	133
133	bnd_dicell	*	-	-
134	bnd_bicell	d7	inout	135
135	bnd_dicell	*	-	-
136	bnd_bicell	d8	inout	137
137	bnd_dicell	*	-	-

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Table 20-3 Boundary Scan Bit Definition*(NOTE: shaded region indicates optional pins)*

Bit	Cell Type	Pin/ Control Cell Name	Pin Type	Control Cell (bit number)
138	bnd_bicell	d9	inout	139
139	bnd_dicell	*	-	-
140	bnd_bicell	d10	inout	141
141	bnd_dicell	*	-	-
142	bnd_bicell	d11	inout	143
143	bnd_dicell	*	-	-
144	bnd_bicell	d12	inout	145
145	bnd_dicell	*	-	-
146	bnd_bicell	d13	inout	147
147	bnd_dicell	*	-	-
148	bnd_bicell	d14	inout	149
149	bnd_dicell	*	-	-
150	bnd_bicell	a3	inout	151
151	bnd_dicell	*	-	-
152	bnd_bicell	a4	inout	153
153	bnd_dicell	*	-	-
154	bnd_bicell	d15	inout	155
155	bnd_dicell	*	-	-
156	bnd_bicell	a5	inout	157
157	bnd_dicell	*	-	-
158	bnd_bicell	d16	inout	159
159	bnd_dicell	*	-	-
160	bnd_bicell	a6	inout	161
161	bnd_dicell	*	-	-
162	bnd_bicell	a7	inout	163
163	bnd_dicell	*	-	-
164	bnd_bicell	d17	inout	165
165	bnd_dicell	*	-	-
166	bnd_bicell	d18	inout	167
167	bnd_dicell	*	-	-
168	bnd_bicell	d19	inout	169
169	bnd_dicell	*	-	-
170	bnd_bicell	d20	inout	171
171	bnd_dicell	*	-	-
172	bnd_bicell	d21	inout	173
173	bnd_dicell	*	-	-
174	bnd_bicell	d22	inout	175

Table 20-3 Boundary Scan Bit Definition*(NOTE: shaded region indicates optional pins)*

Bit	Cell Type	Pin/ Control Cell Name	Pin Type	Control Cell (bit number)
175	bnd_dicell	*	-	-
176	bnd_bicell	a8	inout	177
177	bnd_dicell	*	-	-
178	bnd_bicell	a9	inout	179
179	bnd_dicell	*	-	-
180	bnd_bicell	d23	inout	181
181	bnd_dicell	*	-	-
182	bnd_bicell	a10	inout	183
183	bnd_dicell	*	-	-
184	bnd_bicell	d24	inout	185
185	bnd_dicell	*	-	-
186	bnd_bicell	d25	inout	187
187	bnd_dicell	*	-	-
188	bnd_bicell	a11	inout	189
189	bnd_dicell	*	-	-
190	bnd_bicell	d26	inout	191
191	bnd_dicell	*	-	-
192	bnd_bicell	d27	inout	193
193	bnd_dicell	*	-	-
194	bnd_bicell	d28	inout	195
195	bnd_dicell	*	-	-
196	bnd_bicell	d29	inout	197
197	bnd_dicell	*	-	-
198	bnd_bicell	d30	inout	199
199	bnd_dicell	*	-	-

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

Appendix A Electrical Characteristics

This appendix contains electrical specification tables and reference timing diagrams for the Avalanche microcontroller unit. This section contains detailed information on power considerations, DC/AC electrical characteristics, and AC timing specifications of Avalanche.

The electrical specifications are preliminary and are from previous designs or design simulations. These specifications may not be fully tested or guaranteed at this early stage of the product life cycle, however for production silicon these specifications will be met. Finalized specifications will be published after complete characterization and device qualifications have been completed.

NOTE:

The parameters specified in this MCU document supersede any values found in the module specifications.

A.1 Maximum Ratings

Table A-1 Absolute Maximum Ratings^{1, 2}

Num	Rating	Symbol	Value	Unit
1	Supply Voltage	V_{DD}	– 0.3 to +4.0	V
2	Clock Synthesizer Supply Voltage	V_{DDSYN}	– 0.3 to +4.0	V
3	TOD System and OSC Supply Voltage	V_{DDOSC}	– 0.3 to +4.0	V
4	RAM Memory Standby Supply Voltage	V_{STBY}	– 0.3 to + 4.0	V
5	Digital Input Voltage ³	V_{IN}	– 0.3 to + 4.0	V
6	Instantaneous Maximum Current Single pin limit (applies to all pins) ^{3, 4, 5}	I_D	25	mA
7	Operating Temperature Range (Packaged)	T_A ($T_L - T_H$)	– 40 to 85	°C
8	Storage Temperature Range	T_{stg}	– 65 to 150	°C

NOTES:

1. Functional operating conditions are given in DC Electrical Specifications. Absolute Maximum Ratings are stress ratings only, and functional operation at the maxima is not guaranteed. Stress beyond those listed may affect device reliability or cause permanent damage to the device.
2. This device contains circuitry protecting against damage due to high static voltage or electrical fields; however, it is advised that normal precautions be taken to avoid application of any voltages higher than maximum-rated voltages to this high-impedance circuit. Reliability of operation is enhanced if unused inputs are tied to an appropriate logic voltage level (e.g., either V_{SS} or V_{DD}).
3. Input must be current limited to the value specified. To determine the value of the required current-limiting resistor, calculate resistance values for positive and negative clamp voltages, then use the larger of the two values.
4. All functional non-supply pins are internally clamped to V_{SS} and V_{DD} .
5. Power supply must maintain regulation within operating V_{DD} range during instantaneous and operating maximum current conditions. If positive injection current ($V_{in} > V_{DD}$) is greater than I_{DD} , the injection current may flow out of V_{DD} and could result in external power supply going out of regulation. Insure external V_{DD} load will shunt current greater than maximum injection current. This will be the greatest risk when the MCU is not consuming power (ex; no clock). Power supply must maintain regulation within operating V_{DD} range during instantaneous and operating maximum current conditions.

A.2 Thermal Characteristics

Table A-2 Thermal Characteristics

Num	Rating	Symbol	Value	Unit
	Thermal Resistance TBD	θ_{JA}		$^{\circ}\text{C/W}$

The average chip-junction temperature (T_J) in $^{\circ}\text{C}$ can be obtained from:

$$T_J = T_A + (P_D \times \theta_{JA}) \quad (1)$$

Where:

T_A = Ambient Temperature, $^{\circ}\text{C}$

θ_{JA} = Package Thermal Resistance, Junction-to-Ambient, $^{\circ}\text{C/W}$

P_D = $P_{INT} + P_{I/O}$

P_{INT} = $I_{DD} \times V_{DD}$, Watts - Chip Internal Power

$P_{I/O}$ = Power Dissipation on Input and Output Pins — User Determined

For most applications $P_{I/O} < P_{INT}$ and can be neglected. An approximate relationship between P_D and T_J (if $P_{I/O}$ is neglected) is:

$$P_D = K + (T_J + 273^{\circ}\text{C}) \quad (2)$$

Solving equations 1 and 2 for K gives:

$$K = P_D \times (T_A + 273^{\circ}\text{C}) + \theta_{JA} \times P_D^2 \quad (3)$$

where K is a constant pertaining to the particular part. K can be determined from equation (3) by measuring P_D (at equilibrium) for a known T_A . Using this value of K, the values of P_D and T_J can be obtained by solving equations (1) and (2) iteratively for any value of T_A .

A.3 ESD Protection

Table A-3 ESD Protection Characteristics^{1, 2}

Characteristics	Symbol	Value	Units
ESD Target for Human Body Model	HBM	2000	V
ESD Target for Machine Model	MM	200	V
HBM Circuit Description	R _{series}	1500	W
	C	100	pF
MM Circuit Description	R _{series}	0	W
	C	200	pF
Number of pulses per pin (HBM) positive pulses negative pulses	—	1	—
	—	1	—
Number of pulses per pin (MM) positive pulses negative pulses	—	3	—
	—	3	—
Interval of Pulses	—	1	s

NOTES:

1. All ESD testing is in conformity with CDF-AEC-Q100 Stress Test Qualification for Automotive Grade Integrated Circuits.
2. A device will be defined as a failure if after exposure to ESD pulses the device no longer meets the device specification requirements. Complete DC parametric and functional testing shall be performed per applicable device specification at room temperature followed by hot temperature, unless specified otherwise in the device specification.

A.4 DC Electrical Specifications

Table A-4 DC Electrical Specifications¹

($V_{SS} = V_{SSSYN} = V_{SSF} = V_{SSA} = 0 V_{DC}$) ($T_A = T_L$ to T_H)

Num	Characteristic	Symbol	Min	Max	Unit
1	Input High Voltage	V_{IH}	2.0	$V_{DD} + 0.3$	V
2	Input Low Voltage	V_{IL}	$V_{SS} - 0.3$	0.8	V
3	Input Hysteresis	V_{HYS}	300	—	mV
4	Input Leakage Current $V_{in} = V_{DD}$ or V_{SS} , Input-only pins	I_{in}	-1.0	1.0	μA
5	High Impedance (Off-State) Leakage Current $V_{in} = V_{DD}$ or V_{SS} , All input/output and output pins	I_{OZ}	-1.0	1.0	μA
6	Output High Voltage (All input/output and all output pins) $I_{OH} = -2.0$ mA	V_{OH}	$V_{DD} - 0.5$	—	V
7	Output Low Voltage (All input/output and all output pins) $I_{OL} = 2.0$ mA	V_{OL}	—	0.5	V
8	Weak Internal Pull Up Device Current, tested at V_{IL} Max.	I_{APU}	-10	- 130	μA
9	Weak Internal Pull Down Device Current, tested at V_{IH} Min.	I_{APD}	10	130	μA
10	Input Capacitance ² All input-only pins All input/output (three-state) pins	C_{in}	— —	7 7	pF
11	Load Capacitance (50% Partial Drive) (100% Full Drive)	C_L		25 50	pF
12	Supply Voltage (includes core modules and pads)	V_{DD}	2.7	3.6	V
13	Clock Synthesizer Supply Voltage	V_{DDSYN}	2.7	3.6	V
14	TOD and 32KHz OSC Supply Voltage	V_{OSCY}	2.7	3.6	V
15	RAM Memory Standby Supply Voltage Normal Operation: $V_{DD} > V_{STBY} - 0.5$ V Standby Mode: $V_{DD} < V_{STBY} - 0.5$ V	V_{STBY}	0.0 2.7	3.6 3.6	V
16					

Table A-4 DC Electrical Specifications¹ $(V_{SS} = V_{SSSYN} = V_{SSF} = V_{SSA} = 0 V_{DC}) (T_A = T_L \text{ to } T_H)$

Num	Characteristic	Symbol	Min	Max	Unit
17	Operating Supply Current ³ Master Mode Single Chip Mode WAIT DOZE STOP	I_{DD}	— — — — —	60 40 15 10 10	mA mA mA mA μA
18	Clock Synthesizer Supply Current Normal Operation 8.25 MHz crystal, VCO on, Max f_{sys} STOP (OSC and PLL enabled) STOP (OSC enable, PLL disabled) STOP (OSC and PLL disabled)	I_{DDSYN}	— — — —	4 2 1 10	mA mA mA μA
19	TOD and 32KHz OSC Supply Current	I_{OSC}	—	TBD	mA
20	RAM Memory Standby Supply Current Normal Operation: $V_{DD} > V_{STBY} - 0.5 V$ Transient Condition: $V_{STBY} - 0.5 V > V_{DD} > V_{SS} + 0.5 V$ Standby Operation: $V_{DD} < V_{SS} + 0.5 V$	I_{STBY}	— — —	10 7 20	μA mA μA
21					
22					
23	DC Injection Current ^{2, 4, 5, 6} $V_{NEGCLAMP} = V_{SS} - 0.3 V$, $V_{POSCLAMP} = V_{DD} + 0.3 V$ Single Pin Limit Total MCU Limit, Includes sum of all stressed pins	I_{IC}	-1.0 -10	1.0 10	mA
24	POR Reset Voltage	V_{POR}	0	1.89	V
25	POR Re-arm Voltage	V_{ARM}	0	1.59	V
26	POR V_{DD} Ramp Rate	PRR	0.03	-	V/ms

NOTES:

1. Refer to Table A-5 for additional PLL specifications.
2. This parameter is characterized before qualification rather than 100% tested.
3. Current measured at maximum system clock frequency, all modules active, and default drive strength with matching load.
4. All functional non-supply pins are internally clamped to V_{SS} and their respective V_{DD} .
5. Input must be current limited to the value specified. To determine the value of the required current-limiting resistor, calculate resistance values for positive and negative clamp voltages, then use the larger of the two values.

6. Power supply must maintain regulation within operating V_{DD} range during instantaneous and operating maximum current conditions. If positive injection current ($V_{in} > V_{DD}$) is greater than I_{DD} , the injection current may flow out of V_{DD} and could result in external power supply going out of regulation. Insure external V_{DD} load will shunt current greater than maximum injection current. This will be the greatest risk when the MCU is not consuming power. Examples are: if no system clock is present, or if clock rate is very low which would reduce overall power consumption. Also, at power-up, system clock is not present during the power-up sequence until the PLL has attained lock.

A.5 Phase Lock Loop Electrical Specifications

Table A-5 PLL Electrical Specifications
(V_{DD} and $V_{DDSYN} = 2.7$ to 3.6 V, $V_{SS} = V_{SSSYN} = 0$ V, $T_A = T_L$ to T_H)

Num	Characteristic	Symbol	Min	Max	Unit
P1	PLL Reference Frequency Range				
	Crystal reference	$f_{ref_crystal}$	2	10.0	MHz
	External reference	f_{ref_ext}	2	33.0	
	1:1 Mode	$f_{ref_1:1}$	10	33.0	
P2	System Frequency ¹ External reference On-Chip PLL Frequency	f_{sys}	0 $f_{sys} \times 0.016$	33.0 33.0	MHz
P3	Loss of Reference Frequency ^{2, 4}	f_{LOR}	100	250	kHz
P4	Self Clocked Mode Frequency ^{3, 4}	f_{SCM}	0.5	15	MHz
P5	Crystal Start-up Time ^{4, 5}	t_{cst}	—	10	ms
P6	EXTAL Input High Voltage Crystal Mode All other modes (1:1, Bypass, External)	V_{IHEXT}	$V_{DDSYN} - 1.0$ 2.0	V_{DDSYN} V_{DDSYN}	V
P7	EXTAL Input Low Voltage Crystal Mode All other modes (1:1, Bypass, External)	V_{ILEXT}	V_{SSSYN} V_{SSSYN}	1.0 0.8	V
P8	XTAL Output High Voltage $I_{OH} = 1.0$ mA	V_{OL}	$V_{DDSYN} - 1.0$	—	V
P9	XTAL Output Low Voltage $I_{OL} = 1.0$ mA	V_{OL}	—	0.5	V
P10	XTAL Load Capacitance		5	30	pF
P11	PLL Lock Time ^{4, 6}	t_{lpll}	—	200	μ s
P12	Power-up To Lock Time ^{4, 5, 7} With Crystal Reference (includes P5 time)	t_{lplk}	—	11	ms
	Without Crystal Reference		—	200	μ s
P13	1:1 Clock Skew (between CLKOUT and EXTAL) ⁸	t_{skew}	-2	2	ns
P14	Duty Cycle of reference ⁴	t_{dc}	40	60	% f_{sys}
P15	Frequency un-LOCK Range	f_{UL}	- 1.5	1.5	% f_{sys}
P16	Frequency LOCK Range	f_{LCK}	- 0.75	0.75	% f_{sys}
P17	CLKOUT Period Jitter, ^{4, 5, 7, 9} Measured at f_{sys} Max Peak-to-peak Jitter (Clock edge to clock edge) Long Term Jitter (Averaged over 2 ms interval)	C_{jitter}	— —	5 .01	% f_{sys}

NOTES:

1. All internal registers retain data at 0 Hz.
2. "Loss of Reference Frequency" is the reference frequency detected internally, which transitions the PLL into self clocked mode.
3. Self clocked mode frequency is the frequency that the PLL operates at when the reference frequency falls below f_{LOR} with default MFD/RFD settings.
4. This parameter is characterized before qualification rather than 100% tested.
5. Proper PC board layout procedures must be followed to achieve specifications.
6. This specification applies to the period required for the PLL to relock after changing the MFD frequency control bits in the synthesizer control register (SYNCR).
7. Assuming a reference is available at power up, lock time is measured from the time V_{DD} and V_{DDSYN} are valid to $\overline{RSTOUT}$ negating. If the crystal oscillator is being used as the reference for the PLL, then the crystal start up time must be added to the PLL lock time to determine the total start-up time.
8. PLL is operating in 1:1 PLL mode.
9. Jitter is the average deviation from the programmed frequency measured over the specified interval at maximum f_{sys} . Measurements are made with the device powered by filtered supplies and clocked by a stable external clock signal. Noise injected into the PLL circuitry via V_{DDSYN} and V_{SSSYN} and variation in crystal oscillator frequency increase the Cjitter percentage for a given interval.

A.6 External Interface Timing Characteristics

$D_F = 3.135 \text{ to } 3.465 \text{ V}$, $V_{PP} = 5.25 \text{ to } 5.75 \text{ V}$, $T_A = T_L \text{ to } T_H$)¹

Characteristic	Symbol	Value			Unit
		Minimum	Typical	Maximum	
Number of erase pulses	E_{PULSE}	1	3	<100	—
Erase pulse time	T_{ERASE}		100		ms
Erase recovery time ²	T_{E_OFF}	4.0	4.8	6.0	μs
Number of program pulses	P_{PULSE}	1	75	<150	—
Program pulse time	T_{PROG}	10.7	12.8	32.0	μs
Program recovery time ²	T_{P_OFF}	4.0	4.8	6.0	μs

NOTES:

1. T_L is defined to be -40 C and T_H is defined to be 85 C.
2. See the Flash Specification for more information on the scaled clock period.

Table A-6 External Interface Timing
($V_{DD} = 2.7 \text{ to } 3.6 \text{ V}$, $V_{SS} = 0 \text{ V}$, $T_A = T_L \text{ to } T_H$)¹

NUM	Characteristic	Symbol	Min	Max	Unit
1	CLKOUT Period	t_{CYC}	30	-	ns
2	CLKOUT Low Pulse Width	t_{CLW}	$0.5 t_{CYC} - 1$	-	ns
3	CLKOUT High Pulse Width	t_{CHW}	$0.5 t_{CYC} - 1$	-	ns
4	All Rise Times	t_{CR}	-	3	ns
5	All Fall Times	t_{CF}	-	3	ns
6	CLKOUT High to A[23:0], TSIZ[1:0] Valid ²	t_{CHAV}	-	10	ns
7	CLKOUT High to A[23:0], TSIZ[1:0] Invalid	t_{CHAI}	0	-	ns
8	CLKOUT High to $\overline{CS7-0}$ Asserted ²	t_{CHCA}	-	10	ns
9	CLKOUT High to $\overline{CS7-0}$ Negated	t_{CHCN}	0	-	ns
12	CLKOUT High to TC[2:0], PSTAT[3:0] Valid	t_{CHTV}	-	15	ns
13	CLKOUT High to TC[2:0], PSTAT[3:0] Invalid	t_{CHTI}	0	-	ns
14	CLKOUT High to $\overline{R/W}$ High (hold time)	t_{CHRWV}	0	10	ns
15	CLKOUT High to $\overline{R/W}$ Valid (write) ²	t_{CHRWV}	$0.25 t_{CYC}$	$0.25 t_{CYC} + 8$	ns

Table A-6 External Interface Timing (Continued)
 $(V_{DD} = 2.7 \text{ to } 3.6 \text{ V}, V_{SS} = 0 \text{ V}, T_A = T_L \text{ to } T_H)^1$

NUM	Characteristic	Symbol	Min	Max	Unit
16	CLKOUT High to $\overline{OE}$, $\overline{EB}$ Asserted ^{2, 3}	t_{CHOEA}	$0.25 t_{CYC}$	$0.25 t_{CYC} + 8$	ns
17	CLKOUT High to $\overline{OE}$, $\overline{EB}$ (read) Negated	t_{CHOEN}	0	8	ns
17A	CLKOUT Low to $\overline{EB}$ (write) Negated	t_{CLEN}	$0.25 t_{CYC}$	$0.25 t_{CYC} + 8$	ns
18	CLKOUT Low to $\overline{SHS}$ Low	t_{CLSL}	0	10	ns
19	CLKOUT High to $\overline{SHS}$ High ⁴	t_{CHSH}	0	8	ns
20	CLKOUT Low to Data-out Low Impedance (write/show)	t_{CHDOD}	0	-	ns
21	CLKOUT High to Data-out High Impedance (write/show) ^{4,5}	t_{CHDOZ}	-	$0.25 t_{CYC} + 6$	ns
22	CLKOUT Low to Data-out Valid (write)	t_{CLDOVW}	-	10	ns
22A	CLKOUT Low to Data-out Valid (show)	t_{CLDOVS}	-	10	ns
23	CLKOUT High to Data-out Invalid (write/show) ^{4,5}	t_{CHDOIW}	$0.25 t_{CYC}$	-	ns
24	Data-in Valid to CLKOUT High (read)	t_{DIVCH}	22	-	ns
25	CLKOUT High to Data-in Invalid (read)	t_{CHDII}	2	-	ns
26	$\overline{TA}$, $\overline{TEA}$ Asserted to CLKOUT High	t_{TACH}	18	-	ns
27	CLKOUT High to $\overline{TA}$, $\overline{TEA}$ Negated	t_{CHTN}	2	-	ns

NOTES:

1. All AC timing is shown with respect to 20% V_{DD} and 80% V_{DD} levels unless otherwise noted.
2. A[23:0], TSIZ[1:0], $\overline{CS7-0}$ Valid to $R/\overline{W}$ (write), $\overline{OE}$, $\overline{EB}$ Asserted (minimum) spec is 0ns. (This parameter is characterized before qualification rather than 100% tested.)
3. Write/Show Data High-Z to $\overline{OE}$ Asserted (minimum) spec is 0ns. (This parameter is characterized before qualification rather than 100% tested.)
4. $\overline{SHS}$ High to Show Data or Write Data Invalid (minimum) spec is 2ns. (This parameter is characterized before qualification rather than 100% tested.)
5. Write/Show Data High-Z and Write/Show Data Invalid is 0 ns for synchronous reset conditions.

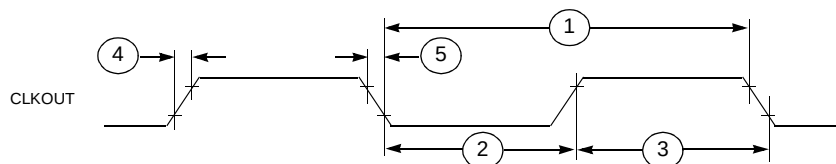


Figure A-1 CLKOUT Timing

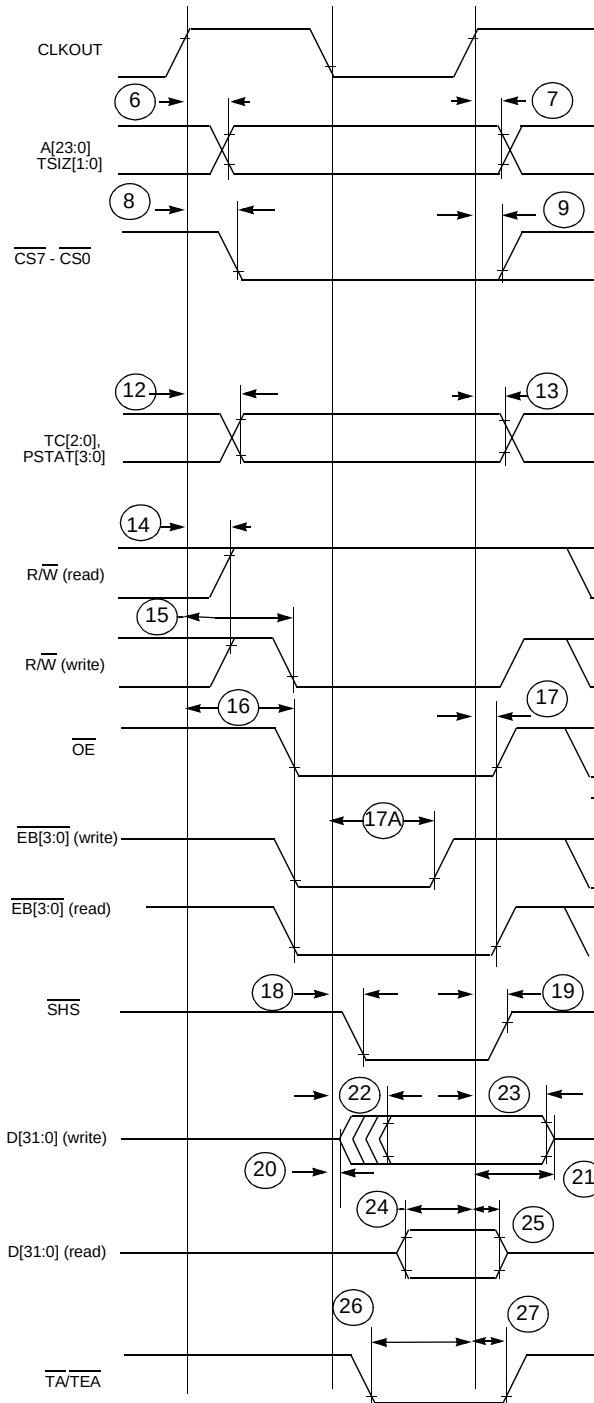


Figure A-2 1-Clock Read/Write Cycle Timing

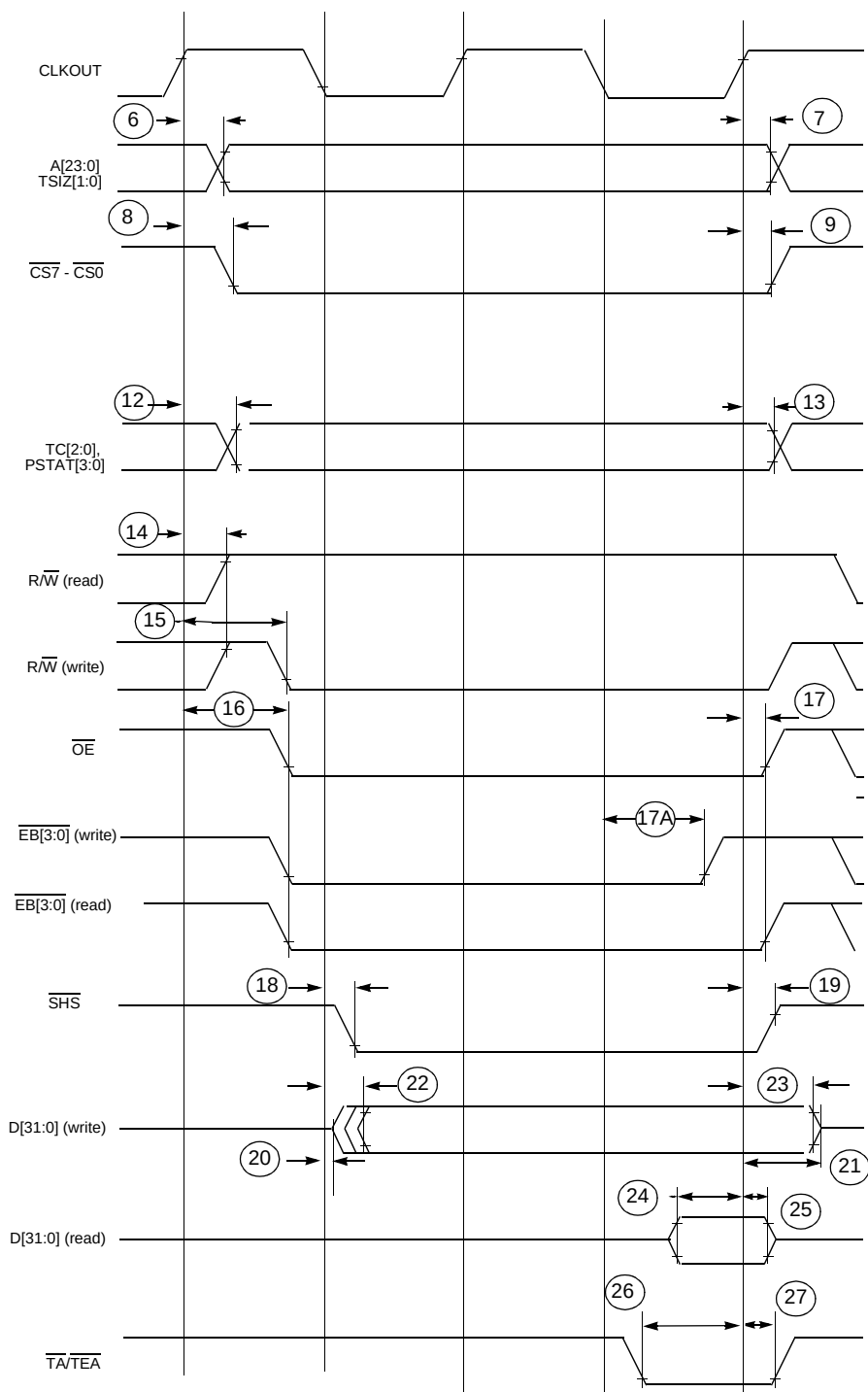


Figure A-3 Read/Write Cycle Timing with Wait States

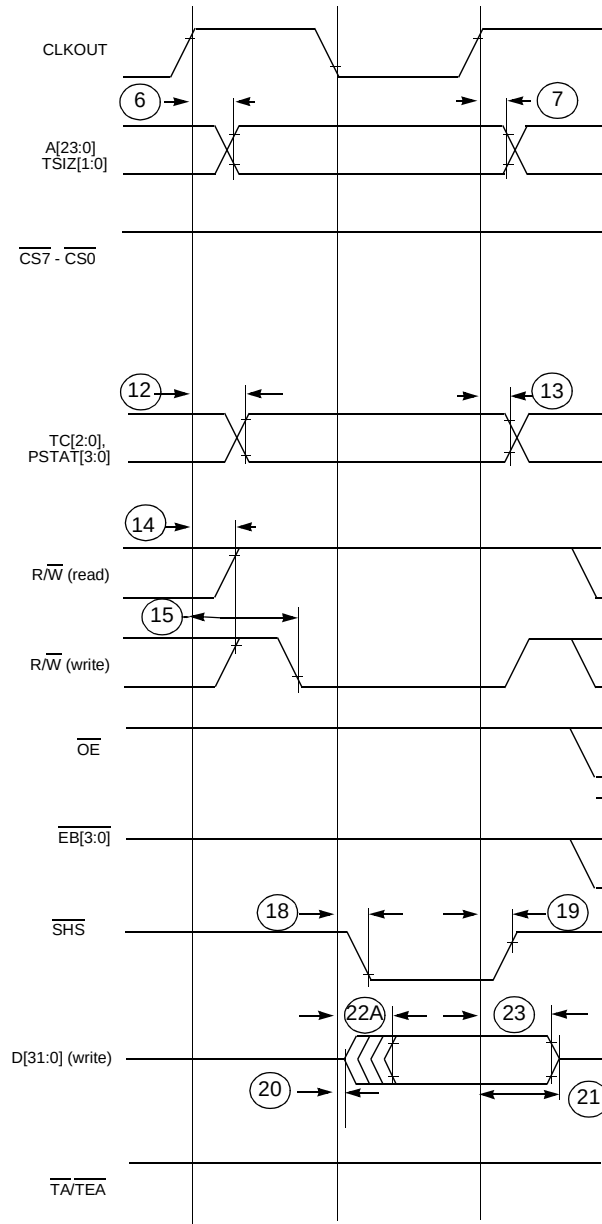


Figure A-4 Show Cycle Timing

A.7 General Purpose I/O Timing

Table A-7 GPIO Timing¹
 $(V_{DD} = 2.7 \text{ to } 3.6 \text{ V}, V_{SS} = 0 \text{ V}, T_A = T_L \text{ to } T_H)^2$

NUM	Characteristic	Symbol	Min	Max	Unit
G1	CLKOUT High to GPIO Output Valid	t_{CHPOV}	-	20	ns
G2	CLKOUT High to GPIO Output Invalid	t_{CHPOI}	0	-	ns
G3	GPIO Input Valid to CLKOUT High	t_{PVCH}	8	-	ns
G4	CLKOUT High to GPIO Input Invalid	t_{CHPI}	8	-	ns

NOTES:

- GPIO pins include: Ports A-I, Edge Port (including $\overline{\text{INT}}$ functions), SPI, SCI1/2 (including SCI functions), Timer1/2 (including Timer functions), PQA, and PQB (input only).
- All AC timing is shown with respect to 20% V_{DD} and 80% V_{DD} levels unless otherwise noted.

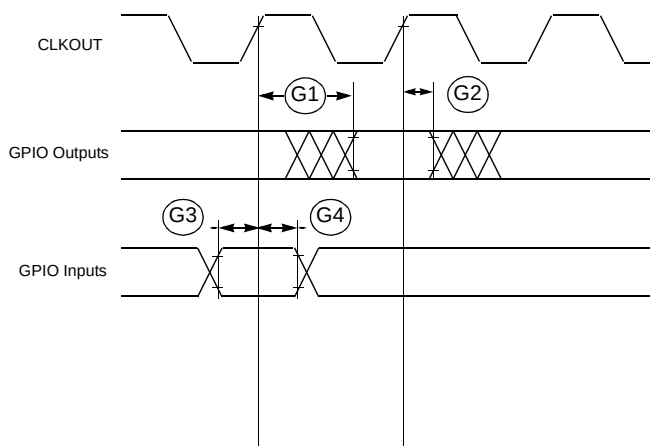


Figure A-5 GPIO Timing

A.8 Reset and Configuration Override Timing

Table A-8 Reset and Configuration Override Timing

($V_{DD} = 2.7$ to 3.6 V, $V_{SS} = 0$ V, $T_A = T_L$ to T_H)¹

NUM	Characteristic	Symbol	Min	Max	Unit
R1	$\overline{\text{RESET}}$ Input Asserted to CLKOUT High	t_{RACH}	8	-	ns
R2	CLKOUT High to $\overline{\text{RESET}}$ Input Negated	t_{CHRN}	8	-	ns
R3	$\overline{\text{RESET}}$ Input Assertion Time ²	t_{RIAT}	5	-	t_{CYC}
R4	CLKOUT High to $\overline{\text{RSTOUT}}$ Valid ³	t_{CHROV}	-	20	ns
R5	$\overline{\text{RSTOUT}}$ Asserted to Config. Overrides Asserted	t_{ROACA}	0	-	ns
R6	Configuration Override Setup Time to $\overline{\text{RSTOUT}}$ Negated	t_{COS}	20	-	t_{CYC}
R7	Configuration Override Hold Time after $\overline{\text{RSTOUT}}$ Negated	t_{COH}	0	-	ns
R8	$\overline{\text{RSTOUT}}$ Negated to Configuration Override High Impedance	t_{RONCZ}	-	1	t_{CYC}

NOTES:

1. All AC timing is shown with respect to 20% V_{DD} and 80% V_{DD} levels unless otherwise noted.
2. During low power **STOP**, the synchronizers for the $\overline{\text{RESET}}$ input are bypassed and $\overline{\text{RESET}}$ is asserted asynchronously to the system. Thus, $\overline{\text{RESET}}$ must be held a minimum of 100 ns.
3. This parameter also covers the timing of the Show Interrupt function.

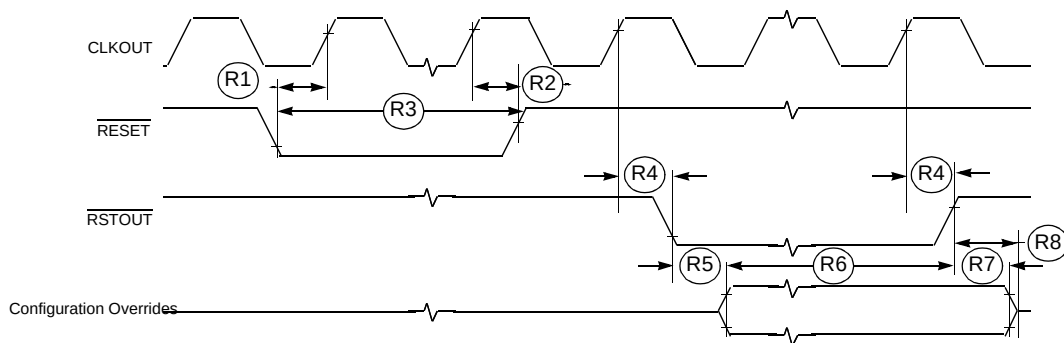


Figure A-6 $\overline{\text{RESET}}$ and Configuration Override Timing

A.9 SPI Timing Characteristics

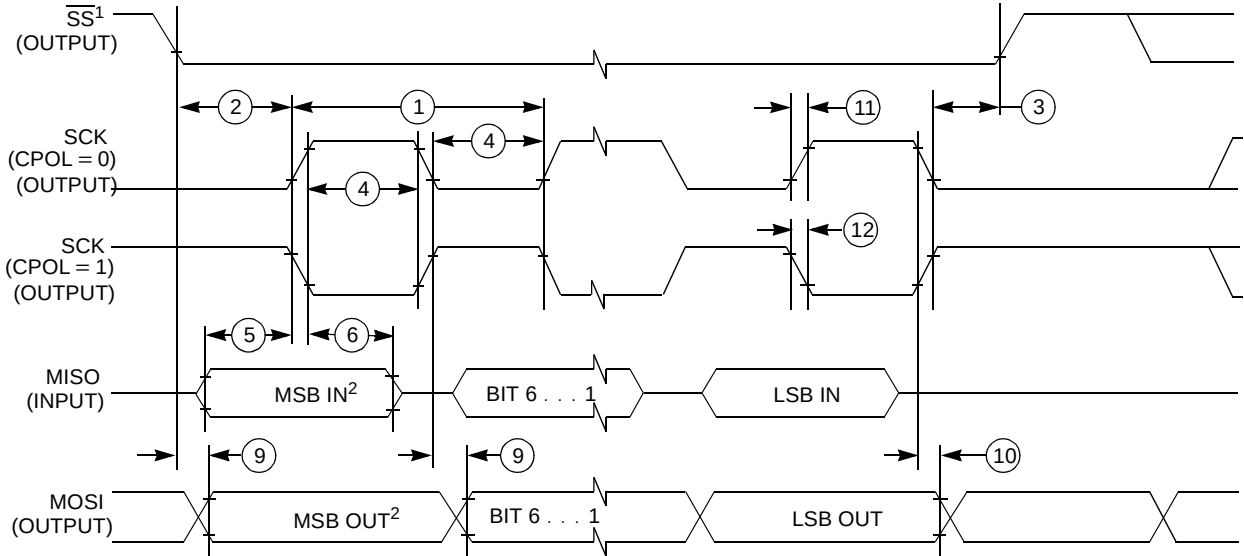
Table A-9 SPI Timing

($V_{DD} = 2.7$ to 3.6 , $V_{SS} = 0$ V, $T_A = T_L$ to T_H) ¹

Num	Function	Symbol	Min	Max	Unit
	Operating Frequency Master Slave	f_{op}	DC DC	1/2 1/2	system frequency
1	SCK Period Master Slave	t_{sck}	2 2	2048 —	t_{cyc} t_{cyc}
2	Enable Lead Time Master Slave	t_{lead}	1/2 1	— —	t_{sck} t_{cyc}
3	Enable Lag Time Master Slave	t_{lag}	1/2 1	— —	t_{sck} t_{cyc}
4	Clock (SCK) High or Low Time Master Slave	t_{wsck}	$t_{cyc} - 30$ $t_{cyc} - 30$	1024 t_{cyc} —	ns ns
5	Data Setup Time (Inputs) Master Slave	t_{su}	25 25	— —	ns ns
6	Data Hold Time (Inputs) Master Slave	t_{hi}	0 25	— —	ns ns
7	Slave Access Time	t_a	—	1	t_{cyc}
8	Slave MISO Disable Time	t_{dis}	—	1	t_{cyc}
9	Data Valid (after SCK Edge) Master Slave	t_v	— —	25 25	ns ns
10	Data Hold Time (Outputs) Master Slave	t_{ho}	0 0	— —	ns ns
11	Rise Time Input Output	t_{ri} t_{ro}	— —	$t_{cyc} - 25$ 25	ns ns
12	Fall Time Input Output	t_{fi} t_{fo}	— —	$t_{cyc} - 25$ 25	ns ns

NOTES:

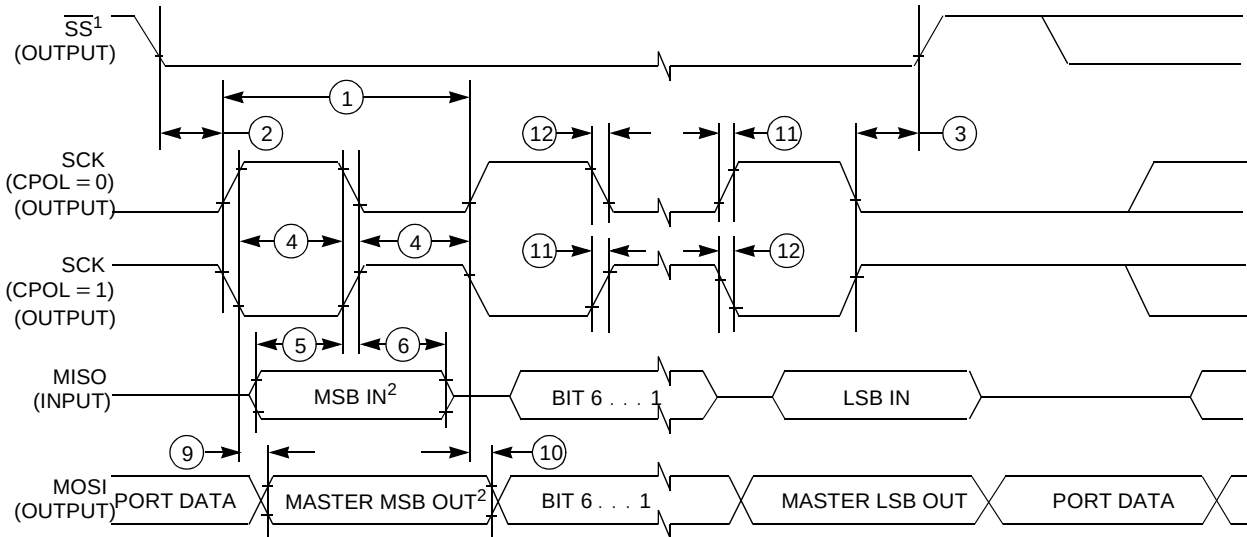
1. All AC timing is shown with respect to 20% V_{DD} and 70% V_{DD} levels unless otherwise noted.
2. Timing is based on Wired-or mode being off. With Wired-or mode on, timing will depend on pull-up value.



1. $\overline{SS}$ output mode (DDSR7 = 1, SSOE = 1).
2. LSBF = 0. For LSBF = 1, bit order is LSB, bit 1, ..., bit 6, MSB.

SPI MASTER CPHA0

A) SPI Master Timing (CPHA = 0)

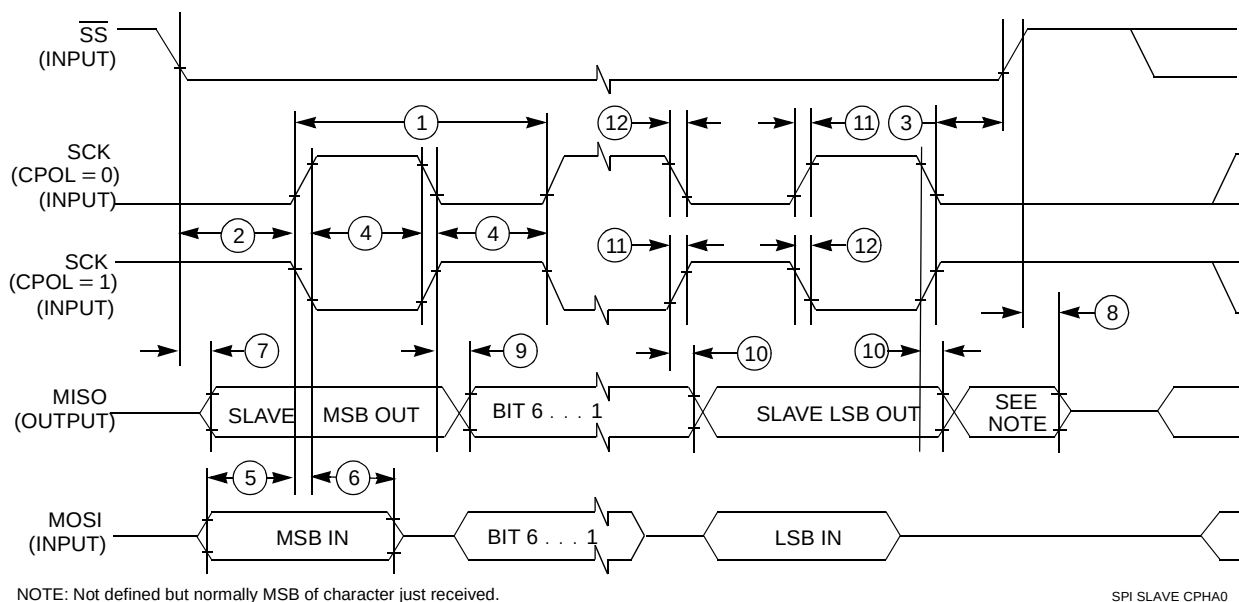


1. $\overline{SS}$ output mode (DDSR7 = 1, SSOE = 1).
2. LSBF = 0. For LSBF = 1, bit order is LSB, bit 1, ..., bit 6, MSB.

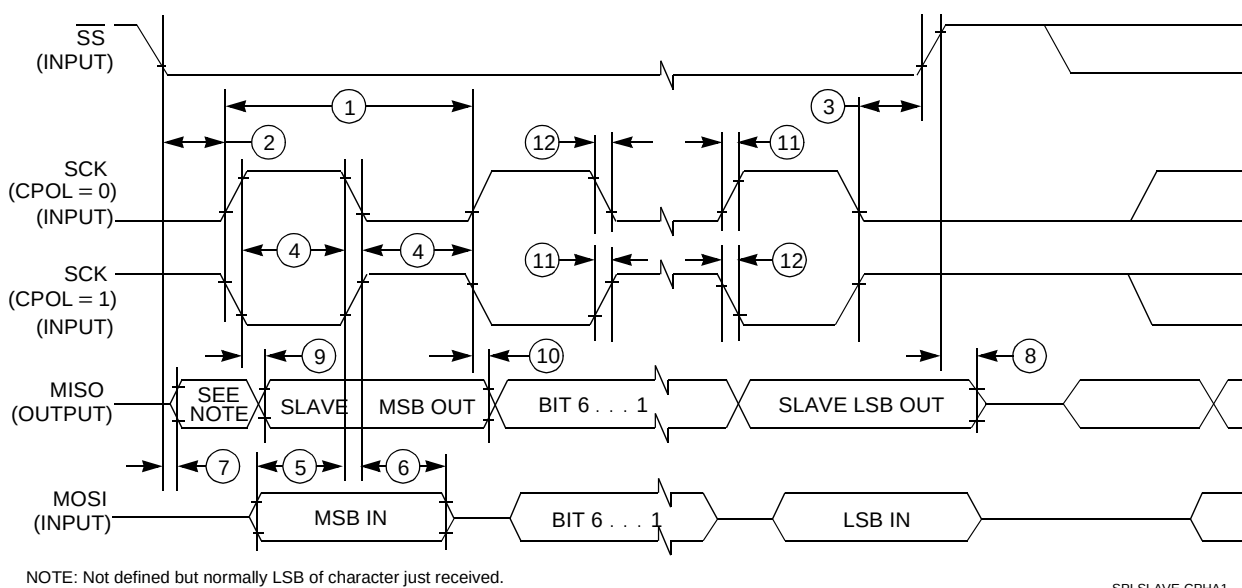
SPI MASTER CPHA1

B) SPI Master Timing (CPHA = 1)

Figure A-7 SPI Timing Diagram (1 of 2)



A) SPI Slave Timing (CPHA = 0)



B) SPI Slave Timing (CPHA = 1)

Figure A-8 SPI Timing Diagram (2 of 2)

A.10 OnCE, JTAG and Boundary Scan Timing

Table A-10 OnCE, JTAG and Boundary Scan Timing

Num	Characteristics	Symbol	Min	Max	Unit
1	TCLK Frequency of Operation	f_{JCYC}	DC	1/2	System frequency
2	TCLK Cycle Period	t_{JCYC}	2	-	System period
3	TCLK Clock Pulse Width	t_{JCW}	25	-	ns
4	TCLK Rise and Fall Times	t_{JCRF}	0.0	3.0	ns
5	Boundary Scan Input Data Setup Time to TCLK Rise	t_{BSDST}	5.0	-	ns
6	Boundary Scan Input Data Hold Time after TCLK Rise	t_{BSDHT}	24.0	-	ns
7	TCLK Low to Boundary Scan Output Data Valid	t_{BSDV}	0.0	40.0	ns
8	TCLK Low to Boundary Scan Output High Z	t_{BSDZ}	0.0	40.0	ns
9	TMS, TDI Input Data Setup Time to TCLK Rise	t_{TAPDST}	7.0	-	ns
10	TMS, TDI Input Data Hold Time after TCLK Rise	t_{TAPDHT}	15.0	-	ns
11	TCLK Low to TDO Data Valid	t_{TDODV}	0.0	16.0	ns
12	TCLK Low to TDO High Z	t_{TDODZ}	0.0	9.0	ns
13	$\overline{TRST}$ Assert Time	t_{TRSTAT}	100.0	-	ns
14	$\overline{TRST}$ Setup Time to TCLK Low	t_{TRSTST}	10.0	-	ns
15	$\overline{DE}$ Input Data Setup Time to CLKOUT Rise	t_{DEDST}	8.0	-	ns
16	$\overline{DE}$ Input Data Hold Time after CLKOUT Rise	t_{DEDHT}	8.0	-	ns
17	CLKOUT Low to $\overline{DE}$ Data Valid	t_{DEDV}	0.0	20.0	ns
18	CLKOUT Low to $\overline{DE}$ High Z	t_{DEDZ}	0.0	10.0	ns

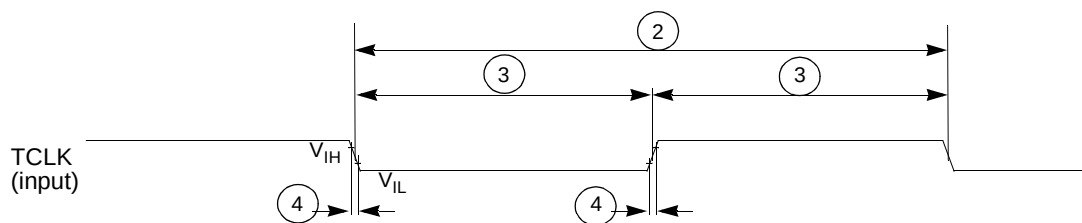


Figure A-9 Test Clock Input Timing

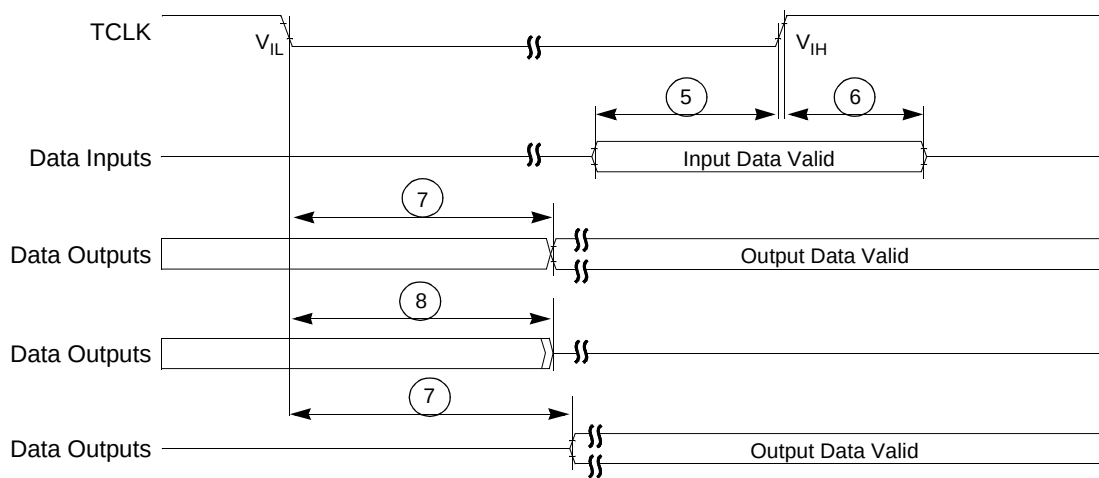


Figure A-10 Boundary Scan (JTAG) Timing

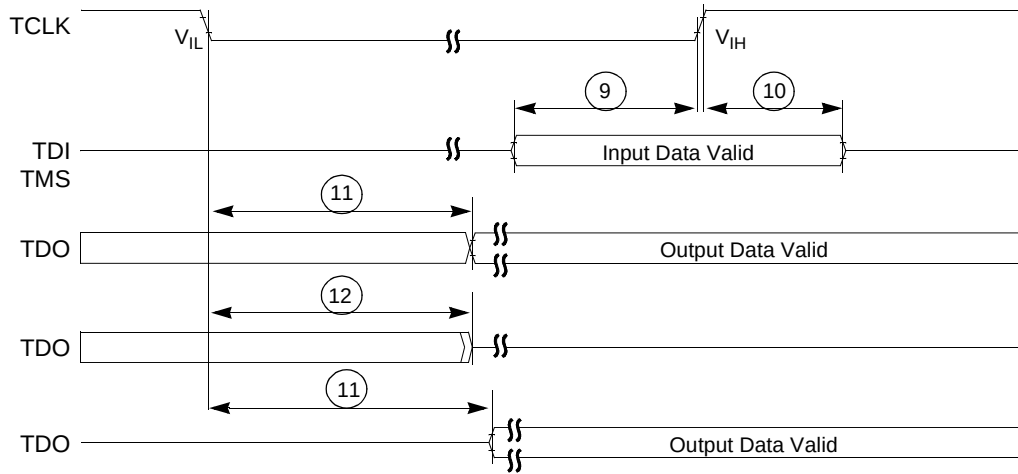


Figure A-11 Test Access Port Timing

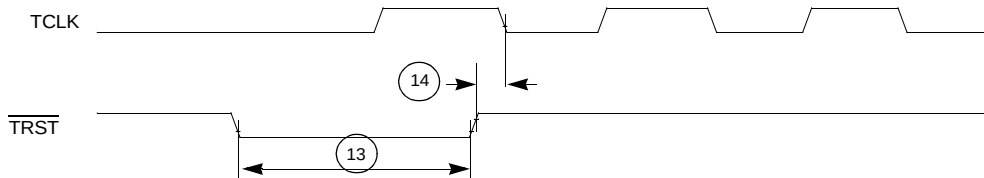


Figure A-12 TRST Timing

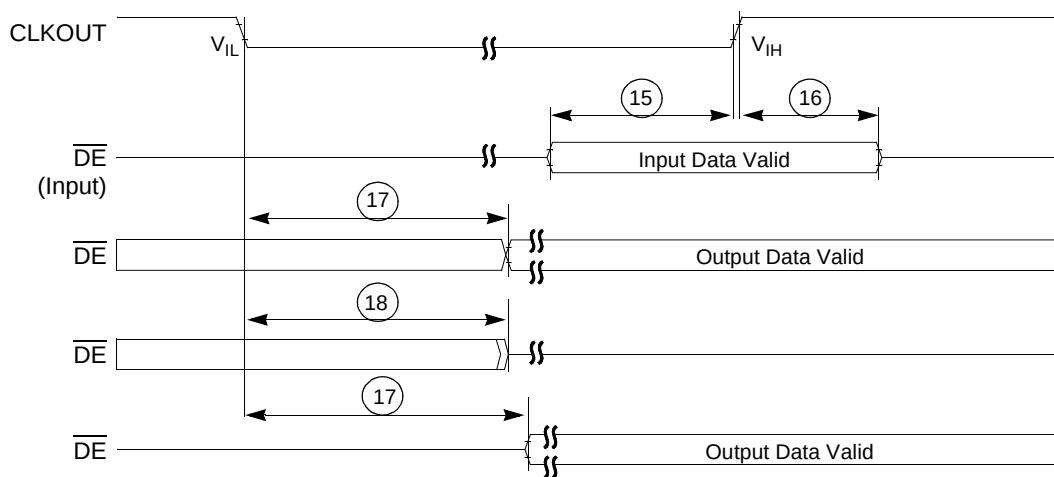


Figure A-13 Debug Event Pin Timing

Appendix B Package

The Avalanche die will be packaged in 192 MAPBGA package. This package supports full expanded mode of operation with 16 dedicated general purpose I/O ports. The Avalanche die has a total of 175 bond pads.

Function name ballmap																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	vsscore	dsrci1	rstout	vsscore	xosc	exosc	vsscore	xtal	extal	vsscore	pb7	pb6	pb5	pa5	pa3	vsscore
B	vddcore	vsscore	dsrci0	trst	vssosc	vddosc	vsscore	vddsyn	vsssyn	vsscore	pb4	pb3	pb2	pb1	pa4	vsscore
C	dsrc0	vddcore	vsscore	reset	de	vsscore	test	vsscore	vsscore	clkout	pa7	pa6	pa1	pa2	vsscore	tms
D	dsrc02	dsrc01	vddcore	vsscore	vddcore	vddcore	vstby	vddcore	vsscore	vddcore	pb0	pa0	vsscore	vsscore	tdo	tdi
E	ta	dsrc04	dsrc03	vddcore									vddcore	tc1k	icoc3	icoc2
F	shs	tea	oe	vddcore									vddcore	icoc1	icoc0	rxd2
G	eb0	eb2	eb3	eb1			vsscore	vsscore	vsscore	vsscore			txd1	txd2	rx1d	mosi
H	cs0	cs1	cs2	vsscore			vsscore	vsscore	vsscore	vsscore			vsscore	miso	sck	ss
J	cs3	cs4	cs7	cs5			vsscore	vsscore	vsscore	vsscore			vsscore	int6	int7	int5
K	cs6	tc0	tc1	vddcore			vsscore	vsscore	vsscore	vsscore			d30	int2	int4	int3
L	tc2	rw	d0	a0									vddcore	int0	int1	d31
M	a1	a2	d3	vddcore									vddcore	a23	d29	d28
N	d1	d2	a3	vsscore	vddcore	vddcore	vddcore	vsscore	vsscore	a15	vddcore	vddcore	vsscore	d27	a22	a21
P	a4	a5	vsscore	d7	d8	a10	d12	a12	d16	d17	d20	d23	vddcore	vsscore	d26	d25
R	d4	vsscore	d6	a7	d9	a9	a11	d15	a14	d19	a17	d22	a19	vddcore	vsscore	d24
T	vsscore	d5	a6	a8	d10	d11	d13	d14	a13	d18	a16	d21	a18	a20	vddcore	vsscore
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Figure B-1 Ball MAP(top view)

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

B.1 Pinout Table

The following table summarizes the pinouts for both packages.

Table 20-4 Ball Map Table

PIN	PAD NAME	BGA ball	PIN	PAD NAME	BGA ball	PIN	PAD NAME	BGA ball	PIN	PAD NAME	BGA ball
N/A	VSSCORE	A1	3	DSRCO[0]	C1	8	TA	E1	12	EB[0]	G1
174	DSRCI[1]	A2	N/A	VDDCORE	C2	7	DSRCO[4]	E2	14	EB[2]	G2
172	RSTOUT	A3	N/A	VSSCORE	C3	6	DSRCO[3]	E3	15	EB[3]	G3
N/A	VSSCORE	A4	171	RESET	C4	N/A	VDDCORE	E4	13	EB[1]	G4
166	XOSC	A5	169	DE	C5						
165	EXOSC	A6	N/A	VSSCORE	C6						
N/A	VSSCORE	A7	163	TEST	C7				N/A	VSSCORE	G7
160	XTAL	A8	N/A	VSSCORE	C8				N/A	VSSCORE	G8
159	EXTAL	A9	N/A	VSSCORE	C9				N/A	VSSCORE	G9
N/A	VSSCORE	A10	156	CLKOUT	C10				N/A	VSSCORE	G10
153	PB[7]	A11	144	PA[7]	C11						
152	PB[6]	A12	143	PA[6]	C12						
151	PB[5]	A13	136	PA[1]	C13	N/A	VDDCORE	E13	117	TXD1	G13
140	PA[5]	A14	137	PA[2]	C14	127	TCLK	E14	119	TXD2	G14
138	PA[3]	A15	N/A	VSSCORE	C15	126	ICOC[3]	E15	118	RXD1	G15
N/A	VSSCORE	A16	130	TMS	C16	125	ICOC[2]	E16	116	MOSI	G16
N/A	VDDCORE	B1	5	DSRCO[2]	D1	11	SHS	F1	18	CS0	H1
N/A	VSSCORE	B2	4	DSRCO[1]	D2	9	TEA	F2	19	CS1	H2
173	DSRCI[0]	B3	N/A	VDDCORE	D3	10	OE	F3	20	CS2	H3
170	TRST	B4	N/A	VSSCORE	D4	N/A	VDDCORE	F4	N/A	VSSCORE	H4
167	VSSOSC	B5	N/A	VDDCORE	D5						
164	VDDOSC	B6	N/A	VDDCORE	D6						
N/A	VSSCORE	B7	162	VSTBY	D7				N/A	VSSCORE	H7
161	VDDSYN	B8	N/A	VDDCORE	D8				N/A	VSSCORE	H8
158	VSSSYN	B9	N/A	VSSCORE	D9				N/A	VSSCORE	H9
N/A	VSSCORE	B10	N/A	VDDCORE	D10				N/A	VSSCORE	H10
150	PB[4]	B11	145	PB[0]	D11						
148	PB[3]	B12	135	PA[0]	D12						
147	PB[2]	B13	N/A	VSSCORE	D13	N/A	VDDCORE	F13	N/A	VSSCORE	H13
146	PB[1]	B14	N/A	VSSCORE	D14	124	ICOC[1]	F14	115	MISO	H14
139	PA[4]	B15	129	TDO	D15	123	ICOC[0]	F15	114	SCK	H15
N/A	VSSCORE	B16	128	TDI	D16	122	RXD2	F16	113	SS	H16

Table 20-4 Ball Map Table

PIN	PAD NAME	BGA ball	PIN	PAD NAME	BGA ball	PIN	PAD NAME	BGA ball	PIN	PAD NAME	BGA ball
21	CS3	J1	28	TC[2]	L1	36	D[1]	N1	42	D[4]	R1
22	CS4	J2	29	RW	L2	37	D[2]	N2	N/A	VSSCORE	R2
25	CS7	J3	35	D[0]	L3	39	A[3]	N3	48	D[6]	R3
23	CS5	J4	32	A[0]	L4	N/A	VSSCORE	N4	51	A[7]	R4
						N/A	VDDCORE	N5	54	D[9]	R5
						N/A	VDDCORE	N6	57	A[9]	R6
N/A	VSSCORE	J7				N/A	VDDCORE	N7	61	A[11]	R7
N/A	VSSCORE	J8				N/A	VSSCORE	N8	65	D[15]	R8
N/A	VSSCORE	J9				N/A	VSSCORE	N9	70	A[14]	R9
N/A	VSSCORE	J10				77	A[15]	N10	74	D[19]	R10
						N/A	VDDCORE	N11	79	A[17]	R11
						N/A	VDDCORE	N12	82	D[22]	R12
N/A	VSSCORE	J13	N/A	VDCORE	L13	N/A	VSSCORE	N13	85	A[19]	R13
111	INT[6]	J14	104	INT[0]	L14	94	D[27]	N14	N/A	VDDCORE	R14
112	INT[7]	J15	105	INT[1]	L15	96	A[22]	N15	N/A	VSSCORE	R15
109	INT[5]	J16	101	D[31]	L16	95	A[21]	N16	91	D[24]	R16
24	CS6	K1	33	A[1]	M1	40	A[4]	P1	N/A	VSSCORE	T1
26	TC[0]	K2	34	A[2]	M2	41	A[5]	P2	47	D[5]	T2
27	TC[1]	K3	38	D[3]	M3	N/A	VSSCORE	P3	50	A[6]	T3
N/A	VDDCORE	K4	N/A	VDDCORE	M4	49	D[7]	P4	52	A[8]	T4
						53	D[8]	P5	55	D[10]	T5
						60	A[10]	P6	56	D[11]	T6
N/A	VDDCORE	K7				62	D[12]	P7	63	D[13]	T7
N/A	VSSCORE	K8				68	A[12]	P8	64	D[14]	T8
N/A	VSSCORE	K9				71	D[16]	P9	69	A[13]	T9
N/A	VSSCORE	K10				72	D[17]	P10	73	D[18]	T10
						80	D[20]	P11	78	A[16]	T11
						83	D[23]	P12	81	D[21]	T12
100	D[30]	K13	N/A	VDDCORE	M13	N/A	VDDCORE	P13	84	A[18]	T13
106	INT[2]	K14	97	A[23]	M14	N/A	VSSCORE	P14	86	A[20]	T14
108	INT[4]	K15	99	D[29]	M15	93	D[26]	P15	N/A	VDDCORE	T15
107	INT[3]	K16	98	D[28]	M16	92	D[25]	P16	N/A	VSSCORE	T16

Specification End Sheet

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

PRINTED VERSIONS ARE UNCONTROLLED EXCEPT WHEN STAMPED "CONTROLLED COPY" IN RED

**FINAL PAGE OF
342
PAGES**